

Develando al Leviatán: Proceso ágil basado en Scrum para gestionar la deuda social en el desarrollo de software ágil



Universidad del Cauca

Luis Miguel Romero Vivas
Manuel Santiago Córdoba Galíndez

Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Departamento de Sistemas
Grupo de investigación en tecnologías de la información – GTI
Línea de investigación ingeniería de software aplicada
Popayán, marzo de 2026

Develando al Leviatán: Proceso ágil basado en Scrum para gestionar la deuda social en el desarrollo de software ágil



Luis Miguel Romero Vivas
Manuel Santiago Córdoba Galíndez

Trabajo de investigación para optar al título de Ingenieros de Sistemas

Director: *César Jesús Pardo Calvache, PhD. MS.c.*

Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Departamento de Sistemas
Grupo de investigación en tecnologías de la información – GTI
Línea de investigación ingeniería de software aplicada
Popayán, marzo de 2026

AGRADECIMIENTOS

A mi familia, por su paciencia, apoyo y amor incondicional durante toda la carrera universitaria.

A mi compañero de tesis, Manuel Santiago Córdoba Galíndez, por su colaboración, apoyo y amistad durante el desarrollo de este trabajo de grado.

A nuestro director de tesis, el profesor PhD. MS.c. César Jesús Pardo Calvache, por su apoyo profesional y personal, su paciencia y motivación durante todo el desarrollo de este trabajo de grado.

A mis amigos que me brindaron su apoyo, paciencia y comprensión durante este proceso.

Luis Miguel Romero Vivas
Pereira, 2026

AGRADECIMIENTOS

A mi familia, por el respaldo constante, la comprensión y el cariño incondicional que me acompañaron a lo largo de toda mi formación universitaria.

A mi compañero de tesis, Luis Miguel Romero Vivas, por su disposición, apoyo y compañerismo durante el desarrollo de este trabajo de grado.

A nuestro director de tesis, el profesor PhD. MS.c. César Jesús Pardo Calvache, por su orientación académica, calidad humana, paciencia y motivación a lo largo de todo el proceso investigativo.

A mis amigos, por su acompañamiento, comprensión y apoyo durante las distintas etapas de este camino.

Manuel Santiago Córdoba Galíndez
Bogotá, 2026

RESUMEN

La deuda social es un concepto que se refiere a las desigualdades y carencias que afectan a ciertos grupos dentro de una sociedad. En este contexto, el presente trabajo aborda la gestión de la deuda social mediante un enfoque ágil, proponiendo un proceso que permite identificar, priorizar y abordar las necesidades sociales de manera efectiva y eficiente.

Objetivo. El objetivo de este trabajo es desarrollar un proceso ágil para la gestión de la deuda social, que permita a las organizaciones y comunidades identificar y abordar las necesidades sociales de manera rápida y efectiva.

Métodos. Se utilizó un enfoque cualitativo basado en la revisión bibliográfica, diseño conceptual y validación mediante juicio de expertos para desarrollar el proceso ágil propuesto.

Resultados.

Conclusiones.

Trabajos futuros.

Palabras clave: Deuda social, gestión ágil, necesidades sociales, proceso ágil, comunidades, organizaciones.

ABSTRACT

The social debt is a concept that refers to the inequalities and needs that affect certain groups within a society. In this context, this work addresses the management of social debt through an agile approach, proposing a process that allows identifying, prioritizing and addressing social needs in an effective and efficient way.

Keywords: Social Debt, Agile, Scrum, Social Scrum.

TABLA DE CONTENIDO

RESUMEN	III
ABSTRACT	IV
CAPÍTULO 1. Introducción	1
1.1. Planteamiento del problema	1
1.2. Objetivos	3
1.3. Método de investigación	4
1.4. Estructura del documento	5
CAPÍTULO 2. Marco teórico y trabajos relacionados	7
2.1. Marco teórico	7
2.1.1. Deuda social en el desarrollo de software	7
2.1.1.1. Analogía y distinción con la deuda técnica	7
2.1.1.2. Community smells (olores comunitarios)	8
2.1.1.3. El impacto de la deuda social en el entorno ágil	8
2.1.1.4. Desafíos en la medición y gestión de la deuda social	9
2.1.2. Síndrome de Burnout	9
2.1.3. Scrum	11
2.1.3.1. Definición de Scrum	11
2.1.3.2. Adaptación de Scrum para la gestión de la deuda social	11
2.2. Trabajos relacionados	11
2.2.1. Protocolo de investigación	12
CAPÍTULO 3. Fundamentos conceptuales de SocialScrum	15
3.1. Introducción	15
3.2. Modelo conceptual de SocialScrum	16
3.2.1. Roles	17
3.2.1.1. Product Owner	17
3.2.1.2. Scrum Master	18
3.2.1.3. Developers (Desarrolladores)	18
3.2.1.4. Social Guide	19

3.2.2.	Eventos adaptados	20
3.2.3.	Artefactos sociales	24
3.3.	Fundamentos teóricos	27
3.3.1.	Necesidades psicológicas y motivación intrínseca	27
3.3.1.1.	Autonomía	28
3.3.1.2.	Competencia	29
3.3.1.3.	Relacionalidad	29
3.3.2.	Confianza en equipos interdependientes	30
3.3.2.1.	Habilidad (Ability)	31
3.3.2.2.	Benevolencia (Benevolence)	31
3.3.2.3.	Integridad (Integrity)	32
3.3.3.	Valores de Scrum aplicados a la deuda social	34
3.3.3.1.	Coraje (Courage)	34
3.3.3.2.	Apertura (Openness)	34
3.3.3.3.	Respeto (Respect)	35
3.3.3.4.	Compromiso (Commitment)	35
3.3.3.5.	Foco (Focus)	35
3.3.4.	Principios empíricos de Scrum aplicados a la gestión de la deuda social	37
3.3.4.1.	Transparencia	37
3.3.4.2.	Inspección	39
3.3.4.3.	Adaptación	40
3.3.5.	El ciclo como sistema	41
3.3.5.1.	Ejemplo práctico del ciclo empírico en SocialScrum	42
3.3.6.	Medición de la salud-social	44
3.4.	Salvaguardas éticas y protecciones	45
3.4.1.	Confidencialidad absoluta	46
3.4.2.	Rendición de cuentas y estructura organizacional	47
3.4.3.	Consentimiento informado del equipo	47
3.4.4.	Anonimización en reportes y métricas	48
3.4.5.	Prevención del “teatro social”	48
3.4.6.	Derecho de auditoría del equipo	49
3.4.7.	Implementación práctica	49
3.5.	Diseño del modelo SocialScrum	50
3.6.	Consideraciones de escalabilidad y adaptación	54
3.6.1.	Adaptabilidad a contextos diversos	54
3.7.	Limitaciones reconocidas	55
3.8.	Conclusiones del capítulo	56

CAPÍTULO 4. SocialScrum: Proceso ágil para la gestión de la deuda social	58
4.1. Consideraciones del capítulo	58
4.2. Adopción inicial de SocialScrum	59
4.2.1. Pre-requisitos organizacionales	60
4.2.1.1. Condiciones necesarias para la adopción	60
4.2.1.2. Cuándo NO implementar SocialScrum	62
4.2.1.3. Evaluación inicial: checklist de viabilidad	63
4.2.2. Sprint 0: Preparación del equipo	64
4.2.2.1. Actividades del Sprint 0	64
4.2.2.2. Criterios de éxito del Sprint 0	66
4.2.2.3. Qué hacer si el equipo rechaza SocialScrum	67
4.2.3. Presentación al equipo y <i>stakeholders</i>	67
4.2.3.1. Objetivos de la presentación	67
4.2.3.2. Gestión de objeciones comunes	68
4.3. El rol del Social Guide: Perfil, competencias y gobernanza	68
4.3.1. Perfil y competencias requeridas	69
4.3.2. Proceso de selección y capacitación	70
4.4. Eventos adaptados de SocialScrum	71
4.4.1. Social Daily Standup	71
4.4.1.1. Protocolo del Social Daily Standup	72
4.4.2. Social Sprint Planning	73
4.4.2.1. Protocolo del Social Sprint Planning	73
4.4.2.2. Double Sprint Goal (opcional)	73
4.4.3. Social Sprint Review	74
4.4.3.1. Protocolo del Social Sprint Review	74
4.4.3.2. Visualización del Health Score	74
4.4.3.3. Confidencialidad en el Social Sprint Review	74
4.4.4. Social Sprint Retrospective	75
4.4.4.1. Estructura de la Social Sprint Retrospective	75
4.4.4.2. Flujo de la Social Sprint Retrospective	75
4.5. Artefactos sociales y su gestión	76
4.5.1. Social Product Backlog	76
4.5.1.1. Naturaleza y propósito	77
4.5.1.2. Estructura de un ítem del Social Product Backlog	77
4.5.1.3. Tipos de ítems y priorización	78
4.5.1.4. Gestión continua del Social Product Backlog	78
4.5.1.5. Relación con el Product Backlog técnico	78
4.5.2. Social Sprint Backlog	79
4.5.2.1. Naturaleza y propósito	79

4.5.2.2.	Contenido del Social Sprint Backlog	79
4.5.2.3.	Integración con el Sprint Backlog técnico	80
4.5.2.4.	Actualización durante el Sprint	81
4.5.2.5.	Qué hacer si no se completan items sociales	81
4.6.	Sistema de medición de salud social	82
4.6.1.	Health Score: Dimensiones y cálculo	82
4.6.1.1.	Frecuencia y formato de medición	83
4.6.2.	Zonas del Health Score y protocolo de respuesta	83
4.6.2.1.	Alertas tempranas	84
4.6.3.	Triangulación de fuentes de datos	84
4.7.	Framework de priorización social-técnica	84
4.7.1.	Triángulo de decisión: valor de negocio, urgencia técnica, salud social	85
4.7.1.1.	Las tres dimensiones de evaluación	85
4.7.1.2.	Principio fundamental: conversación, no fórmula	86
4.7.1.3.	Reglas de oro para casos extremos	86
4.8.	Herramientas y tecnología de soporte	87
4.9.	Simulación de implementación de SocialScrum	87
4.9.1.	Contexto del equipo	88
4.9.1.1.	Situación inicial	88
4.9.1.2.	Diagnóstico inicial (Sprint 0)	88
4.9.2.	Sprint 1: Establecer fundamentos	89
4.9.2.1.	Social Sprint Planning	89
4.9.2.2.	Social Sprint Backlog	90
4.9.2.3.	Ejecución del Sprint	90
4.9.2.4.	Social Sprint Retrospective	91
4.9.2.5.	Resultados Sprint 1	91
4.9.3.	Sprint 2: Abordar smells activos	91
4.9.3.1.	Contexto	91
4.9.3.2.	Intervención: Transferencia de conocimiento	92
4.9.3.3.	Evento inesperado: Crisis de deadline	92
4.9.3.4.	Resultados Sprint 2	93
4.9.4.	Sprint 3: Consolidación y ajustes.	93
4.9.4.1.	Contexto.	93
4.9.4.2.	Intervención: Sincronización FE-BE.	93
4.9.4.3.	Retrospectiva de consolidación.	94
4.9.4.4.	Resultados Sprint 3.	94
4.9.5.	Resumen de resultados	95
4.9.6.	Lecciones aprendidas	95
4.9.6.1.	Factores de éxito	95

4.9.6.2.	Dificultades encontradas	96
4.9.6.3.	Recomendaciones para otros equipos	96
4.10.	Conclusiones del capítulo	97

CAPÍTULO 5. Validación del modelo SocialScrum mediante la técnica de juicio de expertos 98

5.1.	Estrategia de validación del modelo SocialScrum	99
5.2.	Criterios de evaluación considerados	99
5.2.1.	Criterios de evaluación	100
5.2.2.	Escala de medición	100
5.3.	Definición del instrumento de validación	101
5.3.1.	Estructura del cuestionario	101
5.3.2.	Preguntas formuladas	101
5.4.	Perfil y selección de expertos participantes	103
5.4.1.	Perfilamiento de los participantes	103
5.4.2.	Selección de participantes	103
5.5.	Protocolo de validación mediante juicio de expertos	104
5.5.1.	Objetivo de la validación	104
5.6.	Resultados de la validación y análisis cuantitativo	104
5.6.1.	Análisis cuantitativo de las respuestas	104
5.6.1.1.	Claridad	107
5.6.1.2.	Complejidad	107
5.6.1.3.	Idoneidad	108
5.6.1.4.	Facilidad de uso	108
5.6.1.5.	Agilidad	109
5.6.2.	Resultados preguntas abiertas	110
5.7.	Desempeño del modelo y recomendaciones de mejora	110
5.7.0.1.	Fortalezas del modelo SocialScrum	110
5.7.0.2.	Debilidades del modelo SocialScrum	110
5.7.0.3.	Oportunidades de mejora	111
5.7.0.4.	Comentarios sobre la adecuación del Instrumento de Evaluación	111

CAPÍTULO 6. Conclusiones, publicaciones, recomendaciones y trabajos futuros 113

6.1.	Conclusiones	113
6.1.1.	Sobre la caracterización de la deuda social	113
6.1.2.	Sobre el diseño del marco SocialScrum	113
6.1.3.	Sobre la viabilidad práctica	114
6.2.	Limitaciones del trabajo	114

6.3.	Recomendaciones	115
6.3.1.	Para organizaciones que consideren adoptar SocialScrum	115
6.3.2.	Para investigadores	115
6.4.	Trabajos futuros	115
6.5.	Reflexión final	116
A.	Revisión sistemática de la literatura (RSL) sobre la gestión de la deuda social en el desarrollo de software ágil	116
B.	Artículo publicado y aceptado en el evento internacional IEEE AmITIC 2025: Social Debt in Agile Software Development.	130
C.	Acuerdo de Confidencialidad del Social Guide	138
C.1.	Declaración de Confidencialidad y Compromiso Ético	138
C.1.1.	1. Confidencialidad absoluta de información individual	138
C.1.2.	2. Estructura de reporte y rendición de cuentas	139
C.1.2.1.	Tipos de reportes	139
C.1.2.2.	Protocolo si HR solicita información	139
C.1.2.3.	Rendición de cuentas	139
C.1.2.4.	Dedicación del rol	139
C.1.3.	3. Separación total entre salud social y evaluación de desempeño . .	140
C.1.4.	4. Estructura de reporte y rendición de cuentas	140
C.1.5.	5. Excepciones legalmente justificadas	141
C.1.6.	6. Protección de la seguridad psicológica del equipo	141
C.1.7.	7. Derecho de auditoría del equipo	142
C.1.8.	8. Compromiso de mejora continua	142
C.1.9.	9. Consecuencias de la violación de este acuerdo	142
D.	Consentimiento Informado del Equipo para SocialScrum	144
D.1.	Consentimiento Informado para la Implementación de SocialScrum	144
D.1.1.	1. Comprensión del propósito de SocialScrum	144
D.1.2.	2. Información que será recopilada	144
D.1.2.1.	2.1. Información cuantitativa	145
D.1.2.2.	2.2. Información cualitativa	145
D.1.3.	3. Cómo será utilizada la información	145
D.1.4.	4. Quién tendrá acceso a la información	146
D.1.5.	5. Salvaguardas de confidencialidad	146
D.1.6.	6. Excepciones a la confidencialidad	146
D.1.7.	7. Mis derechos como participante	147
D.1.8.	8. Duración del consentimiento	147

D.1.9. 9. Declaración de consentimiento voluntario	147
D.1.10. Nota para el equipo	148
E. Herramientas y Tecnología de Soporte para SocialScrum	149
E.1. Adaptación contextual y escalabilidad	149
E.1.1. Configuración por tamaño de equipo	149
E.1.2. Adaptación por modalidad de trabajo	150
E.2. Herramientas y tecnología	151
E.2.1. Integración con Jira	151
E.2.1.1. Campos personalizados	151
E.2.1.2. Configuración de tipos de incidencias (<i>Issue Types</i>)	151
E.2.1.3. Workflow para ítems sociales	152
E.2.1.4. Filtros JQL para SocialScrum	152
E.2.2. Integración con Azure DevOps	153
E.2.2.1. Campos personalizados en Azure DevOps	153
E.2.2.2. Queries para Azure DevOps	154
E.2.2.3. Integración de Health Score	154
E.2.3. Plantillas operacionales	155
E.2.3.1. Plantilla de User Story Social	155
E.2.3.2. Plantilla para mediación de conflictos	158
E.2.3.3. Plantilla de Health Score Dashboard semanal	159
E.2.3.4. Plantilla de notas de retrospectiva social	161
E.2.3.5. Plantilla de reporte mensual para las partes interesadas (<i>stakeholders</i>)	163
E.2.3.6. Plantilla de registro de <i>community smells</i>	166
E.2.4. Dashboards automatizados	167
E.2.4.1. Arquitectura de datos	167
E.2.4.2. Dashboard en Google Sheets	168
E.2.4.3. Dashboard en Grafana	168
E.2.4.4. Dashboard en Power BI / Tableau	169
E.2.5. Alertas automatizadas	171
E.2.6. Consideraciones de implementación	171
F. Protocolos Operativos de Gobernanza y Ética del Social Guide	173
F.1. 1 Protocolo técnico de protección de datos	173
F.1.1. Confidencialidad absoluta: Principio no negociable	173
F.1.1.1. Definición del principio	173
F.1.1.2. Qué es confidencial vs. qué es reportable	174
F.1.1.3. Ejemplos de aplicación	174
F.1.1.4. Acuerdos formales de confidencialidad	174

F.1.1.5.	Protocolo técnico de protección de datos	175
F.1.2.	Estructura organizacional y rendición de cuentas	176
F.1.2.1.	Principio de reporte	176
F.1.2.2.	Estructura organizacional recomendada	176
F.1.2.3.	Justificación de la estructura	176
F.1.2.4.	Protocolo cuando HR solicita información	177
F.1.2.5.	Separación entre salud social y evaluación de desempeño	177
F.1.3.	Protocolo de conflicto irreconciliable PO-Social Guide	177
F.1.3.1.	Principio rector	178
F.1.3.2.	Protocolo de escalamiento	178
F.1.3.3.	Criterios de decisión para el CTO	178
F.1.3.4.	Prevención de conflictos recurrentes	179
F.1.4.	Protocolo de continuidad del Social Guide	179
F.1.4.1.	Escenarios de salida	179
F.1.4.2.	Compromiso de confidencialidad post-salida	181
F.1.5.	Protocolo de auditoría independiente	181
F.1.5.1.	Definición del principio	181
F.1.5.2.	Selección del auditor	181
F.1.5.3.	Proceso de auditoría	181
F.1.5.4.	Consecuencias de hallazgos negativos	182
F.1.5.5.	Canal de denuncia (whistleblowing)	183
F.1.6.	Prevención del “Teatro Social”	184
F.1.6.1.	Síntomas del “Teatro Social”	184
F.1.6.2.	Causas del “Teatro Social”	184
F.1.6.3.	Estrategias de prevención	185
F.1.6.4.	Qué hacer si se detecta “Teatro Social”	188

G. Guía de Facilitación y Protocolos Operativos 190

G.1.	1. Guía para la presentación inicial de SocialScrum	190
G.1.1.	Estructura de la presentación (45 minutos).	190
G.1.2.	Documentación post-presentación	192
G.2.	2. Programa de capacitación del Social Guide	193
G.2.1.	Fase 1: Fundamentos teóricos (8 horas)	193
G.2.2.	Fase 2: Facilitación y mediación (12 horas)	194
G.2.3.	Fase 3: Medición y análisis (8 horas)	194
G.2.4.	Fase 4: Ética y salvaguardas (12 horas)	194
G.2.5.	Evaluación final post-capacitación (semana 7)	195
G.2.6.	Seguimiento (en primer año)	195
G.3.	3. Protocolos operativos para eventos clave	195

G.3.1. Preguntas sociales recomendadas	195
G.3.2. Preguntas de evaluación de riesgos sociales	196
G.3.3. Documentación de riesgos sociales	196
G.3.4. Contenido de la evaluación de salud social	196
G.3.4.1. Técnicas por fase	197
G.3.4.2. Cierre y documentación	197
G.3.4.3. Frecuencia	198
G.3.4.4. Manejo de situaciones difíciles	198
H. Framework de Priorización Extendido	199
H.1. Guía del Framework de Priorización Extendido	199
H.1.1. Matriz de 18 escenarios de priorización	199
H.1.2. Protocolo de decisión colaborativa	200
H.1.2.1. Participantes y autoridad	200
H.1.2.2. Protocolo de priorización en Social Sprint Planning	200
H.1.2.3. Conversión entre Social Story Points y Story Points	202
H.1.2.4. Protocolo de discrepancia	203
H.1.2.5. Ejemplo completo de priorización	204
I. Sistema de Estimación Social (SSP)	206
I.1. Sistema de estimación de ítems sociales (Social Story Points)	206
I.1.1. Escala de estimación SSP	206
I.1.2. Metodología de estimación en 3 pasos	206
I.1.2.1. Matriz de severidad y SSP base	207
I.1.2.2. Cálculo del SSP final	207
I.1.3. Tabla de referencia rápida	207
I.1.4. Conversión SSP ↔ SP y gestión de capacidad	207
I.1.4.1. Mejores prácticas de estimación	208
J. Catálogo de Estrategias de Mitigación	209
J.1. Estrategias de mitigación de community smells	209
J.1.1. Catálogo de estrategias por categoría	209
J.1.1.1. Estrategias para Community Smells (Olores Comunitarios) de comunicación	211
J.1.1.2. Estrategias para smells de conocimiento	213
J.1.1.3. Estrategias para smells relacionales	215
J.1.1.4. Estrategias para smells organizacionales	216
J.1.1.5. Smells que requieren escalamiento	217
J.1.2. Diseño de experimentos de mejora	217

K. Principio de No-Sobrecarga: Integración sin agregar tiempo	219
K.1. 1. Principio de No-Sobrecarga: Integración sin agregar tiempo	219
K.1.1. Sobrecarga temporal por evento	219
K.1.1.1. Detalle por evento	220
K.1.2. Sesiones adicionales y presupuesto	220
K.1.3. Optimizaciones prácticas	220
K.1.4. Gestión de expectativas	221
K.1.4.1. Comunicación al equipo	221
K.1.4.2. Comunicación a stakeholders	221
K.1.5. Retorno de inversión (ROI) de SocialScrum	221
L. Ejemplos Detallados de Intervenciones	222
L.1. 1 Intervención: <i>Radio Silence</i> (Smell de Comunicación)	222
L.2. 2 Intervención: Truck Factor = 1 (Smell de Conocimiento)	223
L.3. 3 Intervención: Conflicto No Resuelto (Smell Relacional)	224
L.4. 4 Intervención: Meeting Hell (Smell Organizacional)	225
L.5. 5 Experimento: Reducir Knowledge Islands	226
M. Contextualización del modelo SocialScrum	229
M.1. Conceptos clave	229
M.2. Fundamentos teóricos de SocialScrum (síntesis del Capítulo 3)	230
M.2.1. Teoría de la Autodeterminación (TAD)	230
M.2.2. Modelo de confianza de Mayer, Davis y Schoorman	231
M.2.3. Valores de Scrum aplicados a la deuda social	231
M.2.4. Principios empíricos aplicados al ámbito social	232
M.3. Modelo conceptual de SocialScrum	232
M.3.1. Roles	232
M.3.2. Eventos adaptados	233
M.3.3. Artefactos sociales	233
M.4. Operacionalización de SocialScrum (síntesis del Capítulo 4)	234
M.4.1. Adopción inicial	234
M.4.1.1. Sprint 0 (preparación)	234
M.4.2. El Social Guide: perfil y gobernanza	235
M.4.3. Sistema de medición: Health Score	235
M.4.3.1. Cálculo	235
M.4.3.2. Zonas de interpretación y respuesta	235
M.4.3.3. Alertas tempranas	236
M.4.4. Framework de priorización social-técnica	236
M.4.5. Sistema de estimación Social Story Points (SSP)	236
M.4.5.1. Escala y rangos de referencia	236

M.4.5.2. Proceso de estimación	237
M.4.5.3. Integración con Story Points técnicos	237
M.4.6. Salvaguardas éticas	237
M.5. Simulación de implementación: caso del equipo Phoenix	238
M.6. Limitaciones reconocidas	238
REFERENCIAS BIBLIOGRÁFICAS	240
N. Acrónimos	245
Ñ. Glosario	246

ÍNDICE DE FIGURAS

2.1. Secuencia de actividades realizadas para la RSL	12
3.1. Diagrama general del proceso SocialScrum.	16
3.2. Modelo conceptual de SocialScrum.	17
3.3. Flujo detallado del evento Social Sprint Retrospective en SocialScrum. . . .	24
3.4. Arquitectura integrada del modelo SocialScrum (Capas 2-4)	50
3.5. Fundamentos teóricos de SocialScrum.	51
3.6. Principios empíricos de SocialScrum: ciclo de retroalimentación continua. .	52
3.7. Componentes operacionales de SocialScrum: roles, eventos y artefactos adaptados.	53
3.8. Resultados del modelo SocialScrum: prevención y mejora sistémica.	53
4.9. Ciclo de eventos adaptados de SocialScrum	72
4.10. Ejemplo de visualización del Health Score en el Social Sprint Review . . .	74
4.11. Flujo de las cuatro fases de la Social Sprint Retrospective	75
4.12. Triángulo de priorización de SocialScrum	85
4.13. Evolución del Health Score del equipo Phoenix	95
5.1. Estructura del Capítulo 5: Validación del modelo SocialScrum	98
F.1. Estructura organizacional del Social Guide: reporta a CTO/SM, nunca a HR176	
F.2. Protocolo de escalamiento en conflicto PO-Social Guide	178

ÍNDICE DE TABLAS

2.1. Cadenas de búsqueda utilizadas en la revisión sistemática	13
2.2. Resultados por motor de búsqueda en la revisión sistemática	13
2.3. Preguntas de investigación definidas	14
3.1. Eventos de Scrum adaptados en SocialScrum	22
3.2. Sobrecarga temporal de los eventos sociales en SocialScrum	23
3.3. Artefactos de Scrum adaptados en SocialScrum	25
3.4. Community smells mitigados por cada valor de Scrum en SocialScrum [2] .	36
3.5. Estrategias de transparencia social en SocialScrum	38
3.6. Cadencias de inspección social en SocialScrum	40
3.7. Familias de estrategias de mitigación por categoría de community smell . .	41
3.8. Consecuencias de la ausencia de principios empíricos en la gestión de la deuda social	42
4.9. Checklist de viabilidad para la adopción de SocialScrum	63
4.10. Perfil del rol Social Guide	69
4.11. Proceso de selección del Social Guide	70
4.12. Proceso de votación para la selección del Social Guide	71
4.13. Reglas de decisión del proceso de votación para la selección del Social Guide	71
4.14. Estructura del Social Daily Standup	72
4.15. Estructura del Social Sprint Planning	73
4.16. Estructura del Social Sprint Review	74
4.17. Estructura de la Social Sprint Retrospective (90 minutos)	75
4.18. Artefactos sociales de SocialScrum	76
4.19. Matriz de priorización del Social Product Backlog	78
4.20. Dimensiones del Health Score	82
4.21. Instrumentos de medición del Health Score	83
4.22. Zonas del Health Score y protocolo de respuesta	83
4.23. Complementariedad de instrumentos de medición	84
4.24. Perfil del equipo Phoenix	88
4.25. Health Score inicial del equipo Phoenix	89
4.26. Señales detectadas en Social Daily Standup – Sprint 1	90
4.27. Resultados del Sprint 1	91
4.28. Sesiones de transferencia de conocimiento	92

4.29. Resultados Sprint 2	93
4.30. Resultados Sprint 3	94
4.31. Evolución del equipo Phoenix en tres Sprints	95
4.32. Recomendaciones basadas en el caso Phoenix	96
5.1. Criterios de evaluación y número de preguntas en el instrumento de validación	100
5.2. Estructura del instrumento de evaluación	100
5.3. Ejemplos de preguntas del instrumento de evaluación	101
5.4. Perfil de los expertos participantes en la validación del modelo SocialScrum	103
5.5. Ejemplos de preguntas del instrumento de evaluación	104
5.6. Ejemplos de preguntas del instrumento de evaluación	107
5.7. Ejemplos de preguntas del instrumento de evaluación	107
5.8. Ejemplos de preguntas del instrumento de evaluación	108
5.9. Ejemplos de preguntas del instrumento de evaluación	109
5.10. Ejemplos de preguntas del instrumento de evaluación	109
5.11. Observaciones y recomendaciones de los expertos	110
C.2. Reportes del Social Guide	139
C.4. Dedicación del Social Guide según contexto	140
E.2. Configuración de SocialScrum por tamaño de equipo	150
E.4. Adaptaciones de SocialScrum según la modalidad de trabajo	150
E.5. Campos personalizados para SocialScrum en Jira	151
F.2. Información confidencial vs. reportable	174
F.4. Requisitos técnicos de almacenamiento	175
F.6. Matriz de acceso a datos de SocialScrum	175
F.8. Criterios de decisión en conflicto PO–SG	178
F.10. Indicadores de represalias silenciosas	184
F.12. Síntomas del “Teatro Social”	184
F.14. Validación cruzada del Health Score	186
G.2. Tipos de preguntas sociales para el Daily	195
G.4. Preguntas de evaluación de riesgos sociales	196
G.6. Ejemplo de registro de riesgos sociales del Sprint	196
G.8. Elementos de la evaluación de salud social	197
G.10. Manejo de situaciones difíciles en la Social Sprint Retrospective	198
H.2. Matriz de 18 escenarios de priorización	200
H.4. Autoridad de cada rol en la priorización	200
H.6. Diagnóstico del Social Guide	201
H.8. <i>Features</i> priorizadas por valor de negocio (Product Owner)	201

H.10.Propuesta de intervención del Social Guide (Health Score 6.2)	201
H.12.Compromiso final del Sprint tras diálogo de priorización	202
H.14.Capacidad del equipo y compromiso del Sprint	203
H.16.Diagnóstico social inicial del equipo Alpha	205
H.18.Backlog priorizado por valor y urgencia	205
H.20.Intervención social propuesta por el Social Guide	205
H.22.Sprint Backlog final — Equipo Alpha	205
I.2. Escala de Social Story Points (SSP)	206
I.4. Proceso de estimación de SSP	207
I.6. Niveles de severidad y SSP base	207
I.8. Referencia rápida de estimación SSP por <i>community smell</i>	207
I.10. Ejemplo de cálculo de capacidad combinada	207
J.2. Categorías de <i>community smells</i> , ejemplos e intervenciones	210
J.4. Estrategias para smells de comunicación	212
J.6. Estrategias para smells de conocimiento	214
J.8. Estrategias para smells relacionales	215
J.10. Estrategias para smells organizacionales	216
J.12. Smells que requieren escalamiento	217
J.14. Componentes de un experimento de mejora	218
K.2. Sobrecarga temporal de SocialScrum por evento	220
K.4. Detalle de cambios por evento	220
K.6. Optimizaciones para reducir sobrecarga	221
K.8. Inversión anual estimada en sobrecarga de SocialScrum	221
K.10.Análisis de ROI de SocialScrum	221

CAPÍTULO 1. INTRODUCCIÓN

En este capítulo se explica, paso a paso, cuál es el problema que se aborda en la investigación, qué objetivos se buscan alcanzar y cuál fue el método que se usó para hacerlo posible. Además, se presenta cómo está organizado el resto del documento para que el lector tenga una idea clara de su estructura y del camino que sigue el trabajo.

1.1 Planteamiento del problema

El término “deuda social” se acuñó por primera vez hace más de 50 años [1], con un enfoque económico para explicar las obligaciones sociales derivadas del intercambio de favores entre personas en una transacción, es decir, entre acreedores y deudores. En otras palabras, cuanto mayor sea el número de relaciones tensas, mayor será la deuda social [2]. Sin embargo, este concepto no se limita a aspectos económicos, ya que su estudio se ha extendido a otros ámbitos, como el desarrollo de software [2]. En este contexto, la deuda social se refiere a las obligaciones pendientes que afectan la calidad, la colaboración y el bienestar en los proyectos de desarrollo [2], [3]. En esencia, la deuda social encapsula las repercusiones no técnicas derivadas de decisiones apresuradas o inadecuadas en la gestión de equipos y proyectos, lo que impacta negativamente en la cultura organizacional, la comunicación interna, el bienestar del equipo y, en última instancia, en el éxito del proyecto. La deuda social puede compararse con un “Leviatán”, un término que evoca la imagen de un monstruo poderoso y temible que crece de manera progresiva y silenciosa, volviéndose cada vez más difícil de ignorar. En el contexto del desarrollo de software, este “Leviatán” representa la acumulación de obligaciones pendientes y tensiones interpersonales que surgen de decisiones apresuradas o inadecuadas en la gestión de equipos y proyectos. Este fenómeno genera un daño colateral que afecta a toda la estructura jerárquica de desarrollo, causando costos y deudas en múltiples aspectos [2]. Uno de los pilares fundamentales de una organización de desarrollo de software, el talento humano, se ve directamente impactado por la deuda social. Las afectaciones se manifiestan sin importar el cargo o función de una persona, evidenciándose principalmente en su rendimiento y calidad de trabajo, los cuales dependen de un estado de bienestar mental y físico, así como de un ambiente laboral justo y motivador [2]. Una mala gestión de estos factores es la causa principal de la alta rotación de personal, la toma de decisiones erróneas y la entrega de productos de baja calidad. La deuda social surge principalmente cuando se priorizan objetivos a corto

plazo, como cumplir con fechas de entrega ajustadas, reducir costos operativos de manera indiscriminada o incorporar requisitos imprevistos en la planificación estratégica inicial, sacrificando las buenas prácticas de gestión y deteriorando las relaciones interpersonales [4]. Esto conduce a un ambiente de trabajo tóxico, estrés crónico y problemas de salud mental y física, lo que a su vez disminuye la motivación, la productividad y la calidad del trabajo, limitando la capacidad de innovación y creatividad [2], [3], [4], [5]. Para mitigar o gestionar esta deuda, es crucial adoptar un enfoque holístico que promueva la comunicación efectiva, el equilibrio entre trabajo y vida personal, el reconocimiento profesional y un liderazgo que priorice el bienestar del equipo, fomentando un ambiente de trabajo positivo y motivador. Identificar y comprender la naturaleza y el impacto de la deuda social permite a las organizaciones de software desarrollar estrategias más efectivas para promover un ambiente de trabajo saludable y productivo [5]. Esto incluye implementar prácticas de gestión que equilibren las demandas operativas con el bienestar de los empleados, fomentando un clima de trabajo sostenible y propicio para la innovación y la creatividad [2]. Las estrategias para gestionar la deuda social deben ser claras y objetivas, permitiendo identificar su impacto en los proyectos. Estas pueden incluir evaluaciones específicas, como pruebas de calidad en procesos de trabajo, encuestas de satisfacción laboral, análisis de rotación de personal y estudios de clima organizacional que reflejen la realidad del equipo de trabajo. Con la información recopilada, es posible gestionar proactivamente la deuda, ajustando políticas de gestión de recursos humanos y estrategias de desarrollo de software ágil. Además, la transparencia en la comunicación y la participación del equipo en la toma de decisiones son vitales para construir un entorno de confianza y colaboración, lo que mejora la moral del equipo y promueve un mayor sentido de pertenencia y responsabilidad colectiva en la calidad del producto final. En este sentido, invertir en la salud organizacional a través de la gestión y mitigación de la deuda social no solo mejora el bienestar de los empleados, sino que también se traduce en una mayor eficiencia operativa, innovación sostenida, participación creativa constante y una ventaja competitiva en el mercado [2]. Por lo tanto, es esencial que las organizaciones reconozcan la deuda social como un aspecto crítico que requiere atención continua y estrategias bien definidas para su gestión efectiva. Con el fin de comprender mejor esta problemática, se realizó una Revisión Sistemática de la Literatura (RSL) orientada a recopilar, identificar y priorizar estudios relevantes sobre la gestión de la deuda social en equipos de desarrollo de software ágil, dicha RSL permitió realizar un análisis para reconocer algunas metodologías y herramientas que intentan mitigar sus efectos, como prácticas de comunicación efectiva, sesiones de retroalimentación y dinámicas de equipo, no obstante, los resultados evidenciaron una limitada producción científica en este campo, lo que pone de manifiesto una brecha significativa en la atención académica y práctica hacia este tipo de deuda. Esta carencia sugiere la necesidad urgente de desarrollar estrategias concretas que aborden no solo los aspectos técnicos del desarrollo, sino también las dimensiones humanas y organizacionales que influyen directamente

en el rendimiento y la salud del equipo. En este trabajo de investigación, surge la siguiente pregunta: ¿Cómo gestionar la deuda social que surge en equipos de desarrollo de software ágil? Para dar respuesta a este interrogante, se propone una adaptación del marco de trabajo Scrum, orientada a incorporar mecanismos explícitos para la gestión de la deuda social. Esta propuesta contempla la inclusión de sesiones de retrospectiva focalizadas en aspectos sociales del equipo, así como la implementación de sprints dedicados exclusivamente a identificar y resolver conflictos interpersonales y organizacionales. El objetivo es promover un entorno de trabajo más saludable, colaborativo y productivo, en el que la calidad técnica y humana del desarrollo de software se integren de manera armónica. Esta tesis busca, por tanto, contribuir al cuerpo de conocimiento sobre metodologías ágiles, proponiendo un enfoque innovador que permita gestionar de forma efectiva la deuda social en proyectos de software. A través de la adaptación del marco Scrum, se pretende ofrecer una herramienta práctica que fortalezca las relaciones dentro del equipo, mejore la comunicación y fomente una cultura de trabajo más empática y sostenible.

1.2 Objetivos

A continuación, se muestran tanto el objetivo general como los objetivos específicos de este proyecto de investigación. Todos ellos fueron aprobados en el anteproyecto según la resolución 8.4.3-4.4/104 de 2025, emitida el 14 de marzo del mismo año.

- **Objetivo general:** Definir un proceso ágil para gestionar la deuda social generada en proyectos de desarrollo de software ágil, a partir de aspectos fundamentales propuestos en la literatura y de la adaptación de los elementos del enfoque ágil para la gestión de proyectos conocido como Scrum.
- **Objetivos específicos:** A continuación, se presentan los objetivos específicos de acuerdo con las palabras claves para la definición de objetivos propuestas en la taxonomía de Bloom [6]:
 - **Objetivo específico 1:** Identificar los elementos fundamentales para la gestión de la deuda social en el desarrollo de software ágil mediante un análisis de la literatura encontrada en diferentes bases científicas, el cual permita entender y organizar el conocimiento relacionado con la gestión de la deuda social en equipos de desarrollo de software.
 - **Objetivo específico 2:** Construir un proceso basado en Scrum que permita gestionar la deuda social en proyectos de desarrollo de software ágil, incorporando los elementos identificados en el objetivo específico 1 y aquellos aspectos que se consideren fundamentales.

- **Objetivo específico 3:** Evaluar el proceso propuesto en el objetivo específico 2 mediante la técnica de validación de juicio de expertos, y determinar oportunidades de mejora a partir de un coeficiente estadístico.

1.3 Método de investigación

El proyecto se llevará a cabo utilizando el método de Investigación-Acción en múltiples ciclos [7] y la evaluación de la propuesta se efectuará a través de un estudio de caso [8], esto permitió evaluar y analizar este fenómeno con el fin de encontrar soluciones efectivas para su gestión. Siguiendo esta metodología, se llevarán a cabo cuatro ciclos de investigación, que se describen a continuación de manera secuencial e incremental junto con las actividades correspondientes:

- **Ciclo 1: Análisis conceptual.** En esta etapa se realizará una revisión sistemática de la literatura relacionada con la gestión de la deuda social en el desarrollo de software ágil. El objetivo es identificar propuestas y elementos clave para la definición de una solución efectiva.
 - **Actividad 1.1: Investigar la literatura:** Se llevará a cabo una investigación exhaustiva de las propuestas vinculadas con la gestión de la deuda social en el desarrollo de software ágil.
 - **Actividad 1.2: Analizar la literatura:** Se identificarán causas, consecuencias, desafíos y futuras líneas de investigación en relación con la gestión de la deuda social en estas comunidades.
 - **Actividad 1.3: Resumir la literatura seleccionada:** Se analizarán y sintetizarán los estudios relevantes que aborden la gestión de la deuda social en el desarrollo de software ágil.
- **Ciclo 2: Elaboración de la propuesta.** En esta fase se desarrollará una solución basada en Scrum para gestionar la deuda social en el desarrollo de software ágil, promoviendo la sostenibilidad y el bienestar de sus profesionales.
 - **Actividad 2.1: Análisis de la información:** Identificación, análisis y categorización de los elementos que influyen en la gestión de la deuda social en el desarrollo de software ágil.
 - **Actividad 2.2: Definición de la propuesta:** Desarrollo de un proceso basado en Scrum para gestionar la deuda social en el desarrollo de software ágil. Se explorará la utilización de soluciones para la definición de procesos, uno de ellos puede ser el propuesto por D. A. Quintana Guzmán [9].

- **Ciclo 3: Evaluación de la propuesta.** En esta fase se llevará a cabo una evaluación utilizando el juicio de expertos como una técnica de estudio cualitativa. Esta técnica contará con la colaboración de un grupo de profesionales y/o especialistas compuestos por expertos en desarrollo de software ágil.
 - **Actividad 3.1: Planificación:** Se llevará a cabo la capacitación, coordinación, organización y diseño de los expertos.
 - **Actividad 3.2: Acción:** Se llevará a cabo el juicio de expertos según la planificación y el diseño establecidos en la actividad 3.1.
 - **Actividad 3.3: Observación:** Recopilación de datos sobre la ejecución e intervención de los expertos.
 - **Actividad 3.4: Reflexión:** Generación de un informe basado en el análisis de los datos obtenidos durante la ejecución de los expertos.
- **Ciclo 4: Documentación y socialización.** Este ciclo se llevará a cabo de manera transversal a lo largo del proyecto, incluyendo las siguientes actividades:
 - **Actividad 4.1: Elaboración de la monografía:** Desarrollo de la monografía y los anexos resultantes del trabajo de grado o documento final.
 - **Actividad 4.2: Elaboración del artículo de investigación:** Creación de un artículo que describa los resultados obtenidos durante la implementación de la propuesta.
 - **Actividad 4.3: Sustentación:** Presentación y defensa de los resultados obtenidos a lo largo del desarrollo del proyecto.

1.4 Estructura del documento

Este documento está organizado por seis capítulos, cada uno de ellos con sus respectivas secciones y subsecciones, de la siguiente manera:

- **Capítulo 1: Introducción** En este capítulo se presenta el planteamiento del problema, los objetivos, el método de investigación y la estructura del documento.
- **Capítulo 1.4: Marco teórico y trabajos relacionados:** En este capítulo se presentan las bases conceptuales de la investigación: deuda social, síndrome de Burnout y marco de trabajo Scrum.
- **Capítulo 3: Fundamentos conceptuales de SocialScrum:** En este capítulo se presenta la adaptación de Scrum para la gestión de la deuda social, incluyendo su filosofía, principios, valores, eventos, artefactos adaptados y el rol del Social Guide.

- **Capítulo 4: SocialScrum: Proceso ágil para la gestión de la deuda social:** En este capítulo se presenta la aplicación operativa de SocialScrum: proceso por fases, instrumentos, métricas, ejemplos teórico-prácticos y una discusión crítica con limitaciones.
- **Capítulo 5: Validación del modelo SocialScrum mediante la técnica de juicio de expertos:** En este capítulo se presenta la validación del modelo SocialScrum a través del juicio de ocho expertos, utilizando un instrumento con escala Likert y preguntas abiertas que evalúa cinco criterios: claridad, completitud, idoneidad, facilidad de uso y agilidad. Se incluye el análisis de resultados y las oportunidades de mejora identificadas.
- **Capítulo 6: Conclusiones, publicaciones, recomendaciones y trabajos futuros:** En este capítulo se presentan las conclusiones, recomendaciones y trabajos futuros derivados de la investigación.
- **Referencias bibliográficas:** En esta sección se presentan la lista de referencias bibliográficas utilizadas en la elaboración de la investigación.
- **Anexos:** En esta sección se presentan los anexos del documento los cuales son:
 - **Anexo A:** Revisión sistemática de la literatura (RSL) sobre la gestión de la deuda social en el desarrollo de software ágil.
 - **Anexo B:** Artículo publicado y aceptado en el evento internacional IEEE AmITIC 2025: Social Debt in Agile Software Development.
 - **Anexo C:** Acuerdo de Confidencialidad del Social Guide.
 - **Anexo D:** Consentimiento Informado del Equipo para SocialScrum.
 - **Anexo E:** Herramientas y tecnología de soporte para SocialScrum.
 - **Anexo F:** Reporte de juicio de expertos para la aplicación de SocialScrum.
 - **Anexo G:** Consentimiento ético para la participación de los expertos.
 - **Anexo H:** Framework de Priorización Extendido.
 - **Anexo I:** Sistema de estimación de items sociales (Social Story Points).
 - **Anexo J:** Guía de facilitación para eventos
 - **Anexo K:** Estrategias para minimizar la sobrecarga de SocialScrum.
 - **Anexo L:** Ejemplos de intervenciones para diferentes categorías de *Community Smells* (Olores Comunitarios).
 - **Anexo M:** Contextualización del modelo SocialScrum.

CAPÍTULO 2. MARCO TEÓRICO Y TRABAJOS RELACIONADOS

En este capítulo se presentan las bases conceptuales que sustentan la investigación, abordando los conceptos clave relacionados con la deuda social, la comunicación, colaboración y coordinación (3C), el compromiso organizacional, el síndrome de Burnout y el marco de trabajo Scrum. Estos conceptos son fundamentales para comprender el contexto en el que se desarrolla la aplicación de SocialScrum en equipos de desarrollo de software ágil.

2.1 Marco teórico

2.1.1 Deuda social en el desarrollo de software

La deuda social se describe como la acumulación de costos imprevistos, o potenciales costos, en proyectos de software generando así un conjunto de decisiones sociotécnicas deficientes que resultan de procesos de desarrollo subóptimos [2]. Dicho de otro modo, a medida que se toman decisiones erróneas de manera constante, los efectos negativos de la deuda social en los equipos de desarrollo de software se vuelven más pronunciados, y mientras estas malas decisiones persistan, sus efectos se intensificarán con el tiempo [4]. Este concepto, que proviene de la sociología —donde originalmente se usaba para explicar cómo las obligaciones sociales pueden afectar las relaciones interpersonales— ha sido adoptado por los investigadores en ingeniería de software para describir fenómenos similares dentro de los equipos de desarrollo y sus organizaciones [2].

2.1.1.1 Analogía y distinción con la deuda técnica

La deuda social es, en muchos sentidos, análoga a la deuda técnica [10]. Ambas representan el estado de las organizaciones de software como resultado de decisiones acumuladas —que, si bien pueden ofrecer beneficios a corto plazo— generan un “interés” que se manifiesta como costos adicionales a medio y largo plazo [11]. No obstante, existe una diferencia fundamental:

- La deuda técnica se centra en decisiones técnicas postergadas, como refactorizaciones de código o decisiones arquitectónicas que afectan la calidad del software [2].

- En contraste, la deuda social se enfoca en decisiones sociotécnicas incorrectas que moldean el entorno de trabajo y que influyen directamente en el bienestar, las interacciones sociales y el rendimiento de los desarrolladores [2], [4]. Por lo tanto, mientras la deuda técnica se ocupa de la parte técnica del software, la deuda social pone el foco en las “personas”. Cabe señalar que, de forma recíproca, la deuda social puede ser una causa subyacente de la deuda técnica [2], [4].

2.1.1.2 Community smells (olores comunitarios)

La acumulación de deuda social se inicia con una o más decisiones sociotécnicas equivocadas [2]. La presencia de *community smells* es un indicador clave de que algo no funciona correctamente dentro de los equipos [2]. Estos *community smells* son, en esencia, antipatrones sociotécnicos [2], definidos como “conjuntos de circunstancias organizacionales y sociales, con relaciones causales implícitas” [2], [10]. Cuando estos Community Smells (Olores Comunitarios) persisten, se genera una acumulación de deuda social [2]. Las fuentes identificadas de estos Community Smells (Olores Comunitarios) son diversas y se manifiestan a través de modelos de causa y efecto, por ejemplo:

- Decisiones inadecuadas de gestión de la información que resultan en una arquitectura por ósmosis, donde los desarrolladores no tienen acceso a información arquitectónica clave, lo que lleva a la frustración y al desperdicio de tiempo [2].
- Problemas de coordinación de tareas en equipos aislados, conocidos como Organizational Silo (Silo Organizacional), que dificultan la comunicación y la cooperación efectiva [2].
- Comportamientos individuales desafiantes, como el Lone Wolf (Lobo Solitario) donde un desarrollador trabaja de forma independiente sin comunicarse con sus compañeros, generando decisiones arquitectónicas no sancionadas y retrasos en el proyecto [2].
- Estructuras organizacionales excesivamente formales o complejas que dan lugar al Radio Silence (Silencio Radiofónico) o Bottleneck (Cuello de Botella), creando cuellos de botella en la comunicación y retrasos masivos. Estos *community smells* impactan principalmente en las emociones, estresan las interacciones sociales, afectan el rendimiento del equipo y disminuyen la calidad del software [2].

2.1.1.3 El impacto de la deuda social en el entorno ágil

La deuda social tiene un impacto profundo y multifacético en los equipos de desarrollo ágil y sus organizaciones [2]. Entre las consecuencias más notables se incluyen:

- Reacciones humanas adversas y un deterioro del bienestar de los individuos [2].

- Relaciones sociales tensas y conflictos entre compañeros de trabajo, lo que afecta la cohesión del equipo [2].
- Bajo rendimiento del equipo y prácticas de trabajo subóptimas que afectan la calidad del software [2].
- Artefactos de software defectuosos y productos de baja calidad que no cumplen con las expectativas del cliente [2].
- El Impacto económico no se limita a simples cifras: suele traducirse en gastos inesperados que, en algunos casos, pueden amenazar la continuidad misma del negocio [2]. Esto cobra especial relevancia en entornos de proyectos ágiles a gran escala (LSA), donde diversos equipos autónomos colaboran en pos de un objetivo común. En estos escenarios, la coordinación y una gestión eficaz no son opcionales, sino piezas clave para mantener el rumbo [11]. Una alta autonomía de equipo, si no se equilibra con una alineación adecuada, puede llevar a procesos subóptimos que generen deuda social, como duplicación de trabajo o problemas de integración [11]. Como se evidencia en un estudio de caso, la deuda social se manifiesta en discusiones sobre autonomía del equipo, la implicación del liderazgo y la comunicación entre equipos [11].

2.1.1.4 Desafíos en la medición y gestión de la deuda social

Pese a su clara importancia, los investigadores aún enfrentan el desafío de expresar los efectos de la deuda social en términos monetarios precisos [2]. Aunque marcos como DAHLIA [2], [12] han intentado cuantificar la deuda social, principalmente en relación con la ininteligibilidad arquitectónica, se requiere más investigación empírica para generalizar estas medidas a la vasta gama de *community smells* [2]. Por lo tanto, las organizaciones necesitan abordar las decisiones sociotécnicas deficientes y “saldar” esta deuda social a través de estrategias de gestión que incluyen enfoques organizacionales, marcos, modelos, herramientas y directrices [2].

2.1.2 Síndrome de Burnout

El Síndrome de Burnout, descrito inicialmente en 1974 por el psicólogo Herbert Freudenberger [13], es un síndrome relacionado con el trabajo que se manifiesta como agotamiento emocional, despersonalización, sobrecarga laboral y una percepción de escasez de logros personales [13]. Como bien señalan las investigaciones, su alcance trasciende ámbitos específicos —arquitectura, sector educativo, administración pública o ventas— [13], para manifestarse como un fenómeno epidémico en entornos STEM (Science, Technology, Engineering y Mathematics). Dicho de otro modo, su omnipresencia y sus repercusiones son un factor crítico que obstaculiza el avance profesional y subyace al estrés laboral en los diversos campos en general [13]. En la ingeniería de software (SE) y, sobre todo con respecto

a la deuda social, el burnout no es una excepción. De hecho, informes recientes, especialmente tras la pandemia de COVID-19, han puesto de manifiesto que más del 80 % de los desarrolladores encuestados han experimentado o están experimentando alguna forma de burnout [13]. La Organización Mundial de la Salud (OMS) ha clasificado el burnout como un fenómeno ocupacional en las ediciones 10^a y 11^a de la Clasificación Internacional de Enfermedades (ICD-10, ICD-11), lo que intensifica su gravedad y el reconocimiento de su impacto en la salud mental en el ámbito laboral [13]. Los estudios han explorado a fondo las causas y consecuencias del burnout entre los equipos de desarrollo. Las consecuencias fundamentales reportadas del burnout incluyen:

- **Agotamiento:** La sensación de estar abrumado y exhausto de recursos emocionales y físicos [13].
- **Cinismo:** Una actitud negativa, hostil y excesivamente distante hacia el trabajo y los compañeros [13].
- **Ineficacia profesional:** La percepción de una disminución en la competencia, el logro personal y productividad en el trabajo [13].

No obstante, al considerar el burnout como un síntoma de “deuda social”, nuestra atención se dirige hacia las dinámicas subyacentes y las expectativas tácitas o explícitas no satisfechas dentro de las interacciones humanas y organizacionales. El burnout en la deuda social sí describe fenómenos que encajan con esta interpretación, por ejemplo, las prácticas de comunicación deficientes son una causa recurrente de burnout:

- Una menor participación de los empleados en la comunicación organizacional se correlaciona con un mayor agotamiento emocional [13].
- La apertura a la crítica puede causar burnout si no se maneja adecuadamente, ya que la falta de retroalimentación constructiva puede llevar a la frustración y al agotamiento emocional [13].
- Los problemas de comunicación entre gerentes y miembros del equipo, así como los desacuerdos entre compañeros, han sido señalados como factores desencadenantes del burnout [13].

Teniendo en cuenta lo descrito anteriormente, las relaciones tóxicas en equipos de desarrollo son claros detonantes de burnout. Como demuestran estudios recientes, el lenguaje descortés en comunicaciones técnicas (desde “solicitudes agresivas” hasta comentarios despectivos) erosiona la confianza y genera una deuda social acumulada: ese déficit invisible de respeto y apoyo mutuo que pagamos con estrés crónico [13]. Paralelamente, factores como sobrecarga laboral, roles ambiguos o tecnoestrés —estrés causado por la tecnología— aíslan a los profesionales, impidiéndoles conectar con sus equipos [13]. Cuando la presión

ahoga la interacción humana, germina el cinismo y la despersonalización —síntomas clave del burnout. Esta desconexión representa otra deuda organizacional: la que se contrae al sacrificar el bienestar por la productividad inmediata [13]. El desenlace es amargo: ese profesional agotado que abandona la carrera no solo reduce la productividad, sino que cobra simbólicamente la deuda social acumulada. Su partida es el recordatorio más crudo de que ningún proyecto sobrevive cuando quema a su gente [13].

2.1.3 Scrum

2.1.3.1 Definición de Scrum

Scrum es un marco de trabajo que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos; donde su arquitectura en equipos reducidos y autogestionados operan mediante ciclos iterativos —denominados Sprints—, los cuales, tiene una duración máxima de un mes, priorizando la entrega iterativa de valor [14]. En Scrum, tres roles vertebran su estructura: el Scrum Master, que facilita los procesos; el Product Owner, el responsable del valor; y el Equipo de Desarrollo, ejecutores de tareas; a su vez, su núcleo operativo reside en la tríada de transparencia-inspección-adaptación [14]. En eventos estructurados tenemos el Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective [14]. Cabe subrayar que, a diferencia de otras metodologías, Scrum no prescribe herramientas técnicas específicas; su esencia radica en proveer un esqueleto mínimo que catalice la colaboración y la mejora progresiva [15].

2.1.3.2 Adaptación de Scrum para la gestión de la deuda social

En el siguiente capítulo entraremos a definir a fondo cómo Scrum puede ser adaptado para abordar la deuda social. Sin embargo, es importante destacar que Scrum, en su esencia, no está diseñado específicamente para gestionar la deuda social. No obstante, su estructura y principios pueden ser adaptados para abordar este tipo de deuda de manera efectiva. Por ejemplo, los eventos de Scrum, como las reuniones diarias y las retrospectivas del Sprint, pueden ser utilizados para identificar y discutir problemas relacionados con la deuda social. Además, el rol del Scrum Master puede ser clave en la facilitación de la comunicación y colaboración entre los miembros del equipo, lo que puede ayudar a mitigar la deuda social [14].

2.2 Trabajos relacionados

A continuación, se presentan algunos hallazgos documentados en la Revisión Sistemática de la Literatura (RSL) realizada, que identifican las brechas existentes, los objetivos del proyecto y los aportes encontrados. La RSL se llevó a cabo siguiendo el protocolo detallado

en [16] y [17], que incluye las siguientes actividades: (i) aplicación del enfoque GQM (Goal-Question-Metric) [18] y el marco de trabajo S.M.A.R.T (Specific, Measurable, Achievable, Realistic, Time-bound) [19], (ii) definición de la estrategia de búsqueda y selección, (iii) realización de la revisión, y (iv) elaboración del informe de la revisión. La RSL completa está disponible en el Anexo A. Este proceso se realizó con el objetivo de recopilar y categorizar la información existente sobre el tema de investigación. A continuación, en la Figura 2.1 se puede observar la secuencia de actividades realizadas para la RSL:

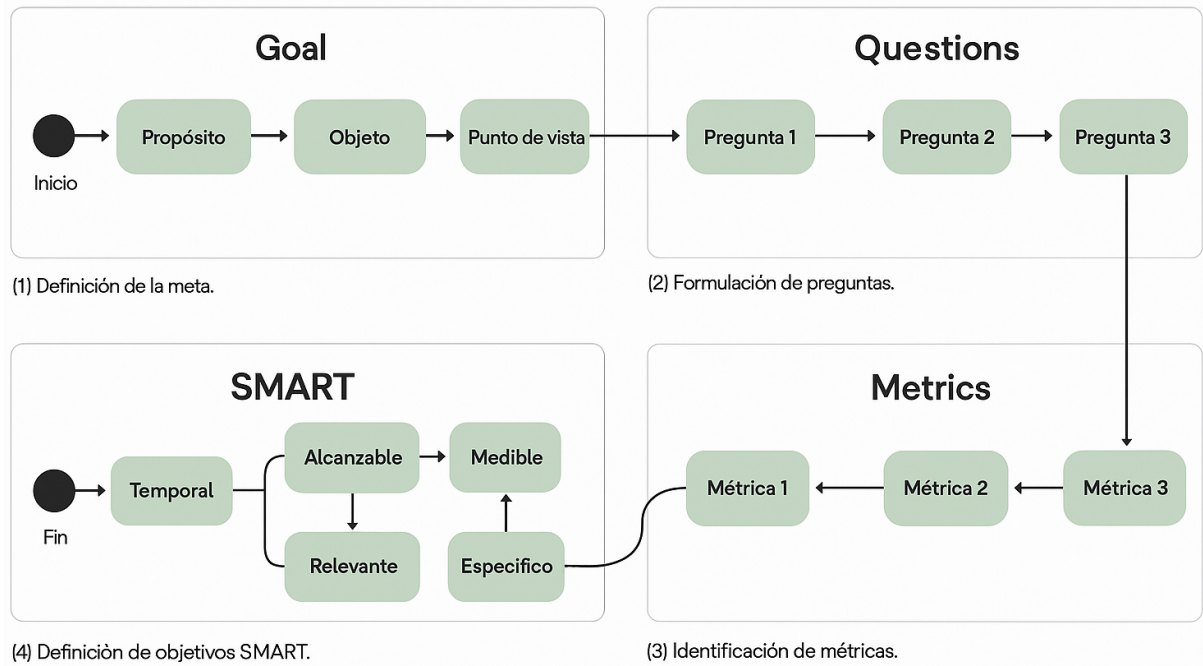


Figura 2.1: Secuencia de actividades realizadas para la RSL

2.2.1 Protocolo de investigación

Para llevar a cabo la RSL, se utilizaron herramientas tecnológicas de inteligencia artificial, como ChatGPT [20] y Microsoft Copilot [21], con el objetivo de obtener diversas formas de definir un mismo concepto y construir la cadena de búsqueda básica. Los documentos a menudo presentan temas de manera dispersa y poco clara, lo que puede generar sesgos y dudas. En este contexto, los chatbots con inteligencia artificial (CIA) resultaron ser recursos valiosos, ya que, mediante prompts específicos, permitieron iniciar conversaciones y obtener información más precisa. A partir de los términos identificados con la ayuda de los CIA, se definió una versión inicial de la cadena de búsqueda básica: “social debt” OR “social software engineering” AND “agile software development” OR “software development teams”. Posteriormente, se seleccionó Google Scholar como la base de datos principal para obtener información o estudios relacionados. Esta elección se basó en varias razones:

- **Amplia cobertura:** Google Scholar indexa una gran variedad de fuentes académicas, incluyendo artículos, tesis, libros y conferencias, lo que permite acceder a una amplia

gama de literatura relevante.

- **Accesibilidad:** Es una herramienta de acceso abierto y gratuito, lo que facilita su uso para investigadores con recursos limitados.
- **Relevancia:** Google Scholar utiliza algoritmos que priorizan artículos altamente citados y relevantes, lo que ayuda a identificar estudios influyentes en el campo de la deuda social y el desarrollo ágil.
- **Integración con otras bases de datos:** Aunque Google Scholar fue la base principal, también se complementó con otras bases de datos especializadas, como IEEE Xplore, SpringerLink y Science Direct, para asegurar una revisión exhaustiva.

La búsqueda manual reveló que las frases clave “social debt” y “agile software development” eran esenciales para definir la cadena de búsqueda básica. Por esta razón, se decidió utilizar el conector lógico “AND” para unir estas dos frases, ya que los artículos relevantes para la investigación las mencionaban conjuntamente. De esta manera, la cadena de búsqueda básica: “social debt” AND “agile software development” se aplicó inicialmente a seis bases de datos científicas: Google Scholar, IEEE Xplore, SpringerLink y Science Direct, como se observa en la Tabla 2.1.

Tabla 2.1: Cadenas de búsqueda utilizadas en la revisión sistemática

No.	Cadena de búsqueda	Fuente	Estado
1	“social debt” AND “agile software development”	Google Scholar	Incluida
2	(“Full Text Only”:social debt) AND (“Full Text Only”:agile software development)	IEEE Xplore	Incluida
3	“social debt” AND “agile software development”	SpringerLink	Incluida
4	‘social debt’ AND ‘agile software development’	Science Direct	Incluida

Acrónimos utilizados: Identificador (No).

La cadena de búsqueda se aplicó a una ventana de tiempo desde 2013 hasta 2024, adaptándose a los filtros y parámetros específicos de cada motor de búsqueda. Tras realizar las modificaciones necesarias a la cadena de búsqueda básica, se inició la búsqueda en cada motor. La Tabla 2.2 muestra el total de artículos encontrados, relevantes, repetidos y primarios.

Tabla 2.2: Resultados por motor de búsqueda en la revisión sistemática

No.	Motor de búsqueda	Encontrados	Relevantes	Relevantes repetidos	Primarios seleccionados
1	Google Scholar	130	15	0	8

Continúa en la siguiente página...

Tabla 2.2 – Continuación

No.	Motor de búsqueda	Encon- trados	Relevantes	Relevantes repetidos	Primarios seleccionados
2	IEEE Xplore	28	3	0	0
3	Science Direct	5	0	0	0
4	SpringerLink	35	2	0	0
Total		198	20	0	8

Acrónimos utilizados: Identificador (No).

En la Tabla 2.3 se presentan las preguntas de investigación definidas que ayudaron a segmentar la información identificada en los artículos primarios, su motivación y la relación que tienen con los objetivos propuestos en la RSL. Debido a la limitación de espacio, la RSL se puede consultar en detalle en el Anexo A.

Tabla 2.3: Preguntas de investigación definidas

Id.	Pregunta de investigación	Motivación
PI1	¿Cuáles son los estudios primarios más citados que han proporcionado según la evidencia aportes importantes e influyentes en el contexto de la gestión de la deuda social en el desarrollo de software ágil?	Identificar y evaluar los estudios fundamentales que han tenido un impacto significativo en el campo de la gestión de la deuda social en el desarrollo de software ágil.
PI2	¿Cuál es la distribución de tiempo de publicación de los estudios primarios relevantes con respecto a la deuda social en el desarrollo de software ágil?	Comprender cómo ha evolucionado el interés y el conocimiento sobre la deuda social en el contexto del desarrollo de software ágil a lo largo del tiempo.
PI3	¿Cuáles son las investigaciones o propuestas desarrolladas cuya idea central sea la gestión de la deuda social en el desarrollo de software ágil hasta la fecha?	Identificar y analizar las contribuciones académicas y propuestas prácticas que se han centrado específicamente en la gestión de la deuda social en el desarrollo de software ágil.
PI4	¿Cuáles son los principales desafíos y oportunidades para la investigación futura sobre la deuda social en el desarrollo de software ágil?	Identificar las barreras y las posibles áreas de crecimiento en el estudio de la deuda social en el desarrollo de software ágil.
PI5	¿Cuáles son las prácticas o enfoques que abordan de manera exitosa la gestión de la deuda social en el desarrollo de software ágil?	Identificar y evaluar las prácticas y enfoques que han demostrado ser efectivos en la gestión de la deuda social dentro de proyectos de desarrollo de software ágil.

Acrónimos utilizados: Identificador (Id.), Pregunta de investigación (PI).

TODO: Agregar lo faltante del anteproyecto de investigación. — EN PROCESO —

CAPÍTULO 3. FUNDAMENTOS CONCEPTUALES DE SOCIALSCRUM

Este capítulo introduce SocialScrum, un proceso ágil que nace como una adaptación del marco Scrum, pensado especialmente para abordar la Deuda Social que suele surgir dentro de los equipos de desarrollo de software. A lo largo de las siguientes páginas, se exploran las razones que dieron origen a esta propuesta, el modelo conceptual que la respalda, los fundamentos teóricos que la sostienen y, finalmente, el diseño completo del proceso: sus roles, eventos y artefactos.

3.1 Introducción

Aunque Scrum ha demostrado ser una herramienta efectiva para gestionar proyectos ágiles, su mirada suele centrarse en lo técnico y deja de lado un aspecto igual de importante: las dinámicas sociales que influyen directamente en la colaboración y el rendimiento del equipo. Con el tiempo, esas tensiones —conocidas como deuda social— pueden acumularse y terminar afectando no solo la calidad del producto, sino también el bienestar y la conexión entre los miembros del equipo [3].

En respuesta a esta necesidad, surge SocialScrum, un proceso que amplía los principios de Scrum para integrar, de manera explícita, la gestión consciente de la deuda social dentro de un ciclo ágil.

SocialScrum es un proceso ágil que adapta los principios, eventos y artefactos de Scrum para identificar, priorizar y mitigar la deuda social en equipos de desarrollo de software. Su propósito es hacer visibles las decisiones socio-técnicas subóptimas que impactan la productividad, calidad del software y cohesión del equipo [3], integrando prácticas de inspección y adaptación en el plano social y organizacional [22]. En síntesis, promueve un entorno de trabajo saludable y sostenible donde las mejoras sociales y del producto evolucionan de forma armónica. Para ello, SocialScrum introduce roles, eventos y artefactos específicos que complementan los de Scrum, enfocándose en aspectos como: la comunicación, colaboración, confianza y bienestar del equipo. A continuación, se presenta un diagrama general del proceso SocialScrum en la Figura 3.1.

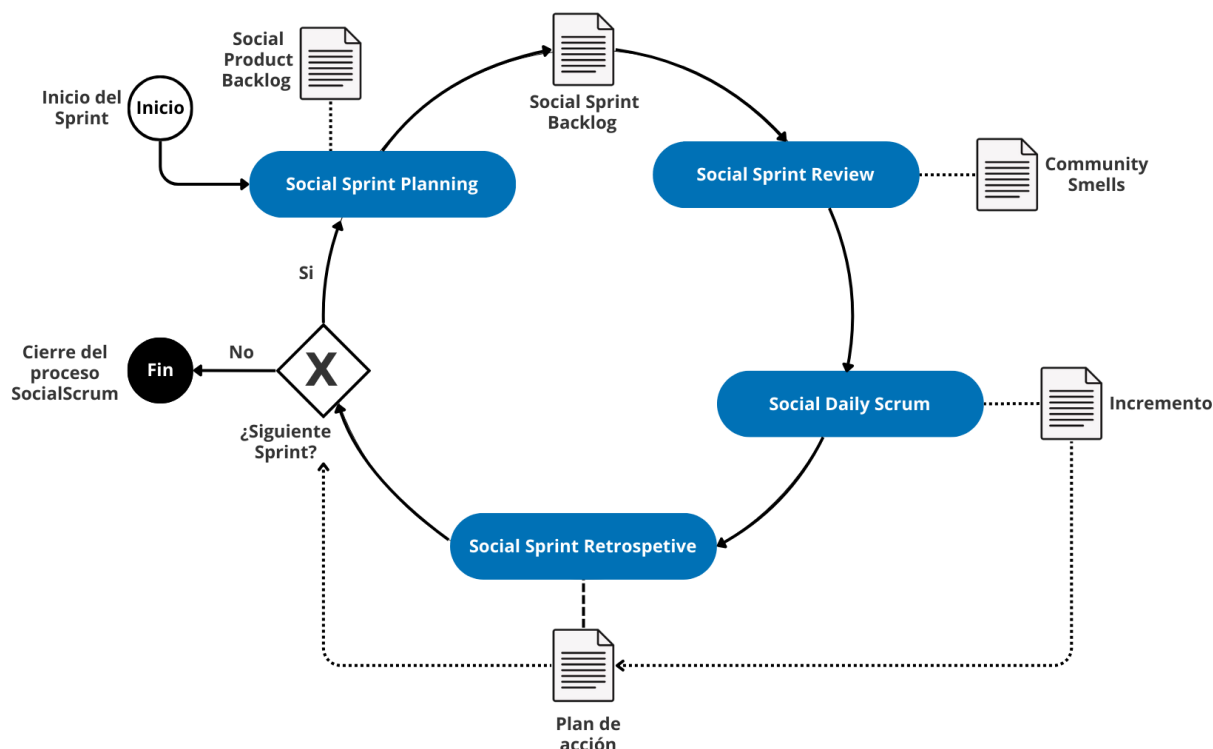


Figura 3.1: Diagrama general del proceso SocialScrum.

3.2 Modelo conceptual de SocialScrum

Si bien el modelo mantiene la estructura esencial de Scrum, incorpora componentes creados específicamente para enfrentar de manera directa la deuda social dentro del equipo. Para lograrlo, se apoya en los tres pilares empíricos de Scrum —transparencia, inspección y adaptación—, aplicándolos no solo al producto o al proceso, sino también al ámbito social y relacional entre los miembros del equipo [14] a través de los eventos de SocialScrum.

A diferencia del Scrum tradicional, que suele centrarse en los aspectos técnicos y del producto, SocialScrum se enfoca en las dinámicas humanas, no solo las visibiliza y las analiza de forma sistemática, sino que también promueve planes de mejora específicos para enfrentar los *community-smells* y reducir la deuda social [2], [10].

El modelo SocialScrum se fundamenta en tres componentes que se integran al marco tradicional de Scrum: (i) roles especializados, (ii) eventos adaptados y (iii) artefactos enriquecidos con una dimensión social. Estos elementos actúan de forma coordinada para identificar, analizar y gestionar la deuda social, fortaleciendo no solo el desempeño técnico, sino también el bienestar colectivo del equipo.

En conjunto, estas tres dimensiones promueven que las dinámicas humanas reciban la misma atención estructurada y rigurosa que los aspectos técnicos del desarrollo, reconociendo que un equipo saludable es tan esencial para el éxito del proyecto como la calidad del producto que entrega.

En la Figura 3.2 se muestra el modelo conceptual de los pilares empíricos de SocialScrum

conformado por los tres componentes mencionados.

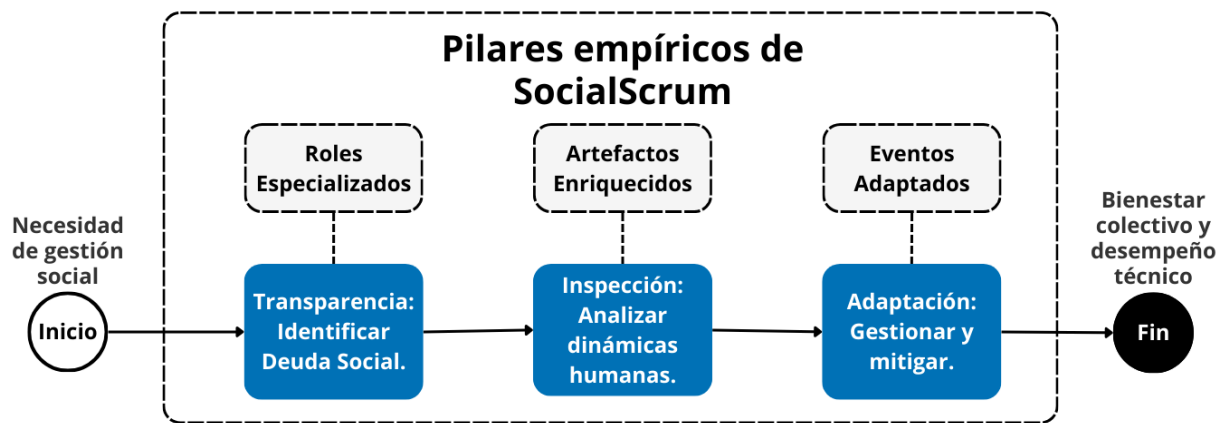


Figura 3.2: Modelo conceptual de SocialScrum.

3.2.1 Roles

SocialScrum mantiene los tres roles esenciales del marco original: el Product Owner, encargado de maximizar el valor del producto; el Scrum Master, responsable de facilitar el proceso y eliminar impedimentos; y los Developers, quienes transforman los elementos del Product Backlog en incrementos de valor [14]. No obstante, el modelo da un paso más allá al incorporar un cuarto rol especializado, el Social Guide, enfocado en atender la dimensión social del equipo y velar por su equilibrio humano y organizacional.

3.2.1.1 Product Owner

En SocialScrum, el Product Owner mantiene su rol esencial de maximizar el valor del producto [14]. Sin embargo, ahora cuenta con una nueva fuente de información proveniente del Social Guide, quien le ofrece una visión clara sobre cómo la deuda social puede afectar la predictibilidad, la colaboración y la velocidad del equipo.

Gracias a esta perspectiva, el Product Owner puede tomar decisiones de priorización más equilibradas y sostenibles, valorando no solo el impacto del producto en el negocio, sino también la salud y estabilidad del equipo que lo hace posible.

Su papel en este ámbito se limita a estar informado y valorar las recomendaciones del Social Guide cuando sea pertinente para la planificación del producto, pero sin involucrarse directamente en la gestión o resolución de las dinámicas sociales internas [22]. En otras palabras, el Product Owner se convierte en un aliado informado, no en un actor operativo dentro de la gestión social del equipo.

En SocialScrum, el Product Owner asume tres nuevas responsabilidades que amplían su rol tradicional e incorporan la dimensión social como parte de la gestión del producto:

1. Durante el *Sprint Planning*, el Product Owner recibe del Social Guide una evaluación del equipo, expresada en una escala del 1 al 10. Si este puntaje resulta crítico, el

Product Owner comprende que el equipo no está en condiciones de asumir *features* de alta complejidad. En ese caso, prioriza en su lugar acciones o elementos orientados a mitigar la deuda social existente.

2. En cada proceso de priorización, el Product Owner busca equilibrar tres dimensiones: (a) el valor de negocio, (b) la urgencia técnica y (c) la salud social del equipo. Estos factores rara vez se alinean de manera perfecta, por lo que la decisión final requiere diálogo y juicio compartido entre el Product Owner, el Social Guide y el Scrum Master.
3. El Product Owner no interviene directamente en las dinámicas sociales internas del equipo. Su papel es mantenerse informado y considerar ese contexto al priorizar, reconociendo que un equipo con buena salud social entrega productos de mayor calidad y de forma más sostenible.

La operacionalización detallada de este rol, incluyendo el framework de priorización (Triángulo de Decisión) y los protocolos esenciales, se presenta en el Capítulo 4, Sección 4.7.

3.2.1.2 Scrum Master

El Scrum Master conserva su papel como facilitador del proceso y protector del equipo ante interrupciones externas [14]. En el contexto de SocialScrum, colabora estrechamente con el Social Guide para identificar de manera temprana los impedimentos sociales que puedan afectar el desarrollo del Sprint.

Mientras el Social Guide aporta una mirada analítica sustentada en datos —por ejemplo, a través de Análisis de Redes Sociales (SNA, Social Network Analysis) o métricas socio-técnicas—, el Scrum Master complementa esta visión con una comprensión más vivencial del grupo, promoviendo conversaciones difíciles y facilitando la resolución constructiva de conflictos [22].

Ambos roles actúan de manera complementaria: combinan la evidencia empírica con la sensibilidad humana para diseñar estrategias que fortalezcan la cohesión y el bienestar del equipo, siempre respetando su capacidad de autoorganización.

Hay que aclarar que el Scrum Master no asume responsabilidades directas en la gestión de la deuda social; esa función recae exclusivamente en el Social Guide. Su rol es más bien de apoyo y colaboración, asegurando que las intervenciones sociales se integren de manera armoniosa dentro del proceso ágil sin generar fricciones ni confusiones en las responsabilidades.

3.2.1.3 Developers (Desarrolladores)

Los Developers mantienen su responsabilidad principal sobre la implementación técnica y la autogestión del trabajo durante el Sprint [14]. No obstante, en SocialScrum asumen

también un rol más activo en el cuidado de la salud social del equipo. Esto implica participar en la detección de *community smells* durante las ceremonias, proponer acciones de mejora que se integren al Social Sprint Backlog y colaborar con el Social Guide aportando retroalimentación sobre aspectos como la comunicación, la colaboración o la distribución del conocimiento [22].

De esta manera, los Developers se convierten en agentes directos del cambio social dentro del equipo, garantizando que las acciones e intervenciones propuestas sean realistas, aplicables y sostenibles en el contexto de su trabajo cotidiano.

3.2.1.4 Social Guide

El Social Guide es la figura central de SocialScrum y su misión es cuidar la dimensión social del equipo. No ocupa un rol jerárquico; más bien, actúa como facilitador y consultor en temas humanos y organizacionales. Su objetivo principal es mantener el equilibrio emocional y relacional del grupo, garantizando que la confianza, la comunicación y la cooperación sean pilares constantes del trabajo.

Sus responsabilidades principales incluyen:

- **Identificación y análisis de *community smells*:** Detecta patrones como el aislamiento, la falta de cooperación o decisiones socio-técnicas poco informadas [22]. Para ello, utiliza herramientas como el Análisis de Redes Sociales (SNA, Social Network Analysis) y otras técnicas que ayudan a evaluar la salud de la comunidad de desarrollo.
- **Facilitación de estrategias de mitigación:** Diseña y aplica acciones específicas para atender los problemas detectados, fomentando una comunicación abierta, un entorno de seguridad psicológica y relaciones basadas en el respeto mutuo [11], [22].
- **Educación y concientización:** Promueve la comprensión colectiva sobre qué es la **deuda social** y cómo impacta en la productividad y calidad del software [10].
- **Colaboración con el Scrum Master:** Trabaja codo a codo con el Scrum Master para identificar y resolver impedimentos sociales, aportando una mirada especializada en la cohesión grupal y la resolución de conflictos [14].
- **Colaboración con el Product Owner:** Informa al Product Owner sobre los efectos de la deuda social en el valor del producto, facilitando una priorización equilibrada entre resultados técnicos y salud del equipo [13], [22].

Es fundamental dejar claro lo que el Social Guide no hace, a continuación se mencionan algunas de sus limitaciones y alcances:

- **No reemplaza al Scrum Master:** el Scrum Master se encarga de facilitar el proceso ágil; el Social Guide, en cambio, se enfoca en facilitar las dinámicas sociales del equipo. Ambos roles se complementan, no se superponen.
- **No evalúa el desempeño individual:** el Social Guide nunca entrega información personal a recursos humanos ni a la gerencia para fines de evaluación. Los datos que maneja son agregados, confidenciales y usados exclusivamente para la mejora del equipo.
- **No actúa como “policía social”:** su labor no es sancionar ni imponer normas. Su función es acompañar y guiar al equipo para que identifique y resuelva sus propios conflictos.
- **No toma decisiones sobre el producto:** el Product Owner define qué se construye; el Social Guide simplemente aporta información sobre cuándo el equipo está en condiciones de comprometerse.

Para desempeñar el rol de manera efectiva, el Social Guide necesita combinar sensibilidad humana con criterio analítico. Algunas competencias clave son:

1. **Experiencia en dinámicas de grupo:** conocimientos en psicología social (idealmente con especialización en gestión de personas), resolución de conflictos y facilitación de equipos.
2. **Comprensión técnica:** conoce el contexto del desarrollo de software como prácticas de *pair programming*, revisiones de código o despliegues, aunque no necesariamente sea un programador experto.
3. **Capacidad de medición:** sabe interpretar escalas psicométricas, analizar tendencias y traducir datos en acciones comprensibles para el equipo.
4. **Neutralidad:** no tiene autoridad ejecutiva; su poder radica en la confianza, la escucha y la capacidad de facilitar sin imponer.

3.2.2 Eventos adaptados

En SocialScrum, los eventos del marco original se ajustan para incorporar la gestión de la deuda social sin modificar su esencia ni su propósito fundamental. Esta adaptación surge de una premisa sencilla pero profunda: si la deuda social es, por naturaleza, algo difícil de ver, que evoluciona con el tiempo y se resiste al cambio —como lo plantean los principios empíricos—, entonces los espacios de inspección y adaptación no pueden quedar sujetos a la casualidad ni a la buena voluntad. Deben estar integrados de manera intencional en la dinámica del equipo, formando parte de su ritmo cotidiano y de su cultura de trabajo.

SocialScrum, por tanto, no añade nuevos eventos; en cambio, aprovecha los mismos espacios del marco Scrum para incluir una dimensión social en ellos. La clave está en que esta integración se logra sin sacrificar tiempo de desarrollo ni alterar el flujo natural del equipo. Cada evento mantiene su propósito original, pero amplía su alcance para incluir la salud social como un aspecto esencial del desempeño colectivo.

Los ajustes sociales se incorporan respetando el Principio de No-Sobrecarga (más detalles en el Anexo K), que busca minimizar el impacto temporal de estas adaptaciones. A continuación, se describen las modificaciones específicas para cada evento:

- **Social Daily Standup:** se añade una pregunta final “¿Hay algún impedimento social?” que toma entre 2 y 3 minutos adicionales.
- **Social Sprint Planning:** se incorpora una breve sección de “Riesgos Sociales” (15 minutos) para identificar posibles tensiones o bloqueos humanos que puedan afectar el Sprint. Este tiempo se compensa optimizando otras secciones, sin afectar la calidad de la planificación.
- **Social Sprint Review:** se reserva un espacio de aproximadamente 10 minutos para presentar el Health Score y discutir su evolución durante el Sprint.
- **Social Sprint Retrospective:** no requiere tiempo adicional; simplemente se re-equilibra el enfoque, pasando de una atención predominantemente técnica (90 %) a una distribución más equilibrada entre lo técnico y lo social (50 %-50 %).

El tiempo adicional estimado es de aproximadamente 45 minutos por cada Sprint de dos semanas, lo que equivale a unos 4.5 minutos por día. En términos prácticos, esto representa menos del 1 % del tiempo total del Sprint, una inversión mínima frente al impacto positivo que genera en la salud y sostenibilidad del equipo.

Cada evento en SocialScrum incorpora una dimensión social concreta, en coherencia con los valores de Scrum y los fundamentos teóricos del modelo. La Tabla 3.1 ilustra cómo cada ceremonia se convierte en un espacio para observar, evaluar y fortalecer la salud social del equipo; un punto de encuentro donde lo humano también se inspecciona, junto con los aspectos técnicos que sostienen el trabajo diario.

Tabla 3.1: Eventos de Scrum adaptados en SocialScrum

Evento	Adaptación en SocialScrum
Social Sprint Planning	Se incorpora una Revisión de Riesgos Sociales , liderada por el Social Guide. En esta etapa, el equipo analiza si su estado social actual —medido mediante el Health Score y los Community Smells (Olores Comunitarios) activos— le permite comprometerse con tareas de alta complejidad. Se identifican posibles obstáculos sociales y se definen acciones preventivas que se integran directamente en el <i>Social Sprint Backlog</i> [14]. Esta práctica fortalece la autonomía del equipo al permitirle reconocer su capacidad real, y promueve la integridad al hacer visibles sus compromisos sociales.
Social Daily Standup	Se añade una pregunta breve pero significativa: “¿Hay algún impedimento social?”. El Social Guide observa las dinámicas del equipo —quién participa, quién se mantiene en silencio, el tono emocional del grupo— como señales tempranas de posibles tensiones. Si surgen fricciones, se documentan para abordarlas fuera del Daily, respetando el <i>time-boxing</i> [22]. Este enfoque normaliza la conversación sobre dificultades interpersonales y fomenta una transparencia constante en las relaciones del equipo.
Social Sprint Review	Además de presentar el Increment, se dedican entre 5 y 7 minutos a una Evaluación de la Dinámica del Equipo . El Social Guide comparte de forma breve la evolución de las métricas sociales —como SNA o distribución de conocimiento—, los Community Smells (Olores Comunitarios) mitigados o nuevos, y cómo las interacciones influyeron en la entrega. Este espacio permite que las partes interesadas (<i>stakeholders</i>) comprendan que la salud social también incide en el valor del producto [14], visibilizando un aspecto a menudo ignorado y mostrando una preocupación genuina por el bienestar colectivo.
Social Sprint Retrospective	Se convierte en el núcleo de la gestión social. El Social Guide lidera una sección específica (de 20 a 30 minutos) estructurada en cuatro fases: revisión del registro de Community Smells (Olores Comunitarios), análisis de causas raíz (por ejemplo, con la técnica de los 5 Whys (5 Porqués)), diseño de experimentos de mejora con hipótesis comprobables y priorización de las acciones resultantes para el siguiente Sprint. Las mejoras sociales más importantes se formalizan con responsables, métricas de éxito y criterios de evaluación [11]. Este evento encarna el ciclo completo de inspección, adaptación y nueva transparencia, y requiere tanto coraje para abordar los temas difíciles como compromiso para sostener los cambios en el tiempo.

Al incorporar nuevas dimensiones sociales dentro de los eventos existentes, SocialScrum busca que la gestión de la deuda social no se perciba como una tarea adicional o un “pendiente” más en la agenda del equipo, sino como una parte natural de su dinámica ágil. El objetivo es que estos espacios sociales se integren orgánicamente en el flujo de trabajo, promoviendo una cultura donde lo humano y lo técnico convivan en equilibrio. Así, se fortalece tanto la calidad del producto como las relaciones que lo hacen posible. La Tabla 3.2 presenta una estimación del tiempo adicional (sobrecarga) que podrían implicar estas adaptaciones, evidenciando que el impacto temporal es mínimo en comparación con los beneficios obtenidos en cohesión, confianza y sostenibilidad del equipo.

Tabla 3.2: Sobrecarga temporal de los eventos sociales en SocialScrum

Evento	Tiempo estimado	Sobrecarga social	Facilitador
Social Daily Standup	15 min	+2-3 min	Social Guide (1 pregunta)
Social Sprint Planning	4 h	+15 min	Social Guide (revisión de riesgos sociales)
Social Sprint Review	2 h	+10 min	Social Guide (evaluación del Health Score)
Social Sprint Retrospective	1.5 h	+0 min	Scrum Master (técnico) + Social Guide (social)

El principio de No Sobrecarga establece que las adaptaciones de SocialScrum deben integrarse dentro de los eventos existentes de Scrum, procurando no incrementar de forma significativa la duración total de las ceremonias. Este principio se operacionaliza en el Capítulo 4, en la Sección 4.4 y se detalla técnicamente en el Anexo K.

El **Social Sprint Retrospective** es, sin duda, el evento que mayor protagonismo tiene dentro del ciclo de gestión social, ya que marca el cierre natural del proceso y el punto de partida para nuevas mejoras. A diferencia de las retrospectivas tradicionales donde los temas humanos suelen abordarse solo de manera tangencial, aquí se convierten en el eje central de la conversación, con una estructura clara, propósito definido y un enfoque deliberado en el bienestar colectivo.

La Figura 3.3 ilustra el flujo completo. El proceso comienza con la presentación del Social Guide, quien comparte tanto datos cuantitativos como métricas del Sprint y tendencias del Health Score además de observaciones cualitativas sobre las interacciones del equipo. A partir de esta base, el grupo analiza de forma colaborativa las causas de los Community Smells (Olores Comunitarios) identificados, evitando soluciones rápidas o paliativas. Las acciones resultantes se formulan como experimentos con hipótesis claras, métricas de éxito y criterios definidos para decidir si continuar, ajustar o descartar el cambio. Esta lógica experimental refleja el espíritu empírico y de aprendizaje continuo que caracteriza a SocialScrum.

En este flujo, todos los roles tienen un papel activo y complementario. El Social Guide aporta el diagnóstico analítico y orienta la conversación hacia los aspectos relacionales; el Scrum Master facilita los diálogos difíciles y protege el ambiente psicológico del grupo; los Developers proponen soluciones prácticas y contextualizadas; y el Product Owner actualiza el *Social Product Backlog* con las acciones resultantes. De esta manera, la gestión social deja de ser responsabilidad de una sola figura y se convierte en un compromiso compartido que refuerza la madurez y cohesión del equipo.

Criterios de éxito:

- Se analizó en profundidad al menos un *smell* crítico, abordando sus causas reales y no solo sus síntomas.

- Las acciones acordadas se formularon como hipótesis comprobables, con métricas claras que permitan evaluar su efectividad.
- El equipo pudo expresar sus inquietudes con libertad y sin temor a represalias, reflejando un entorno de auténtica seguridad psicológica.
- Se identificaron aprendizajes concretos del Sprint anterior, tanto sobre lo que funcionó como sobre aquello que requiere ajuste o mejora.

En la Figura 3.3 se presenta el flujo detallado del evento Social Sprint Retrospective en SocialScrum desde la perspectiva de los roles involucrados, comenzando por el Social Guide y finalizando con la actualización del Social Product Backlog por parte del Product Owner.

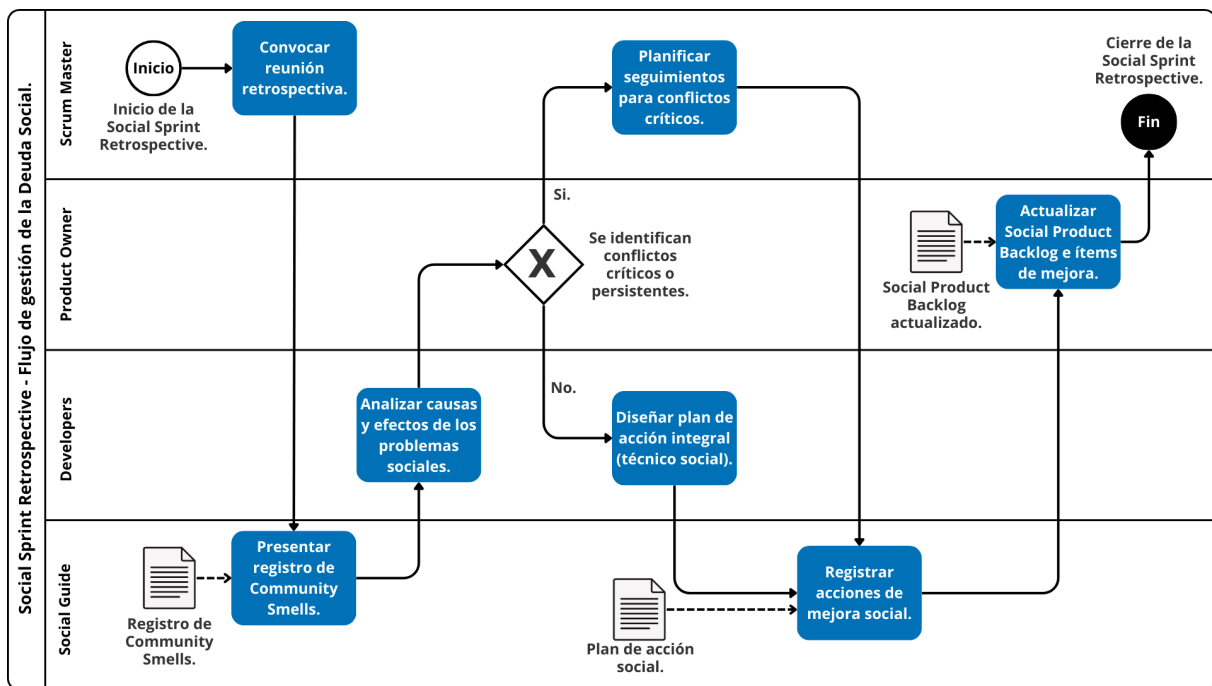


Figura 3.3: Flujo detallado del evento Social Sprint Retrospective en SocialScrum.

Para la implementación técnica exhaustiva del modelo, el catálogo detallado de preguntas guía para la facilitación social se encuentra en el Anexo G. Asimismo, la configuración de las herramientas tecnológicas (Jira y Azure DevOps) y las plantillas operacionales listas para su uso se detallan en el Anexo E.

3.2.3 Artefactos sociales

En Scrum, los artefactos —Product Backlog, Sprint Backlog e Increment— representan el trabajo y el valor generados, y están diseñados para mantener la transparencia en todo momento [14]. En SocialScrum, estos artefactos se amplían para incluir la dimensión social, no como un añadido superficial, sino como una extensión natural del principio de transparencia aplicado a los aspectos humanos del equipo.

Si la deuda social influye directamente en el valor del producto —porque retrasa entregas, degrada la calidad o genera retrabajo—, entonces debe abordarse con el mismo nivel de rigor que la deuda técnica. Los artefactos adaptados en SocialScrum buscan justamente eso: transformar los problemas sociales en elementos visibles, medibles y gestionables, con el mismo nivel de importancia que cualquier funcionalidad técnica.

La Tabla 3.3 ilustra cómo cada artefacto se enriquece con información social, manteniendo su propósito original, pero ampliando su alcance hacia la sostenibilidad del equipo y la salud del entorno de trabajo.

Tabla 3.3: Artefactos de Scrum adaptados en SocialScrum

Artefacto	Adaptación en SocialScrum
Product Backlog	Se amplía con elementos de Deuda Social o habilitadores de salud del equipo: iniciativas destinadas a mejorar la comunicación, reducir Community Smells (Olores Comunitarios) persistentes o fortalecer las habilidades sociales del grupo (por ejemplo: “Establecer protocolos de <i>code review</i> no tóxicos para mitigar Priggish Members (Miembros Remilgados)”). El Product Owner los prioriza junto al Social Guide considerando cuatro criterios: el Health Score actual del equipo, la severidad del smell, el impacto en la velocidad y la viabilidad de mitigación [22]. Este enfoque reconoce que cuidar la salud social también es entregar valor, no una distracción. Refuerza la capacidad del equipo para gestionar su propia complejidad (habilidad) y fomenta la mejora continua mediante herramientas concretas (competencia).
Sprint Backlog	Integra explícitamente las acciones de mitigación social como ítems formales, con descripción clara, responsables asignados, esfuerzo estimado y criterios de aceptación definidos. Ejemplo: “Implementar sesiones semanales de Lunch & Learn (Almuerzo y Aprendizaje) sobre el subsistema X. Owner (Propietario): Developer A + voluntario rotativo. Definition of Done (Definición de Terminado): Al menos tres desarrolladores pueden explicar la arquitectura básica. Métrica: reducción superior al 30 % en preguntas a Developer A (medido en Slack).” Esta visibilidad asegura que el trabajo social no sea “invisible”, sino parte integral del compromiso del Sprint [22]. Formaliza la responsabilidad colectiva (compromiso) y potencia la autonomía del equipo para definir cómo abordar sus propios retos interpersonales.
Increment	Además de cumplir con la Definition of Done (Definición de Terminado) técnica, cada Increment se evalúa también por su impacto en la Salud Social. El Social Guide participa en la definición de criterios de sostenibilidad social, respondiendo preguntas como: ¿el trabajo se completó sin generar Burnout (Síndrome de Agotamiento)? ¿sin crear nuevos Community Smells (Olores Comunitarios) (por ejemplo, Black Cloud (Nube Negra) por falta de documentación)? ¿mejoraron las dinámicas de colaboración? Se analizan indicadores como variaciones en el Health Score, aparición o reducción de Community Smells (Olores Comunitarios), distribución del conocimiento usando Análisis de Redes Sociales (SNA, Social Network Analysis) o niveles de carga emocional (horas extra, rotación). Un Increment que cumple lo técnico pero deteriora el bienestar del equipo debe considerarse deuda social acumulada [22]. Este equilibrio entre entrega técnica y sostenibilidad humana fortalece el valor de foco y refleja la relacionalidad del equipo al medir su cohesión y colaboración efectiva.

Artefactos complementarios específicos de SocialScrum Además de los artefactos adaptados, SocialScrum incorpora tres artefactos complementarios diseñados específicamente para gestionar la deuda social de manera estructurada y continua:

1. **Registro de Community Smells:** Un registro centralizado donde se documentan los Community Smells (Olores Comunitarios) detectados, junto con su fecha, categoría, nivel de severidad, estado actual, acciones tomadas y resultados obtenidos. Este registro no es de uso privado del Social Guide; por el contrario, es completamente transparente para el equipo, bajo la premisa de que “lo que no se mide, no se gestiona”. Cumple también una función de memoria organizacional, evitando la llamada *amnesia social* —cuando se olvidan problemas ya resueltos y se vuelven a repetir—, además de permitir el análisis de tendencias a lo largo del tiempo.
2. **Dashboard de Métricas Sociales:** Una visualización en tiempo real de los principales indicadores de la salud del equipo: evolución del Health Score, distribución de conocimiento usando Análisis de Redes Sociales (SNA, Social Network Analysis), señales tempranas de Burnout (Síndrome de Agotamiento) (por ejemplo, exceso de horas extra o variaciones en la velocidad individual), índice de seguridad psicológica (a partir de encuestas cada 2 o 3 Sprints) y equilibrio en la participación en la toma de decisiones. Estas métricas complementan la visión cualitativa del Social Guide, ayudando a detectar patrones o anomalías antes de que el problema escale, por ejemplo, un miembro que pasa de tener alta participación a permanecer en silencio.
3. **Catálogo de Estrategias Probadas:** Un repositorio evolutivo donde se registran los experimentos realizados: el *smell* abordado, la estrategia implementada, el contexto del equipo, los resultados obtenidos (éxito, fracaso o mixto) y los aprendizajes derivados. No es un manual rígido ni prescriptivo —no se trata de “hacer X para resolver Y”—, sino una base de conocimiento viva que acelera el diseño de futuras intervenciones al evitar repetir intentos fallidos. Representa el enfoque empírico del modelo: cada equipo aprende, adapta y construye su propio conocimiento contextual.

Estos artefactos no son accesorios ni recomendables “solo si hay tiempo”. Son la infraestructura mínima necesaria para que los principios de transparencia, inspección y adaptación funcionen también en el ámbito social. Gracias a ellos, las buenas intenciones se traducen en prácticas sostenibles y medibles que fortalecen la salud del equipo a largo plazo.

La descripción detallada —con plantillas, herramientas digitales y ejemplos de implementación— se presenta en el Anexo E.

3.3 Fundamentos teóricos

Cuando hablamos de gestionar la deuda social en equipos de desarrollo de software, nos enfrentamos a tres preguntas o tres retos concretos, estos son:

1. **El reto motivacional:** ¿Qué hace que un desarrollador decida involucrarse de verdad en detectar y solucionar problemas sociales del equipo, cuando eso va más allá de lo que aparece en su lista de tareas técnicas?
2. **El reto relacional:** ¿Qué es lo que mantiene viva la colaboración genuina en espacios donde todos dependemos unos de otros, pero nadie está vigilando si realmente estamos colaborando o solo aparentando?
3. **El reto preventivo:** ¿Cómo logramos frenar el deterioro de las relaciones y la comunicación antes de que se conviertan en un problema serio, en esa deuda social tan difícil de pagar?

Pues bien, para hacer frente a estos desafíos, SocialScrum se sustenta en dos marcos teóricos que, lejos de oponerse, se complementan de manera orgánica: la Teoría de la Autodeterminación (TAD) [23] y el Modelo Integrador de Confianza de Mayer et al. [24]. Ambos proporcionan herramientas conceptuales precisas para comprender y abordar lo que realmente ocurre dentro de los equipos cuando la deuda social comienza a acumularse.

3.3.1 Necesidades psicológicas y motivación intrínseca

Para abordar la deuda social —que con frecuencia se manifiesta en estrés crónico y en el síndrome de *burnout* [13], [25]—, SocialScrum se fundamenta en la Teoría de la Autodeterminación (TAD) [23]. Esta teoría, considerada un modelo integral de la motivación humana y de la personalidad, pone el acento en las tendencias naturales de crecimiento y en las necesidades psicológicas básicas que todas las personas comparten [23].

Cuando dichas necesidades son satisfechas, los individuos experimentan mayor vitalidad, bienestar y estabilidad emocional [23]. Esto cobra especial relevancia en los equipos de desarrollo de software, donde la motivación interna, el equilibrio emocional y la cohesión social son pilares esenciales que la deuda social puede deteriorar progresivamente.

Durante el diseño de SocialScrum se analizaron marcos teóricos alternativos, como la Teoría de la Autoeficacia de Bandura [26], centrada en la percepción individual de competencia, y la Teoría del Flujo de Csikszentmihalyi [27], enfocada en los estados de inmersión óptima durante la actividad. No obstante, la Teoría de la Autodeterminación (TAD) fue elegida por tres razones clave que la hacen especialmente pertinente para comprender y gestionar la deuda social:

En primer lugar, a diferencia de teorías basadas en incentivos externos —como la Teoría de las Expectativas de Vroom [28]—, la Teoría de la Autodeterminación (TAD) explica por qué las personas pueden actuar sin necesidad de recompensas extrínsecas, impulsadas por el valor intrínseco de la tarea misma [23]. Esta diferencia es crucial: identificar *community smells* implica un acto de vulnerabilidad voluntaria, no una respuesta condicionada por incentivos formales.

Pensémoslo así:

- Los *community smells* no se identifican a través de Indicadores Clave de Desempeño (Key Performance Indicators) (KPI) tradicionales ni mediante sistemas de recompensas.
- Detectar problemas sociales dentro del equipo exige vulnerabilidad voluntaria, no simplemente el cumplimiento de objetivos.
- Mitigar la deuda social de forma sostenible requiere compromiso genuino, no una adhesión superficial por cumplimiento.

En segundo lugar, la evidencia empírica muestra que la frustración de las necesidades psicológicas básicas definidas por la Teoría de la Autodeterminación (TAD) —autonomía, competencia y relacionalidad— predice de manera directa el desarrollo del *burnout* [13], una de las consecuencias más graves y costosas de la deuda social. La falta de autonomía se manifiesta como agotamiento emocional; la ausencia de competencia deriva en sensaciones de ineficacia profesional; y la carencia de relacionalidad alimenta el cinismo y el distanciamiento interpersonal dentro del equipo.

En tercer lugar, a diferencia de otras teorías concebidas para entornos industriales o de manufactura, la Teoría de la Autodeterminación (TAD) ha demostrado su validez empírica en equipos de conocimiento y en contextos complejos [23], donde la autoorganización, la autonomía y la motivación intrínseca son factores decisivos. Justamente, este es el tipo de entorno en el que opera Scrum.

Bajo este enfoque, la Teoría de la Autodeterminación (TAD) reconoce tres necesidades psicológicas universales, cuya satisfacción impulsa tanto la motivación intrínseca como el bienestar colectivo del equipo:

3.3.1.1 Autonomía

La autonomía hace referencia a la necesidad humana de sentirse autor de las propias acciones, de tomar decisiones que reflejen los valores y el criterio personal, en lugar de actuar únicamente por presión externa [23]. En el marco de SocialScrum, esta autonomía se expresa cuando los equipos tienen la libertad de decidir cómo abordar tanto los desafíos técnicos como los sociales, sin que las soluciones les sean impuestas desde fuera.

La falta de autonomía suele traducirse en agotamiento emocional y pérdida de motivación [13]. Cuando los equipos no tienen voz en las decisiones que afectan su trabajo cotidiano, la deuda social se acumula de forma silenciosa: los miembros se desconectan, trabajan de manera mecánica y dejan de proponer mejoras. *Community smells* como Solution Defiance (Desafío a la Solución) o Organizational Rigidity (Rigidez Organizacional) son síntomas claros de entornos donde la autonomía ha sido limitada o erosionada con el tiempo.

En SocialScrum, la autonomía se protege a través de la autoorganización genuina: los equipos deciden qué *community smells* priorizar, cómo diseñar sus estrategias de mitigación y qué compromisos sociales incorporar en el *Social Sprint Backlog*. El Social Guide actúa como facilitador del proceso, pero no como figura de autoridad. De esta manera, el equipo se mantiene como el verdadero responsable y dueño de su salud social.

3.3.1.2 Competencia

La competencia se refiere a la necesidad de sentirse capaz y efectivo en lo que se hace, de percibir que el propio esfuerzo genera progreso y resultados con sentido [23]. No implica perfección, sino la vivencia constante de crecimiento, aprendizaje y dominio gradual frente a los desafíos del entorno.

En los equipos de desarrollo, la falta de competencia suele manifestarse como una sensación de ineficacia profesional: la impresión de que, por más esfuerzo que se invierta, el trabajo no avanza o carece de impacto real [13]. Esta frustración alimenta directamente la deuda social, pues los miembros comienzan a dudar de sus capacidades, evitan pedir ayuda o colaborar por temor a exponerse, y poco a poco se aíslan del grupo.

SocialScrum aborda esta necesidad a través de diversas estrategias: fomenta el *pair programming* y las sesiones de conocimiento compartido, para que los miembros fortalezcan sus habilidades en un entorno de apoyo y aprendizaje mutuo; incorpora en las retrospectivas espacios para reconocer y celebrar los avances sociales, no solo los técnicos; y promueve la transparencia sobre las competencias individuales y colectivas, evitando la cultura del “fingir saber”.

Cuando la competencia se cultiva y se satisface, los equipos desarrollan una confianza mutua basada en habilidades reales y observables, lo que fortalece tanto la colaboración como el sentido de eficacia colectiva.

3.3.1.3 Relacionalidad

La relacionalidad se refiere a la necesidad de sentirse conectado con los demás, de experimentar pertenencia y de establecer vínculos significativos dentro del grupo [23]. Implica reconocer que importamos a otros y que los demás también nos importan; en otras palabras, formar parte de una comunidad donde exista apoyo mutuo, empatía y cuidado genuino.

Cuando esta necesidad se frustra, emergen el cinismo y el distanciamiento emocional, dos de los síntomas más característicos del *burnout* [13]. En entornos donde los miembros se sienten aislados, invisibles o tratados como piezas reemplazables, la deuda social tiende a volverse crónica: las personas reducen su compromiso, evitan las conversaciones difíciles y terminan aceptando la indiferencia como norma.

Community smells como Social Isolation (Aislamiento Social), Lone Wolf (Lobo Solitario) o Toxic Relationships (Relaciones Tóxicas) son expresiones directas de esta carencia de relacionalidad. SocialScrum busca revertirla fortaleciendo los espacios de vínculo humano —por ejemplo, a través de la Social Sprint Retrospective, donde el equipo reflexiona sobre cómo se siente trabajando junto, no solo sobre lo que entrega.

En este punto, el valor de Respeto y el modelo de Confianza se entrelazan de manera natural: solo en un entorno donde las personas se sienten valoradas, escuchadas y cuidadas es posible construir relaciones auténticas que sostengan la colaboración y el bienestar colectivo a largo plazo.

3.3.2 Confianza en equipos interdependientes

Mientras que la Teoría de la Autodeterminación (TAD) se centra en las necesidades internas que impulsan la motivación individual, el modelo de confianza propuesto por Mayer, Davis y Schoorman [24] describe cómo se construyen, sostienen y deterioran las relaciones de interdependencia dentro de los equipos. Según este modelo, la confianza se entiende como la disposición a ser vulnerable frente a las acciones de otra persona, bajo la expectativa de que actuará con competencia, benevolencia e integridad, sin necesidad de control o supervisión constante.

Este enfoque resulta especialmente relevante en equipos de desarrollo de software, donde la colaboración efectiva depende no solo del conocimiento técnico, sino también de la calidad de las relaciones humanas.

- Los *community smells* impactan tanto la confianza técnica —“¿puedo contar con que esta persona hará bien su trabajo?”— como la confianza social —“¿le importa genuinamente el bienestar del equipo?”—.
- La deuda social no surge únicamente de errores técnicos o conflictos interpersonales, sino de la interacción entre ambos niveles: cuando la competencia profesional y la empatía relacional dejan de estar alineadas, la colaboración se erosiona gradualmente.

El modelo de Mayer et al. [24] plantea tres dimensiones fundamentales que determinan el grado de confianza que una persona deposita en otra:

3.3.2.1 Habilidad (Ability)

La habilidad se refiere a la percepción de que otra persona posee las competencias técnicas, conocimientos y destrezas necesarias para desempeñar correctamente su trabajo dentro de un dominio determinado [24]. En términos simples, responde a la pregunta: “¿Esta persona realmente puede hacer lo que dice que hará?”

En los equipos de desarrollo, la habilidad técnica constituye la base inicial de la confianza. Cuando un miembro demuestra de forma consistente su capacidad para resolver problemas complejos, escribir código limpio o diseñar soluciones sólidas, los demás comienzan a confiar en su criterio y ejecución técnica. No obstante, la habilidad —si se ejerce de forma aislada— no garantiza la confianza plena: un desarrollador sumamente competente pero que acapara conocimiento, subestima a los juniors o ignora las contribuciones ajenas puede terminar debilitando la confianza social y fragmentando la colaboración.

SocialScrum promueve el fortalecimiento de esta dimensión a través de prácticas como el *pair programming*, las sesiones de intercambio de conocimiento y la documentación accesible y compartida. Estas acciones no solo elevan las competencias individuales, sino que visibilizan la habilidad técnica de cada miembro ante el resto del equipo, permitiendo que la confianza se fundamente en evidencia observable y no en percepciones o jerarquías implícitas.

3.3.2.2 Benevolencia (Benevolence)

La benevolencia se entiende como la creencia de que otra persona actúa movida por una intención genuina de bienestar hacia los demás; en otras palabras, que busca hacer el bien sin guiarse únicamente por intereses personales o de beneficio propio [24]. Dicho de forma simple: “¿Esta persona realmente se preocupa por mí y por el equipo?”

Esta dimensión es fundamental para el desarrollo de la confianza afectiva, aquella que se construye sobre la empatía, el apoyo mutuo y los vínculos emocionales dentro del grupo [25]. En el contexto de SocialScrum, la benevolencia se traduce en el valor de Respeto y en un conjunto de estrategias destinadas a contrarrestar el cinismo y la desconexión emocional, factores estrechamente relacionados con el Burnout [13].

Cuando los miembros del equipo perciben que sus compañeros actúan con benevolencia —que se interesan genuinamente por su bienestar, que ofrecen ayuda sin esperar retribución, que reconocen los logros de otros con sinceridad—, la confianza se consolida y la colaboración se vuelve más sólida y duradera. Por el contrario, *community smells* como Toxic Relationships (Relaciones Tóxicas) o Priggish Members (Miembros Remilgados) evidencian la erosión de esta dimensión, pues revelan entornos donde predomina la competencia egoísta, la desconfianza o la falta de empatía.

3.3.2.3 Integridad (Integrity)

La integridad se entiende como la percepción de que una persona actúa conforme a principios y valores coherentes, manteniendo una correspondencia entre lo que dice y lo que hace [24]. En términos simples, responde a la pregunta: “¿Esta persona cumple sus compromisos y actúa de acuerdo con sus valores?”

Esta dimensión constituye el núcleo de la confianza sostenida en el tiempo. Un equipo puede perdonar errores técnicos (relacionados con la habilidad) o comprender momentos en los que la empatía disminuye (benevolencia), pero la falta de integridad —es decir, la incongruencia entre el discurso y la acción— produce una erosión profunda de la confianza, difícilmente reversible. Por ejemplo, cuando un miembro promete asistir a una retrospectiva y no lo hace, o cuando un líder predica transparencia pero oculta información relevante, la credibilidad del equipo se resquebraja.

En SocialScrum, la integridad se refuerza a través de tres prácticas centrales:

- La transparencia en la comunicación y en las métricas sociales,
- El cumplimiento verificable de los compromisos incluidos en el Social Sprint Backlog.
- La coherencia ética entre los valores declarados y las conductas observadas del equipo.

El Social Guide encarna esta dimensión al modelar integridad en su propio actuar, demostrando que los principios del modelo se viven, no solo se mencionan.

El modelo de Mayer et al. [24] diferencia claramente la confianza, que implica vulnerabilidad y apertura voluntaria, de la cooperación, que puede nacer de la presión, el control o la conveniencia [24]. Esta distinción es esencial en SocialScrum, pues aclara por qué un equipo puede parecer funcional en la superficie —colabora, entrega resultados— y, sin embargo, acumular deuda social bajo una cooperación forzada. Solo cuando existe confianza genuina —basada en integridad y benevolencia— pueden emerger conversaciones honestas sobre problemas interpersonales, lo que a su vez permite detectar y abordar *community smells* de raíz. La seguridad psicológica, entendida como la libertad para expresar preocupaciones sin miedo a represalias, es un reflejo directo de esa confianza auténtica, no de una coordinación mecánica o por obligación.

El estudio de Wilson et al. [25] valida empíricamente el modelo de Mayer en equipos virtuales y distribuidos, entornos hoy predominantes en el desarrollo de software. Sus hallazgos evidencian que la confianza basada en la habilidad puede surgir rápidamente —incluso entre personas que apenas se conocen—, pero la confianza que proviene de la benevolencia y la integridad requiere tiempo, consistencia y experiencias compartidas. En estos contextos, la calidad de la comunicación —su tono, empatía y coherencia— es determinante para mantener la confianza. De ahí que *community smells* como Radio

Silence (Silencio Radiofónico) o Organizational Silo (Silo Organizacional) resulten tan nocivos: interrumpen los flujos comunicativos y debilitan los lazos interpersonales que sostienen la colaboración y la confianza en equipos distribuidos.

A diferencia de otros enfoques más teóricos o difíciles de aterrizar, el modelo de confianza propuesto por Mayer et al. resulta mucho más claro y aplicable. Sus tres dimensiones —Habilidad, Benevolencia e Integridad (H-B-I)— no solo facilitan entender la confianza dentro de una organización, sino que también permiten medirla y trabajar sobre ella de manera práctica [24].

En el marco de SocialScrum, esta precisión conceptual se convierte en acciones muy concretas:

- El Social Guide puede detectar con claridad cuál de las tres dimensiones está debilitada o afectando la dinámica del equipo.
- Las estrategias para mejorar la confianza pueden diseñarse de forma más intencionada; por ejemplo, implementar sesiones de Pair Programming (Programación en Pareja) para reforzar la percepción de habilidad, o promover retrospectivas más abiertas y sinceras para fortalecer la benevolencia.
- Además, es posible hacer un seguimiento constante del progreso en cada dimensión, obteniendo así una lectura más completa y continua del estado social del equipo.

En los equipos distribuidos, Kim et al. [29] proponen una forma particular de entender la confianza: puede ser rápida (*swift*), basada en el conocimiento (*knowledge-based*) o en la identificación (*identification-based*). Esta clasificación complementa el modelo H-B-I de Mayer y amplía su comprensión.

Ahora bien, como SocialScrum está pensado para funcionar tanto en equipos presenciales como remotos, el modelo de Mayer resulta más flexible y completo. Su enfoque permite mantener una misma lógica conceptual, incluso cuando las condiciones del trabajo cambian según la distancia, el canal de comunicación o la forma de colaboración.

Conviene, sin embargo, hacer una distinción importante entre las necesidades psicológicas que plantea la Teoría de la Autodeterminación (TAD) y las dimensiones de confianza del modelo de Mayer. La TAD se centra en lo que ocurre dentro de cada persona: aquello que sostiene su motivación interna. En cambio, el modelo H-B-I observa lo que ocurre afuera, es decir, cómo los demás perciben a alguien antes de decidir si confiar o no.

Un ejemplo ayuda a entenderlo mejor. En la TAD, la competencia hace referencia a lo que una persona siente sobre su propia capacidad (“siento que puedo hacerlo”); en el modelo H-B-I, la habilidad apunta a cómo otros la perciben (“creo que esta persona puede hacerlo”). Esa diferencia, aunque sutil, es clave: un equipo puede tener integrantes muy competentes en lo individual, pero si no hay claridad ni transparencia sobre sus habilidades, la confianza colectiva difícilmente florecerá.

En este sentido, la unión entre la TAD y el modelo de confianza de Mayer le da a SocialScrum una base sólida y equilibrada. Le permite abordar la llamada “deuda social” desde varios ángulos: la motivación de cada persona, las relaciones entre compañeros y las acciones concretas para mejorar el ambiente del equipo. En pocas palabras, combina lo individual con lo colectivo, la prevención con la intervención, y el análisis con la práctica.

3.3.3 Valores de Scrum aplicados a la deuda social

Scrum se apoya en dos pilares esenciales: el empirismo y el pensamiento Lean [14]. En pocas palabras, promueve aprender de la experiencia y tomar decisiones con base en lo que realmente se observa, evitando suposiciones. Al mismo tiempo, busca eliminar todo aquello que no aporta valor y concentrar los esfuerzos en lo esencial.

Estos principios cobran un sentido especial cuando se habla de la deuda social. Este tipo de deuda, a diferencia de los problemas técnicos evidentes, suele pasar desapercibida: se manifiesta en actitudes, relaciones o dinámicas que, aunque “invisibles”, terminan afectando el bienestar y la colaboración dentro del equipo de desarrollo [2], [22].

En esa línea, SocialScrum retoma los cinco valores fundamentales de Scrum —Compromiso, Foco, Apertura, Respeto y Coraje— [14] y los convierte en una especie de brújula ética y cultural. Son el punto de referencia para guiar las decisiones, fomentar la confianza y, sobre todo, enfrentar de manera proactiva la deuda social antes de que se acumule o deteriore las relaciones del equipo.

3.3.3.1 Coraje (Courage)

El coraje es la base emocional que permite al equipo asumir riesgos y mostrarse vulnerable frente a los demás. Dado que la confianza implica precisamente esa disposición a la vulnerabilidad [24], el coraje se convierte en un requisito indispensable para tomar acciones valientes dentro de las relaciones interpersonales, conocidas en la literatura como Risk Taking in Relationship (RTR, Toma de Riesgos en las Relaciones) [24]. En SocialScrum, este valor cobra especial importancia durante la *Social Sprint Retrospective*, donde se necesita el coraje para abrir conversaciones difíciles, enfrentar tensiones o conflictos personales, y expresar de manera honesta lo que afecta la dinámica del equipo. Solo a través de esa valentía compartida se construye una confianza real y sostenible.

3.3.3.2 Apertura (Openness)

La apertura está estrechamente ligada al principio de transparencia, uno de los pilares fundamentales de Scrum, que exige que tanto el proceso como el trabajo sean visibles para todos los involucrados [14], [25]. Este valor resulta esencial para prevenir la acumulación de deuda social, ya que dicha deuda suele crecer cuando se pierde la transparencia en los aspectos no técnicos, como las interacciones humanas o las decisiones socio-técnicas

que afectan la colaboración [3], [14]. Además, el modo en que se comunica el equipo también influye en la confianza: los mensajes cargados de tensión o tono negativo tienden a ralentizar el desarrollo de la confianza en entornos mediados por tecnología, mientras que una comunicación de apoyo, empática y coherente la fortalece [25]. En este sentido, SocialScrum promueve la apertura al hacer visibles los *community smells*, las fricciones interpersonales y cualquier señal de deterioro en las relaciones, convirtiendo lo invisible en un punto de reflexión y mejora compartida.

3.3.3.3 Respeto (Respect)

El respeto se conecta directamente con la necesidad de relacionalidad propuesta por la Teoría de la Autodeterminación (TAD) y con la benevolencia del modelo de confianza. Es un valor esencial para construir un entorno inclusivo y seguro, donde cada miembro del equipo se sienta escuchado, valorado y cuidado [23]. En SocialScrum, el *Social Guide* desempeña un papel clave en la promoción de este valor, fomentando interacciones basadas en la empatía y el reconocimiento mutuo. Asimismo, el respeto actúa como un antídoto frente al lenguaje descortés y las relaciones tóxicas, factores que suelen detonar tanto el Burnout como la deuda social dentro de los equipos [13].

3.3.3.4 Compromiso (Commitment)

El compromiso del equipo con los objetivos del Sprint y con la mejora continua es un pilar fundamental para prevenir y abordar la deuda social. En SocialScrum, este compromiso va más allá del cumplimiento técnico: implica una participación activa y genuina en la identificación y mitigación de los *community smells*, así como en el fortalecimiento del bienestar colectivo [2]. Cuando todos los miembros del equipo se comprometen con un propósito común —no solo con las metas del producto, sino también con la salud emocional del grupo—, se genera un sentido compartido de responsabilidad que refuerza la cohesión, la confianza y la resiliencia del equipo frente a los desafíos cotidianos.

3.3.3.5 Foco (Focus)

Mantener el foco en la salud social del equipo es tan esencial como concentrarse en la entrega del producto. En este sentido, SocialScrum ayuda a los equipos a equilibrar ambas prioridades, garantizando que las dinámicas humanas no queden relegadas frente a las demandas técnicas [22]. Al mantener este enfoque, el equipo puede identificar y resolver tempranamente los problemas sociales que podrían afectar la productividad, la motivación o la calidad del software [22]. De esta forma, el foco deja de ser únicamente un asunto de cumplimiento técnico y se convierte en un compromiso integral con el bienestar y la sostenibilidad del trabajo en equipo.

La Tabla 3.4 muestra cómo cada valor de Scrum actúa como un antídoto específico frente a distintos community smells, conectando las dimensiones sociales afectadas con los mecanismos concretos que SocialScrum activa para mitigarlos.

Tabla 3.4: Community smells mitigados por cada valor de Scrum en SocialScrum [2]

Valor Scrum	Community Smells mitigados	Dimensión afectada	Mecanismo en SocialScrum
Coraje	Radio Silence (Silencio Radiofónico), Sharing Villainy (Villanía del Compartir), Lack of Conflict Resolution (Falta de Resolución de Conflictos), Decision Incommunicability (Incomunicabilidad de Decisiones), Newbie Ignored (Newbie Ignorado)	Comunicación y transparencia	Social Sprint Retrospective como espacio protegido; modelado de liderazgo; canales seguros para expresar tensiones.
Apertura	Radio Silence (Silencio Radiofónico), Organizational Silo (Silo Organizacional), Cognitive Distance (Distancia Cognitiva), Black Cloud (Nube Negra), Information Island (Isla de Información), Missing Link (Eslabón Perdido), Documentation Silo (Silo de Documentación)	Visibilidad de información y conocimiento	Social Product Backlog visible; métricas sociales compartidas; documentación accesible; canales abiertos de comunicación.
Respeto	Priggish Members (Miembros Remilgados), Lone Wolf (Lobo Solitario), Social Isolation (Aislamiento Social), Toxic Relationships (Relaciones Tóxicas), Power Holder (Concentrador de Poder), Bottleneck (Cuello de Botella), Dismissiveness (Desdén), Unfriendly Environment (Ambiente Hostil)	Relacionalidad y benevolencia	Código de conducta explícito; rotación de roles; feedback estructurado y empático; políticas contra el acoso o la intimidación.
Compromiso	Dissensus (Disenso), Solution Defiance (Desafío a la Solución), Prima Donnas (Primas Donnas), Organizational Rigidity (Rigidez Organizacional), Truck Factor (Key Person Dependency), Unhealthy Contribution Patterns (Patrones de Contribución No Saludables), Absent Stakeholder (Parte Interesada Ausente), Disengagement (Desenganche)	Responsabilidad colectiva y participación	Social Sprint Backlog con ownership; Definition of Done con criterios sociales; distribución equitativa del conocimiento; participación activa en decisiones.

Continúa en la siguiente página...

Tabla 3.4 – Continuación

Valor Scrum	Community Smells mitigados	Dimensión afectada	Mecanismo en SocialScrum
Foco	Work Exhaustion (Agotamiento Laboral), Inefficacy (Ineficacia), Organizational Silo (Silo Organizacional) (intraequipo), Cognitive Distance (Distancia Cognitiva), Overload (Sobrecarga), Time Wasted (Tiempo Perdido), Scattered Development (Desarrollo Disperso)	Equilibrio, priorización y carga de trabajo	Capacidad reservada para salud social; Sprint Goal dual (técnico + social); métricas balanceadas; límites WIP con enfoque social.

La aplicación concreta de estos valores —a través de indicadores tanto cuantitativos como cualitativos, como escalas de medición, preguntas de evaluación y umbrales de acción— se explica con mayor detalle en el Capítulo 4, Sección 4.6, donde se describe el funcionamiento del Health Score y sus protocolos de respuesta.

3.3.4 Principios empíricos de Scrum aplicados a la gestión de la deuda social

Scrum se apoya en tres pilares empíricos fundamentales: Transparencia, Inspección y Adaptación [14]. Aunque estos principios fueron concebidos originalmente para gestionar la complejidad técnica, adquieren un valor aún más profundo cuando se aplican a la deuda social, un fenómeno de naturaleza sociotécnica que plantea desafíos de comprensión y medición mucho más complejos.

A diferencia del código, cuya calidad puede evaluarse mediante métricas objetivas, las dinámicas humanas son difusas, cambiantes y difíciles de estandarizar. En ese contexto, los tres pilares de Scrum ofrecen una base práctica y teórica para hacer visibles los problemas sociales, inspeccionarlos de manera continua y adaptar el comportamiento del equipo frente a ellos.

Esta sección explora cómo cada uno de estos pilares se reinterpreta y adapta dentro de SocialScrum para abordar las particularidades de la deuda social, equilibrando el rigor empírico con la sensibilidad humana que exige la gestión de los aspectos sociales en los equipos de desarrollo.

3.3.4.1 Transparencia

La deuda social suele habitar en las sombras, mientras la deuda técnica deja rastros claros y medibles —como la complejidad ciclomática, las tasas de defectos o el tiempo de compilación—, la deuda social se esconde en tres espacios difíciles de capturar: las conversaciones informales que nunca se documentan, los patrones implícitos de colaboración

entre personas y los estados emocionales —agotamiento, desconfianza, resentimiento— que no aparecen en ningún dashboard de velocidad [10], [22].

Este no es solo un problema práctico de medición: es, ante todo, un problema epistemológico. Si algo no puede observarse, tampoco puede gestionarse mediante un proceso empírico. El modelo de confianza de Mayer et al. [24] lo deja claro: sin visibilidad sobre las habilidades reales, no se puede evaluar la competencia; sin visibilidad de las intenciones, no se construye benevolencia; y sin observar la coherencia entre palabras y acciones, no se valida la integridad. La opacidad social, por tanto, no solo oculta los problemas: erosiona activamente las bases de la confianza.

En SocialScrum, la transparencia no es un ideal, sino una condición de posibilidad. El marco enfrenta este desafío transformando lo difuso en explícito a través de cuatro estrategias complementarias:

- Artefactos sociales que formalizan los problemas humanos como work items gestionables.
- Métricas sociales que cuantifican aspectos cualitativos, como la comunicación o la distribución del conocimiento.
- Espacios institucionales donde expresar tensiones o impedimentos sociales sea algo esperado y seguro.
- Mecanismos de revelación protegida que permiten visibilidad colectiva sin exponer vulnerabilidades individuales (ver Tabla 3.5).

Tabla 3.5: Estrategias de transparencia social en SocialScrum

Estrategia	Función	Qué visibiliza
Artefactos sociales explícitos	Formalizan los problemas sociales como work items con el mismo estatus que la deuda técnica.	Community smells, acciones de mitigación y responsabilidades compartidas.
Métricas sociales objetivas	Cuantifican dinámicas cualitativas mediante indicadores observables.	Patrones de comunicación, distribución de conocimiento y señales tempranas de deterioro social.
Espacios formales de revelación	Institucionalizan momentos seguros para exponer impedimentos sociales.	Fricciones cotidianas, salud del Sprint e impacto en la entrega.
Mecanismos de revelación segura	Permiten visibilidad colectiva sin inhibir el reporte de temas sensibles.	Patrones agregados en lugar de casos individuales.

En última instancia, SocialScrum apuesta por que estas estrategias, aplicadas de forma constante, transformen las dinámicas sociales: de ser simples “rumores de pasillo” a convertirse en objetos legítimos de gestión. Ese paso —hacer visible lo invisible— es el punto de partida de toda intervención sistemática.

3.3.4.2 Inspección

Hacer visible la deuda social es apenas el primer paso; no basta con verla, hay que seguirle el rastro. A diferencia de la deuda técnica, que tiende a acumularse de manera lineal —cada atajo suma un poco más al problema—, la deuda social es dinámica, cambiante y profundamente contextual. Como señalan Caballero et al. [2], los *community smells* no son estáticos: evolucionan con el tiempo, atraviesan fases identificables y pueden incluso transformarse entre sí. Pueden escalar (un desarrollador independiente se vuelve cuello de botella crítico), interactuar (un silo organizacional combinado con una mala comunicación genera efectos multiplicativos) o oscilar entre latencia y manifestación (conflictos “resueltos” que reaparecen bajo presión).

Esta naturaleza volátil invalida los diagnósticos puntuales. Un chequeo anual de salud del equipo sirve tanto como medir la deuda técnica una vez al año: cuando detectas el problema, ya dejó daños profundos. Wilson et al. [25] demostraron que en los equipos distribuidos, la confianza se construye gradualmente; sin monitoreo continuo, las micro-erosiones pasan desapercibidas hasta que la colaboración se derrumba. De manera similar, Tulili et al. [13] evidenciaron que el burnout presenta señales tempranas, pero solo son visibles si alguien las busca de forma sistemática.

Desde la Teoría de la Autodeterminación [23], la inspección continua también permite detectar la frustración progresiva de las necesidades básicas: la autonomía (cuando surge micromanagement), la competencia (cuando aumentan los errores o el feedback negativo), y la relacionalidad (cuando emergen aislamiento o exclusión). Sin esa vigilancia constante, el deterioro se normaliza silenciosamente hasta que es demasiado tarde.

Para enfrentar este desafío, SocialScrum organiza la inspección en múltiples cadencias, entendiendo que distintos fenómenos operan a diferentes velocidades:

- Diaria, para fricciones e impedimentos inmediatos.
- Por Sprint, para analizar la evolución de los smells y la eficacia de las intervenciones.
- Por Release, para observar tendencias sostenidas y riesgos de burnout.
- Continua, mediante monitoreo automatizado de señales digitales que revelan cambios abruptos en patrones establecidos.

En la Tabla 3.6 se resumen estas cadencias, sus focos específicos, los fenómenos detectables y los métodos empleados en cada caso.

Tabla 3.6: Cadencias de inspección social en SocialScrum

Cadencia	Foco	Fenómenos detectables	Métodos
Diaria	Impedimentos inmediatos	Fricciones, bloqueos de comunicación, conflictos emergentes	Observación directa, indagación estructurada.
Por Sprint	Evolución de smells y eficacia de intervenciones	Nuevos smells, cambios en severidad, efectos secundarios	Análisis de métricas, revisión de causas raíz.
Por Release	Tendencias de largo plazo	Deterioro sostenido, patrones recurrentes, riesgo de burnout	Instrumentos validados, evaluaciones longitudinales.
Continua	Anomalías en patrones	Cambios súbitos o desviaciones respecto al baseline	Monitoreo automatizado.

Este enfoque multirresolución permite que SocialScrum detecte tanto las crisis inmediatas como las erosiones silenciosas —ambas igual de peligrosas, solo que en distintas escalas temporales—, garantizando así una visión integral del estado social del equipo.

3.3.4.3 Adaptación

Detectar la deuda social no basta si no se logra mitigarla de manera efectiva. Aquí aparece uno de los mayores desafíos: como señalan Saeeda et al. [22], la deuda social es “más difícil de arreglar que la deuda técnica”. Y esta dificultad no se debe solo al esfuerzo necesario, sino a una diferencia cualitativa en la naturaleza misma del problema.

La deuda técnica tiene soluciones conocidas. Cuando se detecta código con alta complejidad ciclomática, existe un catálogo probado de refactorizaciones. La deuda social, en cambio, no sigue reglas universales. Una estrategia que resulta brillante en un equipo puede ser contraproducente en otro, dependiendo de su cultura, su historia o su composición interna [2]. Peor aún, los sistemas sociales son no lineales: pequeños cambios pueden producir efectos desproporcionados, y una intervención mal diseñada puede agravar el problema que busca resolver [3].

A esto se suma la resistencia intrínseca al cambio humano. Modificar código no exige alterar identidades, hábitos ni vínculos personales. Transformar dinámicas sociales, sí. Y esa transformación suele enfrentarse a una resistencia psicológica mucho más intensa [23]. Por todo ello, SocialScrum no ofrece un “manual de soluciones” para la deuda social. En lugar de prescribir, propone experimentar. Estructura la adaptación como un proceso empírico iterativo, basado en:

- Priorizar según la severidad y la viabilidad de los problemas detectados.
- Diseñar intervenciones contextuales, apoyadas en análisis de causas raíz y exploración de opciones múltiples.
- Implementar experimentos pequeños y reversibles, con hipótesis claras y resultados observables.

- Evaluar sistemáticamente para decidir si pivotar, perseverar, escalar o institucionalizar la intervención.

La Tabla 3.7 no pretende ser un recetario, sino un punto de partida: presenta familias de estrategias documentadas en la literatura que han funcionado en ciertos contextos, pero que cada equipo debe adaptar o descartar según su realidad.

Tabla 3.7: Familias de estrategias de mitigación por categoría de community smell

Categoría	Naturaleza del problema	Familias de intervención
Comunicación	Flujos bloqueados o distorsionados	Canales estructurados, redistribución de intermediación, protocolos asíncronos [25].
Conocimiento	Distribución desigual de expertise	Transferencia por inmersión, incentivos para documentación, rotación de funciones [2].
Relacional	Deterioro de vínculos o exclusión	Mediación, normas explícitas, construcción de propósito compartido [13].
Organizacional	Impedimentos estructurales	Comunidades de práctica, movilidad temporal, escalamiento progresivo [22].

Este enfoque se alinea con los tres principios de la Teoría de la Autodeterminación (TAD) para lograr intervenciones sostenibles [23]: preservar la autonomía (co-crear soluciones, no imponerlas), reforzar la competencia (que el equipo tenga los medios para ejecutarlas) y fortalecer la relacionalidad (fomentar la unión, no la fragmentación). Solo así, la adaptación genera mejora genuina y no simple cumplimiento superficial.

3.3.5 El ciclo como sistema

La Transparencia, Inspección y Adaptación, no son etapas aisladas ni pasos lineales, sino un ciclo continuo de retroalimentación. La transparencia transforma lo difuso en observable; la inspección identifica desviaciones o señales de deterioro; la adaptación actúa experimentalmente sobre ellas, y los resultados de esas intervenciones generan nueva transparencia acerca de lo que realmente funciona.

Aplicado de manera consistente —en distintas cadencias (diaria, Sprint, Release) y a través de métodos mixtos, tanto cuantitativos como cualitativos—, este ciclo convierte la deuda social de una amenaza invisible en un aspecto gestionable dentro del proceso ágil.

Tabla 3.8: Consecuencias de la ausencia de principios empíricos en la gestión de la deuda social

Principio ausente	Patrón de fallo	Evidencia empírica
Transparencia	Acumulación silenciosa de disfunciones hasta la crisis; normalización de comportamientos nocivos.	Silos no detectados que derivan en re-arquitecturas completas [10]; “ambiente de miedo” institucionalizado en Boeing [22].
Inspección	Burnout como desenlace no anticipado; escalamiento exponencial de los costos humanos y técnicos.	Estudios muestran que el burnout presentaba señales meses antes sin inspección continua [13]; detección tardía implica costos significativamente mayores [5].
Adaptación	Intervenciones contraproducentes; persistencia de síntomas por aplicar soluciones superficiales.	Acciones mal diseñadas que agravan el burnout [11]; community smells recurrentes no abordados desde su causa raíz [2].

3.3.5.1 Ejemplo práctico del ciclo empírico en SocialScrum

Para ilustrar cómo opera el ciclo de Transparencia → Inspección → Adaptación dentro de SocialScrum, a continuación se presenta un caso práctico:

Escenario inicial: un problema invisible.

Un equipo de cinco desarrolladores trabaja en el Sprint X. A simple vista, todo parece marchar bien: cumplen con la velocidad planificada y no se reportan errores. Sin embargo, bajo la superficie existe una tensión silenciosa entre Dev A (senior) y Dev B (junior) por desacuerdos en los estándares de código. Nadie lo menciona, pues se asume que “es un asunto entre ellos”.

Fase 1: Transparencia — Hacer visible el problema

Escenario aplicado

Durante la *Sprint Retrospective*, el Social Guide introduce una pregunta clave: **Laura (Social Guide)**:

“¿Hay dinámicas interpersonales que estén afectando nuestro trabajo?”

Antes de esta intervención, el conflicto permanecía oculto y seguía acumulando fricción. Después de la pregunta, Dev A comenta:

Dev A:

“Siento que Dev B no respeta mis comentarios en los *code reviews*”

Dev B:

“Me siento juzgado; muchas veces no entiendo por qué rechazas mis PRs”

Resultado:

El conflicto, antes invisible, ahora es observable. Se ha logrado transparencia.

Fase 2: Inspección — Analizar el problema

Escenario aplicado

El Social Guide facilita el análisis sin emitir juicios personales:

Laura (Social Guide):

“¿Cuánto tiempo se está afectando el proceso de *code review*?” → Debería tomar 45 minutos, pero actualmente tarda entre 2 y 3 horas.

“¿Cuál parece ser la causa?” → Existen expectativas no documentadas sobre los estándares de código.

“¿Qué impacto tiene esto en el equipo?” → Los demás desarrolladores evitan participar en los *reviews*, duplicando el trabajo y reduciendo la colaboración.

Resultado:

La causa raíz queda identificada. Inspección completada.

Fase 3: Adaptación — Implementar cambios

Escenario aplicado

El equipo crea un ítem social dentro del *Sprint Backlog* enfocado en resolver la situación. El Social Guide facilita una serie de sesiones:

Laura (Social Guide):

Sesión 1:1 con Dev A: “¿Qué esperas de un buen PR?”

Sesión 1:1 con Dev B: “¿Qué necesitas para sentirte respetado durante la revisión?”

Sesión conjunta: ambos acuerdan y documentan qué constituye un “buen estándar de código”.

Resultado:

Existe ahora una referencia compartida. Se ha implementado un cambio.
Adaptación lograda.

Retroalimentación: cierre del ciclo

Escenario aplicado

En el siguiente Sprint, los resultados son visibles:

Laura (Social Guide):

Tiempo promedio de *code review*: vuelve a ser de 45 minutos (antes 2–3 horas). ✓

Participación: los demás desarrolladores retoman los *reviews*. ✓

Relación A–B: mejora de 3/10 (tensa) a 6/10 (profesional y respetuosa).
✓

El ciclo continúa: Transparencia → Inspección → Adaptación se convierte en una práctica recurrente que permite identificar y atender nuevos desafíos sociales a medida que surgen. *Reflexión*: sin transparencia, el conflicto habría permanecido oculto; sin inspección, no se habría comprendido su causa; y sin adaptación, nada habría cambiado. En SocialScrum, los tres pilares son interdependientes y forman el corazón del aprendizaje social continuo. El ciclo empírico de SocialScrum no promete eliminar la deuda social —ningún sistema humano podría hacerlo sin caer en la ingenuidad—, pero sí garantiza algo mucho más valioso: que cuando la deuda emerja, será detectada tempranamente, abordada con estrategias basadas en evidencia y gestionada de forma iterativa, aprendiendo de cada experiencia. En ese sentido, SocialScrum transforma la deuda social de una fuerza destructiva invisible en un componente reconocido y gestionable del desarrollo sostenible de software.

3.3.6 Medición de la salud-social

Para que SocialScrum mantenga su carácter *empírico*, necesita métricas observables que permitan evaluar el progreso de manera constante. A diferencia de las métricas técnicas —como la velocidad o la cantidad de defectos—, las métricas sociales se centran en las dinámicas humanas y relacionales del equipo.

En este modelo, la salud social se mide a través del Health Score, un indicador compuesto que evalúa los cinco valores fundamentales de Scrum:

- **Openness / Franqueza**: mide qué tan abierta y honesta es la comunicación dentro del equipo.

- **Compromiso:** evalúa el nivel de compromiso mutuo entre los miembros y su sentido de responsabilidad compartida.
- **Respeto:** refleja el grado de seguridad psicológica presente en el grupo, es decir, si las personas se sienten escuchadas y valoradas.
- **Foco:** determina si el equipo está alineado en torno a un objetivo común y evita distracciones que debiliten su cohesión.
- **Coraje:** analiza si el equipo enfrenta los problemas difíciles o tiende a evitarlos.

Cada valor se califica en una escala del 1 al 10, generando un Health Score promedio que representa la salud social general del equipo. Un puntaje inferior a 4/10 indica una situación crítica que requiere intervención inmediata, mientras que valores altos reflejan un entorno estable y colaborativo.

Ejemplos de preguntas de evaluación:

- *Apertura:* “¿Puedo expresar mi opinión con sinceridad dentro del equipo, sin miedo a consecuencias negativas?” (1 = Nunca, 10 = Siempre).
- *Respeto:* “¿Siento que mis aportes son realmente valorados y reconocidos por mis compañeros?” (1 = Para nada, 10 = Totalmente).
- *Coraje:* “¿El equipo enfrenta las conversaciones difíciles cuando algo no está funcionando?” (1 = Se evitan, 10 = Se abordan abiertamente).

La forma de aplicar esta medición —que incluye el conjunto completo de preguntas, las escalas utilizadas y el seguimiento de las tendencias a lo largo del tiempo— se explica con mayor detalle en el Capítulo 4, Sección 4.6.

Relación con los Social Story Points: mientras el Health Score evalúa la condición actual del equipo, los *Social Story Points* estiman el esfuerzo necesario para abordar un problema social o relacional. Ambos indicadores son complementarios: uno diagnostica el estado del equipo y el otro proyecta la energía requerida para mejorar su bienestar colectivo.

3.4 Salvaguardas éticas y protecciones

SocialScrum incorpora mecanismos pensados para sacar a la luz aquellas dinámicas interpersonales que normalmente pasan desapercibidas: conflictos, tensiones, agotamiento emocional o falta de motivación. El rol del Social Guide implica manejar información sensible sobre las vulnerabilidades del equipo, mientras que los artefactos sociales registran situaciones que, en otros contextos, quedarían solo en conversaciones privadas o, peor aún, nunca se mencionarían.

Esa transparencia es clave para abordar la deuda social de manera real y efectiva. Sin embargo, también plantea un dilema ético importante: si no existen salvaguardas claras, SocialScrum podría transformarse en lo contrario de lo que busca ser —una herramienta de control y vigilancia que termine destruyendo la seguridad psicológica que intenta proteger [24].

Por eso, las salvaguardas éticas no pueden verse como simples recomendaciones o “buenas prácticas”. Son requisitos indispensables para que la implementación de SocialScrum sea legítima y saludable. Sin ellas, el modelo perdería sentido y, en lugar de ayudar al equipo, podría causar el efecto opuesto: desconfianza, miedo y daño emocional.

3.4.1 Confidencialidad absoluta

Toda la información compartida durante los eventos sociales —como el Social Daily Standup, la Social Sprint Retrospective o las sesiones de mediación— debe manejarse con total confidencialidad. Ningún dato puede usarse para evaluaciones de desempeño, decisiones de contratación o despido, ni para cualquier acción que afecte la situación laboral de una persona.

Ahora bien, esta confidencialidad no significa mantener un secreto absoluto, sino actuar con un criterio contextual. El Social Guide puede, por ejemplo, comunicar información general o agregada —como el puntaje promedio de salud del equipo (Health Score) o la aparición de ciertos *community smells*—, pero jamás debe revelar detalles que permitan identificar a alguien o exponer declaraciones personales.

Lo que debe mantenerse confidencial:

- Comentarios o declaraciones personales hechas en las retrospectivas sociales (“María mencionó que...”).
- Conflictos interpersonales concretos (“Juan y Pedro tuvieron una discusión porque...”).
- Información relacionada con el estado emocional, el agotamiento (burnout) o la desmotivación de personas identificables.
- Cualquier dato que pueda ser utilizado para evaluaciones punitivas o afectar la reputación de alguien.

Lo que sí puede compartirse (datos agregados):

- Promedio del Health Score del equipo (“El equipo tiene un puntaje de 4.2 sobre 10”).
- *Community smells* generales, sin mencionar nombres ni casos específicos (“El equipo muestra señales de aislamiento social”).

- Tendencias a lo largo del tiempo (“La cohesión ha mejorado en los últimos tres sprints”).
- Necesidades de intervención sin señalar individuos (“El equipo podría beneficiarse de un taller de cohesión”).

Esta distinción es esencial: permite que la organización conozca el estado social de los equipos sin poner en riesgo la seguridad psicológica de sus integrantes. Si alguien llega a sentir que lo que dice en una retrospectiva puede usarse en su contra, la confianza se rompe... y con ella, se derrumba la transparencia que sostiene a SocialScrum.

3.4.2 Rendición de cuentas y estructura organizacional

El Social Guide reporta directamente al Scrum Master o, en su defecto, a los líderes técnicos (como el CTO o el VP de Ingeniería), pero nunca a Recursos Humanos (HR). Esta restricción existe para evitar conflictos de interés, ya que HR se encarga de evaluaciones de desempeño y decisiones laborales, precisamente los ámbitos donde está prohibido usar la información social.

La estructura organizacional recomendada es la siguiente:

- **Social Guide → Scrum Master → Liderazgo técnico (CTO / VP Engineering)**
- **Nunca:** Social Guide → HR → Evaluaciones de desempeño

Cuando la organización necesita intervenir —por ejemplo, para ofrecer un taller de cohesión o equilibrar la carga de trabajo del equipo—, el Scrum Master o el liderazgo técnico son quienes gestionan la solicitud. De esta forma, pueden actuar sin exponer información sensible ni comprometer la confianza que el equipo deposita en el Social Guide.

3.4.3 Consentimiento informado del equipo

Antes de poner en marcha SocialScrum, el equipo debe otorgar su consentimiento informado de manera explícita. Esto implica que cada miembro entiende con claridad:

- Qué tipo de información se recopilará (Health Score, *community smells*, reflexiones surgidas en las retrospectivas).
- Para qué se usará esa información (exclusivamente para la mejora interna del equipo, nunca con fines de evaluación).
- Quién tendrá acceso a ella (solo el Social Guide y el propio equipo, sin intervención externa).

- Qué salvaguardas existen para proteger la privacidad (confidencialidad, anonimización de los reportes).
- Cuál es su derecho a auditar (el equipo puede revisar en cualquier momento los reportes elaborados por el Social Guide).

El consentimiento no debe verse como un simple requisito administrativo. Es, en realidad, la base de la confianza entre el equipo y el Social Guide. Si este acuerdo no nace de una comprensión real —de los riesgos, los límites y las protecciones involucradas—, SocialScrum corre el riesgo de percibirse como una imposición más. Y cuando eso pasa, la colaboración se desvanece y da paso a la desconfianza.

3.4.4 Anonimización en reportes y métricas

Cuando el Social Guide comparte métricas o *community smells* a nivel organizacional, debe anonimizar toda la información para que ninguna persona pueda ser identificada. Por ejemplo:

- **Prohibido:** “El desarrollador A muestra signos de *burnout* (agotamiento emocional 9/10)”.
- **Permitido:** “El equipo presenta un *community smell* activo: *Work Exhaustion*. El Health Score promedio es de 4.2/10, y el valor “Foco” se encuentra en estado crítico (3/10)”.

Esta anonimización no le quita valor a la información ni reduce su utilidad. La organización sigue pudiendo identificar que hay un problema y tomar medidas —como ajustar la carga de trabajo, ofrecer apoyo o facilitar espacios de mediación—. Lo que se protege, en esencia, es la identidad de las personas, evitando que una vulnerabilidad individual se convierta en una fuente de riesgo laboral.

3.4.5 Prevención del “teatro social”

Uno de los riesgos más difíciles de detectar en SocialScrum es que los equipos aprendan a fingir bienestar social para evitar el escrutinio o la presión de la organización. A esto se le llama “teatro social”: cuando un equipo muestra métricas positivas, evita hablar de los problemas reales en las retrospectivas y proyecta una imagen de cohesión... mientras la deuda social sigue creciendo por debajo de la superficie.

El “teatro social” no surge por mala intención, sino como una respuesta lógica a incentivos mal planteados. Si las personas sienten que reconocer un problema puede traer consecuencias negativas —como evaluaciones desfavorables, presión por “arreglar” el equipo de inmediato o intervenciones que invaden su espacio—, terminarán optando por callar o maquillar los resultados.

Las salvaguardas éticas existen precisamente para evitar esto. Al asegurar que mostrar vulnerabilidad no conlleva castigos, el equipo puede expresarse con libertad. Solo cuando hay confianza en que la información no se usará en su contra, la transparencia deja de ser un riesgo y se convierte en un acto genuino.

3.4.6 Derecho de auditoría del equipo

El equipo tiene pleno derecho a revisar cualquier reporte o métrica que el Social Guide planea compartir fuera del grupo. En otras palabras, antes de comunicar el Health Score o un *community smell* a nivel organizacional, el Social Guide debe mostrar al equipo exactamente qué información se va a divulgar y en qué formato.

Esta auditoría cumple dos propósitos esenciales:

- **Transparencia interna:** el equipo puede confirmar que los datos reportados son justos, precisos y que no se está revelando nada confidencial.
- **Confianza mutua:** el Social Guide respalda con hechos su compromiso ético, mostrando coherencia entre lo que promete (confidencialidad) y lo que entrega (reportes anonimizados).

Si el equipo detecta que un reporte vulnera las salvaguardas acordadas, tiene derecho a pedir ajustes o incluso a bloquear su publicación. Este mecanismo de control compartido actúa como un equilibrio saludable: evita abusos, fomenta la corresponsabilidad y fortalece la seguridad psicológica del grupo.

3.4.7 Implementación práctica

Las salvaguardas éticas no son simples declaraciones de principios: necesitan traducirse en acciones concretas y acuerdos formales. En SocialScrum, dos documentos son fundamentales para poner en práctica estas protecciones:

1. **Acuerdo de confidencialidad del Social Guide:** un compromiso formal en el que el Social Guide declara que no compartirá información individual, que solo reportará datos agregados y que su prioridad será proteger la seguridad psicológica del equipo. Este acuerdo debe firmarse antes de asumir el rol.
2. **Acuerdo del equipo (Consentimiento Informado):** un documento donde el equipo autoriza la implementación de SocialScrum bajo condiciones claras de confidencialidad, anonimización y derecho de auditoría. Cada miembro firma, dejando constancia de que entiende y acepta plenamente estas condiciones.

Estos acuerdos no son un trámite legal más, sino verdaderos mecanismos de commitment: convierten las promesas en compromisos reales y verificables. En esencia, refuerzan la confianza al alinear lo que se dice con lo que se hace —lo que Mayer et al. [24] denominan la dimensión de integridad—, ya que este alineamiento permite que las partes evalúen objetivamente el cumplimiento de los compromisos, fortaleciendo así la cooperación y reduciendo la incertidumbre sobre el comportamiento futuro.

La implementación detallada de estas salvaguardas —incluyendo plantillas de acuerdos, protocolos de reporte y ejemplos prácticos— se presenta en el Anexo F: “Protocolos Operativos de Gobernanza y Ética del Social Guide”.

3.5 Diseño del modelo SocialScrum

La Figura 3.4 ilustra la arquitectura conceptual de SocialScrum, organizada en cuatro capas interdependientes que muestran cómo los fundamentos teóricos se traducen en mecanismos operacionales capaces de prevenir y mitigar la deuda social dentro de los equipos de desarrollo. Esta arquitectura en capas permite que el modelo escale coherentemente desde los fundamentos teóricos (motivación, confianza y valores) hasta el impacto medible en la práctica (prevención de Community Smells (Olores Comunitarios), fortalecimiento de la cohesión, sostenibilidad y bienestar).

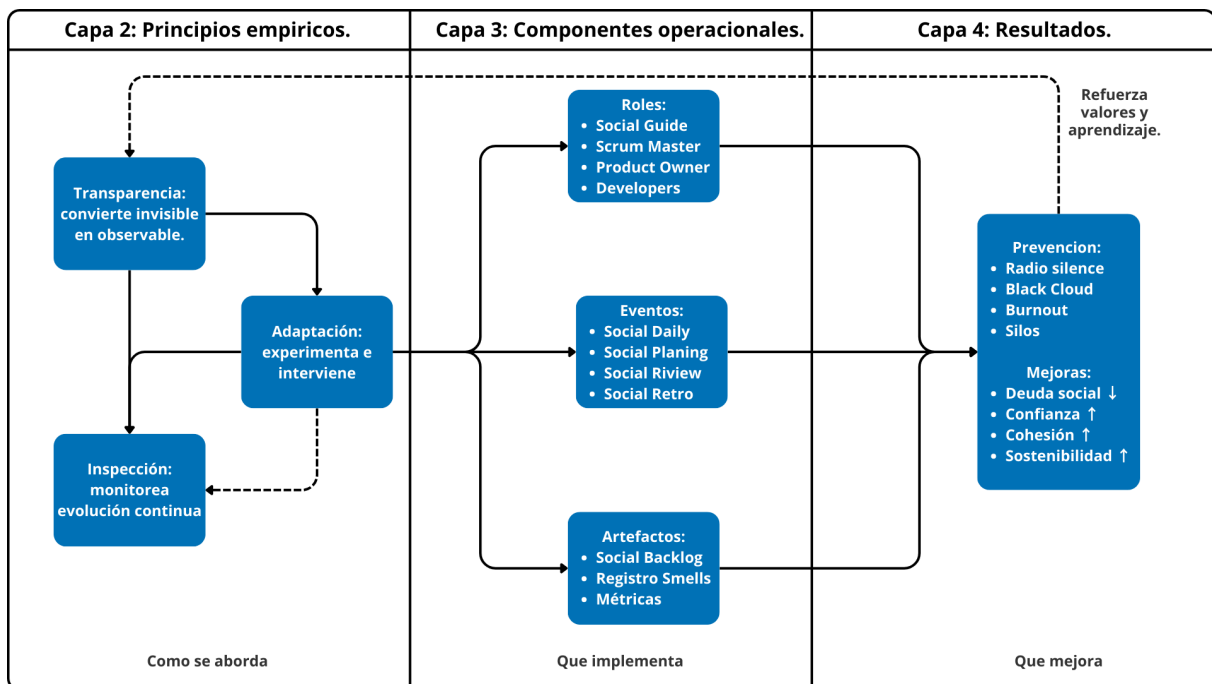


Figura 3.4: Arquitectura integrada del modelo SocialScrum (Capas 2-4)

A continuación, se describen cada una de las capas y su relación entre sí:

Capa 1 (Fundamentos): La Capa 1 integra tres marcos teóricos complementarios que sirven como la base del modelo:

- La Teoría de la Autodeterminación (TAD) explica qué necesitan las personas para mantener una motivación sostenible.
- El modelo de Confianza de Mayer et al. describe qué sostiene las relaciones de interdependencia y cooperación genuina dentro del equipo.
- Finalmente, los Valores de Scrum aportan la brújula ética y cultural que orienta las decisiones y comportamientos del grupo.

Estos tres fundamentos convergen para habilitar los principios empíricos de Scrum —Transparencia, Inspección y Adaptación— que, aplicados al ámbito social, dan origen al enfoque distintivo de SocialScrum. La relación entre estos fundamentos puede observarse en la Figura 3.5.



Figura 3.5: Fundamentos teóricos de SocialScrum.

Capa 2 (Principios empíricos): representa el corazón operativo del modelo, donde los tres pilares de Scrum —Transparencia, Inspección y Adaptación— se articulan en un ciclo de retroalimentación continua. En este nivel, la Transparencia convierte los fenómenos sociales difusos en objetos observables; la Inspección monitorea su evolución en distintas cadencias (diaria, por Sprint, por Release y continua); y la Adaptación actúa mediante experimentación controlada para intervenir en las causas raíz. Los resultados de esta adaptación generan nueva transparencia, cerrando así el ciclo y promoviendo el aprendizaje constante del equipo.

Capa 3 (Componentes operacionales): traduce los principios empíricos en mecanismos concretos que hacen que SocialScrum funcione en la práctica. Esta capa define el

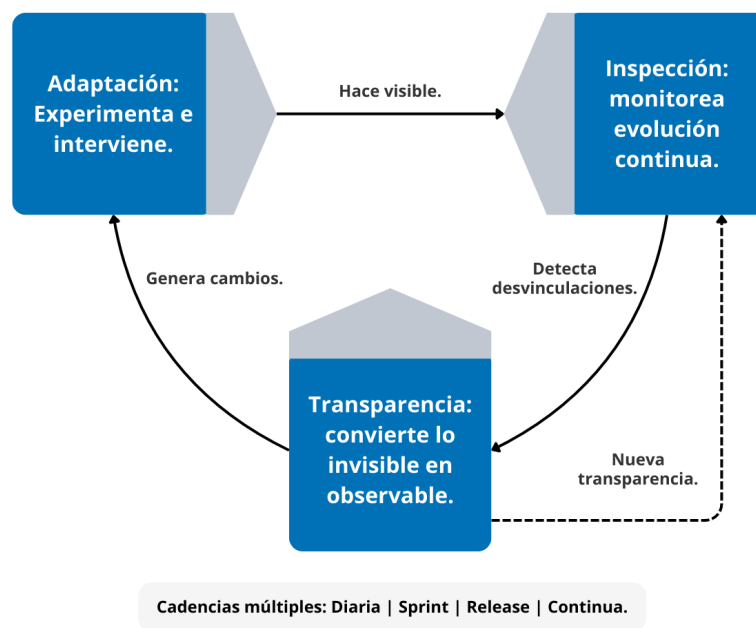


Figura 3.6: Principios empíricos de SocialScrum: ciclo de retroalimentación continua.

“cómo”: los roles que sostienen la dinámica social, los eventos que la institucionalizan y los artefactos que le dan trazabilidad.

En primer lugar, los roles incorporan al **Social Guide** como figura integradora, trabajando junto al **Scrum Master** (facilitador del proceso), el **Product Owner** (priorizador estratégico) y los **Developers** (ejecutores técnicos y sociales). En segundo lugar, los eventos son adaptados para incluir explícitamente dimensiones sociales, siendo la **Retrospective Social** el núcleo del aprendizaje colectivo. Por último, los artefactos se amplían para formalizar la gestión de la deuda social, integrando mecanismos de transparencia y seguimiento continuo.

Capa 4 Resultados (Outcomes): representa los resultados observables del modelo SocialScrum, mostrando tanto los efectos preventivos (reducción de community smells por categoría) como los impactos sistémicos positivos en el equipo y la organización.

En la Figura 3.8, se establece la última capa de SocialScrum donde trasciende la gestión operativa para mostrar su valor sistémico: cómo la aplicación continua de sus principios y componentes reduce la deuda social, fortalece la confianza, previene el burnout y mejora la sostenibilidad del trabajo en equipo. Así, los resultados no son únicamente reactivos (resolver conflictos o eliminar smells), sino también emergentes, reflejando una transformación cultural hacia equipos más cohesionados, autónomos y emocionalmente saludables.

Finalmente, los Feedback Loops (Ciclos de Retroalimentación) (representados con líneas discontinuas en la Figura 3.4) evidencian que el sistema es reflexivo: los resultados positivos retroalimentan los valores y principios que los generaron, haciendo que el ciclo

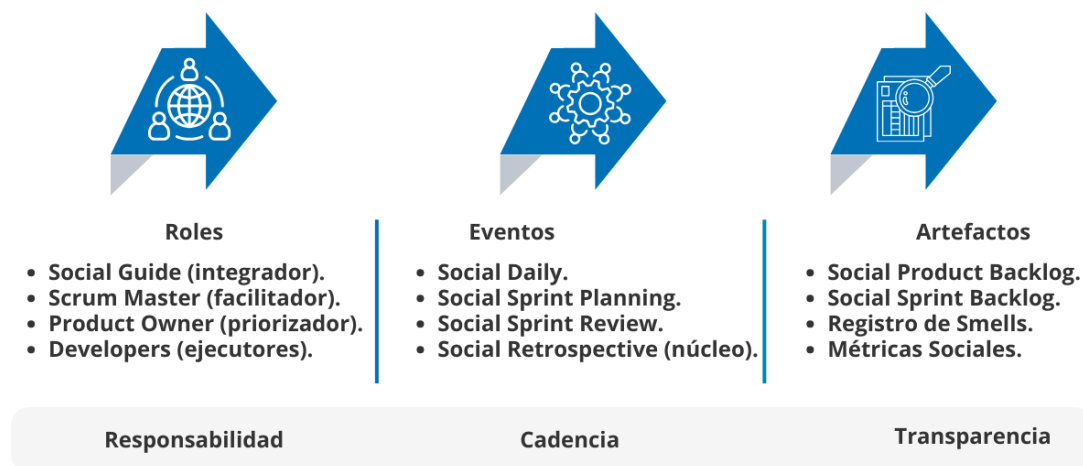


Figura 3.7: Componentes operacionales de SocialScrum: roles, eventos y artefactos adaptados.

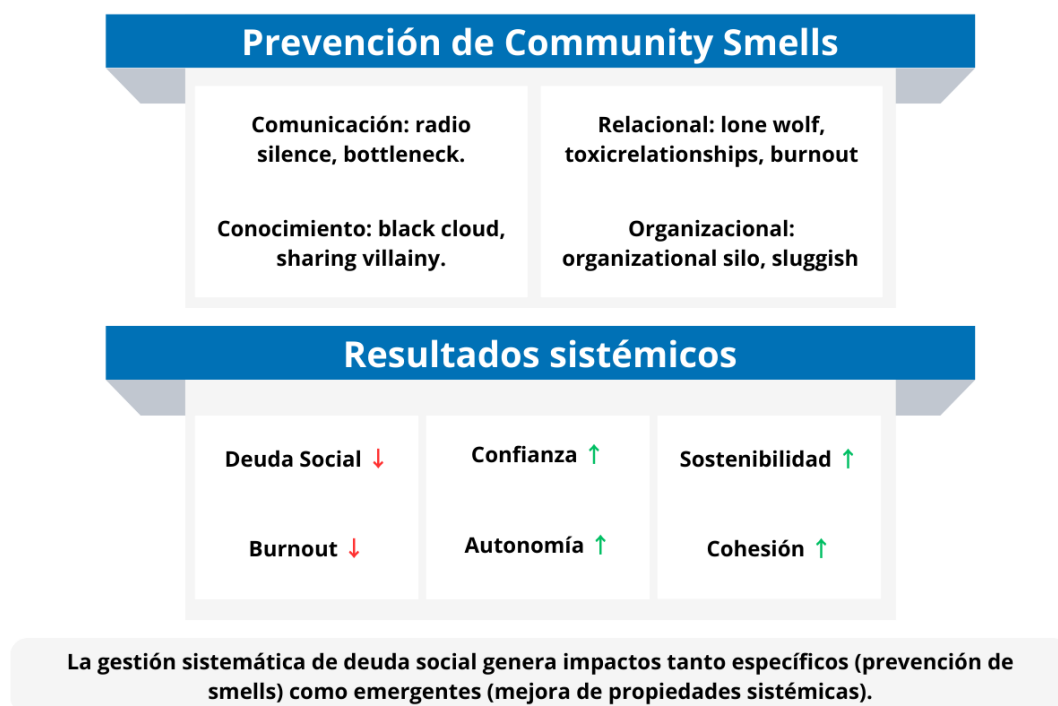


Figura 3.8: Resultados del modelo SocialScrum: prevención y mejora sistémica.

empírico se auto-perfecciona de forma continua. De esta manera, SocialScrum no se limita a un marco operativo, sino que actúa como un sistema adaptativo complejo, capaz de aprender de sí mismo. Esta arquitectura en capas permite que el modelo escale desde los fundamentos teóricos hasta el impacto medible, manteniendo coherencia conceptual en cada nivel.

3.6 Consideraciones de escalabilidad y adaptación

SocialScrum es un framework diseñado para adaptarse a distintos contextos organizacionales y de equipo. Aunque sus principios fundamentales —la Teoría de la Autodeterminación (TAD), la confianza y los valores de Scrum— son universales, su aplicación práctica debe ajustarse a las particularidades de cada entorno.

3.6.1 Adaptabilidad a contextos diversos

Las siguientes preguntas sirven como guía para adaptar SocialScrum de forma responsable y contextual:

1. ¿Cuál es el tamaño del equipo? (¿5 personas o 50?)
2. ¿Dónde está ubicado el equipo? (¿Co-localizado, híbrido o completamente remoto?)
3. ¿Qué nivel de madurez tiene en Scrum? (¿Principiante o experimentado?)
4. ¿Cómo es la cultura organizacional? (¿Jerárquica, horizontal o con dinámicas disfuncionales?)

SocialScrum debe ajustarse a estas variables, buscando mantener su esencia sin imponer estructuras rígidas. Algunos ejemplos prácticos de adaptación incluyen:

- **Equipo pequeño (5 personas):** el rol de Social Guide puede ser rotativo, dedicando cada miembro un pequeño porcentaje de su tiempo (aproximadamente un 10 %) en lugar de tener una figura exclusiva.
- **Equipo completamente distribuido:** los *Social Daily Standup's* pueden realizarse de manera asincrónica (por ejemplo, a través de Slack o herramientas similares), reduciendo la sobrecarga de reuniones sin perder la transparencia diaria.
- **Organización jerárquica:** es recomendable contar con un patrocinio ejecutivo (CTO o CEO) que respalde al Social Guide y garantice la protección de sus reportes frente a posibles interferencias o presiones jerárquicas.
- **Cultura organizacional tóxica:** en entornos con altos niveles de desconfianza o comunicación deteriorada, SocialScrum puede no ser viable hasta que se produzca una transformación organizacional previa que restablezca condiciones básicas de respeto y apertura.

Nota importante: SocialScrum no es un modelo “único para todos”. Su efectividad depende de una adaptación consciente a la realidad de cada equipo y organización.

Las consideraciones más amplias sobre cómo escalar SocialScrum y adaptarlo a diferentes contextos se explican con mayor detalle en el Capítulo 4, Sección ??: “Escalabilidad y Adaptación de SocialScrum”.

3.7 Limitaciones reconocidas

SocialScrum no es una fórmula mágica ni una “bala de plata”. Su efectividad depende, en gran medida, del contexto organizacional en el que se implemente. De hecho, hay situaciones donde el modelo encuentra límites estructurales que es necesario reconocer con honestidad y sin maquillarlos:

Contextos organizacionales severamente tóxicos. En entornos donde predominan la desconfianza sistémica, el abuso jerárquico o una comunicación totalmente fracturada, SocialScrum solo puede actuar de forma limitada. Puede ayudar a reducir el impacto individual y fomentar pequeños espacios de seguridad dentro de ciertos sub-equipos, pero no tiene la capacidad de erradicar la toxicidad organizacional desde la raíz.

En estos escenarios, la implementación debe centrarse en aquellos equipos cuyo liderazgo esté genuinamente comprometido con el cambio, evitando caer en la falsa expectativa de que el modelo por sí solo “arreglará” una cultura disfuncional. Sin una transformación real a nivel ejecutivo, cualquier intento se quedará corto.

Requisito de patrocinio ejecutivo genuino. El Social Guide solo puede desempeñar su labor de manera efectiva si cuenta con un respaldo real y explícito de los niveles ejecutivos (CTO, VP de Ingeniería, CEO). Cuando la alta dirección no garantiza la confidencialidad de los reportes, no asigna recursos para intervenciones o, peor aún, usa la información con fines punitivos, SocialScrum se derrumba y da paso al “teatro social”: los equipos comienzan a aparentar bienestar mientras esconden los problemas de fondo. Sin un compromiso ejecutivo claro y tangible, SocialScrum no debe implementarse.

Riesgo de “teatro social” y simulación de bienestar. Cuando los equipos sienten que hablar de sus problemas puede traer consecuencias negativas —como presión para “arreglar” el equipo de inmediato, intervenciones excesivas o evaluaciones de desempeño afectadas—, desarrollan mecanismos de defensa que vacían de sentido el modelo. Empiezan a reportar métricas positivas, evitan discutir los conflictos reales durante las retrospectivas y proyectan una imagen de cohesión que no refleja su realidad interna.

Este riesgo solo puede prevenirse a través de salvaguardas éticas firmes y de una coherencia constante entre lo que la organización promete y lo que realmente hace. Sin esa congruencia, la transparencia se vuelve una puesta en escena más.

Escalabilidad no lineal y costo del Social Guide. A diferencia del Scrum Master que puede acompañar a varios equipos al mismo tiempo, el Social Guide necesita involucrarse de forma profunda en la realidad particular de cada grupo. Por eso, la escalabilidad

de SocialScrum no es lineal: gestionar la deuda social de diez equipos no se resuelve simplemente contratando diez Social Guides, sino construyendo una cultura organizacional donde los principios del modelo estén verdaderamente integrados.

En estructuras grandes, el costo inicial puede ser alto, pero debe entenderse como una inversión estratégica que solo da frutos con visión y compromiso a largo plazo.

Tensiones operacionales inherentes: ¿Quién cuida al vigilante? SocialScrum no elimina los conflictos estructurales dentro de una organización; simplemente los hace visibles. Algunos de ellos son especialmente delicados:

- **Product Owner vs. Social Guide:** ¿Qué pasa cuando el Product Owner exige “entregar ya”, mientras el Social Guide alerta que el equipo está emocionalmente al límite? El modelo puede mostrar datos claros —como un Health Score en estado crítico—, pero la decisión de priorizar el bienestar sobre la entrega sigue dependiendo de instancias de liderazgo que SocialScrum no controla.
- **Supervisión del Social Guide:** ¿Y si el propio CTO, quien supervisa al Social Guide, forma parte del problema? Si la toxicidad proviene del liderazgo técnico, el Social Guide se enfrenta a un conflicto de interés que no puede resolver desde dentro. En estos casos, se necesita escalar la situación a niveles ejecutivos superiores o, si no hay otra opción, recurrir a Recursos Humanos bajo garantías explícitas de no represalia.

Dependencia de competencias especializadas. El rol del Social Guide exige una combinación poco común: sensibilidad humana, rigor analítico y conocimiento técnico. No todos los psicólogos organizacionales entienden los entornos de desarrollo de software, ni todos los Scrum Masters tienen formación en dinámicas de grupo. Esta carencia de perfiles híbridos limita, de manera natural, la adopción del modelo en el corto plazo. Sin embargo, estas limitaciones no invalidan la propuesta. Lo que sí demandan es honestidad organizacional: SocialScrum solo funciona cuando existen condiciones básicas de confianza, autonomía y apoyo desde la dirección. Si esos cimientos no están, intentar implementarlo puede generar más frustración que progreso.

3.8 Conclusiones del capítulo

El modelo SocialScrum representa una evolución necesaria del marco ágil tradicional, al incorporar explícitamente la gestión de la deuda social como parte integral del proceso de desarrollo. A través de la integración de marcos teóricos complementarios —la Teoría de la Autodeterminación (TAD), el modelo de Confianza de Mayer et al. y los valores y principios empíricos de Scrum— el modelo proporciona una base científica y práctica

para abordar dimensiones humanas que históricamente han permanecido invisibles en la gestión ágil.

SocialScrum demuestra que las dinámicas sociales pueden gestionarse con el mismo rigor empírico que los aspectos técnicos del producto. Mediante la aplicación cíclica de los principios de Transparencia, Inspección y Adaptación, el modelo permite identificar patrones de deterioro social, implementar intervenciones contextualizadas y medir su efectividad a lo largo del tiempo. Esta aproximación convierte la salud social del equipo en un indicador gestionable, equiparable a la calidad del software o la velocidad de entrega.

Más allá de la mitigación de disfunciones, SocialScrum impulsa un cambio cultural: fomenta la motivación intrínseca, la confianza mutua, la cohesión relacional y la sostenibilidad emocional del trabajo colectivo.

Al hacerlo, redefine el éxito en entornos ágiles, equilibrando rendimiento y bienestar, y demostrando que un equipo saludable no es solo un medio para producir mejor software, sino un fin valioso en sí mismo.

CAPÍTULO 4. SOCIALSCRUM: PROCESO ÁGIL PARA LA GESTIÓN DE LA DEUDA SOCIAL

Este capítulo traslada el modelo teórico de SocialScrum (Capítulo 3) al terreno de la aplicación práctica en equipos ágiles. Se muestra cómo sus componentes adaptados (eventos, artefactos y los roles (como también el rol del Social Guide) se materializan en escenarios reales para identificar, priorizar y mitigar la deuda social y los olores comunitarios. Asimismo, se presenta el proceso operativo que permite implementar SocialScrum en equipos reales, incluyendo protocolos, plantillas y salvaguardas éticas.

4.1 Consideraciones del capítulo

El Capítulo 3 presentó el modelo conceptual de SocialScrum, sus bases teóricas y su arquitectura general. Allí se explicó cómo tres marcos teóricos complementarios sustentan la propuesta:

1. **Teoría de la Autodeterminación [30]:** proporciona las necesidades psicológicas (autonomía, competencia, relación) que los eventos sociales deben proteger.
2. **Modelo de confianza de Mayer et al. [31]:** define cómo se construye la confianza mutua (habilidad, benevolencia, integridad) en el equipo.
3. **Valores de Scrum [14]:** establecen el cómo (coraje, apertura, respeto, compromiso, foco) para operacionalizar esas necesidades.

Sin embargo, un modelo sólido sigue siendo solo una idea hasta que se convierte en acciones concretas, protocolos operativos y herramientas que los equipos puedan usar en su día a día. Este capítulo cierra esa brecha, llevando SocialScrum del plano conceptual a la ejecución real mediante una estructura sintética orientada a la ingeniería de procesos. En concreto, incluimos el nuevo rol encargado de guiar el proceso de SocialScrum (Sección 4.3), además, mostramos cómo la autonomía se protege mediante los protocolos operativos de los eventos (Sección 4.4), la confianza se construye a través de los principios de transparencia y confidencialidad (Sección 4.2 y Anexo F), y los valores se practican en la gestión de artefactos y medición (Secciones 4.5 y 4.6). La propuesta se desglosa en tres niveles de operacionalización:

1. **Adopción:** preparación del equipo y checklist de viabilidad (Sección 4.2) además de la implementación del nuevo rol de Social Guide (Sección 4.3).
2. **Ejecución:** gestión de eventos, artefactos, métricas de salud social y el framework de priorización (Secciones 4.4 a 4.7).
3. **Gobernanza:** protección ética y escalabilidad, cuyos detalles procedimentales se desplazan a los Anexos E y F para preservar el enfoque práctico del capítulo.

¿Por qué es importante este capítulo? Porque responde las preguntas que cualquier equipo y organización se plantean antes de adoptar SocialScrum: ¿es viable en nuestro contexto? ¿Cómo se ejecuta en la vida diaria? ¿Qué hace exactamente el Social Guide? ¿Cómo se equilibran los ítems sociales con los técnicos sin parálisis? ¿Cómo se mide el progreso? ¿Y qué garantías existen para evitar que esto termine pareciendo vigilancia organizacional? Estas no son preguntas menores: tocan la viabilidad ética y operativa del modelo completo. Implementar SocialScrum no es tan sencillo como añadir una herramienta al flujo de desarrollo. Gestionar la deuda social implica transformaciones profundas en la cultura organizacional, la forma de relacionarse y la manera en que el equipo entiende su propio bienestar. De hecho, una mala implementación puede ser peor que no hacer nada. Un Social Guide sin salvaguardas éticas puede convertirse en una herramienta de vigilancia que socava la seguridad psicológica que el modelo intenta proteger. Por eso, antes de adoptar SocialScrum, todo equipo y organización deben verificar si existen condiciones mínimas (ver la Sección 4.2.1: liderazgo genuino, confidencialidad garantizada, autonomía del equipo, cultura de confianza). Si falta alguno de estos pilares, lo mejor es esperar e invertir en construirlos primero. SocialScrum no arregla culturas disfuncionales; las requiere para funcionar.

Dada esta complejidad, este capítulo se organiza como un manual práctico y riguroso a través de cuatro componentes centrales: (1) **protocolos paso a paso** donde se presentan instrucciones claras para facilitar cada evento y cada decisión de priorización; (2) **plantillas y artefactos listos para usar** donde se proveen formatos para Jira, Confluence o Slack que evitan improvisación; (3) **ejemplos contextualizados**, a través de casos reales y escenarios simulados que muestran aciertos y errores frecuentes y (4) **salvaguardas éticas obligatorias** conformada por un conjunto de principios indispensables para proteger la confidencialidad y seguridad psicológica. Estos cuatro componentes se entretajan a lo largo de las secciones que siguen.

4.2 Adopción inicial de SocialScrum

La adopción de SocialScrum no es un trámite administrativo ni una decisión que la gerencia pueda imponer por sí sola. Es un proceso cuidadoso, consensuado y con un fuerte

sustento ético, que requiere preparación, transparencia y un compromiso real tanto del equipo como de la organización. A diferencia de la incorporación de una herramienta técnica —donde basta con capacitarse y configurarla—, SocialScrum implica un cambio cultural mucho más profundo donde el equipo debe estar dispuesto a mostrar vulnerabilidades, hablar abiertamente de tensiones y asumir la responsabilidad compartida sobre su propia salud social. Sin esa disposición, cualquier intento de adopción terminará convirtiéndose en “teatro social”: una apariencia de bienestar sin cambios reales.

Esta sección explica el proceso de adopción inicial, organizado en tres fases: evaluación de los pre-requisitos organizacionales, preparación del equipo (Sprint 0) y presentación formal a los Stakeholders (Partes Interesadas). Cada fase incluye criterios de éxito concretos y puntos de decisión que permiten detener el proceso si las condiciones aún no están dadas.

4.2.1 Pre-requisitos organizacionales

Antes de empezar la implementación de SocialScrum, es clave verificar si el contexto organizacional realmente cuenta con las condiciones mínimas para sostenerlo. No todos los equipos —ni todas las empresas— están listos para adoptar este modelo, y forzarlo en un entorno inadecuado no solo genera frustración: también puede empeorar los problemas que pretende resolver.

4.2.1.1 Condiciones necesarias para la adopción

SocialScrum exige, como punto de partida, que existan al menos cuatro condiciones organizacionales fundamentales.

a. Compromiso ejecutivo genuino. El (Chief Technology Officer (Director de Tecnología) (CTO)), Vicepresidente (VP) de Ingeniería o roles equivalentes, deben respaldar el modelo de forma clara y explícita. Ese respaldo no puede quedarse en discursos bienintencionados; tiene que verse reflejado en acciones concretas:

- Asignar presupuesto para el rol del Social Guide, incluyendo tiempo protegido y capacitación especializada.
- Aceptar el principio de no-sobrecarga: comprender que SocialScrum puede reducir temporalmente la capacidad técnica del equipo (entre un 10 % y un 20 %) mientras se atienden olores comunitarios críticos.
- Comprometerse con las salvaguardas éticas: garantizar formalmente que la información recopilada por el Social Guide jamás se usará para evaluaciones de desempeño ni decisiones de empleo.

- Estar dispuesto a intervenir cuando sea necesario: por ejemplo, ajustar la carga de trabajo, aprobar talleres especializados o apoyar la mediación en conflictos organizacionales.

Si este compromiso no existe, el Social Guide quedará sin autoridad moral y sin los recursos necesarios para actuar. Una señal evidente de falta de respaldo es cuando el liderazgo afirma: “Sí, implementen SocialScrum... pero sin afectar la velocidad ni pedir recursos adicionales”. Eso no es compromiso; es delegar sin ofrecer apoyo real.

b. Cultura organizacional mínimamente saludable. SocialScrum no es una herramienta de rescate para organizaciones profundamente tóxicas. Si la cultura presenta los siguientes síntomas de manera sistémica, la adopción debe posponerse hasta que ocurra una intervención más profunda a nivel organizacional:

- **Desconfianza institucionalizada:** la gerencia utiliza información en contra de los empleados de forma recurrente (por ejemplo, convertir comentarios de retrospectivas en sanciones o llamados de atención).
- **Abuso jerárquico normalizado:** hay patrones evidentes de intimidación, acoso o micro-management destructivo que no tienen consecuencias para quienes los ejercen.
- **Rotación extrema:** más del 30 % del equipo se va cada año por razones vinculadas al clima laboral (y no por oportunidades de crecimiento o factores de mercado).
- **Comunicación totalmente fracturada:** la información crítica se oculta deliberadamente entre niveles jerárquicos, generando desinformación constante.

En contextos así, SocialScrum simplemente no puede operar: la seguridad psicológica —la base misma del modelo [32]— no existe. Edmondson demostró que sin seguridad psicológica, los equipos no reportan problemas reales por temor a represalias, lo que invalida cualquier intento de gestión de deuda social. Intentar implementarlo sería como tratar de apagar un incendio con un vaso de agua: no solo es inútil, sino que puede empeorar la situación.

c. Equipo con autonomía técnica y organizacional. El equipo necesita contar con un nivel mínimo de autonomía para decidir cómo trabaja. SocialScrum depende de esa capacidad de autogestión, porque requiere que el propio equipo pueda:

- Priorizar ítems sociales dentro del Social Sprint Backlog sin tener que pedir autorización externa.
- Ajustar su capacidad técnica según lo que refleje el Health Score.

- Definir cómo abordar los *community smells* (por ejemplo, implementar un daily asíncrono, modificar procesos de comunicación o establecer nuevas normas internas).

Si el equipo opera bajo un esquema de comando y control, donde cada decisión pasa por una aprobación gerencial, SocialScrum simplemente no podrá funcionar. La autoorganización no es un extra ni un ideal aspiracional: es un requisito operativo sin el cual el modelo no tiene espacio para florecer.

d. Disponibilidad de un Social Guide calificado. El rol del Social Guide no puede dejarse al azar. Se necesita una persona con un perfil híbrido poco común: alguien que entienda las dinámicas humanas —psicología organizacional, facilitación, mediación— pero que también conozca cómo funciona el desarrollo de software en la práctica. Si la organización no cuenta con alguien así, o no está dispuesta a formarlo, lo más sensato es esperar antes de adoptar el modelo.

Un error frecuente es asignarle este rol al Scrum Master o al Product Owner. Aunque pueden trabajar de la mano, el Social Guide debe ser una figura independiente para evitar choques de responsabilidades. El Scrum Master se encarga de facilitar el proceso ágil; el Social Guide vela por la salud social del equipo. Son roles que se acompañan, sí, pero no pueden sustituirse entre sí.

4.2.1.2 Cuándo NO implementar SocialScrum

Hay situaciones en las que intentar poner en marcha SocialScrum es más dañino que útil. Los siguientes casos son señales claras de que es mejor detenerse y replantear antes de avanzar:

1. **Equipo en crisis severa:** cuando el Health Score está por debajo de 3/10, el equipo difícilmente tiene la estabilidad emocional para participar en un proceso de este tipo. En estos casos, lo urgente es estabilizar: reducir carga, buscar apoyo de mediación profesional o incluso considerar cambios de personal.
2. **Liderazgo hostil al modelo:** si el CTO o el VP de Ingeniería perciben SocialScrum como “burocracia” o “pérdida de tiempo”, la implementación está destinada al fracaso. Sin respaldo real desde arriba, el Social Guide se convierte en una figura simbólica sin autoridad ni legitimidad.
3. **Equipos temporales o en transición:** SocialScrum requiere cierta continuidad (al menos 3–4 Sprints trabajando juntos). Equipos que se forman y se disuelven con frecuencia no alcanzan a generar las relaciones ni la estabilidad que el modelo necesita para operar.

4. **Ausencia de salvaguardas éticas:** si la organización no garantiza confidencialidad, separación total frente a evaluaciones de desempeño e independencia del Social Guide respecto a HR, entonces la respuesta es clara: **NO implementar**. Sin ética, SocialScrum deja de ser una herramienta de bienestar y se convierte en vigilancia disfrazada.

4.2.1.3 Evaluación inicial: checklist de viabilidad

Antes de avanzar con la implementación, el equipo y el liderazgo deben completar la siguiente evaluación. Si más de dos de las primeras seis preguntas reciben una respuesta negativa, lo más prudente es posponer la adopción hasta que las condiciones mejoren. La Tabla 4.9 presenta este checklist de viabilidad para la adopción de SocialScrum.

Tabla 4.9: Checklist de viabilidad para la adopción de SocialScrum

Criterio	Sí/No	Indicador observable
¿El liderazgo técnico (CTO/VP Eng) respalda de forma clara SocialScrum?	[]	CTO/VP asistió al Sprint 0; aprobó presupuesto por escrito; documentó compromisos de confidencialidad.
¿La organización puede garantizar la confidencialidad de la información social?	[]	Revisión de retrospectivas pasadas: ¿el equipo compartió críticas? ¿hubo represalias? ¿se implementaron cambios?
¿El equipo tiene autonomía real para priorizar ítems sociales en el Sprint?	[]	Decisiones previas: ¿el equipo ajustó carga sin pedir permiso? ¿decidió cómo abordar <i>community smells</i> ?
¿Existe un candidato viable para ejercer como Social Guide?	[]	Revisión de perfil: formación en facilitación o psicología organizacional; conocimiento práctico de desarrollo de software.
¿La cultura organizacional permite conversaciones honestas sin temor a represalias?	[]	Evidencia histórica: críticas expresadas y atendidas sin consecuencias negativas para quienes las plantearon.
¿El equipo cuenta con estabilidad mínima (tres o más Sprints trabajando juntos)?	[]	Historial del equipo: ausencia de cambios frecuentes en personal o estructura durante los últimos Sprints.
¿El Health Score actual es $\geq 4/10$?	[]	Última medición disponible o encuesta anónima rápida realizada antes de la adopción.
¿La rotación anual del equipo es menor al 30 %?	[]	Datos de RRHH: número de salidas en el último año y causas asociadas.

Nota: Los campos vacíos en la columna “Sí/No” deben completarse durante una reunión conjunta entre el Social Guide, el Scrum Master y el liderazgo técnico. Si más de dos respuestas son negativas, se recomienda detener el proceso y trabajar en las áreas débiles antes de intentar la adopción.

4.2.2 Sprint 0: Preparación del equipo

Cuando los pre-requisitos están claros y validados, el equipo dedica un “Sprint 0” —o una iteración exclusivamente preparatoria— a construir las bases operativas y éticas de SocialScrum. Este no es un Sprint orientado a entregar *features*. Su propósito es otro: preparar al equipo para trabajar de una forma distinta, más consciente y más humana.

4.2.2.1 Actividades del Sprint 0

El Sprint 0 dura entre 1 y 2 semanas y debe completar cinco actividades clave.

Actividad 1: Capacitación en fundamentos de SocialScrum (4 horas). El Social Guide (ya seleccionado) y el Scrum Master facilitan una sesión formativa que cubre:

- Qué es la deuda social y por qué es relevante, con evidencia concreta sobre su impacto.
- Qué son los *community smells* y cómo suelen manifestarse en el día a día.
- Cómo funciona SocialScrum: roles, eventos adaptados, artefactos y dinámicas centrales.
- Qué **NO** es SocialScrum: no es evaluación de desempeño, no es vigilancia y no es terapia grupal.
- Salvaguardas éticas esenciales: confidencialidad, separación total frente a evaluaciones y derecho a opt-out.

Esta sesión no debe ser una clase magistral. Es un espacio para conversar abiertamente, donde el equipo pueda hacer preguntas difíciles sin filtro: “¿Qué pasa si digo que estoy en burnout? ¿Esto puede afectar mi estabilidad laboral?”

Las respuestas deben ser claras, documentadas y respaldadas por políticas formales de la organización. Allí es donde se empieza a construir la confianza que SocialScrum necesita para funcionar.

Actividad 2: Establecimiento de la línea base (*baseline*) de salud social (2 horas). En esta etapa, el Social Guide realiza la primera medición del Health Score para definir un punto de partida claro. Esta evaluación incluye tres componentes:

1. Una encuesta anónima sobre los cinco valores de Scrum (Apertura, Compromiso, Respeto, Foco y Coraje), calificados en una escala de 1 a 10.
2. Entrevistas 1:1 opcionales para quienes quieran compartir contexto adicional o experiencias que no se sienten cómodos expresando en grupo.

3. Una identificación preliminar de *community smells* a partir de observación directa: patrones de comunicación en Slack, interacciones en GitHub, dinámicas en Jira, etc.

Protocolo anti-sesgo: La encuesta debe administrarse de forma anónima mediante plataforma online (Google Forms, Typeform, etc.), sin presencia de liderazgo o managers. El link se envía con 24 horas de anticipación para evitar presiones o influencias de grupo. El resultado de esta actividad es un Health Score inicial (por ejemplo, 5.8/10) acompañado de una lista preliminar de smells activos. Este punto de referencia será clave para comparar avances en los próximos Sprints y para entender cómo evoluciona la salud social del equipo a lo largo del tiempo.

Cálculo del Health Score: El score basal es el promedio de las 5 dimensiones alineadas a los valores de Scrum (Apertura, Compromiso, Respeto, Foco y Coraje), cada una medida en escala 1-10. La fórmula exacta y su fundamentación en las necesidades psicológicas y la confianza se detallan en la Sección 4.6.1. Este número es crucial para medir el cambio en Sprints posteriores.

Actividad 3: Firma de acuerdos de consentimiento informado (1 hora). En este punto, cada integrante del equipo firma dos documentos fundamentales:

1. **Acuerdo de Confidencialidad del Social Guide:** aquí, el Social Guide declara formalmente que no compartirá información individual, que solo reportará datos agregados y que su prioridad será proteger la seguridad psicológica del equipo.
2. **Consentimiento Informado del Equipo:** cada persona confirma que entiende qué es SocialScrum, qué tipo de información se va a recopilar, cómo será utilizada y cuáles son sus derechos —incluyendo la opción de retirarse del proceso en cualquier momento.

Si menos del 70 % del equipo firma el consentimiento, SocialScrum no debe implementarse. Este umbral garantiza que exista un apoyo mayoritario real y evita que una minoría presione a una mayoría que no está convencida.

Cabe resaltar que hay un permiso psicológico implícito en la firma del consentimiento: Aquí pueden decir cosas incómodas sin miedo. Si dicen 'estoy quemado', eso NO va a su expediente laboral. Este entendimiento es clave para que el equipo se sienta seguro al compartir vulnerabilidades.

Nota: Las plantillas completas de ambos acuerdos se encuentran en los Anexos C y D de este documento.

Actividad 4: Configuración de herramientas y artefactos (3 horas). En esta actividad, el equipo deja lista toda la infraestructura operativa que permitirá que SocialScrum funcione de manera consistente y transparente. Esto incluye:

- Crear el **Social Product Backlog** en Jira (o una herramienta equivalente así como Slack o Azure DevOps), incorporando campos personalizados como: Social Story Points (SSP), tipo de *Community Smell* e impacto en la salud del equipo (*Health Impact*).
- Configurar un **Dashboard de Health Score** en Confluence o en la herramienta de visualización que use la organización.
- Establecer el **Registro de *community smells***: un documento vivo donde se registre el historial de smells identificados, su evolución y las acciones tomadas para mitigarlos.
- Crear plantillas para *Social User Stories* y para reportes de retrospectivas sociales, de modo que el equipo tenga formatos claros y consistentes desde el primer día.

Esta configuración asegura que SocialScrum no dependa de la memoria del equipo ni de procesos informales. Todo debe quedar documentado, accesible y visible para garantizar transparencia y trazabilidad.

Actividad 5: Primera retrospectiva social simulada (2 horas). Para que el equipo se familiarice con el proceso, se realiza una retrospectiva social “en seco”, es decir, sin la presión de estar en pleno Sprint o de tener que entregar resultados inmediatos. El Social Guide guía las cuatro fases del evento —presentación de métricas, análisis de smells, diseño de experimentos y priorización de acciones— utilizando el *baseline* inicial como referencia. El propósito de esta sesión no es empezar a resolver problemas reales, sino practicar el protocolo: aprender a hablar de temas sociales sin culpa ni señalamientos, ensayar cómo proponer acciones de mejora y entender cómo comprometerse de manera respetuosa y segura para todos.

4.2.2.2 Criterios de éxito del Sprint 0

El Sprint 0 se considera exitoso cuando se cumplen los siguientes puntos:

- Al menos el 70 % del equipo firmó el consentimiento informado.
- Se logró establecer un Health Score *baseline* con la participación de todos los miembros.
- Los artefactos operativos quedaron configurados, documentados y disponibles para el equipo.
- El equipo tiene claridad sobre qué es SocialScrum y, sobre todo, qué **no** es.

- Se identificó al menos un *community smell* activo para trabajar durante el primer Sprint real.

Si alguno de estos criterios no se cumple, el equipo debe considerar extender el Sprint 0 para cerrar los vacíos pendientes. En situaciones más complejas, puede ser necesario reevaluar si este es realmente el momento adecuado para implementar SocialScrum.

4.2.2.3 Qué hacer si el equipo rechaza SocialScrum

Si menos del 70 % del equipo firma el consentimiento, el proceso se detiene. Pero eso no es el final: es el inicio de una investigación acerca de las causas o dudas.

1. Validación en privado Conversaciones 1:1 confidenciales (no con liderazgo) para confirmar si el rechazo es genuino o hay presión externa.

2. Investigación de causas ¿Es por:

- Falta de comprensión? → Repetir sesión de capacitación
- Desconfianza en el Social Guide? → Explorar otro candidato
- Miedo a la sobrecarga? → Reducir sobrecarga (ej: Social Daily asincrónica)
- Resistencia cultural? → Posponer 3-6 meses

3. Comunicación a liderazgo “SocialScrum requiere autonomía genuina. Forzar la adopción violaría Autonomía (TAD 3.3.1.1) y garantizaría fracaso”.

4. Sigüientes pasos Documentar razones. Crear plan concreto para mejorar condiciones. Reintentar en momento más favorable, si es viable.

4.2.3 Presentación al equipo y *stakeholders*

Cuando el Sprint 0 llega a su cierre, el Social Guide y el Scrum Master realizan una presentación formal dirigida a los *stakeholders* clave: el Product Owner, otros equipos con los que se interactúa, la gerencia técnica y, cuando sea relevante, Recursos Humanos (únicamente para informar, no para delegar autoridad ni acceso a información).

4.2.3.1 Objetivos de la presentación

Esta presentación cumple tres propósitos centrales:

1. **Transparencia:** explicar de manera clara qué es SocialScrum, por qué el equipo decidió adoptarlo y cómo se integrará en el trabajo del día a día.

2. **Gestión de expectativas:** dejar explícito que SocialScrum puede generar una reducción temporal en la velocidad técnica (aproximadamente entre el 10 y el 20 %), especialmente mientras se atienden *community smells* críticos. Esta disminución no es un fallo del proceso, sino una inversión para mejorar el rendimiento y la salud del equipo en el mediano y largo plazo.
3. **Compromiso mutuo:** solicitar y asegurar el respaldo del liderazgo, tanto en recursos (tiempo, presupuesto, capacitación) como en decisiones operativas (ajustes de carga, priorización de acciones sociales, intervención cuando sea necesaria).

4.2.3.2 Gestión de objeciones comunes

Durante la presentación pueden surgir resistencias. Una de las tres objeciones más frecuentes y sus respuestas: (1) “¿Esto no va a quitar tiempo de desarrollo?” — Sí, en el corto plazo. Pero la deuda social ya consume tiempo en re-trabajo, conflictos y rotación. SocialScrum vuelve visible ese costo y lo reduce estructuradamente. (2) “¿No es esto responsabilidad de HR?” — No. HR se enfoca en políticas laborales y contratación; SocialScrum se ocupa de dinámicas operativas: colaboración, comunicación y carga socio-técnica diaria. (3) “¿Qué pasa si el equipo no quiere participar?” — No se implementa. El umbral mínimo es 70 % de aceptación. Si menos del 70 % no está de acuerdo, hay un protocolo de investigación documentado en la sección 4.2.2.3.

4.3 El rol del Social Guide: Perfil, competencias y gobernanza

El Social Guide es la figura central de SocialScrum. Sin este rol, el modelo se reduce a Scrum con buenas intenciones pero sin mecanismos concretos para gestionar la deuda social. El Social Guide facilita seguridad psicológica mediante tres mecanismos:

1. Motivación intrínseca: Crea condiciones donde el equipo quiere colaborar (no se siente obligado) respetando autonomía, competencia y relacionalidad (TAD) [33].
2. Confianza interpersonal: Demuestra habilidad (sabe mediación), benevolencia (busca bien del equipo) e integridad (honesto, confidencial) según modelo Mayer et al. [31].
3. Seguridad psicológica: Genera espacios donde es seguro hablar de conflictos, vulnerabilidades y errores sin temor a represalias [34].

4.3.1 Perfil y competencias requeridas

El Social Guide requiere un perfil híbrido: comprensión de dinámicas humanas con conocimiento técnico de desarrollo de software, en la Tabla 4.10 se detalla su perfil ideal.

Tabla 4.10: Perfil del rol Social Guide

Área	Requisito mínimo	Cómo evaluar	Nivel aceptable
Formación base	Pregrado en Psicología, Trabajo Social, Comunicación, o Ingeniería con especialización en gestión de equipos	Revisión de CV, transcripción académica y entrevista	Título oficial y relevancia demostrada para el rol
Formación complementaria	Certificación en facilitación, mediación (CNV, transformativa) o coaching (ICF)	Entrevista teórica sobre técnicas	Al menos una certificación; puede completarse en los primeros seis meses
Experiencia mínima	Dos o más años trabajando con equipos de desarrollo (devs, SM, PO, People Ops)	Referencias laborales y entrevista de contexto	Mínimo un año si cuenta con certificaciones avanzadas
Habilidades duras (hard skills)			
Análisis SNA (Social Network Analysis)	Conocimiento en teoría de grafos aplicada a organizaciones y capacidad de interpretar métricas de centralidad e intermediación en redes de comunicación.	Caso práctico: análisis de un sociograma	Produce al menos tres insignias válidas a partir de un sociograma pequeño en 30 minutos
Medición cualitativa	Dominio de técnicas de codificación de datos, síntesis de entrevistas y análisis de contenido para transformar percepciones subjetivas en indicadores.	Entrevista semi-estructurada observada	Transcribe y analiza usando dos o tres códigos principales coherentes
Facilitación de grupos	Conocimiento sólido de dinámicas de grupo y metodologías de diseño de sesiones colaborativas (como Design Thinking o Liberating Structures).	Facilitación de retrospectiva simulada	El equipo reporta sensación de seguridad psicológica
Mediación de conflictos	Formación o dominio de técnicas de resolución de conflictos, específicamente el modelo de Comunicación No Violenta (CNV).	Role-play con observación del mediador	Mantiene neutralidad y genera opciones de resolución
Dominio de Scrum y enfoques ágiles	Comprensión profunda de la Guía de Scrum y experiencia práctica en el ciclo de vida de desarrollo de software (SDLC) para entender el contexto técnico de los desarrolladores	Explicación de cómo un <i>community smell</i> afecta un Sprint	Propone una intervención ágil concreta y coherente
Habilidades blandas (soft skills)			

Continúa en la siguiente página...

Tabla 4.10 – Continuación

Área	Requisito mínimo	Cómo evaluar	Nivel aceptable
Neutralidad emocional	Capacidad de mantener una postura imparcial y no jerárquica, separando los juicios de valor personales de los hechos observados en el equipo.	Observación durante una retrospectiva	El equipo valida que no toma partido
Empatía calibrada	Habilidad para conectar con las emociones y necesidades del equipo sin perder la objetividad ni validar conductas destructivas.	Role-play de escucha en situación de conflicto	Conecta emocionalmente sin validar conductas destructivas
Coraje profesional	Disposición para enfrentar conversaciones difíciles, señalar <i>community smells</i> invisibles y confrontar al liderazgo si se vulnera la salud del equipo.	Caso ético: “¿Qué harías si el equipo quiere expulsar a alguien?”	Propone soluciones sin atacar a la persona
Integridad ética	Apego inquebrantable a los principios de confidencialidad absoluta y anonimización de datos sensibles	Entrevista sobre confidencialidad	Compromiso explícito con la privacidad
Humildad intelectual	Capacidad de reconocer los límites de su competencia y proponer la derivación a especialistas (psicólogos clínicos o recursos humanos) cuando el problema exceda el alcance de SocialScrum.	Pregunta abierta sobre los límites del rol	Reconoce límites y propone derivación cuando corresponde

4.3.2 Proceso de selección y capacitación

En la Tabla 4.11 se detalla el proceso de selección del Social Guide, que consta de cinco fases clave para garantizar que el candidato seleccionado cumple con los requisitos técnicos, éticos y culturales necesarios para desempeñar este rol crítico.

Tabla 4.11: Proceso de selección del Social Guide

Fase	Nombre	Duración	Actividades
1	Identificación de candidatos	1 semana	Búsqueda de candidatos internos o externos que cumplan con el perfil definido.
2	Evaluación técnica	1 semana	Caso práctico sobre <i>community smells</i> , facilitación simulada y entrevista teórica.
3	Entrevista ética	1 semana	Entrevistas con Scrum Master, Development Team y CTO; análisis de dilemas éticos críticos.
4	Decisión consensuada	–	Requiere consenso entre Scrum Master, Development Team (mayoría simple) y CTO (ver Tabla 4.12).
5	Período de prueba	3 Sprints	Ejercicio del rol bajo supervisión; se requiere una votación de aprobación $\geq 70\%$ para la confirmación definitiva.

En la Tabla 4.12 se describen las preguntas clave y los criterios de votación que cada autoridad debe considerar durante la fase de decisión consensuada (Fase 4) del proceso de selección del Social Guide.

Tabla 4.12: Proceso de votación para la selección del Social Guide

Votante	Autoridad	Pregunta	Voto si
Scrum Master	1 voto	¿Puede facilitar procesos y proteger el marco de trabajo?	Sí / No
Development Team (colectivo)	1 voto (mayoría $\geq 50\%$)	¿Lo aceptamos como un par confiable para el equipo?	Mayoría $\geq 50\%$
CTO / VP Engineering	1 voto + veto por causa legal documentada	¿Es viable a nivel organizacional y legal?	Sí / No (o veto con justificación formal)

Resultado final: En la Tabla 4.13 se presentan las reglas de decisión basadas en los votos emitidos por cada autoridad involucrada en el proceso de selección del Social Guide.

Tabla 4.13: Reglas de decisión del proceso de votación para la selección del Social Guide

Resultado	Sí	No / Veto	Decisión
3 Sí	3	0	Aprobado para Fase 5.
2 Sí + 1 NO	2	1	Aprobado si el NO no corresponde a veto del CTO.
2 Sí + 1 VETO CTO	2	1 (Veto)	Rechazado (causa documentada).
≤ 1 Sí	0-1	≥ 2	Rechazado.

4.4 Eventos adaptados de SocialScrum

Los eventos adaptados de SocialScrum mantienen la estructura básica de Scrum, pero incorporan componentes sociales clave para abordar la deuda social. La Figura 4.9 ilustra cómo cada evento tradicional se enriquece con prácticas específicas para monitorear y mejorar la salud social del equipo.

4.4.1 Social Daily Standup

El Daily Standup tradicional responde tres preguntas: ¿qué hice ayer?, ¿qué haré hoy?, ¿qué impedimentos tengo? Estas preguntas capturan el estado del trabajo técnico, pero ignoran el estado emocional y relacional del equipo. SocialScrum añade una cuarta dimensión: una pregunta social breve (2-3 minutos adicionales) que permite detectar señales tempranas de malestar.



Figura 4.9: Ciclo de eventos adaptados de SocialScrum

4.4.1.1 Protocolo del Social Daily Standup

En la Tabla 4.14 se detalla la estructura del Social Daily Standup, que consta de dos fases principales: técnica y social.

Tabla 4.14: Estructura del Social Daily Standup

Fase	Duración	Facilitador	Contenido
Técnica	12–13 min	Scrum Master	Tres preguntas tradicionales del Daily Standup.
Social	2–3 min	Social Guide	Pregunta social del día orientada a detectar señales tempranas.

La respuesta es siempre opcional, el Social Guide no profundiza en el Daily; si detecta un problema significativo, ofrece conversación privada posterior. A continuación se presentan las reglas de facilitación, variantes para equipos distribuidos y el resultado esperado del Social Daily Standup.

Reglas de facilitación

1. **No profundizar:** El Daily detecta problemas; no los resuelve.
2. **No juzgar:** Si alguien dice “todo bien” cuando no lo está, no confrontar públicamente.
3. **Rotar preguntas:** Variar para evitar respuestas automáticas.

4. **Registrar patrones:** Si múltiples personas mencionan temas similares, hay un patrón que requiere atención.

Variantes para equipos distribuidos. En equipos remotos: (1) registro de entrada (*check-in*) con emoji que representa estado; (2) pregunta semanal rotativa en lugar de diaria; (3) canal dedicado *#team-health* separado de reportes técnicos.

Resultado del Social Daily Standup. Señales tempranas de malestar, datos cualitativos para el Health Score, y agenda de seguimiento con conversaciones privadas a realizar.

4.4.2 Social Sprint Planning

El Sprint Planning tradicional responde: ¿ qué puede entregarse? y ¿ cómo se logrará? Estas preguntas asumen condiciones sociales estables, lo cual frecuentemente no es cierto. SocialScrum añade una tercera dimensión: evaluación de riesgos sociales (15 minutos adicionales) después del Planning técnico.

4.4.2.1 Protocolo del Social Sprint Planning

En la Tabla 4.15 se detalla la estructura del Social Sprint Planning, que consta de tres fases principales: técnica, social y ajuste del Sprint Backlog.

Tabla 4.15: Estructura del Social Sprint Planning

Parte	Duración	Contenido
1. Planning técnico	Estándar	El Product Owner presenta los ítems priorizados y el equipo selecciona el Sprint Backlog y define el Sprint Goal.
2. Evaluación de riesgos sociales	15 min	Respuesta a cuatro preguntas orientadas a identificar riesgos sociales relevantes para el Sprint.
3. Ajuste del Sprint Backlog	5 min	Ajuste de alcance, inclusión de ítems sociales y agendamiento de intervenciones necesarias.

4.4.2.2 Double Sprint Goal (opcional)

Opcionalmente, el equipo puede definir un Double Sprint Goal: uno técnico (“Completar módulo de checkout”) y uno social (“Reducir *Radio Silence* implementando *code reviews* rotativos”). El Social Goal solo se define cuando hay un *smell* de alta severidad; no todos los Sprints lo requieren.

4.4.3 Social Sprint Review

El Sprint Review tradicional inspecciona el Incremento técnico, pero ignora cómo se construyó desde una perspectiva social. SocialScrum añade una evaluación del Health Score (10 minutos adicionales) después de la demostración técnica.

4.4.3.1 Protocolo del Social Sprint Review

En la Tabla 4.16 se detalla la estructura del Social Sprint Review, que consta de tres partes principales: revisión técnica, evaluación social y preguntas de *stakeholders*.

Tabla 4.16: Estructura del Social Sprint Review

Parte	Duración	Contenido
1. Review técnica	Estándar	Demostración del Incremento y recopilación de feedback de <i>stakeholders</i> .
2. Evaluación de la salud social	10 min	Revisión del Health Score, <i>community smells</i> abordados, riesgos materializados y tendencias observadas.
3. Preguntas de <i>stakeholders</i>	Variable	Respuestas limitadas a información agregada y anónima, sin referencias individuales.

4.4.3.2 Visualización del Health Score

La Figura 4.10 muestra un ejemplo de cómo se presenta el Health Score durante el Social Sprint Review. Se utilizan gráficos de barras y líneas para ilustrar la evolución del puntaje a lo largo del tiempo, destacando áreas de mejora y éxitos alcanzados.

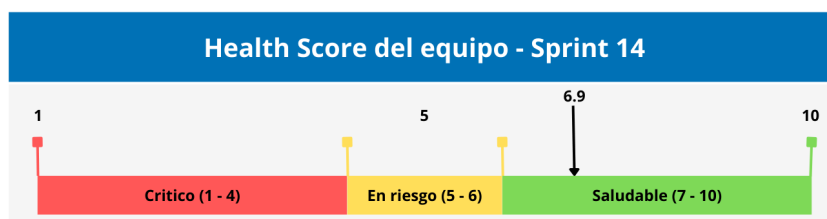


Figura 4.10: Ejemplo de visualización del Health Score en el Social Sprint Review

4.4.3.3 Confidencialidad en el Social Sprint Review

El reporte siempre es agregado, anónimo y enfocado en patrones. Nunca se presenta: nombres de personas involucradas en conflictos, detalles de conversaciones privadas, resultados individuales de encuestas, ni especulaciones sobre causas personales. Si un stakeholder solicita información individual, el Social Guide rechaza citando las salvaguardas éticas de SocialScrum.

4.4.4 Social Sprint Retrospective

La Social Sprint Retrospective es el evento central de SocialScrum, mientras los otros eventos añaden extensiones breves, la Retrospective se transforma sustancialmente para abordar la deuda social con rigurosidad, generando acciones concretas que se integran al Social Sprint Backlog siguiente.

4.4.4.1 Estructura de la Social Sprint Retrospective

Duración: 90 minutos para Sprints de 2 semanas (ajustable proporcionalmente), en la Tabla 4.17 se detalla su estructura en cuatro fases principales.

Tabla 4.17: Estructura de la Social Sprint Retrospective (90 minutos)

Fase	Nombre	Tiempo	Actividades principales
1	Preparación del espacio	10 min	Registro emocional inicial, recordatorio de acuerdos (confidencialidad, ausencia de culpa, buena fe) y verificación de seguridad psicológica.
2	Recolección de datos	25 min	Uso de cuatro cuadrantes sociales o línea de tiempo emocional; notas específicas basadas en eventos observables del Sprint.
3	Análisis de patrones	30 min	Priorización mediante votación por puntos, aplicación de la técnica de los cinco porqués para identificar causas raíz y mapeo a <i>community smells</i> .
4	Generación de compromisos	25 min	Definición de acciones SMART, estimación en SSP y priorización de un máximo de una a tres acciones para el siguiente Sprint.

4.4.4.2 Flujo de la Social Sprint Retrospective

La Figura 4.11 ilustra el flujo de las cuatro fases de la Social Sprint Retrospective, destacando la transición entre recolección de datos, análisis y generación de compromisos concretos.

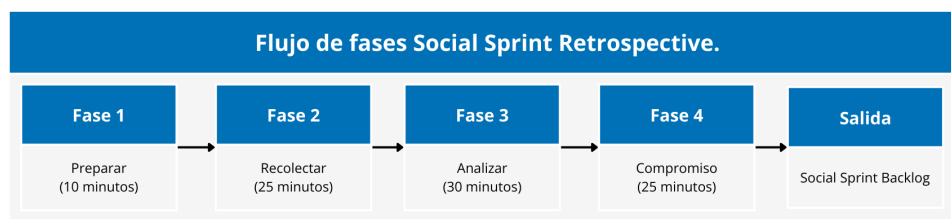


Figura 4.11: Flujo de las cuatro fases de la Social Sprint Retrospective

4.5 Artefactos sociales y su gestión

Los artefactos de Scrum —Product Backlog, Sprint Backlog e Incremento— proporcionan transparencia sobre el trabajo técnico: qué se construyó, qué falta por construir y qué se entregará al final del Sprint. Esta transparencia permite la inspección y adaptación continua que define a Scrum como marco empírico. Sin embargo, estos artefactos ignoran completamente la dimensión social del trabajo. Un equipo puede tener un Product Backlog impecablemente priorizado mientras acumula *community smells* [2] —patrones negativos de colaboración que constituyen deuda social invisible— que eventualmente saboteará su capacidad de entrega. Palomba et al. [17] demuestran correlación directa entre *community smells* e intensidad de defectos de código, validando que estos problemas sociales tienen impacto técnico mensurable.

SocialScrum introduce cuatro artefactos complementarios que hacen visible la salud social del equipo con el mismo rigor que Scrum aplica al trabajo técnico. Estos artefactos no reemplazan a los existentes; los extienden, proporcionando la información necesaria para gestionar la deuda social de forma sistemática, en la Tabla 4.18 se describen estos artefactos.

Tabla 4.18: Artefactos sociales de SocialScrum

Artefacto	Propósito	Se actualiza en	Responsable
Social Product Backlog.	Inventario priorizado de items de deuda social.	Social Sprint Planning, Social Retrospective.	Social Guide + PO.
Social Sprint Backlog.	Compromisos sociales del Sprint actual.	Social Sprint Planning.	Development Team.
Registro de <i>Community Smells</i> .	Historial de smells detectados y su evolución.	Continuo (Daily, Retro).	Social Guide.
Dashboard de Métricas Sociales.	Visualización del Health Score y tendencias.	Semanal.	Social Guide.

Cada artefacto cumple una función específica dentro del ciclo de gestión de deuda social, y su diseño respeta el Principio de No-Sobrecarga: la información se captura como sub-producto natural de los eventos existentes, no mediante trabajo administrativo adicional.

4.5.1 Social Product Backlog

El Social Product Backlog es el inventario ordenado de todos los items de deuda social que el equipo ha identificado y decidido gestionar. Funciona como el equivalente social del Product Backlog técnico: contiene todo lo que el equipo podría hacer para mejorar su salud social, priorizado según criterios explícitos.

4.5.1.1 Naturaleza y propósito

A diferencia del Product Backlog técnico, que contiene funcionalidades que entregan valor al usuario final, el Social Product Backlog contiene intervenciones que entregan valor al equipo mismo. Estas intervenciones abordan *community smells* activos, previenen *smells* emergentes, o fortalecen capacidades relacionales del grupo.

El propósito del Social Product Backlog es triple:

1. **Hacer visible la deuda social:** Sin un inventario explícito, la deuda social permanece en conversaciones informales, quejas de pasillo o frustración silenciosa. El Social Product Backlog la convierte en items concretos que pueden discutirse, priorizarse y abordarse.
2. **Forzar priorización explícita:** Cuando la deuda social compite formalmente por capacidad con el trabajo técnico, el equipo debe decidir conscientemente qué problemas abordar primero. Esta decisión, aunque incómoda, es preferible a la alternativa: ignorar los problemas hasta que exploten.
3. **Proporcionar trazabilidad:** El Social Product Backlog registra qué se identificó, cuándo, por qué se priorizó (o no), y qué ocurrió después. Esta trazabilidad permite aprender de intervenciones pasadas y demostrar progreso a *stakeholders*.

4.5.1.2 Estructura de un item del Social Product Backlog

Cada item del Social Product Backlog sigue una estructura estandarizada con los siguientes campos obligatorios:

- **Identificador y metadatos:** ID único, fecha de creación, origen (evento donde se identificó).
- **Título y descripción:** Descripción clara del problema o intervención propuesta.
- **Criterios de aceptación:** Condiciones verificables que definen cuándo el item está completado.
- **Clasificación:** *Community smell* asociado, severidad (Alta/Media/Baja), dimensión del Health Score afectada.
- **Planificación:** Estimación en SSP (ver Anexo I), prioridad, dependencias.
- **Historial:** Registro de cambios de estado y decisiones.

4.5.1.3 Tipos de items y priorización

El Social Product Backlog puede contener tres tipos de items: (1) **mitigación de smells** —intervenciones para reducir o eliminar *community smells* activos—; (2) **fortalecimiento preventivo** —acciones que previenen problemas futuros cuando el Health Score es alto—; y (3) **investigación** —actividades de diagnóstico cuando existe sospecha de un problema pero no hay claridad sobre su causa raíz.

La priorización es responsabilidad compartida entre el Social Guide y el Product Owner, considerando cuatro factores: severidad del smell (Alta > Media > Baja), impacto en dimensiones del Health Score actualmente bajas, esfuerzo requerido (preferir *quick wins*), y ventanas de oportunidad, como se puede observar en la Tabla 4.19.

Tabla 4.19: Matriz de priorización del Social Product Backlog

Severidad	Esfuerzo bajo (1-3 SSP)	Esfuerzo medio (4-8 SSP)	Esfuerzo alto (9+ SSP)
Severidad alta	Hacer inmediatamente	Hacer este Sprint	Planificar para próximo Sprint
Severidad media	Hacer este Sprint	Evaluar capacidad	Posponer si hay severidad alta pendiente
Severidad baja	Incluir si hay capacidad	Mantener en backlog	Re-evaluar necesidad

4.5.1.4 Gestión continua del Social Product Backlog

El Social Guide refina el backlog semanalmente, eliminando items obsoletos, actualizando estimaciones, y re-priorizando según cambios en el Health Score. Como regla general, SocialScrum recomienda reservar entre el 10 % y el 20 % de la capacidad del Sprint para items sociales cuando el Health Score está por debajo de 7; cuando es 7 o superior, esta reserva puede reducirse al 5-10 %.

4.5.1.5 Relación con el Product Backlog técnico

El Social Product Backlog y el Product Backlog técnico son artefactos separados pero interrelacionados. Durante el Sprint Planning, ambos backlogs compiten por la capacidad del equipo:

- El Product Owner presenta items técnicos prioritarios.
- El Social Guide presenta items sociales prioritarios.
- El Development Team decide cuánta capacidad asignar a cada tipo, considerando el Health Score actual y los compromisos de negocio.

Como regla general, SocialScrum recomienda reservar entre el 10 % y el 20 % de la capacidad del Sprint para items sociales cuando el Health Score está por debajo de 7. Cuando

el Health Score es 7 o superior, esta reserva puede reducirse al 5-10 % para mantenimiento preventivo.

4.5.2 Social Sprint Backlog

El Social Sprint Backlog contiene los items del Social Product Backlog que el equipo se ha comprometido a completar durante el Sprint actual, junto con el plan para lograrlos. Es el equivalente social del Sprint Backlog técnico: un compromiso visible y accionable.

4.5.2.1 Naturaleza y propósito

El Social Sprint Backlog hace explícito qué trabajo social ocurrirá durante el Sprint. Sin este artefacto, los items sociales quedan como “buenas intenciones” que se posponen indefinidamente ante la presión del trabajo técnico.

El propósito del Social Sprint Backlog es:

1. **Convertir intenciones en compromisos:** Un item en el Social Product Backlog es algo que “podríamos hacer”. Un item en el Social Sprint Backlog es algo que “haremos este Sprint”.
2. **Hacer visible el trabajo social:** Cuando los items sociales aparecen en el mismo tablero que los items técnicos, el equipo y los *stakeholders* ven que la salud social recibe atención real, no solo discursos.
3. **Permitir seguimiento de progreso:** Durante el Sprint, el equipo puede ver qué items sociales están “en progreso” y cuáles están “completados”, igual que con los items técnicos.

4.5.2.2 Contenido del Social Sprint Backlog

El Social Sprint Backlog contiene:

- **Items sociales seleccionados:** Típicamente 1-3 items del Social Product Backlog, dependiendo de la capacidad y el Health Score.
- **Tareas asociadas:** Cada item se descompone en tareas concretas que pueden completarse y verificarse.
- **Responsables:** Aunque el trabajo social es responsabilidad del equipo completo, cada tarea tiene un responsable que asegura su ejecución.
- **Criterios de “Done” social:** Condiciones que deben cumplirse para considerar el item completado.

Ejemplo de item en Social Sprint Backlog:

Protocolo operativo

Ítem del Social Sprint Backlog

Título

Reducir *Radio Silence* en la comunicación cross-funcional

Estimación

5 SSP (Social Story Points)

Criterio de Done

- Los mensajes cross-equipo aumentan de aproximadamente 2 a 5 o más por día.
- Al menos un riesgo es reportado de forma proactiva durante el Sprint.

Tareas

- Añadir pregunta de riesgos al Daily (Responsable: Social Guide).
- Crear canal #risks en Slack o herramienta equivalente (Responsable: Scrum Master).
- Facilitar primera sesión de sincronización Frontend y Backend (Responsable: Social Guide).
- Medir comunicación cross-funcional al final del Sprint (Responsable: Social Guide).

Estado actual

En progreso (2 de 4 tareas completadas)

4.5.2.3 Integración con el Sprint Backlog técnico

Existen dos modelos para integrar el Social Sprint Backlog con el técnico:

Modelo integrado (recomendado) Los items sociales aparecen en el mismo tablero que los items técnicos, diferenciados por etiquetas o colores. Esto maximiza visibilidad y normaliza el trabajo social como “trabajo real”.

- **Ventaja:** El equipo ve todo su trabajo en un solo lugar. Los items sociales no se “escondan” en un backlog separado.
- **Implementación:** Por ejemplo, en Jira, usar etiqueta “social” o tipo de incidencias (*issue*) “Social Item”. En tablero físico, usar notas adhesivas (post-its) de color diferente.

Modelo separado Los items sociales se gestionan en un tablero aparte, visible solo para el equipo y el Social Guide. Los *stakeholders* ven únicamente el tablero técnico.

- **Ventaja:** Mayor privacidad para temas sensibles. Útil en organizaciones donde hay riesgo de que *stakeholders* malinterpreten el trabajo social.
- **Desventaja:** Reduce visibilidad y puede perpetuar la percepción de que el trabajo social es “secundario”.

SocialScrum recomienda el modelo integrado salvo que existan razones específicas para mantener separación.

4.5.2.4 Actualización durante el Sprint

El Social Sprint Backlog se actualiza durante el Sprint de la misma forma que el técnico:

- **Social Daily Standup:** Si hay tareas sociales en progreso, se reporta avance brevemente. El Social Guide puede mencionar: “La sesión de sincronización FE-BE (Frontend-Backend) está programada para mañana a las 3pm”.
- **Durante el Sprint:** Cuando se completan tareas, se marcan como “Done”. Si surgen impedimentos, se discuten.
- **Social Sprint Review:** Se reporta qué items sociales se completaron y cuál fue su impacto.

4.5.2.5 Qué hacer si no se completan items sociales

Si al final del Sprint hay items sociales incompletos, se aplican las mismas reglas que para items técnicos:

1. **No se consideran “Done”:** Items parcialmente completados vuelven al Social Product Backlog para re-priorización.
2. **Se analiza la causa:** ¿Se subestimó el esfuerzo? ¿Hubo impedimentos inesperados? ¿El equipo priorizó trabajo técnico sobre social?
3. **Se ajusta para futuros Sprints:** Si consistentemente los items sociales no se completan, puede indicar que el equipo no está comprometido con SocialScrum, o que la estimación en SSP es incorrecta.

Los artefactos sociales de SocialScrum proporcionan la infraestructura de información necesaria para gestionar la deuda social con rigor. Sin ellos, la gestión de salud social queda en el ámbito de las buenas intenciones y las conversaciones informales. Con ellos, se convierte en un proceso sistemático, trazable y demostrable. El Social Product Backlog hace

visible qué se puede mejorar; el Social Sprint Backlog convierte intenciones en compromisos; el Registro de *community smells* proporciona memoria histórica; y el Dashboard de Métricas Sociales permite inspección continua. Juntos, estos artefactos cierran el ciclo de transparencia-inspección-adaptación que define a Scrum, extendiéndolo a la dimensión social del trabajo en equipo.

Los artefactos capturan información, pero ¿cómo se genera esa información de manera confiable? El Health Score que aparece en el Dashboard, los *community smells* que se registran, los indicadores de tendencia —todos requieren un sistema de medición que transforme observaciones cualitativas en datos utilizables. La siguiente sección describe este sistema: cómo se operacionaliza el Health Score, qué instrumentos se utilizan para medirlo, y cómo se interpretan los resultados para tomar decisiones concretas.

4.6 Sistema de medición de salud social

SocialScrum mide la salud social mediante el Health Score, un indicador agregado en escala 1-10 basado en los cinco valores de Scrum. El diseño del Health Score está fundamentado en la Teoría de la Autodeterminación de Ryan y Deci [33], que postula que los individuos requieren satisfacer tres necesidades psicológicas —autonomía, competencia y relacionalidad— para mantener motivación intrínseca. Adicionalmente, incorpora el modelo de confianza organizacional de Mayer et al. [31], que especifica tres componentes: habilidad, benevolencia e integridad. El sistema combina encuestas estructuradas, observación participante y análisis de redes sociales (SNA) para triangular datos y reducir sesgos individuales.

4.6.1 Health Score: Dimensiones y cálculo

El Health Score evalúa cinco dimensiones clave de la salud social, cada una alineada con un valor de Scrum y respaldada por teorías psicológicas y organizacionales como se explica en la tabla 4.20.

Tabla 4.20: Dimensiones del Health Score

Dimensión	Indicadores positivos	Smells relacionados
Apertura	Riesgos reportados temprano; errores discutidos abiertamente; desacuerdo constructivo	Radio Silence (Silencio Radiofónico), Information Hoarding (Acaparamiento de Información)
Compromiso	Acuerdos cumplidos; ayuda a compañeros bloqueados; Sprint Goal tratado colectivamente	Lone Wolf (Lobo Solitario), Free-Riding (Polizones), Disengagement (Desenganche)

Continúa en la siguiente página...

Tabla 4.20 – Continuación

Dimensión	Indicadores positivos	Smells relacionados
Respeto	Ideas evaluadas por mérito; reconocimiento explícito; desacuerdos sin ataques personales	Prima Donnas (Primas Donnas), Toxic Leadership (Liderazgo Tóxico), Cliques (Cliques)
Foco	Una tarea a la vez; reuniones con propósito claro; protección contra demandas externas	Context Switching (Cambio de Contexto), Meeting Hell (Infierno de Reuniones)
Coraje	Ideas no convencionales propuestas; errores reportados voluntariamente; feedback a líderes	Fear of Speaking Up (Miedo a Expresarse), Blame Culture (Cultura de la Culpa)

El Health Score se calcula promediando las puntuaciones de las cinco dimensiones, cada una medida en una escala de 1 a 10. La fórmula es:

$$\text{Health Score} = \frac{\text{Apertura} + \text{Compromiso} + \text{Respeto} + \text{Foco} + \text{Coraje}}{5} \quad (4.1)$$

4.6.1.1 Frecuencia y formato de medición

El Health Score se mide mediante dos instrumentos complementarios como se detalla en la tabla 4.21:

Tabla 4.21: Instrumentos de medición del Health Score

Tipo	Frecuencia	Contenido	Duración	Tasa mínima
Pulso semanal	Cada viernes	5 preguntas (1 por dimensión), escala 1-10	2-3 min	70 %
Sprint completa	Fin de Sprint	20 preguntas + 3 abiertas	5-8 min	80 %

4.6.2 Zonas del Health Score y protocolo de respuesta

El Health Score se interpreta en tres zonas que indican el estado de salud social del equipo y guían las acciones correctivas necesarias como se observa en la tabla 4.22.

Tabla 4.22: Zonas del Health Score y protocolo de respuesta

Rango	Zona	Significado	Capacidad social	Acciones
7.0–10.0	Saludable	Funciona bien, problemas menores	5–10 %	Mantenimiento preventivo, revisión mensual.
5.0–6.9	En riesgo	Fricción que puede escalar	15–20 %	Atención activa, revisión semanal, intervenciones específicas.
1.0–4.9	Crítico	Crisis seria, productividad afectada	30–50 %	Intervención inmediata, escalar a CTO, reducir alcance técnico.

4.6.2.1 Alertas tempranas

El sistema genera alertas automáticas cuando se detectan patrones preocupantes en el Health Score:

- **Caída abrupta:** Health Score baja >1.5 puntos en una semana $\rightarrow$ investigar inmediatamente.
- **Tendencia negativa:** 3 semanas consecutivas de caída $\rightarrow$ activar protocolo En riesgo.
- **Dimensión crítica:** Cualquier dimensión <4.0 $\rightarrow$ priorizar esa dimensión.
- **Baja participación:** $<50\%$ responde encuestas por 2 semanas $\rightarrow$ revisar proceso.

4.6.3 Triangulación de fuentes de datos

El Health Score se complementa con observación participante (durante eventos Scrum) y análisis de redes sociales (SNA, usando metadatos de Slack/GitHub). La triangulación aumenta confiabilidad: si las tres fuentes convergen en el mismo diagnóstico, la confianza es alta; si divergen, se requiere investigación adicional, en la Tabla 4.23 se muestra la complementariedad de los instrumentos.

Tabla 4.23: Complementariedad de instrumentos de medición

Aspecto	Encuestas	Observación	SNA
Percepciones subjetivas	+++	++	-
Comportamientos observables	-	+++	+
Patrones estructurales	-	+	+++

Los símbolos indican el grado relativo de capacidad del instrumento para captar cada aspecto (— nulo, + bajo, ++ medio, +++ alto).

Limitaciones del Health Score: Es una aproximación, no una verdad objetiva. Depende de la honestidad de respuestas, puede inducir conductas de optimización estratégica de la métrica (gaming), especialmente si se usa para evaluación externa, y es sensible al momento. Siempre debe complementarse con información cualitativa.

4.7 Framework de priorización social-técnica

En Scrum tradicional, el Product Owner prioriza el Product Backlog según un criterio principal: el valor de negocio. Los items que generan mayor retorno para el cliente o la organización se ubican arriba; los demás esperan. Este enfoque funciona cuando el

único recurso limitado es el tiempo de desarrollo. Sin embargo, SocialScrum introduce una variable adicional: la salud social del equipo. Un ítem de alto valor técnico puede ser contraproducente si el equipo que debe implementarlo está en crisis. De modo inverso, invertir en restaurar la salud social puede parecer “improductivo” en el corto plazo pero habilitar entregas sostenibles en el largo plazo.

Esta sección presenta un framework de priorización que integra tres dimensiones: valor de negocio, urgencia técnica y salud social. El framework no proporciona una fórmula matemática que calcule automáticamente la prioridad de cada ítem. En su lugar, estructura una conversación entre Product Owner, Scrum Master, Social Guide y Development Team, asegurando que las tres dimensiones se consideren explícitamente antes de comprometer trabajo en un Sprint.

4.7.1 Triángulo de decisión: valor de negocio, urgencia técnica, salud social

El modelo conceptual del framework es un triángulo donde cada vértice representa una dimensión de priorización, en la Figura 4.12 se ilustra el triángulo de priorización.

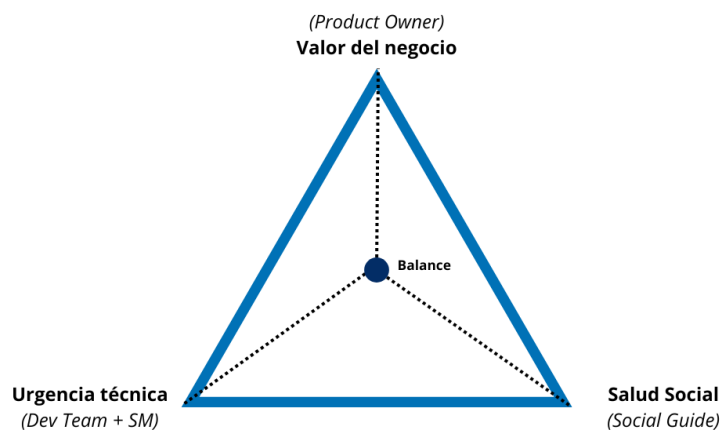


Figura 4.12: Triángulo de priorización de SocialScrum

La interpretación del triángulo es directa: cada ítem del Product Backlog se evalúa en las tres dimensiones, y la priorización final emerge de considerar las tres simultáneamente. No existe un algoritmo que pondere automáticamente las dimensiones; la decisión requiere juicio humano colaborativo entre los responsables de cada vértice.

4.7.1.1 Las tres dimensiones de evaluación

Cada ítem se evalúa en tres dimensiones usando una escala de 1-10:

1. **Valor de negocio** (responsable: Product Owner): ¿Cuánto beneficio genera para el cliente u organización? Un 10 indica que sin este ítem el negocio se detiene; un 1-3 indica mejoras marginales.

2. **Urgencia técnica** (responsable: Development Team): ¿Cuán necesario es desde la perspectiva de arquitectura o calidad? Un 10 indica bloqueo total; un 1-3 indica mejoras deseables sin riesgo inmediato.
3. **Salud social** (responsable: Social Guide): ¿Es necesario para restaurar la capacidad del equipo? Un 10 indica crisis activa (burnout, conflicto severo); un 1-3 indica mantenimiento preventivo con Health Score ≥ 7 .

Las escalas detalladas y ejemplos de evaluación para cada dimensión se presentan en el Anexo H.

4.7.1.2 Principio fundamental: conversación, no fórmula

El framework no utiliza una ecuación matemática. Una fórmula del tipo Prioridad = $w_1 \times \text{Valor} + w_2 \times \text{Urgencia} + w_3 \times \text{Salud}$ sería problemática por cuatro razones: las dimensiones no son conmensurables, los pesos serían arbitrarios, el contexto quedaría ignorado, y la precisión sería falsa.

En su lugar, SocialScrum propone una conversación estructurada donde cada responsable presenta su evaluación, se discuten las implicaciones, y se llega a una decisión por consenso.

4.7.1.3 Reglas de oro para casos extremos

Tres escenarios requieren reglas explícitas porque las consecuencias de decidir mal son significativas:

1. **Regla 1: Health Score $< 5 \rightarrow$ No comprometer features de alta complejidad.** Un equipo en zona crítica no tiene capacidad para trabajo complejo. El Sprint Goal debe enfocarse en restaurar la salud social antes de comprometer features mayores.
2. **Regla 2: Valor de Negocio 10 + Salud Social 10 $\rightarrow$ Escalar a CTO.** Este conflicto genuino entre entregar valor crítico y proteger al equipo requiere perspectiva organizacional. Las opciones típicas incluyen recursos externos, negociación con cliente, o reducción de alcance.
3. **Regla 3: Health Score $\geq 8 \rightarrow$ Priorizar por Valor + Urgencia Técnica.** Cuando el equipo está saludable, SocialScrum se comporta como Scrum tradicional. No hay necesidad de reservar capacidad para items sociales ni de moderar el alcance técnico.

Los escenarios detallados de aplicación de estas reglas, junto con la matriz completa de 18 combinaciones posibles, se presentan en el Anexo H.

En síntesis, el framework de priorización social-técnica transforma la priorización de una decisión unidimensional (solo valor de negocio) a una conversación tridimensional que considera el estado real del equipo. El triángulo de decisión proporciona un modelo mental compartido, y las reglas de oro establecen límites claros para casos extremos.

4.8 Herramientas y tecnología de soporte

SocialScrum puede implementarse con herramientas simples (documentos compartidos, hojas de cálculo), pero la integración con sistemas de gestión de proyectos existentes reduce la fricción operacional. El principio rector es integración, no duplicación: los items sociales deben vivir en el mismo sistema que los items técnicos.

El Anexo E presenta configuraciones detalladas para:

- **Jira:** Campos personalizados (Item Type, SSP, Community Smell, Health Dimension, Severity), workflows específicos para items sociales, y filtros JQL para monitoreo.
- **Azure DevOps:** Campos equivalentes, queries personalizadas, e integración con Power BI.
- **Plantillas operacionales:** Social User Story, mediación de conflictos, Health Score Dashboard semanal, notas de retrospectiva, reportes para *stakeholders*, y registro de *community smells*.
- **Dashboards automatizados:** Arquitectura de datos, implementaciones en Google Sheets, Grafana, y Power BI/Tableau.
- **Alertas automatizadas:** Configuraciones para Health Score crítico, dimensiones en caída, smells sin asignar, y capacidad social excedida.

La sofisticación de las herramientas debe ser proporcional al valor que genera: si una hoja de cálculo simple funciona para el equipo, no hay necesidad de implementar infraestructura compleja.

4.9 Simulación de implementación de SocialScrum

Esta sección presenta una simulación que ilustra la aplicación de SocialScrum durante tres Sprints consecutivos. El escenario se basa en patrones observados en equipos reales de desarrollo, aunque los nombres y detalles específicos son ficticios. El propósito es demostrar cómo los componentes del marco —eventos adaptados, artefactos sociales, sistema de medición y rol del Social Guide— se integran en la práctica diaria de un equipo.

4.9.1 Contexto del equipo

El equipo Phoenix es un equipo Scrum de 7 personas que desarrolla una plataforma SaaS de gestión de inventarios. Tiene 18 meses de experiencia con Scrum estándar, pero ha enfrentado problemas de comunicación y colaboración que han afectado su productividad y moral, en la Tabla 4.24 se presenta su perfil detallado.

Tabla 4.24: Perfil del equipo Phoenix

Característica	Descripción
Nombre del equipo	Equipo Phoenix
Tamaño	7 personas (1 Product Owner, 1 Scrum Master, 5 desarrolladores)
Producto	Plataforma SaaS de gestión de inventarios
Modalidad	Híbrida (3 días en oficina, 2 días remotos)
Duración del Sprint	2 semanas
Madurez en Scrum	18 meses practicando Scrum estándar
Social Guide	Laura (Scrum Master con capacitación adicional; rol compartido al 50 %)

4.9.1.1 Situación inicial

Antes de implementar SocialScrum, el equipo presentaba los siguientes síntomas:

- Velocidad inconsistente: fluctuaciones de 30 % entre Sprints sin causa técnica clara.
- Dos desarrolladores (Carlos y Miguel) evitaban trabajar juntos desde un conflicto no resuelto hace 4 meses.
- Ana, la desarrolladora más senior, concentraba el 60 % del conocimiento crítico del sistema.
- Las retrospectivas se habían vuelto “ceremoniales”: respuestas genéricas, sin mejoras reales.
- El PO reportaba fricción frecuente al solicitar cambios de prioridad.

4.9.1.2 Diagnóstico inicial (Sprint 0)

Laura realizó el diagnóstico inicial usando las herramientas de SocialScrum como se muestra en la Tabla 4.25.

Tabla 4.25: Health Score inicial del equipo Phoenix

Dimensión	Puntuación	Observación
Apertura	4.8	Los problemas se ocultan hasta el último momento
Compromiso	6.2	Aceptable, pero es frecuente la actitud de “ese no es mi código”
Respeto	5.1	Tensión visible entre Carlos y Miguel
Foco	5.5	Interrupciones frecuentes y cambio constante de contexto
Coraje	4.2	Nadie menciona explícitamente el conflicto entre Carlos y Miguel
Health Score	5.2	Zona “en riesgo”

Los *community smells* identificados fueron:

1. **Radio Silence** (Severidad Alta): Carlos y Miguel no se comunican directamente.
2. **Organisational Silo** (Severidad Media): Frontend y Backend trabajan aislados.
3. **Lone Wolf** (Severidad Media): Ana trabaja sola en módulos críticos.

4.9.2 Sprint 1: Establecer fundamentos

4.9.2.1 Social Sprint Planning

Durante el Planning, Laura facilitó la revisión de riesgos sociales:

Escenario aplicado
Laura (Social Guide): “Antes de estimar, revisemos el contexto social. El Health Score es 5.2 y tenemos tres <i>community smells</i> activos. ¿Ven algún riesgo para este Sprint?” Product Owner: “Necesito que Carlos y Miguel trabajen juntos en el módulo de reportes.” Laura (Social Guide): “Eso representa un riesgo. Propongo que, antes de asignar esa tarea, realicemos una sesión de alineación de una hora. ¿Están de acuerdo?” Resultado: El equipo acepta la propuesta y se agenda la sesión de alineación para el día 2 del Sprint.

4.9.2.2 Social Sprint Backlog

Capacidad asignada: 8 SSP de 50 puntos totales del Sprint (16 %).

- Sesión de alineación entre Carlos y Miguel (3 SSP).
- Pair programming rotativo entre Ana y otro desarrollador (3 SSP).
- Creación y adopción del canal #social-phoenix en Slack (2 SSP).

4.9.2.3 Ejecución del Sprint

Social Daily Standup's: En este nuevo evento se implementó la pregunta social diaria donde los miembros del equipo respondían a una pregunta rotativa diseñada para detectar señales tempranas de fricción o malestar. En la Tabla 4.26 se presentan las señales detectadas durante el Sprint 1.

Tabla 4.26: Señales detectadas en Social Daily Standup – Sprint 1

Día	Pregunta	Señal detectada
3	¿Cómo te sientes?	Miguel: “Tenso” (post-sesión con Carlos)
5	¿Necesitas algo?	Ana: “Ayuda con documentación”
8	¿Quién te ayudó?	Carlos: “Miguel me explicó el módulo”

Intervención:

A continuación se detalla la sesión de alineación entre Carlos y Miguel, facilitada por Laura, la cual fue una intervención clave para abordar el community smell en un escenario aplicado.

Escenario aplicado

Sesión de alineación Carlos–Miguel

Laura facilita una sesión de alineación de 90 minutos con el objetivo de reducir fricción y habilitar trabajo conjunto durante el Sprint.

1. **Contexto.** Cada participante expone su perspectiva del conflicto original, sin búsqueda de culpables ni juicios personales.
2. **Impacto.** Ambos reconocen que evitar la colaboración estaba afectando al equipo, en particular la velocidad y la gestión de dependencias.
3. **Acuerdos.** Se establecen tres normas de trabajo conjunto:
 - Comunicación escrita para decisiones relevantes.

- Escalamiento inmediato a Laura ante señales de fricción.
- Retrospectiva privada conjunta al final del Sprint.

Resultado. La sesión permite habilitar la colaboración controlada entre Carlos y Miguel para el resto del Sprint.

4.9.2.4 Social Sprint Retrospective

Fase 3 (Análisis) reveló patrón: “No decimos cuando algo nos molesta hasta que explota.”
Análisis de causa raíz (5 Por qué):

1. ¿Por qué no hablamos? Miedo a reacciones negativas.
2. ¿Por qué hay miedo? Experiencias pasadas de reacciones defensivas.
3. ¿Por qué hubo reacciones defensivas? No existía un espacio seguro para dar feedback.
4. ¿Por qué no había un espacio seguro? Nunca se estableció explícitamente.
5. **Causa raíz:** Falta de normas explícitas para dar feedback crítico.

Compromiso: Crear “Acuerdo de Feedback” con 5 reglas básicas (3 SSP para Sprint 2).

4.9.2.5 Resultados Sprint 1

En la Tabla 4.27 se presentan los resultados cuantitativos del Sprint 1.

Tabla 4.27: Resultados del Sprint 1

Métrica	Antes	Sprint 1	Cambio
Health Score	5.2	5.8	+0.6
Velocity (SP completados)	38	42	+10 %
Smells activos	3	3	=
Smells en observación	0	1	+1

4.9.3 Sprint 2: Abordar smells activos

4.9.3.1 Contexto

El conflicto Carlos-Miguel estaba en observación. El foco ahora era el *smell* “Lone Wolf” de Ana y la falta de documentación.

Social Sprint Backlog

Capacidad asignada: (10 SSP de 50 SP totales = 20 %):

- Crear “Acuerdo de Feedback” (3 SSP)
- Sesiones de transferencia de conocimiento Ana → equipo (5 SSP)
- Documentar módulo de inventarios (2 SSP)

4.9.3.2 Intervención: Transferencia de conocimiento

Ana facilitó 3 sesiones de 90 minutos cada una donde se habló sobre arquitectura, patrones de integración y debugging de casos críticos. En la Tabla 4.28 se presentan las sesiones realizadas.

Tabla 4.28: Sesiones de transferencia de conocimiento

Sesión	Tema	Participantes
1	Arquitectura del módulo de inventarios	Todo el equipo
2	Patrones de integración con ERP	Carlos, Pedro
3	Debugging de casos críticos	Miguel, Sara

Resultado:

- Bus factor del módulo crítico pasó de 1 (solo Ana) a 3 (Ana + Carlos + Miguel).
- Ana reportó “alivio” por no ser la única responsable.
- Se creó wiki con 12 páginas de documentación técnica.

4.9.3.3 Evento inesperado: Crisis de deadline

Día 7: El PO anunció que un cliente importante necesitaba una feature para el día 10 (3 días antes del fin de Sprint).

Escenario aplicado

Product Owner:

“Necesitamos priorizar esta iniciativa. Es crítica para el cliente.”

Social Guide (Laura):

“El Health Score actual es 6.1. El equipo puede absorber presión temporal. Sin embargo, propongo reducir el Social Sprint Backlog a 5 SSP cancelando una sesión de transferencia de conocimiento, para liberar capacidad técnica.”

Equipo:

Acepta la propuesta.

La sesión de transferencia número 3 se re-programa para el Sprint 3.

Este escenario ilustra cómo SocialScrum permite flexibilidad: la salud social no es “sagrada”; puede ajustarse cuando hay presión legítima, siempre que el Health Score lo permita.

4.9.3.4 Resultados Sprint 2

En la Tabla 4.29 se presentan los resultados cuantitativos del Sprint 2.

Tabla 4.29: Resultados Sprint 2

Métrica	Sprint 1	Sprint 2	Cambio
Health Score	5.8	6.4	+0.6
Velocity	42	45	+7 %
Smells activos	3	2	-1
Smells mitigados	0	1 (<i>Radio Silence</i>)	+1

4.9.4 Sprint 3: Consolidación y ajustes.

4.9.4.1 Contexto.

Health Score en 6.4 (zona “En riesgo” pero cerca de “Saludable”). Dos smells activos: *Organisational Silo* (FE-BE) y *Lone Wolf* (reducido a Media).

Social Sprint Backlog

Capacidad asignada: (6 SSP de 50 SP = 12 %):

- Completar sesión 3 de transferencia (2 SSP).
- Ritual de sincronización FE-BE (Frontend-Backend) semanal (2 SSP).
- Retrospectiva profunda de 3 Sprints (2 SSP).

4.9.4.2 Intervención: Sincronización FE-BE.

Se implementó reunión semanal de 30 minutos entre Frontend y Backend:

- Agenda fija: ¿Qué cambios de API vienen? ¿Qué necesita FE de BE? ¿Dónde hay fricción?
- Rotación de facilitador (FE una semana, BE la siguiente).
- Output: Lista de dependencias para el Sprint Planning.

Resultado: Integraciones tardías se redujeron de 4 por Sprint a 1.

4.9.4.3 Retrospectiva de consolidación.

Laura facilitó una retrospectiva extendida (2 horas) para evaluar los resultados obtenidos a lo largo de tres Sprints consecutivos.

Aspectos que funcionaron.

- El Social Daily Standup permitió detectar tensiones antes de que escalaran.
- La sesión de alineación entre Carlos y Miguel evitó el bloqueo de una feature crítica.
- La transferencia de conocimiento redujo la dependencia técnica de Ana.
- El equipo comenzó a mencionar problemas de forma explícita en retrospectivas.

Aspectos que no funcionaron.

- La sobrecarga inicial se percibió como elevada (la percepción mejoró en los Sprints 2 y 3).
- Algunas preguntas del Social Daily Standup se volvieron rutinarias con el tiempo.
- El Product Owner mostró resistencia inicial a asignar capacidad a ítems sociales.

Ajustes acordados.

- Reducir el Social Sprint Backlog al 10 % como nivel de mantenimiento.
- Rotar la pregunta social semanalmente, en lugar de diariamente.
- Incluir al Product Owner en una revisión mensual del Health Score.

4.9.4.4 Resultados Sprint 3.

En la Tabla 4.30 se presentan los resultados cuantitativos del Sprint 3.

Tabla 4.30: Resultados Sprint 3

Métrica	Sprint 2	Sprint 3	Cambio
Health Score	6.4	7.1	+0.7
Velocity	45	48	+6 %
Smells activos	2	1	-1
Smells mitigados	1	2	+1

4.9.5 Resumen de resultados

Los resultados presentados son un resumen de la evolución del equipo Phoenix durante los tres Sprints de implementación de SocialScrum. En la Tabla 4.31 se sintetizan las métricas clave, y en la Figura 4.13 se visualiza la mejora del Health Score a lo largo del tiempo.

Tabla 4.31: Evolución del equipo Phoenix en tres Sprints

Métrica	Inicial	Sprint 1	Sprint 2	Sprint 3
Health Score	5.2	5.8	6.4	7.1
Velocity (SP)	38	42	45	48
Smells activos	3	3	2	1
Capacidad social (%)	0 %	16 %	20 %	12 %

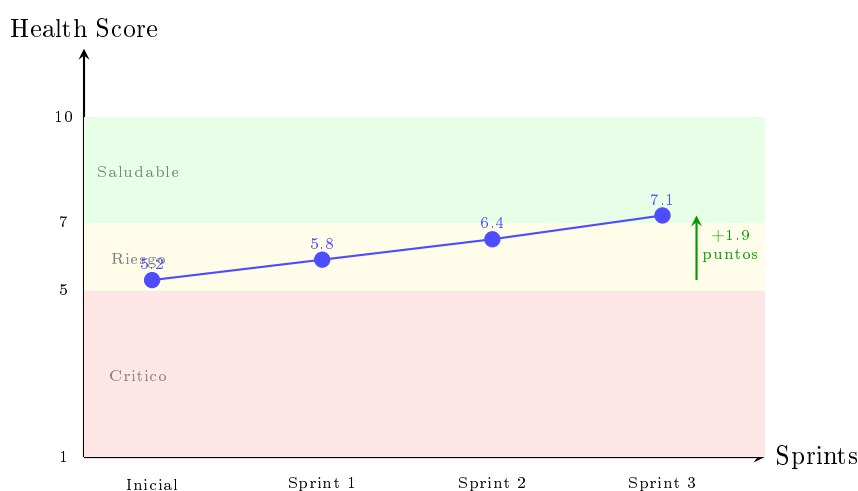


Figura 4.13: Evolución del Health Score del equipo Phoenix

4.9.6 Lecciones aprendidas

Durante la implementación de SocialScrum en el equipo Phoenix, se identificaron varios factores clave que contribuyeron al éxito del enfoque, así como dificultades que surgieron y recomendaciones para otros equipos interesados en adoptar prácticas similares.

4.9.6.1 Factores de éxito

1. **Compromiso visible del liderazgo:** El CTO aprobó dedicar hasta 20 % de capacidad a ítems sociales en los primeros Sprints.
2. **Intervención temprana en conflicto:** La sesión Carlos-Miguel en Sprint 1 evitó que el conflicto bloqueara una feature crítica.

3. **Flexibilidad ante presión:** Sprint 2 demostró que SocialScrum puede adaptarse a deadlines urgentes sin abandonar completamente la agenda social.
4. **Transferencia de conocimiento proactiva:** Reducir el bus factor de Ana benefició tanto la salud social como la resiliencia técnica.

4.9.6.2 Dificultades encontradas

1. **Resistencia inicial del PO:** Percibía los SSP como “tiempo robado” al desarrollo. Se resolvió mostrando correlación velocity-Health Score.
2. **Fatiga de preguntas sociales:** El equipo se “acostumbró” a las preguntas del Daily. Se resolvió rotando preguntas semanalmente.
3. **Dificultad para cuantificar ROI:** Es difícil probar causalidad entre intervenciones sociales y mejora de velocity. Se aceptó correlación como evidencia suficiente.

4.9.6.3 Recomendaciones para otros equipos

En la Tabla 4.32 se resumen recomendaciones prácticas basadas en la experiencia del equipo Phoenix.

Tabla 4.32: Recomendaciones basadas en el caso Phoenix

Recomendación	Justificación
Empezar con 15–20 % de capacidad social	Permite intervenciones significativas sin colapsar la velocity. Reducir a 10 % una vez estabilizado el equipo.
Priorizar conflictos activos	Son los factores que más bloquean el trabajo. Resolverlos temprano libera capacidad para acciones preventivas.
Medir Health Score semanalmente	Permite detectar deterioro antes de que se convierta en crisis. La medición mensual es insuficiente en equipos en riesgo.
Involucrar al PO desde el Sprint 0	La resistencia inicial es predecible. La educación temprana facilita entender la correlación entre salud y productividad.
Documentar intervenciones	Permite aprender qué prácticas funcionan para el equipo específico y justificar inversiones futuras.

El caso del equipo Phoenix demuestra que SocialScrum puede implementarse de forma incremental, con resultados medibles en 3 Sprints. La mejora del Health Score de 5.2 a 7.1 (+36 %) coincidió con un aumento de velocity de 38 a 48 SP (+26 %). Aunque la correlación no prueba causalidad, la consistencia de ambas tendencias sugiere que abordar la deuda social tiene beneficios tanto para el bienestar del equipo como para su productividad.

4.10 Conclusiones del capítulo

Este capítulo ha presentado el modelo operativo completo de SocialScrum, una extensión de Scrum diseñada para gestionar la deuda social en equipos de desarrollo de software. A continuación se sintetizan los aportes principales:

Componentes del marco. SocialScrum introduce cinco extensiones interconectadas: (1) adaptaciones a los eventos de Scrum que integran una dimensión social sin reemplazar su propósito original; (2) tres artefactos nuevos —Social Backlog, Social Sprint Backlog y Social Debt Register— que hacen visible y gestionable la deuda social; (3) un rol especializado, el Social Guide, que facilita intervenciones sin convertirse en terapeuta organizacional; (4) un sistema de medición basado en Health Score; y (5) un catálogo de herramientas y técnicas adaptadas de otras disciplinas.

Integración con Scrum. El diseño respeta los principios fundamentales de Scrum: empirismo, autoorganización y mejora continua. Las extensiones son aditivas, no sustitutivas. Un equipo puede adoptar SocialScrum de forma incremental, comenzando con el Health Score y añadiendo componentes según su madurez y necesidades.

Viabilidad práctica. El caso de estudio simulado del equipo Phoenix demuestra que el marco puede implementarse en 3 Sprints con resultados medibles: mejora del Health Score de 5.2 a 7.1, reducción de *community smells* activos de 3 a 1, y aumento de velocidad de 38 a 48 SP. Aunque los datos son simulados, el escenario se basa en patrones observados en equipos reales.

Limitaciones reconocidas. SocialScrum no ha sido validado empíricamente en múltiples organizaciones. La sobrecarga de 15-20% de capacidad social puede ser prohibitiva para equipos bajo presión extrema de entrega. La efectividad del Social Guide depende de habilidades que no todos los Scrum Masters poseen. Estas limitaciones se abordarán en el capítulo de trabajo futuro.

Contribución al campo. Este capítulo traduce el concepto abstracto de “deuda social” a un conjunto de prácticas concretas y aplicables. A diferencia de enfoques anteriores que se limitan a diagnosticar *community smells*, SocialScrum proporciona un ciclo completo de detección, priorización, intervención y seguimiento, integrado en el flujo natural de trabajo de un equipo ágil.

CAPÍTULO 5. VALIDACIÓN DEL MODELO SOCIALSCRUM MEDIANTE LA TÉCNICA DE JUICIO DE EXPERTOS

Este capítulo presenta la validación del modelo SocialScrum, con el fin de analizar su coherencia y alineación con los principios de la agilidad. La validación se aborda desde una perspectiva conceptual, considerando tanto la evaluación de la agilidad del modelo como la valoración experta de su diseño, previo a una eventual implementación empírica en contextos organizacionales reales.

Es importante señalar que SocialScrum prioriza deliberadamente los factores sociales y organizacionales del trabajo en equipo por encima de prácticas técnicas específicas de ingeniería de software. En consecuencia, los resultados de esta validación deben interpretarse como una validación conceptual del modelo desde la perspectiva ágil, quedando la evaluación de su desempeño técnico y operativo como una línea de trabajo futuro. La estructura de este capítulo se organiza de la siguiente manera y se puede visualizar en la figura 5.1:

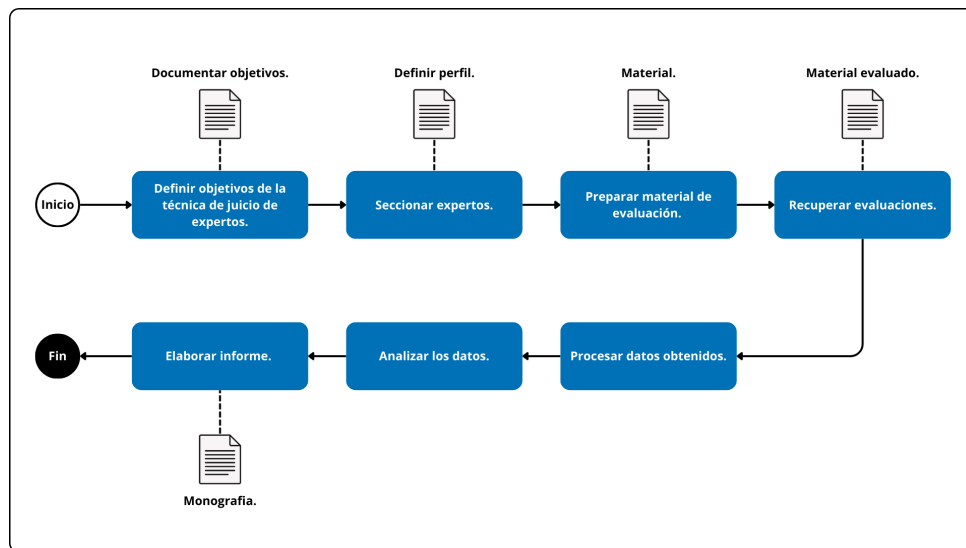


Figura 5.1: Estructura del Capítulo 5: Validación del modelo SocialScrum

1. Protocolo de validación mediante juicio de expertos
2. Perfil y selección de expertos participantes
3. Diseño del instrumento de validación

4. Procedimiento de aplicación

5. Análisis de resultados y retroalimentación

5.1 Estrategia de validación del modelo SocialScrum

La validación del modelo SocialScrum se fundamenta en un enfoque metodológico mixto que busca contrastar la viabilidad conceptual, la pertinencia operativa y el cumplimiento de los principios ágiles. Dado que el modelo propone una extensión socio-organizacional de Scrum, la estrategia de validación se dividió en dos dimensiones complementarias: (i) una evaluación técnica de contenido mediante el juicio de expertos, y (ii) un análisis de alineación con el Manifiesto Ágil utilizando el framework AgilityPRO [35].

Para garantizar que SocialScrum no solo sea una propuesta teórica, sino una herramienta de ingeniería ejecutable, el modelo se somete a la evaluación de cinco criterios de calidad fundamentales detallados en la Sección 5.3. Estos criterios permiten medir aspectos clave como la claridad, completitud, idoneidad, facilidad de uso y agilidad del modelo. La evaluación se realiza a través de un instrumento estructurado que combina preguntas cerradas con escala Likert y preguntas abiertas para capturar tanto valoraciones cuantitativas como cualitativas.

El uso de AgilityPRO (Agility in Processes) [35], es crucial para determinar si SocialScrum mantiene la esencia de los valores y principios del Manifiesto Ágil a pesar de su enfoque en la deuda social. Este framework proporciona un soporte formal para evaluar la agilidad de los procesos mediante el análisis del cumplimiento de los principios ágiles, independientemente de la metodología base utilizada. En esta validación, se utiliza el modelo de referencia AgilityRef para mapear cómo los elementos de SocialScrum (como la Social Sprint Retrospective o el Social Product Backlog) actúan como “aspectos de agilidad” que evidencian la implementación de uno o más principios ágiles. De este modo, la validación no depende de una percepción subjetiva, sino de un marco de referencia que garantiza que el modelo apoya el desarrollo iterativo, la inspección frecuente y la centralidad en las personas.

5.2 Criterios de evaluación considerados

La validación del modelo SocialScrum se llevó a cabo mediante un cuestionario estructurado que incluyó tanto preguntas cuantitativas como cualitativas. El cuestionario se diseñó para evaluar diferentes aspectos del modelo, tales como su coherencia con los principios ágiles, su aplicabilidad práctica y su capacidad para mejorar la colaboración en equipos distribuidos.

5.2.1 Criterios de evaluación

Se definieron cinco criterios de evaluación clave para guiar la valoración del modelo SocialScrum por parte de los expertos:

Tabla 5.1: Criterios de evaluación y número de preguntas en el instrumento de validación

Criterio	Descripción
Claridad	Permite verificar si los conceptos, roles (especialmente el Social Guide) y procesos están definidos de forma precisa, con una semántica adecuada que evite ambigüedades en la ejecución.
Complejidad	Evalúa si el modelo abarca todas las dimensiones necesarias para la gestión de la deuda social, integrando adecuadamente los eventos adaptados y los artefactos sociales requeridos.
Idoneidad	Valora la pertinencia de la propuesta frente a las necesidades reales de los equipos de desarrollo y su capacidad para abordar problemas de mejora en entornos de software.
Facilidad de uso	Determina si los protocolos y herramientas propuestas pueden ser implementados de manera sencilla por el equipo, sin generar una complejidad técnica o administrativa innecesaria.
Agilidad	Este criterio es el eje central de la validación metodológica y se operacionaliza a través del framework AgilityPRO.

En cuanto al criterio relacionado con la agilidad de la tabla 5.1 no se fundamenta en percepciones subjetivas, sino que tiene como propósito validar la pertinencia y el cumplimiento de los 33 indicadores de agilidad (IA_01 a IA_33) definidos en la capa metodológica del framework AgilityPRO [35]. Específicamente, las preguntas de esta dimensión buscan que el experto determine si los componentes de SocialScrum —como el rol del Social Guide, los Eventos Adaptados y los Artefactos Sociales— operan efectivamente como “aspectos de agilidad”. Al utilizar el modelo de referencia AgilityRef, esta validación garantiza que SocialScrum no solo gestiona la deuda social, sino que lo hace manteniendo la alineación formal con los valores y principios del Manifiesto Ágil bajo un marco de referencia internacionalmente reconocido.

5.2.2 Escala de medición

Para la evaluación cuantitativa, se utilizó una escala Likert de de 1 a 5 puntos, donde 1 representaba “Totalmente en desacuerdo” y 5 “Totalmente de acuerdo”, esto permitió a los expertos expresar su grado de acuerdo o desacuerdo con afirmaciones relacionadas con cada criterio de evaluación. La escala se definió de la siguiente manera en la Tabla 5.2:

Tabla 5.2: Estructura del instrumento de evaluación

Interpretación	Valor
Totalmente en desacuerdo	1

Continúa en la siguiente página...

Tabla 5.2 – Continuación

Interpretación	Valor
Parcialmente en desacuerdo	2
Neutralidad	3
Parcialmente de acuerdo	4
Totalmente de acuerdo	5

5.3 Definición del instrumento de validación

La validación del contenido del marco ágil SocialScrum se organizó en torno a un instrumento diseñado específicamente para este fin, denominado Instrumento de evaluación por juicio de expertos - marco ágil SocialScrum. Este instrumento del tipo cuestionario en formato digital fue diseñado como un medio para valorar la calidad técnica y la aplicabilidad metodológica del proceso propuesto además de un documento de contextualización entregado a los expertos (ver Anexo M).

5.3.1 Estructura del cuestionario

El instrumento de evaluación del marco ágil SocialScrum se estructuró como un cuestionario en formato digital, compuesto por 15 preguntas cerradas y 2 preguntas abiertas. Las preguntas cerradas fueron diseñadas para ser valoradas por los expertos en función de cinco criterios de calidad presentados anteriormente, utilizando la escala Likert. Las preguntas abiertas permitieron a los expertos proporcionar comentarios cualitativos y sugerencias de mejora.

5.3.2 Preguntas formuladas

A continuación se presenta el cuestionario dividido en Parte A (preguntas cerradas con escala Likert de 5 puntos), y Parte B (preguntas abiertas). Cada pregunta está identificada con un código (P1–P15 para ítems Likert, PA1–PA2 para abiertas):

Tabla 5.3: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta
Parte A – Preguntas Cerradas		
P1	Claridad	¿Considera que los conceptos del modelo SocialScrum están claramente definidos?
P2	Claridad	¿El rol del Social Guide está claramente diferenciado de otros roles (por ejemplo, con el Scrum Master, Product Owner o el departamento de RRHH) evitando conflictos de autoridad?

Continúa en la siguiente página...

Tabla 5.3 – Continuación

Id.	Criterio	Pregunta
P3	Claridad	¿El modelo establece mecanismos claros de gobernanza para resolver desacuerdos entre el Social Guide y otros roles (por ejemplo: protocolo de escalamiento PO-Social Guide, mediación del Scrum Master o arbitraje del CTO), evitando impactos negativos en la toma de decisiones del equipo?
P4	Complejidad	¿Cree que el modelo SocialScrum cubre todos los aspectos necesarios para la gestión ágil de proyectos?
P5	Complejidad	¿Los artefactos sociales introducidos (por ejemplo, el Social Product Backlog, las historias de usuario sociales y el registro de <i>community smells</i>) están bien definidos y facilitan el seguimiento de la salud del equipo?
P6	Complejidad	¿El modelo incluye mecanismos de auditoría y transparencia que permiten al equipo verificar la actuación del Social Guide, previniendo posibles abusos de autoridad o desviaciones éticas?
P7	Idoneidad	¿Considera que SocialScrum aborda de manera pertinente las necesidades reales de los equipos de desarrollo de software ágil en cuanto a la gestión de la deuda social (por ejemplo, la detección de <i>community smells</i> , la mitigación de tensiones interpersonales y la mejora de la colaboración)?
P8	Idoneidad	¿Los mecanismos de inspección y adaptación social propuestos por SocialScrum (como el Health Score, el Registro de <i>Community Smells</i> y las retrospectivas sociales) son adecuados para identificar y mitigar los problemas socio-técnicos que generan deuda social en los equipos?
P9	Idoneidad	¿El rol del Social Guide, tal como está definido en SocialScrum, resulta pertinente y necesario para facilitar la gestión de la deuda social, cubriendo funciones que no son atendidas por los roles existentes en Scrum (Scrum Master, Product Owner, Developers)?
P10	Facilidad de Uso	¿El perfil y las competencias del Social Guide están claramente definidos y son viables, complementando los roles existentes en el equipo sin solapamiento de responsabilidades?
P11	Facilidad de Uso	¿La implementación de SocialScrum en equipos ágiles reales es factible y no requiere cambios organizacionales o culturales impracticables?
P12	Facilidad de Uso	¿La sobrecarga operacional de SocialScrum es sostenible para equipos ágiles a mediano/largo plazo?
P13	Agilidad	¿Los eventos adaptados de SocialScrum (Social Daily Standup, Social Sprint Planning, Social Sprint Review y Social Sprint Retrospective) promueven la inspección y adaptación continua de las dinámicas sociales del equipo, en coherencia con los principios ágiles del Manifiesto Ágil?
P14	Agilidad	¿El sistema de estimación Social Story Points (SSP) es compatible con las prácticas ágiles estándar y resulta comprensible para todos los miembros del equipo?
P15	Agilidad	¿El mecanismo de priorización socio-técnica de SocialScrum (integrando intervenciones sociales junto con tareas técnicas) es práctico y logra equilibrar la mejora del equipo con la entrega de valor al producto?
Parte B – Preguntas Abiertas		
PA1	—	¿Qué elementos del modelo SocialScrum considera usted que faltan o están poco desarrollados?
PA2	—	¿Qué sugerencias de mejora específicas propondría para el modelo SocialScrum? Justifique su respuesta con fundamentos teóricos o prácticos.

Acrónimos utilizados: Identificador (Id.), Pregunta Cerrada (P) y Pregunta Abierta (PA).

5.4 Perfil y selección de expertos participantes

5.4.1 Perfilamiento de los participantes

Para la validación del modelo SocialScrum, se seleccionaron expertos con experiencia en metodologías ágiles, desarrollo de software y gestión de proyectos. Los criterios de selección incluyeron:

- Experiencia académica o profesional en el área de desarrollo de software.
- Experiencia mínima de 2 años en la aplicación de metodologías ágiles.
- Participación en proyectos ágiles.
- Conocimiento en herramientas de colaboración y gestión de proyectos ágiles.
- Preferiblemente, experiencia en equipos distribuidos o remotos.

5.4.2 Selección de participantes

Se contactaron a 8 expertos potenciales a través de redes profesionales y académicas. De estos, los 8 aceptaron participar en la validación del modelo SocialScrum. Los participantes provinieron de diversas industrias, incluyendo desarrollo de software, consultoría ágil y educación en Ingeniería de Software. A continuación, se presenta un resumen del perfil de los expertos participantes en la validación del modelo SocialScrum (ver Tabla 5.4).

Tabla 5.4: Perfil de los expertos participantes en la validación del modelo SocialScrum

Id.	Último título obtenido	Años de experiencia	Cargo actual	Metodologías ágiles conocidas
E1	Ingeniero de sistemas	4 años	Desarrollador Senior de software	Scrum, Kanban, XP
E2	Ingeniero de sistemas	4 años	Desarrollador Semi-Senior de software	Scrum, Lean, Kanban
E3	Ingeniero de sistemas	9 años	Desarrollador Senior de software	Scrum, Kanban
E4	Ingeniero de sistemas	5 años	Desarrollador Senior de software	Scrum, XP, Lean
E5	Ingeniero de sistemas	4 años	Desarrollador Semi-Senior de software	Scrum, Kanban
E6	Ingeniero en Telemática	5 años	Ingeniero DevOps Senior	Scrum, Kanban, SAFe
E7	Ingeniero de sistemas	3 años	Desarrollador Semi-Senior de software	Scrum, Lean
E8	Ingeniero de sistemas	5 años	Desarrollador Senior de software	Scrum, Kanban, XP, Lean

Acrónimos utilizados: Identificador (Id.) y Experto (E).

5.5 Protocolo de validación mediante juicio de expertos

Para la evaluación del modelo SocialScrum, se diseñó un protocolo de validación basado en la técnica de juicio de expertos. Para la evaluación y validación de este modelo se incluyó un documento entregado a los participantes con la contextualización del tema a evaluar (ver Anexo M). Esta técnica permite obtener una valoración cualitativa y cuantitativa del modelo a partir de la experiencia y conocimientos de profesionales en metodologías ágiles y desarrollo de software. El protocolo se estructuró en varias etapas, que se describen a continuación.

5.5.1 Objetivo de la validación

El objetivo principal de esta validación es evaluar la adecuación del modelo SocialScrum en términos de su alineación con los principios ágiles, su aplicabilidad en contextos reales de desarrollo de software y su capacidad para fomentar la colaboración y eficiencia en equipos distribuidos.

5.6 Resultados de la validación y análisis cuantitativo

En esta sección se presentan los resultados obtenidos a partir de la validación del modelo SocialScrum. El propósito central de este análisis es determinar en qué medida el proceso cumple con los criterios de calidad previamente definidos —claridad, completitud, idoneidad, facilidad de uso y agilidad—, así como identificar percepciones cualitativas que permitan reconocer sus fortalezas y oportunidades de mejora

5.6.1 Análisis cuantitativo de las respuestas

Se recopilieron las respuestas de los 8 expertos participantes, quienes evaluaron cada una de las 15 preguntas del instrumento de validación utilizando la escala Likert propuesta. A continuación, se presentan los resultados cuantitativos obtenidos para cada criterio de evaluación.

Tabla 5.5: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P1	Claridad	¿Considera que los conceptos del modelo SocialScrum están claramente definidos?					

Continúa en la siguiente página...

Tabla 5.5 – Continuación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P2	Claridad	¿El rol del Social Guide está claramente diferenciado de otros roles (por ejemplo, con el Scrum Master, Product Owner o el departamento de RRHH) evitando conflictos de autoridad?					
P3	Claridad	¿El modelo establece mecanismos claros de gobernanza para resolver desacuerdos entre el Social Guide y otros roles (por ejemplo: protocolo de escalamiento PO-Social Guide, mediación del Scrum Master o arbitraje del CTO), evitando impactos negativos en la toma de decisiones del equipo?					
P4	Complejidad	¿Cree que el modelo SocialScrum cubre todos los aspectos necesarios para la gestión ágil de proyectos?					
P5	Complejidad	¿Los artefactos sociales introducidos (por ejemplo, el Social Product Backlog, las historias de usuario sociales y el registro de community smells) están bien definidos y facilitan el seguimiento de la salud del equipo?					
P6	Complejidad	¿El modelo incluye mecanismos de auditoría y transparencia que permiten al equipo verificar la actuación del Social Guide, previniendo posibles abusos de autoridad o desviaciones éticas?					
P7	Idoneidad	¿Considera que SocialScrum aborda de manera pertinente las necesidades reales de los equipos de desarrollo de software ágil en cuanto a la gestión de la deuda social (por ejemplo, la detección de <i>community smells</i> , la mitigación de tensiones interpersonales y la mejora de la colaboración)?					
P8	Idoneidad	¿Los mecanismos de inspección y adaptación social propuestos por SocialScrum (como el Health Score, el Registro de <i>Community Smells</i> y las retrospectivas sociales) son adecuados para identificar y mitigar los problemas socio-técnicos que generan deuda social en los equipos?					

Continúa en la siguiente página...

Tabla 5.5 – Continuación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P9	Idoneidad	¿El rol del Social Guide, tal como está definido en SocialScrum, resulta pertinente y necesario para facilitar la gestión de la deuda social, cubriendo funciones que no son atendidas por los roles existentes en Scrum (Scrum Master, Product Owner, Developers)?					
P10	Facilidad de Uso	¿El perfil y las competencias del Social Guide están claramente definidos y son viables, complementando los roles existentes en el equipo sin solapamiento de responsabilidades?					
P11	Facilidad de Uso	¿La implementación de SocialScrum en equipos ágiles reales es factible y no requiere cambios organizacionales o culturales impracticables?					
P12	Facilidad de Uso	¿La sobrecarga operacional de SocialScrum es sostenible para equipos ágiles a mediano/largo plazo?					
P13	Agilidad	¿Los eventos adaptados de SocialScrum (Social Daily Standup, Social Sprint Planning, Social Sprint Review y Social Sprint Retrospective) promueven la inspección y adaptación continua de las dinámicas sociales del equipo, en coherencia con los principios ágiles del Manifiesto Ágil?					
P14	Agilidad	¿El sistema de estimación Social Story Points (SSP) es compatible con las prácticas ágiles estándar y resulta comprensible para todos los miembros del equipo?					
P15	Agilidad	¿El mecanismo de priorización socio-técnica de SocialScrum (integrando intervenciones sociales junto con tareas técnicas) es práctico y logra equilibrar la mejora del equipo con la entrega de valor al producto?					

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

A continuación, se presentan los resultados organizados por cada uno de los criterios de evaluación considerados en el instrumento: (i) claridad, (ii) completitud, (iii) idoneidad, (iv) facilidad de uso y (v) coherencia. En las subsecciones siguientes se muestran las tablas y figuras que resumen el comportamiento de las respuestas para cada criterio, lo

cual permite disponer de una visión diferenciada del nivel de conformidad alcanzado en cada dimensión del modelo SocialScrum.

5.6.1.1 Claridad

Tabla 5.6: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P1	Claridad	¿Considera que los conceptos del modelo SocialScrum están claramente definidos?					
P2	Claridad	¿El rol del Social Guide está claramente diferenciado de otros roles (por ejemplo, con el Scrum Master, Product Owner o el departamento de RRHH) evitando conflictos de autoridad?					
P3	Claridad	¿El modelo establece mecanismos claros de gobernanza para resolver desacuerdos entre el Social Guide y otros roles (por ejemplo: protocolo de escalamiento PO-Social Guide, mediación del Scrum Master o arbitraje del CTO), evitando impactos negativos en la toma de decisiones del equipo?					

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

5.6.1.2 Completitud

Tabla 5.7: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P4	Completitud	¿Cree que el modelo SocialScrum cubre todos los aspectos necesarios para la gestión ágil de proyectos?					
P5	Completitud	¿Los artefactos sociales introducidos (por ejemplo, el Social Product Backlog, las historias de usuario sociales y el registro de community smells) están bien definidos y facilitan el seguimiento de la salud del equipo?					

Continúa en la siguiente página...

Tabla 5.7 – Continuación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P6	Complejidad	¿El modelo incluye mecanismos de auditoría y transparencia que permiten al equipo verificar la actuación del Social Guide, previniendo posibles abusos de autoridad o desviaciones éticas?					

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

5.6.1.3 Idoneidad

Tabla 5.8: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P7	Idoneidad	¿Considera que SocialScrum aborda de manera pertinente las necesidades reales de los equipos de desarrollo de software ágil en cuanto a la gestión de la deuda social (por ejemplo, la detección de <i>community smells</i> , la mitigación de tensiones interpersonales y la mejora de la colaboración)?					
P8	Idoneidad	¿Los mecanismos de inspección y adaptación social propuestos por SocialScrum (como el Health Score, el Registro de <i>Community Smells</i> y las retrospectivas sociales) son adecuados para identificar y mitigar los problemas socio-técnicos que generan deuda social en los equipos?					
P9	Idoneidad	¿El rol del Social Guide, tal como está definido en SocialScrum, resulta pertinente y necesario para facilitar la gestión de la deuda social, cubriendo funciones que no son atendidas por los roles existentes en Scrum (Scrum Master, Product Owner, Developers)?					

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

5.6.1.4 Facilidad de uso

Tabla 5.9: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P10	Facilidad de Uso	¿El perfil y las competencias del Social Guide están claramente definidos y son viables, complementando los roles existentes en el equipo sin solapamiento de responsabilidades?					
P11	Facilidad de Uso	¿La implementación de SocialScrum en equipos ágiles reales es factible y no requiere cambios organizacionales o culturales impracticables?					
P12	Facilidad de Uso	¿La sobrecarga operacional de SocialScrum es sostenible para equipos ágiles a mediano/largo plazo?					

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

5.6.1.5 Agilidad

Tabla 5.10: Ejemplos de preguntas del instrumento de evaluación

Id.	Criterio	Pregunta	Número de respuestas por nivel de la escala				
			Escala 5	Escala 4	Escala 3	Escala 2	Escala 1
P13	Agilidad	¿Los eventos adaptados de SocialScrum (Social Daily Standup, Social Sprint Planning, Social Sprint Review y Social Sprint Retrospective) promueven la inspección y adaptación continua de las dinámicas sociales del equipo, en coherencia con los principios ágiles del Manifiesto Ágil?	5	0	0	0	0
P14	Agilidad	¿El sistema de estimación Social Story Points (SSP) es compatible con las prácticas ágiles estándar y resulta comprensible para todos los miembros del equipo?	4	1	0	0	0
P15	Agilidad	¿El mecanismo de priorización socio-técnica de SocialScrum (integrando intervenciones sociales junto con tareas técnicas) es práctico y logra equilibrar la mejora del equipo con la entrega de valor al producto?	3	2	0	0	0

Acrónimos utilizados: Identificador (Id.) y Pregunta Cerrada (P).

5.6.2 Resultados preguntas abiertas

El presente análisis se enfoca en las respuestas abiertas proporcionadas por los expertos durante la validación del modelo SocialScrum. Estas respuestas cualitativas ofrecen una perspectiva enriquecedora sobre las fortalezas y áreas de mejora del marco ágil propuesto. A continuación, se presentan las observaciones y recomendaciones más relevantes extraídas de las respuestas de los expertos

Tabla 5.11: Observaciones y recomendaciones de los expertos

Id.	Observación / Recomendación	Calificación	Justificación
PA1	El modelo SocialScrum presenta una estructura clara y bien definida, lo que facilita su comprensión y aplicación en equipos distribuidos.	Si	[Pendiente de consolidación tras la recepción de formularios]
PA2	Sería beneficioso proporcionar guías adicionales sobre cómo adaptar el modelo a las particularidades de cada equipo o proyecto.	No	[Pendiente de consolidación tras la recepción de formularios]

Acrónimos utilizados: Identificador (Id.).

5.7 Desempeño del modelo y recomendaciones de mejora

5.7.0.1 Fortalezas del modelo SocialScrum

- **Claridad y estructura:** Los expertos destacaron que el modelo SocialScrum presenta una estructura clara y bien definida, lo que facilita su comprensión y aplicación en equipos distribuidos.
- **Alineación con principios ágiles:** Se reconoció que el modelo está alineado con los valores y principios del Manifiesto Ágil, promoviendo la colaboración y la adaptabilidad.
- **Enfoque en la colaboración:** Los expertos valoraron positivamente el énfasis del modelo en fomentar la colaboración entre los miembros del equipo, especialmente en contextos distribuidos.

5.7.0.2 Debilidades del modelo SocialScrum

- **Falta de ejemplos prácticos:** Algunos expertos señalaron la ausencia de ejemplos prácticos o casos de estudio que ilustren la implementación del modelo en diferentes contextos.

- **Adaptabilidad limitada:** Se mencionó que el modelo podría beneficiarse de guías adicionales sobre cómo adaptarlo a las particularidades de cada equipo o proyecto.
- **Clarificación de roles:** Algunos expertos indicaron que ciertos roles y responsabilidades dentro del modelo podrían no estar suficientemente claros, lo que podría generar confusiones durante su implementación.
- **Falta de herramientas de soporte:** Se observó que el modelo no proporciona recomendaciones sobre herramientas tecnológicas que puedan facilitar su aplicación en equipos distribuidos.
- **Complejidad en la implementación:** Algunos expertos consideraron que la implementación del modelo podría resultar compleja para equipos con poca experiencia en metodologías ágiles.

5.7.0.3 Oportunidades de mejora

- **Incorporación de ejemplos prácticos:** Se recomendó incluir ejemplos prácticos o casos de estudio que ilustren la implementación del modelo en diferentes contextos.
- **Guías de adaptación:** Los expertos sugirieron proporcionar guías adicionales sobre cómo adaptar el modelo a las particularidades de cada equipo o proyecto.
- **Clarificación de roles:** Algunos expertos señalaron la necesidad de clarificar los roles y responsabilidades dentro del modelo para evitar confusiones durante su implementación.
- **Herramientas de soporte:** Se propuso la inclusión de recomendaciones sobre herramientas tecnológicas que puedan facilitar la aplicación del modelo en equipos distribuidos.

5.7.0.4 Comentarios sobre la adecuación del Instrumento de Evaluación

En general, los expertos consideraron que el Instrumento de Evaluación utilizado para validar el modelo SocialScrum es adecuado y cubre los aspectos esenciales necesarios para una evaluación exhaustiva. Sin embargo, se sugirieron algunas mejoras para fortalecer su efectividad:

- **Claridad en las preguntas:** Algunos expertos recomendaron revisar la redacción de ciertas preguntas para asegurar que sean claras y comprensibles para todos los evaluadores.
- **Incorporación de más preguntas abiertas:** Se sugirió aumentar el número de preguntas abiertas para permitir a los expertos proporcionar comentarios más detallados y específicos sobre el modelo.

- **Equilibrio entre aspectos cualitativos y cuantitativos:** Algunos expertos señalaron la importancia de equilibrar las preguntas cuantitativas con aspectos cualitativos para obtener una visión más completa del desempeño del modelo.

En síntesis, la validación del modelo SocialScrum por parte de expertos ha proporcionado valiosos insights sobre sus fortalezas y áreas de mejora. Las recomendaciones recibidas servirán como base para futuras iteraciones del modelo, con el objetivo de optimizar su efectividad y facilitar su adopción en equipos ágiles distribuidos.

CAPÍTULO 6. CONCLUSIONES, PUBLICACIONES, RECOMENDACIONES Y TRABAJOS FUTUROS

6.1 Conclusiones

Esta investigación propuso SocialScrum, un marco metodológico que extiende Scrum para gestionar la deuda social en equipos de desarrollo de software. A continuación se presentan las conclusiones principales, organizadas según los objetivos específicos planteados.

6.1.1 Sobre la caracterización de la deuda social

El análisis de literatura permitió identificar y categorizar los *community smells* más prevalentes en equipos de desarrollo ágil. Se encontró que:

- Los *community smells* se agrupan en cuatro categorías: comunicación, conocimiento, relacional y organizacional.
- Existe una correlación documentada entre smells sociales y métricas técnicas negativas (aumento de defectos, reducción de velocity, rotación de personal).
- Los marcos ágiles existentes, incluido Scrum, carecen de mecanismos explícitos para detectar y mitigar estos problemas de manera sistemática.

6.1.2 Sobre el diseño del marco SocialScrum

El diseño de SocialScrum logró integrar la gestión de la deuda social en la estructura existente de Scrum sin agregar eventos adicionales, respetando el Principio de No-Sobrecarga. Las contribuciones principales incluyen:

- **Rol del Social Guide:** Definición de un perfil híbrido (competencias psicológicas + conocimiento técnico) con gobernanza clara que protege la confidencialidad.
- **Health Score:** Métrica compuesta de cinco dimensiones (basadas en valores Scrum) que permite cuantificar y monitorear la salud social.
- **Artefactos sociales:** Social Product Backlog, Registro de Smells y Dashboard de Health Score para documentar y gestionar la deuda social.

- **Framework de priorización:** Modelo de triángulo que integra valor de negocio, urgencia técnica y salud social en decisiones de Sprint Planning.
- **Sistema de estimación SSP:** Social Story Points para dimensionar esfuerzo de intervenciones no técnicas.
- **Salvaguardas éticas:** Protocolos de confidencialidad, auditoría y rendición de cuentas que protegen a los miembros del equipo.

6.1.3 Sobre la viabilidad práctica

El caso de estudio simulado (3 Sprints) demostró que SocialScrum es operativamente viable:

- La sobrecarga de tiempo se mantuvo dentro del 10-15 % adicional por evento.
- El Health Score sirve como indicador útil para guiar decisiones de priorización.
- Las estrategias de mitigación del catálogo proporcionan puntos de partida concretos para intervenciones.

Sin embargo, al ser un caso simulado, estos resultados requieren validación empírica en equipos reales.

6.2 Limitaciones del trabajo

Esta investigación presenta las siguientes limitaciones que deben considerarse al interpretar los resultados:

1. **Ausencia de validación empírica:** SocialScrum no ha sido implementado en equipos de desarrollo reales. Las conclusiones sobre viabilidad se basan en análisis teórico y simulación.
2. **Dependencia del contexto:** El marco asume equipos con cultura organizacional mínimamente abierta al cambio. En contextos altamente jerárquicos o resistentes, la adopción podría ser inviable.
3. **Rol del Social Guide:** La efectividad del marco depende de encontrar personas con el perfil híbrido requerido, lo cual puede ser difícil en mercados laborales específicos.
4. **Instrumentos de medición:** Las escalas de Health Score no han sido validadas psicométricamente (validez de constructo, confiabilidad test-retest).
5. **Sesgo cultural:** El marco se desarrolló desde una perspectiva latinoamericana y podría requerir adaptaciones para otros contextos culturales.

6.3 Recomendaciones

6.3.1 Para organizaciones que consideren adoptar SocialScrum

1. **Evaluar madurez organizacional:** SocialScrum requiere apertura a discutir temas sociales. Organizaciones con cultura de “solo resultados” pueden necesitar trabajo previo de cambio cultural.
2. **Comenzar con piloto pequeño:** Implementar en un equipo dispuesto durante 3-6 Sprints antes de escalar.
3. **Invertir en el Social Guide:** No asignar el rol a alguien sin formación. La capacitación de 40 horas propuesta es un mínimo.
4. **Proteger la confidencialidad:** Establecer desde el inicio la separación entre datos del Social Guide y evaluaciones de HR.
5. **Medir y ajustar:** Usar el Health Score como guía, no como objetivo. Adaptar el marco al contexto específico.

6.3.2 Para investigadores

1. **Validar empíricamente:** Realizar estudios de caso múltiples con equipos reales para evaluar efectividad.
2. **Desarrollar instrumentos validados:** Construir y validar psicométricamente escalas de Health Score y detección de smells.
3. **Estudiar factores contextuales:** Investigar cómo varía la efectividad de SocialScrum según tamaño de equipo, industria, cultura nacional, etc.
4. **Comparar con alternativas:** Evaluar SocialScrum contra otras intervenciones de salud organizacional (coaching, team building tradicional, etc.).

6.4 Trabajos futuros

A partir de esta investigación, se identifican las siguientes líneas de trabajo futuro:

1. **Validación empírica:** Implementar SocialScrum en 3-5 equipos de desarrollo durante 6-12 meses, midiendo Health Score, métricas técnicas (velocity, defectos, rotación) y satisfacción del equipo.
2. **Desarrollo de herramientas:** Crear software de código abierto que automatice la recolección del Health Score, genere dashboards y facilite el análisis de SNA.

3. **Formación de Social Guides:** Diseñar y validar un programa de certificación para Social Guides, incluyendo currículo, evaluación de competencias y ética profesional.
4. **Integración con otros marcos:** Explorar la compatibilidad de SocialScrum con SAFe, LeSS y otros marcos de escalado ágil.
5. **Extensión a equipos remotos:** Adaptar los protocolos de observación y detección de smells para equipos completamente distribuidos.
6. **Automatización de detección:** Desarrollar algoritmos de machine learning que detecten *community smells* automáticamente a partir de datos de comunicación (Slack, GitHub, Jira).

6.5 Reflexión final

La deuda social en equipos de desarrollo de software es un problema real con consecuencias medibles. Los conflictos interpersonales, la falta de comunicación, el aislamiento y las dinámicas tóxicas no son “problemas de recursos humanos” desconectados de la ingeniería; son impedimentos que afectan directamente la capacidad del equipo de entregar software de calidad.

SocialScrum representa un intento de abordar este problema de manera sistemática, integrando la gestión de la deuda social en el flujo de trabajo ágil existente. No pretende ser la solución definitiva, sino un punto de partida para que equipos y organizaciones tomen en serio la salud social como factor de productividad y sostenibilidad.

El éxito de SocialScrum, como el de cualquier marco, dependerá de la voluntad genuina de las organizaciones de invertir en el bienestar de sus equipos, no como gesto simbólico, sino como estrategia de negocio. Si este trabajo contribuye a esa conversación, habrá cumplido su propósito.

Desvelando al Leviatán: Revisión sistemática de estrategias para la gestión la deuda social en el desarrollo de software ágil.

Abstract— This systematic literature review aims to identify, evaluate, and synthesize existing research on social debt in agile software development. The process includes formulating specific research questions, establishing inclusion and exclusion criteria, conducting a comprehensive search in academic databases, critically evaluating the quality of selected studies, and synthesizing the results using qualitative and quantitative techniques. The findings highlight the importance of adopting a holistic approach that promotes effective communication, work-life balance, and professional recognition to mitigate the negative effects of social debt. Despite limitations such as the lack of consensus on metrics and evaluation methods, this review provides a solid foundation for evidence-based decision-making and underscores the need for well-defined strategies for effective management of social debt in agile software development teams.

Keywords—social debt, agile software development, systematic literature review.

I. INTRODUCCIÓN

La realización de una revisión sistemática de la literatura (RSL) sobre la deuda social en el desarrollo de software ágil es una iniciativa crítica para comprender las complejas dinámicas que afectan a los equipos y proyectos de desarrollo de software actual. Esta revisión se justifica por la necesidad de abordar las consecuencias no técnicas que emergen de decisiones precipitadas y la adopción de procesos de desarrollo que, aunque eficientes a corto plazo, pueden generar repercusiones negativas en la cultura organizacional, la comunicación interna y el bienestar general del equipo. Es importante identificar y comprender la naturaleza y el impacto de la deuda social, ya que las organizaciones pueden desarrollar estrategias más efectivas para promover un ambiente de trabajo saludable y productivo. Esto incluye la implementación de prácticas de gestión que equilibren las demandas operativas con el bienestar de los empleados, fomentando así un clima de trabajo que no solo sea sostenible, sino también propicio para la innovación y la creatividad. La revisión sistemática permitirá a las organizaciones e investigadores diagnosticar la presencia y el impacto de la deuda social en sus proyectos. Además, promoverá un liderazgo que priorice la salud organizacional y el bienestar del equipo, fomentando la transparencia y la participación en la toma de decisiones. Esta revisión es un paso esencial hacia la creación de un entorno laboral que valore y respalde a sus empleados, lo que se traduce en beneficios tangibles para la organización en términos de eficiencia, calidad y competitividad en el mercado. Hay que reconocer que la salud organizacional y la gestión efectiva de la deuda social son fundamentales para el éxito a largo plazo es crucial. El propósito de esta revisión sistemática de la literatura es identificar, evaluar y sintetizar la investigación existente sobre la deuda social en el desarrollo de software ágil. La estructura aplicada incluye la formulación de preguntas de investigación específicas y bien definidas, el establecimiento de criterios de inclusión y exclusión para seleccionar estudios pertinentes,

una búsqueda exhaustiva en bases de datos académicas, la evaluación crítica de la calidad de los estudios elegidos, y la síntesis de los resultados utilizando técnicas cualitativas o cuantitativas. Este enfoque asegura que la revisión sea completa, transparente y replicable, proporcionando una base sólida para la toma de decisiones fundamentadas en la evidencia.

II. PREGUNTAS DE INVESTIGACIÓN

A. Definición de las preguntas de investigación

Para la ejecución de la revisión sistemática de la literatura (RSL), se establecieron inicialmente cuatro objetivos de búsqueda (OB) y nueve preguntas de investigación (PI) con el fin de dirigir de manera apropiada los esfuerzos de investigación. Sin embargo, es importante destacar que los objetivos y las preguntas experimentaron ajustes y modificaciones a lo largo del proceso, mediante un enfoque de evaluación que se detallará más adelante. Este proceso dinámico de refinamiento evidencia el compromiso constante con la mejora y adaptación del protocolo propuesto, permitiendo una alineación más precisa y menos subjetiva con los OB y la evolución de la investigación en curso.

Para establecer los objetivos de búsqueda se utilizaron dos herramientas: la primera es propuesta por Doran con el enfoque conocido como SMART [1], el cual está conformado por un conjunto de criterios para definir objetivos y metas los cuales son: específicos, medibles, alcanzables, relevantes y temporales (en inglés Specific – S, Measurable – M, Achievable – A, Relevant – R, Time based – T), lo que aumenta la probabilidad para que los objetivos sean cumplidos. La segunda herramienta utilizada es el enfoque propuesto por Basili y Rombach [2] conocido como Goal-Question-Metric (GQM), el cual permitió establecer una estructura que apoya en el cumplimiento, avance y trazabilidad de los objetivos e intereses de la investigación propuestos, el enfoque sugiere tres niveles, el establecimiento de metas, la definición de preguntas, y finalmente, el establecimiento de métricas para medir el avance como los resultados de algún proyecto, esta última parte no se consideró en esta primera versión del RSL, se contemplará para futuros proyectos.

En síntesis, los objetivos se definieron aplicando el marco de trabajo SMART y el enfoque GQM, cabe aclarar qué; según el enfoque GQM, se puede inferir que los objetivos cumplen con los criterios de calidad definidos, sin embargo, en el siguiente párrafo se puede observar los objetivos donde indica si un OB es incluido o excluido de la RSL.

Objetivo 1: Este objetivo está incluido e identifica fuentes de información e investigación fundamentales en la literatura existente, tales como revistas o conferencias revisadas por pares, así como establecer los límites desde una perspectiva demográfica para la gestión de la deuda social usando el enfoque de desarrollo de software ágil.

- S (Específicos): Se cumple este criterio debido a que define dos tareas específicas: Identificar fuentes y establecer los límites.
- M (Medibles): Se cumple este criterio debido a que se puede establecer un indicador de progreso, como por ejemplo el número de artículos relevantes.
- A (Alcanzables): Se cumple este criterio debido a que usa fuentes de información disponibles y existentes.
- R (Relevantes): Se cumple este criterio debido a que contribuye al desarrollo del objetivo general que es la gestión de la deuda social.
- T (Temporales): Se cumple este criterio debido a que, aunque no se establece un plazo fijo, implícitamente tiene un periodo de vida definido por su propósito.

Objetivo 2: Este objetivo está incluido y ayuda a los investigadores y a las partes interesadas a conocer aquellos trabajos que propongan una solución que establezca un conjunto de actividades para gestionar o mitigar todo el trabajo relacionado con la deuda social.

- S (Específicos): Se cumple este criterio debido a que la idea es ayudar a los interesados a conocer más sobre el tema.
- M (Medibles): Se cumple este criterio debido a que se puede establecer el número de interesados que conocen el tema.
- A (Alcanzables): Se cumple este criterio debido a que es alcanzable si se dispone de los recursos y el tiempo necesarios para identificar, recopilar y difundir los trabajos.
- R (Relevantes): Se cumple este criterio debido a que su objetivo es buscar, gestionar o mitigar la deuda social.
- T (Temporales): Se cumple este criterio debido a que, aunque no se establece un plazo fijo, implícitamente tiene un periodo de vida definido por su propósito.

Objetivo 3: Este objetivo está incluido y explora las tendencias relevantes en la investigación actual, trabajos en curso y trabajos futuros relacionados con la gestión de la deuda social en el desarrollo de software ágil.

- S (Específicos): Se cumple este criterio debido a que define una tarea específica cuyo fin es explorar las tendencias relevantes en la investigación actual.
- M (Medibles): Se cumple este criterio debido a que se puede establecer un indicador de progreso, como por ejemplo el número de tendencias relevantes.
- A (Alcanzables): Se cumple este criterio debido a que define un conjunto de fuentes en donde es posible encontrar la información relevante.
- R (Relevantes): Se cumple este criterio debido a que el encontrar las tendencias relevantes en la investigación actual da una perspectiva más amplia con respecto a la gestión de la deuda social.
- T (Temporales): Se cumple este criterio debido a que implícitamente se define un tiempo, ya que la

investigación actual tiene como fin un número de artículos finitos.

Objetivo 4: Este objetivo está excluido y evalúa el progreso de las propuestas en términos de los logros alcanzados, utilizando como indicadores de medición la cantidad de propuestas que han alcanzado sus objetivos y el impacto de las propuestas en el contexto en el que se aplican.

- S (Específicos): Se cumple este criterio debido a que la idea es evaluar el progreso de las propuestas en términos de los logros alcanzados.
- M (Medibles): Se cumple este criterio debido a que es medible en términos de logros alcanzados.
- A (Alcanzables): Se cumple este criterio debido a que es alcanzable si se dispone de los recursos y el tiempo necesarios para recopilar la información sobre los logros alcanzados por las propuestas.
- R (Relevantes): Se cumple este criterio debido a que es relevante dado que su objetivo es que se presenten propuestas para lograr los resultados esperados.
- T (Temporales): Se cumple este criterio debido a que el tiempo establecido es el que los autores de los artículos hayan tomado con el fin de lograr sus objetivos.

Es relevante destacar que, aunque no se estableció una herramienta de evaluación específica para los objetivos de búsqueda (OB), sí se definió una para las preguntas de investigación (PI). Dicho proceso se presenta con mayor detalle en la Figura 2, sin embargo, con la evaluación se obtuvo como resultado la exclusión de las 2 PI que conformaban el OB4 ya que no cumplieron con ninguno de los criterios establecidos en la herramienta de evaluación de la siguiente manera:

Características: Centrado e Investigable

Criterios iniciales: ¿Es centrado en un único tema? ¿Se puede responder utilizando fuentes creíbles? ¿No se basa en juicios de valor?

Criterios finales: ¿Trabaja junto al problema de investigación para cumplir el objetivo central? ¿Se puede responder utilizando datos reales y fuentes creíbles? ¿Evita emitir juicios personales usando expresiones subjetivas como “bueno” o “malo”?

Características: Factible y Específico

Criterios iniciales: ¿Se puede responder dentro de los límites prácticos? ¿Utiliza conceptos específicos y bien definidos? ¿No exige una solución, política o línea de actuación concluyente?

Criterios finales: ¿Se puede responder dentro de los límites prácticos? ¿Utiliza conceptos específicos y bien definidos? ¿Ayuda a comprender el tema en lugar de imponer una solución preestablecida?

Características: Complejo y Discutible

Criterios iniciales: ¿No se puede responder con sí o no? ¿No se puede responder con datos fáciles de encontrar?

Criterios finales: ¿Requiere una respuesta elaborada en lugar de una respuesta directa con sí o no? ¿Requiere

investigaciones extensas para obtener la información necesaria para responder?

Características: Relevante y Original

Criterios iniciales: ¿Aborda un problema relevante? ¿Contribuye a un debate social o académico oportuno? ¿La PI no ha sido resuelta en otros trabajos y/o por otros autores?

Criterios finales: ¿Aborda una problemática relevante sobre el tema central de la investigación? ¿Aporta conocimientos que puedan ser útiles para futuros debates en ese campo de investigación? ¿Contribuye de manera significativa al tema central de la investigación?

Dando como resultado 0 puntos de calificación para cada una de las PI, por consiguiente, aunque el objetivo se definió teniendo en cuenta los criterios del marco de trabajo SMART y el en-foque GQM se procedió a la exclusión de este ya que no terminó siendo lo suficientemente relevante para su replanteamiento, el flujo del marco de trabajo SMART y el enfoque GQM se puede observar en la Figura 1.

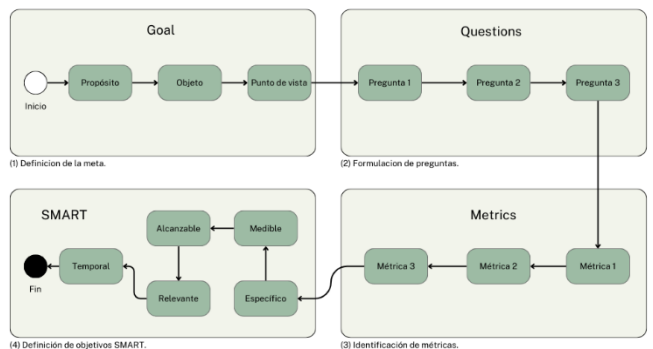


Fig. 1. Diagrama de flujo del marco de trabajo SMART y el enfoque GQM.

Este proceso de inclusión o exclusión se realizó con el fin de mantener una coherencia metodológica y garantizar que los objetivos seleccionados estén alineados de manera efectiva con el alcance y propósito general de la investigación.

Las preguntas de investigación fueron evaluadas llevando a cabo la adaptación del marco de trabajo definido por McCombes [3], el cual propone cuatro características específicas para evaluar la calidad de las preguntas de investigación, las características son: (i) centrado e investigable, (ii) factible y específico, (iii) complejo y discutible, y (iv) relevante y original, cada una de estas características está compuesta por criterios que garantizan su cumplimiento.

Si bien McCombes [3] establece un marco de trabajo específico, este no deja de ser un marco cualitativo, por consiguiente; se decidió acoplarlo con un sistema de calificación numérico y una categoría dependiendo de la calificación de la pregunta, por ejemplo: cuando una pregunta de investigación “SI” cumple completamente con un criterio, se califica con un valor de 1 (punto), en caso de que la pregunta “NO” cumpla en su totalidad con dicho criterio, se califica con un valor de 0 (puntos).

Cada una de las preguntas en total puede recibir como mínimo una puntuación de 0 (puntos) y como máximo una puntuación de 11 (puntos), solo aquellas preguntas que superen o igualen una calificación de 8 (puntos) son consideradas aceptables. La escala completa de este sistema de calificación se puede observar a continuación en la Tabla 1.

TABLA 1. PUNTAJE Y CRITERIOS DE LA PREGUNTA.

Puntaje	Descripción
0	Pregunta sin relevancia o claridad.
1	Pregunta mínimamente relevante o clara.
2	Pregunta con alguna relevancia, pero falta claridad.
3	Pregunta relevante y clara en parte.
4	Pregunta relevante y relativamente clara.
5	Pregunta clara y relevante, pero con margen de mejora.
6	Pregunta clara y bastante relevante.
7	Pregunta muy clara y relevante, pero con espacio para mejoras adicionales.
8	Pregunta altamente clara y muy relevante.
9	Pregunta excepcionalmente clara y altamente relevante.
10	Pregunta casi perfecta en claridad y relevancia.
11	Pregunta perfecta en claridad y relevancia.

Con base en la propuesta del marco de trabajo de McCombes [3] se creó un instrumento de evaluación mediante un formulario de Google Forms. Éste se compartió a cuatro expertos en la materia encargados de evaluar cada una de las nueve PI iniciales. En el formulario se proporcionó a los expertos información detallada sobre la naturaleza del instrumento, se explicaron las cuatro características existentes, y se describieron los objetivos de cada uno de los criterios que conformaban dichas características, además, se instruyó a los expertos acerca del sistema de calificación numérica. Los resultados obtenidos de este ejercicio resultaron altamente enriquecedores, ya que la valiosa retroalimentación proporcionada por los expertos contribuyó significativamente a mejorar el instrumento. Asimismo, además de recibir críticas positivas sobre el trabajo realizado, se observaron mejoras sustanciales en la definición de los criterios de calificación en todas las características. Aunque el enfoque de cada criterio no experimentó alteraciones, se modificó la redacción de estos. En algunos casos, los expertos tuvieron que volver a leer las definiciones para recordar el propósito de los criterios. Por esta razón, se tomó la decisión de redactarlos de manera autodescriptiva, esto con el objetivo de minimizar el riesgo de la mala interpretación de un criterio y, por ende, de una evaluación incorrecta de una PI. Este enfoque garantizó una mayor claridad y precisión en la evaluación de las PI.

La adaptación del marco de trabajo de McCombes [3] ayudó a definir preguntas de investigación más sólidas y menos subjetivas, además de excluir aquellas PI que no cumplieron con los criterios, sin embargo, no es 100% seguro que lo propuesto no sea subjetivo, pero someter el planteamiento a una autoevaluación constante, en nuestro caso, ayudó a identificar oportunidades de mejora y disminuir el sesgo en la investigación. El resultado del proceso

mencionado anteriormente se puede observar más adelante en la Figura 2.

En el siguiente párrafo se puede observar que, para los 4 objetivos de búsqueda propuestos, se definieron 9 preguntas de investigación que ayudan a dar cumplimiento a cada uno de los objetivos propuestos. Además, también destaca qué PI cumplieron con las características de calidad establecidas por el instrumento de evaluación definido. Como resultado de este proceso de evaluación, se logró la inclusión de un total de 5 PI, las cuales demostraron adherirse a los criterios de calidad previamente establecidos. Por otro lado, se identificaron 4 PI que fueron excluidas debido a no cumplir con las características mencionadas. Este análisis detallado y selectivo permite no solo una comprensión clara de las preguntas incluidas en la revisión, sino también una evaluación rigurosa de su calidad, asegurando así la robustez y confiabilidad de los resultados obtenidos.

Objetivo 1:

- PI1 (Incluido): ¿Cuáles son los estudios primarios más citados que han proporcionado según la evidencia, aportes importantes e influyentes en el contexto de la gestión de la deuda social en el desarrollo de software ágil?
- PI2 (Excluido): ¿Cuáles son las fuentes de publicación más recurrentes de los estudios primarios identificados en las bases de datos seleccionadas previamente?
- PI3 (Incluido): ¿Cuál es la distribución de tiempo de publicación de los estudios primarios relevantes con respecto a la deuda social en el desarrollo de software ágil?

Objetivo 2:

- PI4 (Incluido): ¿Cuáles son las investigaciones o propuestas desarrolladas cuya idea central sea la gestión de la deuda social en el desarrollo de software ágil hasta la fecha?
- PI5 (Excluido): ¿Cuál es la calidad con base en la rigurosidad metodológica y presentación de resultados de los estudios seleccionados?

Objetivo 3:

- PI6 (Incluido): ¿Cuáles son las prácticas o enfoques que abordan de manera exitosa la gestión de la deuda social en el desarrollo de software ágil?
- PI7 (Incluido): ¿Cuáles son los principales desafíos y oportunidades para la investigación futura sobre la deuda social en el desarrollo de software ágil?

Objetivo 4:

- PI8 (Excluido): ¿Existen diferencias en el éxito de las propuestas según su área temática o contexto de aplicación?
- PI9 (Excluido): ¿Cuáles son las mejores prácticas para el desarrollo de propuestas exitosas?

Este nuevo marco facilita la autocrítica al analizar las razones para incluir o excluir preguntas de investigación. Para garantizar transparencia, se compartieron abiertamente las justificaciones de la exclusión de las PI 2, 5, 8 y 9. Esta medida

garantiza un enfoque abierto y claro, permitiendo a los interesados comprender de manera completa y detallada el razonamiento detrás de cada decisión de inclusión o exclusión. Este nivel de transparencia fortalece la credibilidad del proceso de revisión y asegura una evaluación crítica y fundamentada de cada pregunta de investigación involucrada en el estudio.

- PI2: Si bien la pregunta aporta significativamente desde el punto de vista demográfico, la categorización de las fuentes de publicación en una línea de tiempo específica no genera un debate sustancial ni una relevancia del tema. Esto se debe a que, con el tiempo, los resultados podrían verse afectados por la cantidad de artículos que se publiquen a partir de la fecha actual. Esta observación resalta la importancia de considerar la dinámica cambiante del campo de investigación y la necesidad de adoptar enfoques que permitan una adaptabilidad continua a medida que evolucionan las tendencias y la producción de nuevas investigaciones.
- PI5: Aunque se han definido características específicas que establecen el tipo de calidad que se desea obtener, esta pregunta de investigación no deja de ser subjetiva, dado que se estableció el nivel de calidad deseado. Sin embargo, es importante reconocer que se debe tener un estándar de calidad definido, un estándar no solo refleja la confiabilidad de las fuentes de información utilizadas en la investigación, sino que también asegura la relevancia y solidez de los resultados obtenidos. Debido a esto, nuestra investigación implementa una herramienta para evaluar la calidad de los artículos, como se detalla más adelante en el artículo. Esta herramienta de evaluación actúa como un respaldo fundamental para asegurar el logro del objetivo de la pregunta de investigación de manera implícita, proporcionando un marco objetivo y estructurado para la evaluación de la calidad de la información recopilada.
- PI8 y PI9: Ambas PI son subjetivas ya que se centran en evaluar el éxito de sus resultados, además de ser situacionales. Para responder a estas preguntas, se requiere un entorno y aplicabilidad específicos en un área y condiciones determinadas, lo que limita su utilidad y las hace menos relevantes con relación del objetivo general de la investigación. En consecuencia, como resultado de la exclusión de ambas PI también se concluyó que el OB4 perdió su relevancia, lo que ocasionó la exclusión de este.

En la Figura 2 se puede observar el resultado de aplicar el marco de trabajo con el sistema de evaluación cuantificable a las preguntas de investigación, cabe resaltar que en esta figura cuadros con un check (☑) dentro significan que la PI “SI” cumple con el criterio y por consiguiente recibirá 1 punto, y en caso de que el cuadro tenga una equis (☒) quiere decir que la PI “NO” cumple con el criterio por ende recibirá 0 puntos.

ID	OB-P1	OB1-P11	OB1-P12	OB1-P13	OB2-P14	OB2-P15	OB3-P16	OB3-P17	OB4-P18	OB4-P19
Contrada e Investigable	¿Trabaja junto al problema de investigación para cumplir el objetivo central?	✓	✓	✓	✓	✓	✓	✓	✗	✗
	¿Se puede responder utilizando datos reales y fuentes creíbles?	✓	✓	✓	✓	✓	✓	✓	✗	✗
	¿Evita emitir juicios personales usando expresiones subjetivas como "bueno" o "malo"?	✓	✓	✓	✓	✗	✓	✓	✗	✗
Factible y Específico	¿Se puede responder dentro de los límites prácticos?	✓	✓	✓	✓	✓	✓	✓	✗	✗
	¿Utiliza conceptos específicos y bien definidos?	✓	✓	✓	✓	✗	✓	✓	✗	✗
	¿Ayuda a comprender el tema en lugar de imponer una solución preestablecida?	✓	✓	✓	✓	✗	✓	✓	✗	✗
Complejo y Discutible	¿Requiere una respuesta elaborada en lugar de una respuesta directa con sí o no?	✓	✓	✓	✓	✓	✓	✓	✗	✗
	¿Requiere investigaciones extensas para obtener la información necesaria para responder?	✓	✗	✗	✓	✓	✓	✓	✗	✗
Relevante y Original	¿Aborda una problemática relevante sobre el tema central de la investigación?	✓	✗	✓	✗	✓	✓	✓	✗	✗
	¿Aporta conocimientos que pueden ser útiles para futuros debates en ese campo de investigación?	✓	✗	✓	✗	✗	✓	✓	✗	✗
	¿Contribuye de manera significativa al tema central de la investigación?	✓	✗	✗	✓	✓	✓	✗	✗	✗
Resultado	Puntaje Total	11	7	9	9	7	11	10	0	0

Fig. 2. Preguntas de investigación.

B. Ejecucion de la cadena de busqueda

Como investigación inicial para obtener una primera aproximación a los conceptos y términos abordados en la gestión de la deuda social, así como los problemas y retos a los que se enfrentan, se utilizó el enfoque de la literatura gris, también llamada literatura no convencional, el cual es un término que se refiere a documentos que no se publican de manera tradicional. Estos documentos pueden ser informes de investigación, tesis, patentes, entre otros, y pueden ser difíciles de encontrar [4].

1) *Búsqueda manual, términos y frases clave:* Para llevar a cabo la búsqueda de artículos, se usaron herramientas tecnológicas desarrolladas con inteligencia artificial denominadas chatbots con IA (CIA) como lo son: ChatGPT y Copilot, esto se llevó a cabo con el fin de obtener diferentes formas de definir un mismo concepto. En ocasiones, los temas tratados en estos documentos no se presentan de manera centrada y concisa, sino más bien; se conforman por definiciones escuetas y poco claras, lo que genera sesgos y dudas que obstaculizan la comprensión y buen entendimiento del tema principal de la investigación. Es aquí donde los CIA se convierten en recursos valiosos, ya que se pueden emplear cada uno de ellos con contextos adecuados y preguntas específicas iniciales denominadas prompts; que se deben proporcionar al CIA para iniciar una conversación o solicitud de información, dando como resultado diferentes escenarios que ayudaron a ser más críticos y analíticos al momento de definir conceptos importantes, tales como: la deuda social en la Ingeniería de Software, los desafíos con respecto a la deuda social, la gestión de la deuda social en equipos de desarrollo de software ágil, congruencia sociotécnica en los equipos de desarrollo de software, entre otros. Estos CIA resultaron ser herramientas bastante prácticas y esenciales en el proceso de comprensión, contextualización y análisis de la investigación inicial.

A partir de los términos identificados en la investigación realizada con ayuda de las CIA, se definió una primera versión de cadena de búsqueda básica: “social debt” OR “social software engineering” AND “agile software development” OR “Software development teams”. Luego, se estableció la base de datos (BD) Google Scholar como fuente principal de información, ya que es un motor de búsqueda de literatura académica que permite a los usuarios buscar una

amplia variedad de disciplinas y fuentes, así como: artículos, tesis, libros, resúmenes, entre otros. Aunque Google Scholar no indexa a otros buscadores, sí indexa bibliotecas, bases de datos bibliográficas o editoriales, entre otros documentos. Además, Google Scholar es una de las bases de datos más grandes y completas de literatura académica, con millones de artículos indexados, lo que la convierte en una fuente de información relevante y confiable para dar respuesta a los objetivos principales de la investigación. Asimismo, Google Scholar es compatible con la herramienta Publish or Perish (PoP), la cual permite listar y filtrar artículos de manera más fácil, así como también brinda métricas de citas, por ejemplo, número de artículos, total de citas y el índice h (un indicador que evalúa la producción científica de un investigador). PoP, al ser un software gratuito que recupera y analiza citas académicas de diversas fuentes de datos [5], ayudó en esta revisión de la literatura en primera instancia a listar los artículos encontrados, aplicar de manera más sencilla los criterios de inclusión y exclusión, y obtener las referencias en formatos bibliográficos, optimizando así el tiempo y el esfuerzo en estas actividades.

Una vez se estableció la BD, con la cadena de búsqueda básica definida, se procedió a realizar pruebas en Google Scholar para analizar qué tipo de artículos resultaban, y un primer vistazo de los posibles artículos encontrados. Una vez identificados los artículos potencialmente relevantes, se procedió a realizar las lecturas detenidas de títulos y resúmenes de dichos artículos con el fin de conocer los términos y definiciones recurrentes que relacionan directa o indirectamente la deuda social en el desarrollo de software ágil. Como resultado de estas pruebas, se identificaron otros términos como: “social challenges” y “ASD” (en inglés: Agile Software Development), que inicialmente se tomaron como potencialmente relevantes, sin embargo, fueron descartados, debido a distintas justificaciones como se muestra en el siguiente párrafo.

El término “deuda social” es el más importante para la búsqueda de artículos relevantes para la RSL, debido a que se refiere específicamente a los problemas sociales que pueden surgir del desarrollo de software ágil por lo tanto está incluida.

El termino “social software engineering” está excluida dado que según la evidencia, este término está más relacionado con la interacción humano-computador (HCI) que con la gestión de la deuda social.

El termino “agile software development” está incluido dado que este término agrupa temas de relevancia como lo son el desarrollo de software ágil y el uso de metodologías ágiles en equipos de desarrollo de software, por lo que el término es relevante para el objetivo de la investigación.

El termino “software development teams” está excluido dado que los artículos encontrados solo son potencialmente relevantes cuando se cita en conjunto con “social debt”, por lo que el término por sí solo puede generar ruido.

El termino “social challenges” esta excluido dado que existen una escasa cantidad de artículos que podrían ser relevantes y son específicamente los que citan en conjunto el término “social debt”, pero en su mayoría los artículos encontrados utilizan el término para referirse a una amplia gama de

problemas sociales que no se dan específicamente a un entorno de desarrollo de software ágil. Otro ejemplo de esto es el término “social impact”.

El termino “ASD” esta excluido dado que el acrónimo “ASD” es usado para referirse a “agile software development”, pero no es exclusivo de este término. Encontramos que “ASD” hace referencia a muchos otros términos que en su mayoría no pertenecen ni siquiera al campo de la ingeniería, por esta razón se decidió que genera más ruido que aportes a la investigación.

Para evaluar la relevancia de estos términos, se realizó un análisis del contexto y del enfoque en el que se usaban en los diferentes artículos encontrados, así como el tipo de relación que tenían entre sí. Este análisis permitió identificar los términos que realmente abordaban el tema de investigación de manera relevante, aquellos términos que no se consideraron relevantes, fueron excluidos de la cadena de búsqueda, y como resultado de este proceso se obtuvo la depuración de cada uno de las términos y frases clave encontradas, resultando así la cadena de búsqueda básica presentada en la siguiente sección.

a) *Búsqueda automática:* La búsqueda manual permitió identificar que las frases clave: “social debt” y “agile software development” eran relevantes para la definición de la cadena de búsqueda básica, además, se optó por utilizar el conector lógico “AND” para unir estas dos frases, ya que los artículos encontrados que eran relevantes para la investigación hacían referencia a ellas de manera conjunta.

Por lo tanto, la cadena de búsqueda: “social debt” AND “agile software development” se aplicó a seis bases de datos científicas: Google Scholar, IEEE Xplore, SpringerLink, Scopus, Science Direct y Web Of Science. Además, se tuvieron en cuenta los siguientes criterios o filtros de búsqueda: (i) artículos en inglés, (ii) artículos exclusivos en áreas de conocimiento como: Computer Science, Software Engineering y Software Development, y (iii) artículos de revistas o actas de eventos científicos. Asimismo, se consideró todo estudio dentro de la temática publicado entre los años 2013 y 2024, dado que el concepto de deuda social relacionado con el desarrollo de software y su gestión fue definido por primera vez en 2013 en el trabajo realizado por Tamburri et al. [6].

Por otro lado, es importante aclarar que la cadena de búsqueda básica no sirvió para todas las bases de datos, en la mayoría de los casos esta cadena debió ser modificada de acuerdo con las especificaciones y configuraciones de cada BD. Para la ejecución de dicho proceso y con el objetivo de reducir la subjetividad en el trabajo, el primer autor adaptó y ejecutó la cadena de búsqueda básica, mientras que el segundo autor verificó la ejecución de la búsqueda y los resultados obtenidos, resultado de este proceso se obtuvieron 171 artículos clasificados como encontrados.

Sin embargo, una vez se analizaron los resultados, se concluyó que los artículos encontrados en algunas base de datos eran muy pocos y ya se encontraban listados en la base de datos principal, en nuestro caso: Google Scholar, por lo cual se tomó la decisión de excluir las base de datos que no arrojaron artículos relevantes y nuevos, dejando como bases de datos principales las siguientes: Google Scholar, IEEE Xplore, Science Direct y SpringerLink (ver en el siguiente

párrafo), dando como resultado un total de 160 artículos encontrados. En este sentido, las bases de datos: Scopus y Web of Science fueron excluidas.

- “social debt” AND “agile software development” - Fuente: Google Scholar - Estado: Incluida.
- (“Full Text Only”:social debt) AND (“Full Text Only”:agile software development) - Fuente: IEEE Xplore - Estado: Incluida.
- “social debt” AND “agile software development” AND (LIMIT-TO(SUBJAREA, “COMP”) OR LIMIT-TO(SUBJAREA, “ENGI”)) AND (LIMIT-TO(DOCTYPE, “ar”) OR LIMIT-TO(DOCTYPE, “re”)) AND (LIMIT-TO(LANGUAGE, “English”)) AND (LIMIT-TO(OA, “all”)) - Fuente: Scopus - Estado: Excluida.
- “social debt” AND “agile software development” - Fuente: SpringerLink - Estado: Incluida.
- “social debt” AND “agile software development” - Fuente: Science Direct - Estado: Incluida.
- ALL=(“social debt” AND “agile software development”) - Fuente: Web Of Science - Estado: Excluida.

C. Selección de artículos primarios

La Figura 3 presenta el proceso de selección de los artículos primarios que se llevó a cabo, este proceso se realizó en cuatro etapas, el cual permitió realizar una selección de artículos primarios de manera ordenada y eficiente. En la primera etapa se organizó la literatura inicial, la cual fue el resultado de las búsquedas en las bases de datos seleccionadas aplicando los filtros de búsqueda, en la segunda etapa, se identificaron los estudios primarios que cumplían con al menos uno de los criterios de inclusión (CI) presentados en la Tabla 2. Por otro lado, en la tercera etapa se procedió a identificar y excluir todos los estudios repetidos, y finalmente; la cuarta etapa permitió excluir los estudios que cumplían con alguno de los criterios de exclusión (CE) presentados en la Tabla 2.

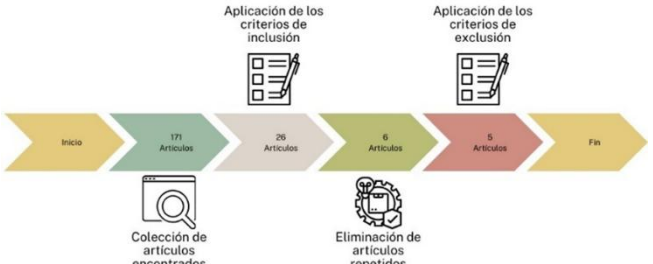


Fig. 3. Proceso de selección de artículos primarios.

En la Tabla 2 se puede observar el listado de los criterios de inclusión utilizados para identificar los estudios primarios que cumplieron con los lineamientos establecidos para la investigación.

TABLA 2. CRITERIOS DE INCLUSIÓN.

Id	Criterio de inclusión (CI)
CI1	Aquellos artículos publicados entre 2013 y 2024.
CI2	Aquellos artículos que hayan sido escritos en inglés.
CI3	Aquellos artículos publicados en revistas o conferencias revisados por pares.
CI4	Aquellos artículos que en su tema principal proponga una solución para gestionar la deuda social.
CI5	Aquellos artículos que analizan las consecuencias de la deuda social y que sugieren o proponen una solución.

En la Tabla 3 se puede observar el listado de los criterios de exclusión utilizados para identificar los estudios que no cumplieron con los lineamientos establecidos para la investigación.

TABLA 3. CRITERIO DE EXCLUSIÓN.

Id	Criterio de exclusión (CE)
CE1	Aquellos artículos que aborden el tema principal de investigación de manera superficial.
CE2	Aquellos artículos que aparecen duplicados en la búsqueda inicial de Google Scholar
CE3	Aquellos artículos que sean libros o capítulos de libros
CE4	Aquellos artículos con acceso restringido que requieran pagos externos o vinculaciones con terceros.
CE5	Tomaremos como base aquellos artículos encontrados inicialmente en Google Scholar, a partir de esta selección inicial, excluirémos cualquier artículo que aparezca repetido en otras bases de datos.

Acrónimos utilizados: Identificador (Id), Criterio de exclusión (CE).

Considerando el número de artículos primarios resultantes, se optó por utilizar la estrategia definida por Jalali, S., y Wohlin, C. [7] llamada: Snowballing Backward, también conocida como: “revisión bibliográfica retrospectiva”. Esta estrategia implica comenzar con documentos relevantes y buscar las referencias citadas en ellos para descubrir otros trabajos pertinentes. De esta manera, se pueden encontrar investigaciones que podrían haber sido pasadas por alto en búsquedas convencionales, en la respuesta de la P11 se hace énfasis en esta estrategia y que resultados se obtuvieron.

A diferencia de su contraparte, Snowballing Forward, que también busca identificar estudios relevantes, pero en lugar de revisar las citas de los documentos seleccionados, se enfoca en examinar los artículos que citan a estos documentos. Este método permite descubrir investigaciones más recientes que han construido sobre los estudios iniciales, proporcionando una perspectiva actualizada y ampliada del tema de investigación, pero debido a que las investigaciones del tema de investigación no son cuantiosas ni recientes es su totalidad se optó por Snowballing Backward por su eficacia para detectar investigaciones relevantes que podrían haberse pasado por alto en búsquedas convencionales. Sin embargo, se decidió no utilizar la técnica de Snowballing Forward, ya que esta puede generar un gran número de artículos no directamente relevantes para las preguntas de investigación. Además, esta estrategia puede desviar el enfoque de los estudios relevantes al incluir trabajos más recientes que aún no han sido suficientemente validados por la comunidad académica. Por lo tanto, se optó por centrarse en la revisión retrospectiva, lo que permite un mejor control sobre la relevancia y calidad de los estudios incluidos en esta revisión.

D. Evaluación de la pertinencia y relevancia de los estudios primarios

Para evaluar la pertinencia y relevancia de los artículos primarios, se decidió utilizar las categorías propuestas por Dybå y Dingsøyr [8], resultando en un modelo de evaluación conformado por cinco categorías: claridad, credibilidad, rigor, relevancia y aplicabilidad, en la Tabla 4 se puede observar las categorías y sus descripciones.

TABLA 4. DEFINICIÓN DE LOS CRITERIOS DE LA PERTINENCIA Y RELEVANCIA.

No.	Categoría	Descripción
1	Claridad	Esta categoría permite entender cuán relacionado está cada artículo primario con la deuda social en el desarrollo de software ágil, destacando claramente el contexto y el propósito de la investigación.
2	Credibilidad	Esta categoría permite determinar el grado de significancia, validez y eficacia de los resultados de la investigación mediante la evaluación de los métodos científicos utilizados, la discusión de las limitaciones del proceso de investigación y el análisis de los resultados obtenidos [9].
3	Rigor	Esta categoría ayuda a comprender el grado de confiabilidad de los hallazgos mediante la evaluación de los métodos de investigación, las herramientas de recolección de datos y los métodos de análisis empleados.
4	Relevancia	Esta categoría ayuda a determinar el nivel de importancia e impacto de los hallazgos en dos ámbitos: (i) la industria del software, mediante la aplicación de las propuestas, y (ii) el ámbito académico, al ofrecer la posibilidad de impulsar futuras investigaciones similares o relacionadas [10].
5	Aplicabilidad	Esta categoría se enfoca en evaluar cómo las soluciones propuestas en una investigación pueden ser aplicadas en la práctica para abordar la deuda social en el desarrollo de software ágil. Esto implica considerar si se ofrecen ejemplos claros de implementación, su impacto en la eficiencia, calidad y satisfacción de las personas en el desarrollo de software, y si se exploran las barreras para una implementación efectiva.

Acrónimos utilizados: Identificador (No.)

Como sistema de calificación, se estableció que cada criterio tuviera una escala de tres puntos, con una puntuación de 1 punto para “Sí”, 0.5 puntos para “Parcialmente”, y 0 puntos para “No”. En este sentido, cada artículo primario puede obtener una puntuación total entre 0 y 15 puntos, que se utilizan para determinar su nivel de pertinencia y relevancia para la investigación sobre la deuda de social en el desarrollo de software ágil, este modelo se puede observar en la Tabla 5.

TABLA 5. Criterios de evaluación.

Criterios de evaluación	Categoría
¿El artículo aborda el fenómeno de la deuda social en el desarrollo de software ágil?	Claridad
¿El artículo proporciona una descripción clara y concisa del problema y los objetivos de la investigación?	
¿El artículo sigue un proceso de investigación definido y apoyado en referentes?	Rigor
¿El artículo ofrece una definición clara de la deuda social?	
¿El artículo propone características que puedan ser útiles para evaluar la deuda social en el desarrollo de software ágil?	
¿El artículo propone un método, forma o herramienta para evaluar la deuda social en el desarrollo de software ágil?	
¿El artículo presenta una evaluación de su propuesta, explicando clara y detalladamente los resultados obtenidos?	Relevancia
¿El artículo describe claramente la importancia de su propuesta para la	

industria y/o para el mundo académico sobre la Deuda social en el desarrollo de software ágil?	
¿El artículo sugiere claramente futuros trabajos o alternativas de investigación sobre la deuda social en el desarrollo de software ágil?	
¿El artículo proporciona una descripción detallada de los métodos utilizados?	Credibilidad
¿El artículo analiza críticamente los resultados obtenidos para llegar a conclusiones sólidas?	
¿El artículo explica claramente cómo los sesgos podrían influir en los resultados?	
¿El artículo proporciona ejemplos claros de cómo se podrían aplicar los hallazgos de la investigación en el contexto de la deuda social en el desarrollo de software ágil?	Aplicabilidad
¿El artículo evalúa cómo las soluciones propuestas impactan la eficiencia, calidad y satisfacción de las personas en el desarrollo de software ágil con relación a la deuda social?	
¿Se exploran desafíos o barreras que podrían obstaculizar la implementación efectiva de las soluciones propuestas en los artículos seleccionados?	

III. RESPUESTAS A PREGUNTAS DE INVESTIGACIÓN

La investigación nos ha proporcionado aportes importantes e influyentes en el contexto de la gestión de la deuda social en el desarrollo de software ágil, donde se incluyen investigaciones sobre: habilidades blandas en los equipos de desarrollo de software, los roles comunitarios y estrategias de mitigación de la deuda social. Estos estudios se centran en identificar y establecer la definición de la deuda social y cuál es su impacto en el desarrollo de software ágil.

A continuación, se presentan los resultados obtenidos para cada una de las preguntas de investigación establecidas en esta revisión sistemática. Cada estudio se acompaña de su correspondiente referencia, facilitando a los lectores interesados en explorar más a fondo el tema.

PI1: ¿Cuáles son los estudios primarios más citados que han proporcionado según la evidencia aportes importantes e influyentes en el contexto de la gestión de la deuda social en el desarrollo de software ágil?

Como un Leviatán, la deuda social ha venido asechando los grupos de desarrollo de software a lo largo de los años como un mal que a menudo es invisible pero inevitable, y que va creciendo progresivamente hasta el punto de ser tan colosal que se vuelve imposible ignorarlo, y que al final se refleja en un daño colateral que afecta toda una estructura jerárquica de desarrollo, causando costos y deudas en todos los aspectos incluso socialmente. La deuda social afecta uno de los pilares fundamentales en una organización o empresa de desarrollo de software; el talento humano, las afectaciones producidas se generan sin importar el cargo o función que desempeña una persona, y se evidencia principalmente en su rendimiento y calidad de trabajo, debido a que éstos dependen de un estado de paz y armonía no solo mental y físico, sino también de un ambiente laboral armonioso y motivador con tareas justas y acopladas a las capacidades de cada profesional, una mala gestión de estos factores son la causa

principal de: la alta rotación de personal, mala toma de decisiones y la mala calidad de trabajo entregado, entre otros. En este sentido, es importante profundizar en el desarrollo de proyectos en esta temática, sin embargo, se evidencia poca investigación con respecto a la gestión de la deuda social en el desarrollo de software ágil. Los resultados presentados a continuación, se presentan con el objetivo de darles visibilidad a los trabajos en esta temática, y de esta manera; develar al leviatán a la que se ve enfrentada, la industria de software.

En el siguiente párrafo se presentan los estudios primarios seleccionados, donde se puede observar su título, el número de citas, año de publicación, fuente de publicación, motor de búsqueda utilizado para encontrar el artículo y el tipo de búsqueda utilizado (cadena de búsqueda o Snowballing). Además, se puede observar que no todos los artículos primarios tienen una gran cantidad de citas, esto se debe en gran parte al año de publicación, es decir; que los artículos más citados son aquellos que fueron publicados hace 10 años, y los artículos menos citados son aquellos publicados en los últimos 4 años. Cabe resaltar que; aunque la gestión de la deuda social es un tema que surgió por primera vez en el año 2013, los estudios que sugieren una metodología o estrategia para gestionar o mitigar la deuda social son muy pocos, y no son tan visibles como otros tipos de deudas que siguen siendo los más estudiados y abordados por la comunidad científica, por ejemplo: la deuda técnica y de arquitectura. En la columna: “Fuente de publicación”, se puede observar que existen diferentes fuentes, siendo la más relevante IEEE xplora, con 2 de los artículos más citados (artículo 1 y 2). Por otra parte, y basándonos en la evidencia, es posible declarar que Google Scholar es el motor de búsqueda que más aportes arroja en la investigación. También se puede evidenciar que; si bien es importante definir la cadena de búsqueda más apropiada, se debe establecer una basada en iteraciones, el uso de estrategias como la búsqueda por snowballing es un enfoque complementario que permite identificar artículos potencialmente relevantes, dado que con su aplicación se pudo identificar dos de los artículos base más importantes de la investigación.

El primero, titulado “What is social debt in software engineering?”, fue publicado en 2013 en IEEE Xplore, con 117 citas y utilizando la búsqueda Snowballing Backward en Google Scholar. El segundo, “Beyond technical aspects: How do community smells influence the intensity of code smells?”, también publicado en IEEE Xplore en 2018, obtuvo 120 citas y utilizó el mismo tipo de búsqueda. El tercer artículo, “The Second Vice is Lying, the First is Running into Debt.” Antecedents and Mitigating Practices of Social Debt: an Exploratory, se publicó en 2021 en Scholar Space con solo 6 citas, utilizando una cadena de búsqueda en Google Scholar. En 2023, se publicó Community smells—The sources of social debt: A systematic literature review en Sciencedirect, con 9 citas, también basado en una cadena de búsqueda. Finalmente, en 2024 se presentó A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt en E-Informatica software engineering journal, sin citas hasta la fecha, con una búsqueda de cadena en Google Scholar.

PI2: ¿Cuál es la distribución de tiempo de publicación de los estudios primarios relevantes con respecto a la deuda social en el desarrollo de software ágil?

Según Espinosa et al. [11], el término deuda social se acuñó por primera vez hace más de 50 años con un enfoque económico y con el fin de explicar las obligaciones sociales que involucran un reembolso de favores intercambiados entre las personas participantes en una transacción, es decir; acreedores y deudores, dicho en otras palabras; cuanto mayor sea el número de relaciones tensas, mayor es la deuda social. Sin embargo, el concepto de deuda social en el desarrollo de software ágil es tema de conversación sólo desde el año 2013 [6], dato curioso es que en la última década los estudios que tratan el tema de una manera más profunda no superan los 21 artículos, una cifra preocupante para un problema tan recurrente en los equipos de desarrollo, sin mencionar que solo 5 de ellos plantean o mencionan una solución o estrategia para gestionar o mitigar la deuda social.

La Figura 4 presenta la cantidad de artículos relevantes (barras negras) y no relevantes representadas (con barras grises) publicados por año en la última década.

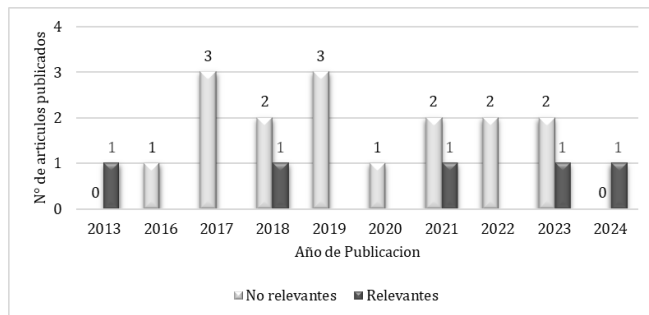


Fig. 4. Artículos relevantes y no relevantes publicados en la última década.

Esta RSL se enfoca únicamente en los artículos primarios relevantes y analiza estudios publicados entre 2013 y 2024. Es importante aclarar que al examinar la base de datos de Google Scholar se evidenció un incremento en la producción de artículos relacionados con la deuda social en el desarrollo de software ágil a partir del 2018 en adelante. La Figura 5 presenta la dispersión y distribución de los artículos primarios seleccionados durante el período 2013 hasta 2024, ambos años inclusive.

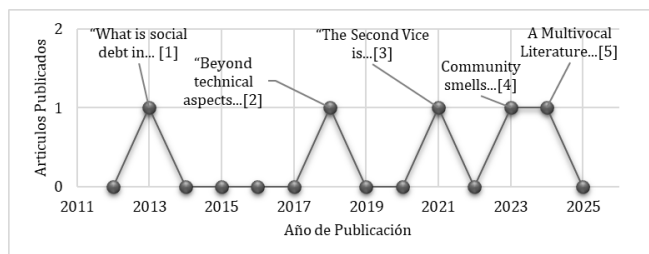


Fig. 5. Intervalo de tiempo de publicación de los artículos primarios.

Con base en la Figura 3 se observa que en el año 2013 se publicó el primer artículo relevante con respecto a la deuda social en el desarrollo de software titulado: "What is social debt in software engineering?", el cual establece el tema de la deuda social como una problemática que debe ser identificada y gestionada [6]. Este artículo es un referente ya

que estableció lo que conocemos como los cimientos que sirvieron para las nuevas definiciones de conceptos y perspectivas sobre deuda social. Por otro lado, se puede observar que hay un período de 5 años en el cual no se evidencia ningún artículo relevante sobre el tema en las fuentes de datos consultadas. En el año 2018 se publicó el artículo: "Beyond technical aspects: How do community smells influence the intensity of code smells?" [12], donde se abrió una nueva ventana de investigación con relación a la deuda social, ya que en este artículo se define uno de los principales conceptos para categorizarla: los "community smells", también conocidos en español como: "olores comunitarios", donde Tamburri et al. [13] la definen como "conjuntos de circunstancias organizativas y sociales que tienen relaciones causales implícitas". Los olores comunitarios producen principalmente incomodidad entre los individuos, estrés en las interacciones sociales, afectan el desempeño del equipo y disminuyen la calidad del software. Por lo tanto, mientras no se gestionen, los olores comunitarios inciden en un costo adicional de los proyectos software. Después del 2018 la continuidad en las publicaciones de artículos relevantes ha sido fluctuante, pero de cierta manera constante, variando entre 1 y 2 años de distancia entre publicaciones. Por último, se observa que desde el 2021 hasta la fecha de consulta de esta revisión sistemática, se identificaron 3 artículos ([6], [14] y [15]) relevantes que tienen 15 citas en total hasta la fecha, esto quiere decir que; si bien existen nuevas formas y perspectivas para abordar esta temática de investigación, los investigadores siguen optando por los artículos 1 y 2 como referencia a pesar de tener una distancia de tiempo de 11 y 6 años, respectivamente.

Finalmente, en términos de porcentaje, de los 21 artículos publicados en la última década (2013-2024), solo el 23.8% de los artículos son relevantes para el estudio, ya que son los únicos que plantean una solución basada en estrategias para mejorar la comunicación entre los individuos afectados dentro del equipo de desarrollo, así como la identificación y clasificación de olores comunitarios relevantes que afectan el rendimiento de los equipos de desarrollo.

PI3: ¿Cuáles son las investigaciones o propuestas desarrolladas cuya idea central sea la gestión de la deuda social en el desarrollo de software ágil hasta la fecha?

Teniendo en cuenta los artículos mencionados anteriormente, se evidenció que el 60% (artículos: [13], [14] y [16]) de los estudios se centran en: identificar, clasificar, analizar la deuda social y medir el impacto de las decisiones sociotécnicas en equipos de desarrollo de software subóptimos, así como establecer factores críticos para la efectividad del trabajo en equipo, por ejemplo: Caballero et al. [11] nos dice que la detección de olores comunitarios indica problemas dentro de los equipos de desarrollo de software, así como conflictos entre miembros del equipo o la falta de estándares claros. Con el tiempo, la persistencia de estos olores comunitarios resulta en la acumulación de deuda social, lo que afecta negativamente la colaboración, la eficiencia y la calidad del trabajo realizado en el proyecto de desarrollo de software. Asimismo, se mencionan nueve factores críticos del trabajo, así como: con-texto, cultura, comunicación, cooperación, coordinación, conflicto, control, compromiso y cambio, cada uno de estos elementos puede

tener un impacto en la manera en la que las interacciones sociales y las decisiones técnicas influyen en el desempeño del equipo de desarrollo de software. Al examinar las dinámicas de la comunidad bajo la perspectiva de estos factores, se puede obtener una comprensión más detallada de cómo estos olores afectan la dinámica interna del trabajo en equipo, y, por ende, en el rendimiento de los equipos de desarrollo de software. Esta perspectiva puede proporcionar información valiosa sobre las causas y efectos de los olores de la comunidad en el entorno laboral y en la calidad del trabajo realizado por los equipos de desarrollo de software. Además, se identifican 30 olores comunitarios que surgen de situaciones sociales y organizativas adversas causadas por decisiones sociotécnicas incorrectas. Los efectos de estas decisiones incluyen interacciones sociales tensas, mala conducta personal y profesional, así como artefactos de software defectuosos como consecuencia. Los olores comunitarios identificados se listan a continuación y se puede ver sus causas y consecuencias en la P4.

- Arquitectura por osmosis
- Capucha de arquitectura
- Nube negra
- Cognición de clase
- Código rojo
- Distancia cognitiva
- Desarrollo de recetario
- Choque de DevOps
- Desvinculación
- Dispersión
- Disenso
- Hiper comunidad
- Exceso de informalidad
- Isomorfismo institucional
- Arquitectura invisible
- Técnico sobrante
- Lobo solitario
- Arquitectura solitaria
- Recién llegados que no aportan
- Arquitectura obstruida
- Silos organizacionales
- Escaramuza organizacional
- Distancia de poder
- Miembros pedantes
- Prima donnas
- Silencio radiofónico o cuello de botella
- Villanía de compartir
- Desafío de soluciones
- Distorsión temporal
- Desaprendizaje

En la sección PI4 se describe cada olor en detalle. Por otra parte en cuanto a los artículos [6] y [15] correspondientes al 40% de los artículos restantes proponen diferentes soluciones, por lo que Dressen et al. [17] propone un modelo para reducir la deuda social basado en el modelo de las 3C's, es decir; comunicación, colaboración y coordinación, esto ayuda a categorizar los diferentes tipos de olores comunitarios dentro de una de las 3C's para así poder establecer el tipo de estrategia con el cual se procederá a gestionar y mitigar la deuda social. Este modelo también se

centra en mejorar la comunicación y colaboración para reducir la deuda social por medio de una estrategia de entrevistas individuales con los individuos de más relevancia en el equipo de desarrollo de software ágil, si bien no se hace con todos los miembros del equipo, la recomendación es involucrar la mayor parte de individuos para un mejor entendimiento de la situación y una mejor contextualización del ambiente laboral. Por otro lado, Saeeda et al. [15] presenta un estudio que investiga varios tipos de deuda no técnicas en el desarrollo de software como: la deuda de procesos enfocados en los aspectos sociales, deuda de la gente, deuda organizacional y deuda cultural. Las deudas no técnicas se relacionan con problemas similares a la deuda técnica, pero surgen de no prestar atención a aspectos no técnicos del desarrollo. Descuidar estos factores puede llevar a resultados negativos en los equipos de desarrollo, incluyendo lo que se conoce como deuda social, donde sus causas y estrategias para mitigarla son abordadas en la PI4. En este artículo se identificaron tres elementos generadores de causas: limitaciones sociales, desafíos organizacionales y problemas comunitarios, a partir de los cuales se propusieron ocho estrategias de mitigación, los cuales son: i) análisis de redes sociales, ii) comunicación colaborativa, iii) gestión de la composición del equipo y mejora en la descripción de decisiones arquitectónicas, iv) entornos de trabajo combinados (híbridos), v) métricas para la comunicabilidad de la arquitectura del software, vi) marcos de trabajo específicos como lo son CAFFEA, Tácticas arquitectónicas, y DAHLIA y el Marco de calidad socio-técnica, vii) herramientas como GEEZMO, CodeFace4Smell, YOSHI, viii) modelos del tipo estadísticos y de redes sociales.

PI4: ¿Cuáles son los principales desafíos y oportunidades para la investigación futura sobre la deuda social en el desarrollo de software ágil?

Los principales desafíos en la investigación futura sobre la deuda social en el desarrollo de software ágil incluyen la comprensión de su naturaleza compleja de esta temática, el desarrollo de métricas efectivas para su evaluación, la integración de enfoques interdisciplinarios y la identificación de estrategias de mitigación. Estos desafíos requieren una profunda comprensión de los aspectos técnicos y sociales involucrados, así como la capacidad de medir y abordar la deuda social de manera oportuna y precisa. Dressen et al. [17] nos dice que actualmente se desconoce qué causa exactamente el aumento de la deuda social y qué mecanismos ayudan a mitigar o disminuir sus efectos adversos. Caballero et al. [11] identifican 30 olores comunitarios nombrados en la PI3, que se analizan mediante un modelo que vincula causas y consecuencias. En el siguiente párrafo se representará estas causas y consecuencias.

Los olores comunitarios en el desarrollo de software son diversas situaciones que pueden afectar la productividad y calidad del equipo. Algunos de ellos incluyen “Arquitectura por osmosis”, que ocurre cuando las decisiones arquitectónicas se toman sin una adecuada gestión de la información, lo que lleva a una arquitectura inestable y a malentendidos. “Capucha de arquitectura” describe las dificultades en la cooperación entre equipos remotos, resultando en decisiones inconsistentes. “Nube negra” hace referencia a la falta de condiciones para una comunicación

efectiva entre compañeros, lo que disminuye la moral y productividad. “Cognición de clase” es el desafío de entender clases y estructuras complejas, lo que puede generar errores por interpretaciones distintas. “código rojo” se refiere a las clases complejas gestionadas por pocos desarrolladores, lo que genera presión y estrés. “Distancia cognitiva” implica diferencias entre desarrolladores que pueden causar conflictos y malentendidos. “Desarrollo de recetario” describe la resistencia a nuevas metodologías y falta de innovación. “Choque de DevOps” refiere a los conflictos entre los equipos de desarrollo y operaciones que afectan la integración y despliegue del software.

El “Desvinculación” ocurre cuando se percibe que el software está maduro y se entrega prematuramente, afectando la calidad. “Dispersión” surge cuando las correcciones fragmentan el equipo, dificultando la coordinación. “Disenso” es la incapacidad de alcanzar consenso, dejando problemas sin resolver, lo que afecta la calidad del software. “Hiper comunidad” se refiere a la conexión excesiva dentro del equipo, lo que retrasa las decisiones y disminuye la productividad. “Exceso de informalidad” es la falta de protocolos formales, que genera desbordamiento de información y afecta la eficiencia. “Isomorfismo institucional” implica similitudes en los procesos que limitan la innovación. “Arquitectura invisible” ocurre cuando las decisiones arquitectónicas no se documentan adecuadamente, lo que causa errores y falta de alineación. “Técnico sobrante” es la comunicación deficiente entre desarrolladores y técnicos, lo que disminuye la motivación y productividad. El “Lobo solitario” es el trabajo aislado de ciertos miembros, lo que dificulta la cohesión y aumenta la duplicación de esfuerzos. “Arquitectura solitaria” describe decisiones tomadas por no arquitectos, lo que genera problemas de escalabilidad. “Recién llegados que no aportan” es la dependencia excesiva de los nuevos miembros del equipo, afectando la productividad y cohesión. “Arquitectura obstruida” es la falta de visión clara en sub-equipos, lo que dificulta la implementación de cambios. “Silos organizacionales” se refiere a la falta de coordinación entre tareas, lo que lleva a duplicación de esfuerzos. “Escaramuza organizacional” describe las diferencias culturales entre equipos que afectan la gestión del proyecto. “Distancia de poder” es la percepción de desigualdad, lo que interrumpe la comunicación y afecta la toma de decisiones. “Miembros pedantes” se refiere a la exigencia excesiva de conformidad, lo que frustra a los compañeros. “Prima donnas” describe la resistencia al cambio y el trabajo aislado, lo que afecta la innovación. “Silencio radiofónico o cuello de botella” es la comunicación ineficiente en estructuras complejas, lo que genera retrasos. “Villanía de compartir” ocurre cuando no se comparte eficazmente el conocimiento, disminuyendo la cohesión y eficiencia. “Desafío de soluciones” es cuando los conflictos entre sub-equipos impiden la resolución de problemas, afectando el progreso. “Distorsión temporal” se refiere a los malentendidos después de cambios organizacionales, causando retrasos y problemas con los clientes. Finalmente, “Desaprendizaje” ocurre cuando los empleados experimentados resisten el cambio, lo que lleva a la pérdida de conocimiento y errores repetidos.

Teniendo en cuenta lo anterior, también se describe una serie de fases que permiten observar cómo surgen los “olores comunitarios” las cuales son:

- Fase 1, Inicio: los líderes toman decisiones sociotécnicas que generan un ambiente tenso, lo que puede llevar a malestar entre los miembros del equipo. A pesar de que este estrés comienza a manifestarse, no es fácilmente detectado por los gerentes.
- Fase 2, Impacto directo de los problemas comunitarios: el equipo experimenta más episodios de emociones negativas, lo que lleva a ignorar procesos importantes y reducir la calidad del trabajo. Aunque los efectos son más notorios, aún pueden pasar desapercibidos por los líderes.
- Fase 3, Difusión del equipo: los miembros afectados por estrés comienzan a influir negativamente en el resto del equipo, lo que provoca baja productividad y la acumulación de deuda social. Aunque los gerentes pueden detectar los problemas, la solución dependerá de sus habilidades y estrategias.
- Fase 4, Impacto en la organización: el deterioro de la calidad del software afecta a más equipos, y los líderes pueden identificar fallos técnicos que requieren atención urgente para evitar que los problemas se extiendan.
- Fase 5, Problemas avanzados: los usuarios externos comienzan a sufrir las consecuencias de los problemas, lo que afecta la reputación de la empresa y provoca pérdidas económicas debido a la pérdida de clientes.

En particular durante la fase 3, denominada “Difusión del Equipo”. En esta fase, se destaca que los miembros del equipo que sufren episodios recurrentes de emociones negativas comienzan a manifestar sus estados psicológicos internos. Esto tiene un impacto directo en el resto del equipo, derivado de decisiones sociotécnicas que afectan a la cohesión grupal y contribuyen al aumento de la deuda social. Sin embargo, los líderes de los proyectos pueden detectar la presencia de estos efectos, pero el desarrollo de un método preciso para diagnosticar y solucionar los problemas depende de la experiencia del líder y los enfoques de gestión disponibles. Estos líderes tienen una oportunidad significativa para prevenir posibles daños organizativos al supervisar y mitigar periódicamente los efectos de los olores comunitarios dentro de los equipos de desarrollo de software. No obstante, se presentan oportunidades para investigar nuevas tecnologías y herramientas, explorar la relación entre la deuda social y el rendimiento del equipo, estudiar el papel de la cultura organizacional y evaluar el impacto a largo plazo de la deuda social en la dinámica del equipo y la calidad del software. Estas oportunidades pueden conducir a una comprensión más completa de la deuda social en entornos ágiles y al desarrollo de estrategias más efectivas para su gestión y mitigación.

PI6: ¿Cuáles son las prácticas o enfoques que abordan de manera exitosa la gestión de la deuda social en el desarrollo de software ágil?

Dressen et al. [17] sugieren una estrategia para abordar la deuda social, fundamentada en el modelo de las 3C's. Este modelo enfatiza las deficiencias en comunicación,

colaboración y coordinación. El enfoque de mitigación sugerido se concentra particularmente en mejorar la comunicación y la colaboración, destacando los siguientes puntos:

- “Comunicación honesta y abierta”, la cual es fundamental en equipos de desarrollo de software ágil distribuido para mitigar la deuda social. La comunicación honesta y abierta se define por su franqueza y transparencia, lo que ayuda a construir confianza mutua entre los miembros del equipo. Esta confianza facilita la resolución efectiva de conflictos sociales, ya que los miembros del equipo pueden abordar los problemas directamente. En última instancia, esta comunicación directa contribuye a crear un ambiente de trabajo constructivo y ayuda a reducir la acumulación de deuda social en el equipo.
- “Intensidad de la comunicación relacionada con las tareas”, se refiere a la naturaleza altamente enfocada y orientada a objetivos de la interacción en el trabajo remoto. Las interacciones y la comunicación no son aleatorias, sino que están determinadas o preparadas de antemano para completar tareas específicas, reduciendo las distracciones. Este enfoque intensificado en la comunicación relacionada con las tareas puede ayudar a mitigar el efecto de los factores que contribuyen a la deuda social en equipos de desarrollo de software ágil distribuido (ASD).
- Para la colaboración, se identifica la: “Intensidad de la colaboración relacionada con las tareas”, se refiere a cómo las interacciones durante el trabajo remoto están altamente enfocadas en tareas y objetivos específicos. Esto significa que todas las acciones colaborativas están orientadas a completar la tarea en cuestión, reduciendo las distracciones. Este enfoque intensificado en la colaboración relacionada con las tareas puede tener un efecto tanto en promover como en mitigar la deuda social en equipos de desarrollo de software ágiles distribuidos. Por un lado, puede conducir a una comunicación más directa y eficiente, pero por otro, puede resultar en una falta de interacción informal que es importante para la construcción de relaciones y la resolución de conflictos.
- Se mencionan que los: “Costos iniciales reducidos para iniciar interacciones o comunicaciones”, se refieren a la capacidad de establecer rápidamente un entorno para la colaboración y comunicación uno a uno sin restricciones. Esto es beneficioso en un contexto de trabajo remoto, ya que las salas de reuniones libres ya no son un recurso escaso y se puede configurar la comunicación sin casi ninguna limitación. Este factor ayuda a mitigar la deuda social en equipos de desarrollo de software ágil distribuido al facilitar la resolución de conflictos y promover una comunicación más directa y abierta entre los miembros del equipo.

IV. DISCUSIÓN

La deuda social en el desarrollo de software ágil se refiere a las repercusiones no técnicas derivadas de decisiones

apresuradas o inadecuadas, que afectan la cultura organizacional, la comunicación interna y el bienestar del equipo. Esta deuda surge cuando se priorizan objetivos a corto plazo, como cumplir con fechas de entrega o reducir costos, sacrificando buenas prácticas de gestión y relaciones interpersonales. Esto puede llevar a un ambiente de trabajo tóxico, estrés crónico y problemas de salud, disminuyendo la motivación, productividad y calidad del trabajo, y limitando la innovación y creatividad. Para mitigarla, es crucial adoptar un enfoque holístico que promueva la comunicación efectiva, el equilibrio trabajo-vida, el reconocimiento profesional y un liderazgo que priorice el bienestar del equipo. Además, es fundamental establecer estrategias claras para identificar el impacto de la deuda social, como encuestas de satisfacción laboral y análisis de rotación de personal. La transparencia en la comunicación y la participación del equipo en la toma de decisiones son vitales para construir un entorno de confianza y colaboración. Invertir en la salud organizacional mejora el bienestar de los empleados y se traduce en mayor eficiencia operativa, innovación y competitividad. Sin embargo, la revisión sistemática ha identificado limitaciones en la literatura existente, como la falta de estrategias concretas para la gestión de la deuda social y la ausencia de consenso sobre métricas de evaluación. A pesar de estas limitaciones, es esencial que las organizaciones reconozcan la deuda social como un aspecto crítico que requiere atención continua y estrategias bien definidas para su gestión efectiva.

V. AGRADECIMIENTOS

Agradezco a nuestra familia por su apoyo constante, a mis mentores por su guía, y a mis colegas por su colaboración. También, gracias a todos aquellos que, de una u otra forma, han contribuido al desarrollo de este trabajo.

VI. REFERENCIAS

- [1] Doran, G. T. “There’s a S.M.A.R.T. Way to Write Management’s Goals and Objectives.” *Management Review*, vol. 70, pp. 35-36, 1981.
- [2] V. R. B. G. Caldiera and H. D. Rombach, “The goal question metric approach,” *Encyclopedia of Software Engineering*, pp. 528-532, 1994.
- [3] S. McCombes, “Writing strong research questions | Criteria & examples,” Scribbr, Agosto 2023, [En línea]. Disponible: <https://www.scribbr.com/research-process/research-questions/>
- [4] “La literatura gris”, *Formación Universitaria*, vol. 4, no. 6, pp. 1–2, 2011. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.4067/s0718-50062011000600001>
- [5] T. L. Baldwin, “Publish or perish: An evaluation of the quality, quantity, ethics and review process of IEEE/PES: Advanced technology for assisting the review process”, en *IEEE Power and Energy Society General Meeting - Conversion and Delivery of Electrical Energy in the 21st Century*, Pittsburgh, PA, USA, 20–24 de julio de 2008. IEEE, 2008, pp. 1-2 Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1109/pes.2008.4596777>
- [6] D. A. Tamburri, P. Kruchten, P. Lago y H. van Vliet, “What is social debt in software engineering?”, en *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, San Francisco, CA, USA, 25 de mayo de 2013, pp. 93-96. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1109/chase.2013.6614739>
- [7] S. Jalali y C. Wohlin, “Systematic literature studies: database searches vs. backward snowballing”, en *Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement (ESEM ’12)*. Association for Computing Machinery, Lund, Sweden, 19–20 de septiembre de 2012. New York, USA: ACM Press, 2012, p. 29-39. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1145/2372251.2372257>

- [8] T. Dybå y T. Dingsøyr, "Strength of evidence in systematic reviews in software engineering", en Second ACM-IEEE International Symposium on Empirical Software engineering and measurement (ESEM '08). Association for computing Machinery, Kaiserslautern, Germany, 9–10 de octubre de 2008. New York, USA: ACM Press, 2008, p. 178-187. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1145/1414004.1414034>
- [9] L. Yang et al., "Quality assessment in systematic literature reviews: A software engineering perspective", Information and Software Technology, vol. 130, p. 106397, febrero de 2021. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1016/j.infsof.2020.106397>
- [10] M. Ivarsson y T. Gorschek, "A method for evaluating rigor and industrial relevance of technology evaluations", Empirical Software Engineering, vol. 16, no. 3, pp. 365–395, octubre de 2010. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1007/s10664-010-9146-4>
- [11] E. A. Caballero Espinosa, J. C. Carver y K. Stowers, "Community smells—The sources of social debt: A systematic literature review", Information and Software Technology, vol. 153, p. 107078, septiembre de 2022. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1016/j.infsof.2022.107078>
- [12] F. Palomba, D. Andrew Tamburri, F. Arcelli Fontana, R. Oliveto, A. Zaidman y A. Serebrenik, "Beyond technical aspects: ¿How do community smells influence the intensity of code smells?", IEEE Transactions on Software Engineering, vol. 47, no. 1, pp. 108–129, enero de 2021. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1109/tse.2018.2883603>
- [13] D. A. Tamburri, P. Kruchten, P. Lago y H. V. Vliet, "Social debt in software engineering: Insights from industry", Journal of Internet Services and Applications, vol. 6, art. no. 10, mayo de 2015. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.1186/s13174-015-0024-6>
- [14] A. Martini, V. Stray y N. B. Moe, "Technical-, social- and process debt in large-scale agile: An exploratory case-study", en Agile Processes in Software Engineering and Extreme Programming – Workshops. Cham: Springer Int. Publishing, 2019, pp. 112–119. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: https://doi.org/10.1007/978-3-030-30126-2_14
- [15] H. Saeeda, M. Ovais Ahamd y T. Gustavsson, "A multivocal literature review on non-technical debt in software development: an insight into process, social, people, organizational, and culture debt", e-Informatica Software Engineering Journal, vol. 18, no. 1, p. 240101, 2024. Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.37190/e-inf240101>
- [16] E. A. Caballero Espinosa, "Understanding Social Debt in Software Engineering". Tesis doctoral, Departamento de Ciencias de la Computación, Universidad de Alabama, Tuscaloosa, Alabama, EE. UU, 2021. [En línea]. Disponible: <http://ir.ua.edu/handle/123456789/8278>
- [17] T. Dreesen, P. Hennel, C. Rosenkranz y T. Kude, "The second vice is lying, the first is running into debt." antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams", en Hawaii International Conference on System Science, 2021, p. 6826-6835 Accedido el 24 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.24251/hicss.2021.818>
- [18] R. L. Baskerville, "Investigating information systems with action research", Communications of the Association for Information Systems, vol. 2, no. 19, 1999. Accedido el 25 de septiembre de 2024. [En línea]. Disponible: <https://doi.org/10.17705/1cais.00219>

Deuda Social en el Desarrollo Ágil de Software

Social Debt in Agile Software Development

Luis Romero
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
luisrom@unicauca.edu.co

Santiago Córdoba
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
santiago9739@unicauca.edu.co

César Pardo
University of Cauca
GTI Research Group
Popayán, Cauca, Colombia
cpardo@unicauca.edu.co
<https://orcid.org/0000-0002-6907-2905>

Eydy Suárez Brieva
Popular University of Cesar
GISICO Research Group
Valledupar, Cesar, Colombia
eydysuarez@unicesar.edu.co
<https://orcid.org/0000-0002-6296-0117>

Vladimir Villarreal
Technological University of
Panama
Grupo GITCE - UTP
Chiriquí, Panamá
vladimir.villarreal@utp.ac.pa
<https://orcid.org/0000-0003-4678-5977>

Abstract— In agile software development, suboptimal sociotechnical decisions can lead to adverse conditions that negatively impact the social and emotional well-being of teams. This phenomenon, known as social debt, manifests through deteriorated dynamics that undermine communication, collaboration, coordination, and team cohesion. This systematic literature review aims to analyze the current state of knowledge on social debt in agile contexts, identifying its causes, manifestations, consequences, and mitigation strategies. A total of 171 articles were reviewed, of which 21 were deemed relevant; five of these are primary studies. The findings reveal the presence of thirty community smells resulting from sociotechnical dysfunctions, along with nine critical factors influencing team effectiveness. Various types of non-technical debt were also classified, including process debt, people-related debt, organizational debt, and cultural debt. Additionally, three root cause elements and eight mitigation strategies were identified. The results underscore the need to develop specific metrics, adopt interdisciplinary approaches, and validate practical strategies for managing social debt. This review provides a solid conceptual and empirical foundation for understanding and addressing social debt in agile software development environments.

Keywords—Social debt, agile software development, systematic literature review, community odors, sociotechnical factors, organizational health

Resumen— En el desarrollo ágil de software, las decisiones sociotécnicas subóptimas pueden generar condiciones adversas que afectan el bienestar social y emocional de los equipos. Este fenómeno, denominado deuda social, se manifiesta en dinámicas deterioradas que comprometen la comunicación, la colaboración, la coordinación y la cohesión del equipo. Esta revisión sistemática de la literatura tiene como objetivo analizar el conocimiento actual sobre la deuda social en contextos ágiles, identificando sus causas, manifestaciones, consecuencias y estrategias de mitigación. Se identificaron 171 artículos, de los cuales 21 fueron considerados relevantes; cinco de ellos corresponden a estudios primarios. Los

resultados evidencian la existencia de treinta olores comunitarios derivados de disfunciones sociotécnicas, así como nueve factores críticos que afectan la efectividad del trabajo en equipo. Se clasificaron también distintos tipos de deuda no técnica (de procesos, de personas, organizacional y cultural), tres elementos generadores de causas y ocho estrategias de mitigación. Los hallazgos subrayan la necesidad de desarrollar métricas específicas, adoptar enfoques interdisciplinarios y validar estrategias prácticas. Esta revisión aporta una base conceptual y empírica para comprender y gestionar la deuda social en entornos ágiles.

Palabras clave—Deuda social, desarrollo ágil de software, revisión sistemática de literatura, olores comunitarios, factores sociotécnicos, salud organizacional

I. INTRODUCCIÓN

La deuda social (DS) en el desarrollo de software se refiere a las consecuencias acumuladas no gestionadas, derivadas de decisiones sociotécnicas subóptimas, que deterioran progresivamente la cultura organizacional, la comunicación interna, el bienestar del equipo y la calidad técnica del producto [1], [6]. A diferencia de la deuda técnica, la DS tiene su origen en dinámicas humanas mal gestionadas —como conflictos interpersonales, falta de transparencia, deficiencias en la coordinación y comunicación, o una distribución desigual de la carga de trabajo— que generan impactos tangibles sobre los miembros del equipo, manifestándose en estrés, desconfianza, irritabilidad y otros riesgos psicosociales que afectan tanto la salud física como emocional de los profesionales [7]. Uno de los síntomas más visibles de la DS es la aparición de olores comunitarios (*community smells*), entendidos como patrones disfuncionales en las relaciones o dinámicas de equipo. Si no se identifican y abordan a tiempo, estos pueden escalar y desencadenar crisis organizacionales [6], [13], por ejemplo: cuando los equipos priorizan de manera sistemática el

cumplimiento de plazos por encima del cuidado de las relaciones laborales, se acumula este tipo de deuda, lo que a largo plazo puede traducirse en estrés crónico, rotación de personal y fallos en el software [14], [7], [15].

En la última década, la adopción generalizada de metodologías ágiles ha intensificado este fenómeno. Dado que estas metodologías dependen de buena comunicación, cooperación y coordinación constantes, la falta de gestión adecuada de estos aspectos puede afectar negativamente los procesos de desarrollo. Diversos estudios revelan que muchos equipos ágiles operan con DS no diagnosticada, lo que se traduce en una disminución de la productividad y un incremento de los errores en el producto final [6], [9]. A pesar de su impacto tangible, la DS continúa siendo un fenómeno poco comprendido y escasamente abordado de forma sistemática. Asimismo, se identificó que persisten vacíos críticos en la literatura: no existe consenso sobre cómo medirla, las estrategias de mitigación carecen de validación empírica, y sus causas se abordan de manera fragmentada [13], lo que limita la capacidad de las organizaciones para intervenir eficazmente.

En este contexto, se justifica la realización de una revisión sistemática de la literatura (RSL) orientada a comprender las dinámicas no técnicas —especialmente aquellas relacionadas con fallas en la comunicación, coordinación y cooperación— que inciden en los equipos de desarrollo de software [16], [17]. Adicionalmente, esta RSL tiene como finalidad identificar las causas, efectos y consecuencias asociadas a la DS, consolidar las investigaciones existentes sobre el tema, reconocer los trabajos futuros sugeridos por la comunidad académica, y establecer un marco de prácticas necesarias para gestionar su acumulación en las organizaciones. Comprender la naturaleza y el impacto de la DS permite a las organizaciones diseñar estrategias más efectivas para promover entornos de trabajo sostenibles, saludables e innovadores, equilibrando las exigencias operativas con el bienestar de los equipos [13]. Este artículo está organizado de la siguiente manera: la Sección 2 describe el protocolo de investigación adoptado; la Sección 3 presenta el análisis detallado de los resultados; la Sección 4 discute los hallazgos en relación con la literatura existente; y la Sección 5 expone las conclusiones junto con las recomendaciones para investigaciones futuras.

II. PROTOCOLO DE INVESTIGACIÓN

Para este estudio, se recopilaron los principales artículos de investigación mediante una Revisión Sistemática de Literatura (RSL). Esta técnica permitió identificar, evaluar y analizar eficientemente artículos relacionados con problemas de investigación. Por lo tanto, una RSL facilita la recopilación de investigación primaria sobre un tema específico [5]. Para llevar a cabo el diseño de esta RSL, se siguió el protocolo propuesto por Kitchenham y Charters [18] y Budgen *et al.* [19], que proporciona directrices para la realización de una revisión sistemática. El protocolo comprendió las siguientes etapas: (i) definición de las preguntas de investigación; (ii) realización de la búsqueda de artículos relevantes; (iii) selección de artículos primarios aplicando criterios de inclusión y exclusión; (iv) evaluación de la calidad de los artículos primarios; y (v) extracción de información.

A. Definición de las preguntas de investigación

Para establecer los objetivos de búsqueda se utilizaron dos herramientas: la primera es propuesta por Doran con el enfoque conocido como SMART [20], el cual está conformado por un conjunto de criterios para definir objetivos y metas los cuales son: específicos, medibles, alcanzables, relevantes y temporales (en inglés Specific – S, Measurable – M, Achievable – A, Relevant – R, Time based - T), lo que aumenta la probabilidad para que los objetivos sean cumplidos. La segunda herramienta utilizada es el enfoque propuesto por Basili y Rombach [21] conocido como Goal-Question-Metric (GQM), el cual permitió establecer una estructura que apoya en el cumplimiento, avance y trazabilidad de los objetivos e intereses de la investigación propuestos, el enfoque sugiere tres niveles, el establecimiento de metas, la definición de preguntas, y finalmente, la definición de métricas para medir el avance así como los resultados de algún proyecto, esta última parte no se consideró en esta primera versión del RSL, se contemplará para futuros proyectos. La evaluación de la calidad de las preguntas de investigación se llevó a cabo siguiendo una adaptación del marco de trabajo definido por McCombes [22], el cual propone cuatro características específicas para evaluar la calidad de las preguntas de investigación, las características son: (i) centrado e investigable, (ii) factible y específico, (iii) complejo y discutible, y (iv) relevante y original, cada una de estas características está compuesta por criterios que garantizan su cumplimiento. A través del siguiente link: <https://zenodo.org/records/15654060>, se provee información complementaria acerca del proceso llevado a cabo para evaluar las preguntas de investigación formuladas.

A continuación, se pueden observar los objetivos (OB) incluidos en la RSL.

OB1: Identificar las investigaciones y propuestas existentes sobre la deuda social en el desarrollo de software ágil. Esto incluye la caracterización de los "olores comunitarios", sus causas, consecuencias y los factores críticos que influyen en el desempeño de los equipos.

OB2: Explorar los principales enfoques y prácticas utilizados para la gestión y mitigación de la deuda social en equipos de desarrollo ágil. Esto abarca modelos, tácticas, marcos de trabajo y herramientas específicas propuestas en la literatura.

OB3: Determinar los desafíos actuales y las oportunidades de investigación futura en el ámbito de la deuda social en el desarrollo de software ágil. Esto implica identificar las brechas en el conocimiento, como la ausencia de métricas estandarizadas, la necesidad de enfoques interdisciplinarios y la exploración de tecnologías emergentes para su diagnóstico y manejo.

Con base en los objetivos propuestos, se plantearon las siguientes preguntas de investigación (PI), correspondientemente:

PI1: ¿Cuáles son las investigaciones o propuestas desarrolladas?

PI2: ¿Cuáles son los principales desafíos y oportunidades para la investigación futura?

PI3: ¿Cuáles son las practicas o enfoques que abordan la gestión de la deuda social en el desarrollo de software ágil?

B. Ejecucion de la cadena de busqueda

En el protocolo de investigación también se utilizó el enfoque de revisión de literatura gris, también llamada literatura no convencional, y se refiere a la revisión de documentos que no se publican de manera tradicional. Estos documentos pueden ser informes de investigación, tesis, patentes, entre otros, y pueden ser difíciles de encontrar [4], esto permitió obtener una primera aproximación a los conceptos y términos abordados en la gestión de la deuda social, así como los problemas y retos a los que se enfrentan.

A partir de los términos identificados en la revisión de literatura gris, se definió la cadena de búsqueda básica: **“social debt” AND “agile software development”** la cual se aplicó a seis bases de datos científicas: Google Scholar, IEEE Xplore, SpringerLink, Scopus, Science Direct y Web Of Science. Es importante aclarar que la cadena de búsqueda básica debió ser adaptada de acuerdo con las especificaciones y configuraciones de cada base de datos. Asimismo, se tuvieron en cuenta los siguientes criterios o filtros de búsqueda de las diferentes bases de datos: (i) artículos en inglés, (ii) artículos exclusivos en áreas de conocimiento como: Computer Science, Software Engineering y Software Development, y (iii) artículos de revistas o actas de eventos científicos. Además, se consideró todo estudio dentro de la temática publicado entre los años 2013 y 2024, dado que el concepto de deuda social relacionado con el desarrollo de software y su gestión fue definido por primera vez en 2013 en el trabajo realizado por Tamburri *et al.* [1].

C. Selección de artículos primarios

En la Tabla 1 se puede observar el listado de los criterios de inclusión utilizados para identificar los estudios primarios que cumplieron con los lineamientos establecidos para la investigación. La Tabla 2 presenta el listado de los criterios de exclusión utilizados para identificar los estudios que no cumplieron con los lineamientos establecidos para la investigación.

TABLA 1. CRITERIOS DE INCLUSIÓN

Id	Criterio de inclusión (CI)
CI1	Aquellos artículos publicados entre 2013 y 2024.
CI2	Aquellos artículos que hayan sido escritos en inglés.
CI3	Aquellos artículos publicados en revistas o conferencias revisados por pares.
CI4	Aquellos artículos que en su tema principal proponga una solución para gestionar la deuda social.
CI5	Aquellos artículos que analizan las consecuencias de la deuda social y que sugieren o proponen una solución.

Acrónimos utilizados: Identificador (Id), Criterio de inclusión (CI).

TABLA 2. CRITERIOS DE EXCLUSIÓN

Id	Criterio de exclusión (CE)
CE1	Aquellos artículos que aborden el tema principal de investigación de manera superficial.
CE2	Aquellos artículos que aparecen duplicados en la búsqueda inicial de Google Scholar
CE3	Aquellos artículos que sean libros o capítulos de libros
CE4	Aquellos artículos con acceso restringido que requieran pagos externos o vinculaciones con terceros.
CE5	Tomaremos como base aquellos artículos encontrados inicialmente en Google Scholar, a partir de esta selección inicial, excluirémos cualquier artículo que aparezca repetido en otras bases de datos.

Acrónimos utilizados: Identificador (Id), Criterio de exclusión (CE).

Para la ejecución de dicho proceso y con el objetivo de reducir la subjetividad en el trabajo, se adaptó y ejecutó la

cadena de búsqueda básica a cada base de datos, asimismo, se verificó la ejecución de la búsqueda y los resultados obtenidos, resultado de este proceso se obtuvieron 171 artículos clasificados como encontrados. A partir de los artículos encontrados, 21 estudios fueron considerados como relevantes, y sólo 3 fueron considerados primarios. Considerando el número de artículos primarios resultantes, se optó por utilizar la estrategia definida por Jalali, S. y Wohlin, C. [2] denominada Snowballing Backward, también conocida como: “revisión bibliográfica retrospectiva”, aumentando el número de artículos, para un total de 5 estudios primarios que se pueden consultar en la Tabla 3. Cada uno de ellos fue elegido tras un proceso cuidadoso de revisión, considerando no solo su calidad metodológica, sino también el valor que aportan al entendimiento de la deuda social en equipos ágiles.

TABLA 3. ARTICULOS PRIMARIOS

Id	Nombre del artículo	Ref.
Artículos Snowballing		
EP1	“What is social debt in software engineering?”. Año 2013	[6]
EP2	“Beyond technical aspects: How do community smells influence the intensity of code smells?”. Año 2018.	[12]
Artículos primarios encontrados		
EP3	“The Second Vice is Lying, the First is Running into Debt.” Antecedents and Mitigating Practices of Social Debt: an Exploratory Study in Distributed Software Development Teams. Año 2021.	[17]
EP4	Community smells—The sources of social debt: A systematic literature review. Año 2023.	[11]
EP5	A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt. Año 2024.	[15]

Acrónimos utilizados: Identificador (Id), Referencia (Ref.).

III. ANÁLISIS DE LOS RESULTADOS

A continuación, se presentan los resultados obtenidos para cada una de las preguntas de investigación establecidas en esta revisión sistemática.

A. PII: ¿Cuáles son las investigaciones o propuestas desarrolladas?

A partir del análisis de los estudios primarios encontrados, se evidenció que el 60% (artículos: [6], [8], [9], [10], [11], [17], [23]) de los estudios se centran en: identificar, clasificar, analizar la deuda social y medir el impacto de las decisiones sociotécnicas en equipos de desarrollo de software subóptimos, así como: establecer factores críticos para la efectividad del trabajo en equipo, por ejemplo: Caballero *et al.* [6] menciona que la detección de olores comunitarios indica problemas dentro de los equipos de desarrollo de software, así como conflictos entre miembros del equipo o la falta de estándares claros. Con el tiempo, la persistencia de estos olores comunitarios resulta en la acumulación de deuda social, lo que afecta negativamente la colaboración, la eficiencia y la calidad del trabajo realizado en el proyecto de desarrollo de software. Asimismo, se mencionan nueve factores críticos del trabajo, así como: (i) contexto, (ii) cultura, (iii) comunicación, (iv) cooperación, (v) coordinación, (vi) conflicto, (vii) control, (viii) compromiso y (ix) cambio, cada uno de estos elementos puede tener un impacto en la manera en la que las interacciones sociales y las decisiones técnicas influyen en el desempeño del equipo de desarrollo de software. Al examinar las dinámicas de la comunidad bajo la perspectiva de estos factores, se puede obtener una comprensión más detallada de cómo estos olores afectan la dinámica interna del trabajo en equipo, y, por ende; en el rendimiento de los

equipos de desarrollo de software. Esta perspectiva puede proporcionar información valiosa sobre las causas y efectos de los olores de la comunidad en el entorno laboral y en la calidad del trabajo realizado por los equipos de desarrollo de software.

Por otra parte, en Caballero *et al.* [6] se identifican 30 olores comunitarios que surgen de situaciones sociales y organizativas adversas causadas por decisiones sociotécnicas incorrectas. Los efectos de estas decisiones incluyen interacciones sociales tensas, mala conducta personal y profesional, así como artefactos de software defectuosos como consecuencia. Los olores comunitarios identificados, junto con las causas y consecuencias se presentan en detalle en la Tabla 5.

Respecto a los artículos EP1 y EP5 (ver Tabla 3), éstos proponen diferentes soluciones, por su parte; Dressen *et al.* [13] proponen un modelo para reducir la deuda social basado en el modelo de las 3C's, es decir; comunicación, colaboración y coordinación, esto ayuda a categorizar los diferentes tipos de olores comunitarios dentro de una de las 3C's para así poder establecer el tipo de estrategia con el cual se procederá a gestionar y mitigar la deuda social. Este modelo también se centra en mejorar la comunicación y colaboración para reducir la deuda social por medio de una estrategia de entrevistas individuales con los miembros de más relevancia en el equipo de desarrollo de software ágil, si bien no se hace con todos los miembros del equipo, la recomendación es involucrar la mayor parte de individuos para un mejor entendimiento de la situación y una mejor contextualización del ambiente laboral. Por otro

lado, Saeeda *et al.* [11] investigan varios tipos de deuda no técnicas en el desarrollo de software, así como: (i) *la deuda de procesos* enfocados en los aspectos sociales, (ii) *deuda de la gente*, (iii) *deuda organizacional* y (iv) *deuda cultural*. Las deudas no técnicas se relacionan con problemas similares a la deuda técnica, pero surgen al no prestar atención a aspectos no técnicos del desarrollo. Descuidar estos factores puede llevar a resultados negativos en los equipos de desarrollo, incluyendo lo que se conoce como deuda social —algunas causas y estrategias para mitigarla son abordadas en la PI2—.

Asimismo, Saeeda *et al.* [11] identifican tres elementos generadores de causas en la deuda social: (i) limitaciones sociales, (ii) desafíos organizacionales y (iii) problemas comunitarios, a partir de los cuales se proponen ocho estrategias de mitigación: (1) análisis de redes sociales para una mejor comprensión de la deuda, hasta la (2) comunicación colaborativa, efectiva dentro del equipo, la (3) gestión de la composición del equipo y la mejora de decisiones arquitectónicas, (4) entornos de trabajo híbridos que combinan modalidades presenciales y remotas, (5) métricas específicas para la comunicabilidad de la arquitectura del software y la implementación de (6) marcos de trabajo para la mitigación de la deuda social, como CAFFEA, diseñado para empresas con productos complejos, asimismo, se destacan (7) tácticas arquitectónicas y frameworks sociotécnicos como DAHLIA, y la introducción de (8) herramientas específicas como GEEZMO para monitorear el estado de ánimo, CodeFace4Smell para

TABLA 5. OLORES COMUNITARIOS

No.	Nombre del olor	Causa	Consecuencia
1	Arquitectura por osmosis	Decisiones arquitectónicas sin una adecuada gestión de la información.	Arquitectura inestable y malentendidos.
2	Capucha de arquitectura	Dificultades en la cooperación entre equipos remotos.	Decisiones inconsistentes.
3	Nube negra	Falta de condiciones para una comunicación efectiva entre compañeros.	Disminución de la moral y productividad.
4	Cognición de clase	Desafío de entender clases y estructuras complejas.	Errores por interpretaciones distintas.
5	Código rojo	Clases complejas gestionadas por pocos desarrolladores.	Presión y estrés.
6	Distancia cognitiva	Diferencias entre desarrolladores.	Conflictos y malentendidos.
7	Desarrollo de recetario	Resistencia a nuevas metodologías y falta de innovación.	Falta de innovación.
8	Choque de DevOps	Conflictos entre los equipos de desarrollo y operaciones.	Afecta la integración y despliegue del software.
9	Desvinculación	Percepción de que el software está maduro y se entrega prematuramente.	Afecta la calidad.
10	Dispersión	Las correcciones fragmentan el equipo.	Dificulta la coordinación.
11	Disenso	Incapacidad de alcanzar consenso.	Problemas sin resolver, lo que afecta la calidad del software.
12	Hiper comunidad	Conexión excesiva dentro del equipo.	Retrasa las decisiones y disminuye la productividad.
13	Exceso de informalidad	Falta de protocolos formales.	Desbordamiento de información y afecta la eficiencia.
14	Isomorfismo institucional	Similitudes en los procesos.	Limita la innovación.
15	Arquitectura invisible	Decisiones arquitectónicas no se documentan adecuadamente.	Causa errores y falta de alineación.
16	Técnico sobran	Comunicación deficiente entre desarrolladores y técnicos.	Disminuye la motivación y productividad.
17	Lobo solitario	Trabajo aislado de ciertos miembros.	Dificulta la cohesión y aumenta la duplicación de esfuerzos.
18	Arquitectura solitaria	Decisiones tomadas por no arquitectos.	Genera problemas de escalabilidad.
19	Recién llegados que no aportan	Dependencia excesiva de los nuevos miembros del equipo.	Afecta la productividad y cohesión.
20	Arquitectura obstruida	Falta de visión clara en sub-equipos.	Dificulta la implementación de cambios.
21	Silos organizacionales	Falta de coordinación entre tareas.	Lleva a duplicación de esfuerzos.
22	Escaramuza organizacional	Diferencias culturales entre equipos.	Afectan la gestión del proyecto.
23	Distancia de poder	Percepción de desigualdad.	Interrumpe la comunicación y afecta la toma de decisiones.
24	Miembros pedantes	Exigencia excesiva de conformidad.	Frustra a los compañeros.
25	Prima donnas	Resistencia al cambio y el trabajo aislado.	Afecta la innovación.
26	Silencio radiofónico o cuello de botella	Comunicación ineficiente en estructuras complejas.	Genera retrasos.
27	Villanía de compartir	No se comparte eficazmente el conocimiento.	Disminuye la cohesión y eficiencia.
28	Desafío de soluciones	Conflictos entre sub-equipos.	Impiden la resolución de problemas, afectando el progreso.
29	Distorsión temporal	Malentendidos después de cambios organizacionales.	Causando retrasos y problemas con los clientes.
30	Desaprendizaje	Empleados experimentados resisten el cambio.	Lleva a la pérdida de conocimiento y errores repetidos.

Acronimos utilizados: Número (No.).

detectar causas en silos organizacionales, y YOSHI para mejorar la transparencia en comunidades de código abierto.

B. PI2: ¿Cuáles son los principales desafíos y oportunidades para la investigación futura?

Los principales desafíos en la investigación futura sobre la deuda social en el desarrollo de software ágil incluyen la comprensión de su naturaleza compleja de esta temática, el desarrollo de métricas efectivas para su evaluación, la integración de enfoques interdisciplinarios y la identificación de estrategias de mitigación. Estos desafíos requieren una profunda comprensión de los aspectos técnicos y sociales involucrados, así como la capacidad de medir y abordar la deuda social de manera oportuna y precisa. Dressen *et al.* [17] menciona que actualmente se desconoce qué causa exactamente el aumento de la deuda social y qué mecanismos ayudan a mitigar o disminuir sus efectos adversos. Caballero *et al.* [6] identifican 30 olores comunitarios, estos se presentan en la Tabla 5, junto con una posible causa y consecuencias. Los olores comunitarios en el desarrollo de software se deben entender como las distintas situaciones que pueden afectar la productividad y calidad del equipo.

Al analizar la Tabla 5, es posible observar la variedad de causas y consecuencias descritas en los olores comunitarios identificados, y tal parece que muchos de los “olores” y sus problemas derivan de deficiencias en: (i) Comunicación y Colaboración, (ii) Gestión de la Arquitectura y el Conocimiento, (iii) gestión del conocimiento, (iii) Estructurales y Organizacionales, (iv) Gestión del Talento y Dinámicas de Equipo Individuales, (v) Calidad y Entrega del Software. Con base en este análisis, es posible categorizar las causas y consecuencias en temas más amplios para comprender mejor las principales fuentes e impactos de estos “olores”. La Tabla 6 muestra los problemas y/o deficiencias en categorías genéricas con los números de entrada de los olores correspondientes a la Tabla 5.

TABLA 6. CAUSAS GENÉRICAS DE LOS OLORES

Id.	Categorías genéricas de las Causas	Id. del Olor (ver Tabla 5)
1	Problemas de Comunicación y Colaboración	2, 3, 6, 12, 13, 16, 17, 26, 27
2	Deficiencias en la Gestión de la Arquitectura y el Conocimiento	1, 4, 15, 18, 20, 30
3	Problemas Estructurales y Organizacionales	7, 8, 14, 21, 22, 23, 28, 29
4	Gestión del Talento y Dinámicas de Equipo Individuales	5, 10, 11, 19, 24, 25
5	Calidad y Entrega del Software	9, 11

Acronimos utilizados: Identificador (Id).

A continuación, se presenta una descripción de cada una de las categorías y las relaciones inferidas con los olores relacionados de la Tabla 6:

1) Problemas de Comunicación y Colaboración. Esta categoría engloba los "olores" donde la falta de comunicación efectiva y la dificultad para trabajar en equipo son las causas centrales, llevando a malentendidos, ineficiencia y baja moral. La relación inferida surge por la comunicación deficiente, por ejemplo: la falta de canales adecuados o una transmisión clara de información y la falta de colaboración, por ejemplo: la incapacidad de los equipos o individuos para trabajar juntos de manera efectiva, son las raíces de muchos problemas. Estas fallas se manifiestan en inconsistencias en las decisiones, baja productividad, conflictos y retrasos.

2) Deficiencias en la Gestión de la Arquitectura y el Conocimiento. Esta categoría agrupa los “olores” donde la claridad, documentación y evolución del diseño del software y el conocimiento asociado son insuficientes. La relación inferida surge por una gestión inadecuada de la arquitectura, por ejemplo: decisiones no documentadas o tomadas por personas no cualificadas y una falta de claridad en el conocimiento técnico, por ejemplo: dificultad para entender estructuras complejas o resistencia a aprender llevan a arquitecturas inestables, errores constantes, problemas de escalabilidad y la imposibilidad de adaptarse a nuevas prácticas.

3) Problemas Estructurales y Organizacionales. Esta categoría aborda los desafíos derivados de la organización interna, los procesos y las dinámicas entre diferentes grupos o jerarquías. La relación inferida surge por la falta de coordinación entre departamentos o equipos, los conflictos inter-equipo (especialmente entre diferentes roles como desarrollo y operaciones), y una cultura organizacional rígida o jerárquica, por ejemplo: resistencia a nuevas metodologías o percepción de desigualdad, son causas directas de duplicación de esfuerzos, gestión de proyectos ineficaz y limitación de la innovación.

4) Gestión del Talento y Dinámicas de Equipo Individuales. Esta categoría se centra en cómo las actitudes, las responsabilidades individuales y la gestión de los miembros del equipo impactan la cohesión y el rendimiento. La relación inferida surge por la presión excesiva sobre pocos desarrolladores, la dependencia de nuevos miembros sin el apoyo adecuado, la incapacidad de llegar a consensos y las actitudes individuales problemáticas, por ejemplo: la resistencia al cambio o exigencia de conformidad, son factores que contribuyen directamente al estrés, la baja productividad, la frustración y una cohesión deficiente del equipo.

5) Calidad y Entrega del Software. Esta categoría se enfoca directamente en los “olores” que afectan el resultado final del desarrollo: la calidad del producto y su liberación. La relación inferida surge por la liberación prematura de software inmaduro y la persistencia de problemas técnicos o de diseño debido a la falta de resolución de conflictos son causas directas de la baja calidad del producto. Estos “olores” a menudo son una consecuencia de problemas en las categorías anteriores.

Al analizar las categorías de la Tabla 6, es posible observar que muchos “olores” no existen de forma aislada, por ejemplo: una comunicación deficiente (Categoría 1) puede llevar a una gestión de la arquitectura inadecuada (Categoría 2), lo que a su vez genera problemas estructurales (Categoría 3) y afecta la motivación del equipo (Categoría 4), culminando en una baja calidad del software (Categoría 5), por lo tanto, una deficiencia en la comunicación, a menudo exacerba otro tipo de problemas en la gestión de la arquitectura o las dinámicas del equipo. Por otra parte, abordar la dinámica interpersonal, podría ser un área clave para reducir la aparición y el impacto de olores en el desarrollo de software. En este sentido, abordar estos problemas y/o deficiencias, es crucial un enfoque integral u holístico que identifique no solo el olor superficial, sino también las causas profundas y las relaciones subyacentes entre ellos.

La Tabla 7 muestra las consecuencias y/o impactos en categorías genéricas con los números de entrada de los olores correspondientes a la Tabla 5. En resumen, al analizar cada olor

con su consecuencia directa, se refuerza la idea de que los "olores comunitarios" son síntomas de disfunciones sistémicas que impactan múltiples facetas de un equipo de desarrollo, desde su eficiencia operativa hasta la calidad de su producto y su capacidad de innovar.

TABLA 7. CONSECUENCIAS GENÉRICAS DE LOS OLORES

Id.	Categorías genéricas	Id. del Olor (ver Tabla 5)
1	Impacto en la Productividad y Eficiencia Operativa	3, 12, 13, 16, 17, 19, 21, 26, 27, 29
2	Impacto en la Calidad y Robustez del Software	1, 4, 15, 18, 20, 30
3	Impacto en la Cohesión y el Clima Laboral	7, 8, 14, 21, 22, 23, 28, 29
4	Impacto en la Innovación y Adaptabilidad	5, 10, 11, 19, 24, 25
5	Impacto en la Toma de Decisiones y Resolución de Problemas	9, 11

Acónimos utilizados: Identificador (Id).

A continuación, se presenta una descripción de cada una de las categorías y las relaciones inferidas con los olores relacionados de la Tabla 5:

1) *Impacto en la Productividad y Eficiencia Operativa*. En esta categoría es posible observar que varios olores tienen consecuencias directas en la productividad, eficiencia, retrasos y duplicación de esfuerzos. Esto sugiere que la mayoría de los "olores" tienen un costo operativo tangible en el día a día del equipo. La falta de comunicación, la mala coordinación y las dinámicas de equipo ineficaces son los principales impulsores de esta pérdida de eficiencia.

2) *Impacto en la Calidad y Robustez del Software*. Olores como: Arquitectura por osmosis, Cognición de clase, Choque de DevOps, Desvinculación, Disenso, Arquitectura invisible, Arquitectura solitaria y Desaprendizaje, afectan directamente la calidad del software, la estabilidad de la arquitectura, la integración, el despliegue, la escalabilidad y la aparición de errores. Esto resalta la importancia de una buena gestión técnica y de procesos para asegurar un producto final sólido.

3) *Impacto en la Cohesión y el Clima Laboral*. Consecuencias como la disminución de la moral, conflictos, frustración, dificultad en la coordinación y la cohesión, indican que los "olores" tienen un fuerte componente humano y social. Un ambiente de trabajo negativo o desorganizado afecta directamente el bienestar de los miembros del equipo y su capacidad para colaborar eficazmente.

4) *Impacto en la Innovación y Adaptabilidad*. La falta de innovación es una consecuencia recurrente, a menudo ligada a la resistencia al cambio o a procesos rígidos. Esto sugiere que los "olores" no solo afectan el presente, sino que también comprometen la capacidad de la organización para evolucionar y mantenerse relevante.

5) *Impacto en la Toma de Decisiones y Resolución de Problemas*. Olores como: Disenso, Hiper comunidad, Distancia de poder y Desafío de soluciones muestran cómo las dinámicas internas pueden impedir la toma de decisiones efectiva y la resolución de problemas, lo que a su vez genera retrasos y afecta el progreso general del proyecto.

El análisis de la interacción entre las causas y las consecuencias de los "olores comunitarios" revela una compleja trama de retroalimentación negativa que afecta directamente la eficiencia y la calidad en los equipos de desarrollo de software. Asimismo, es posible observar que las deficiencias en la

comunicación y la colaboración actúan como catalizadores primarios, desencadenando una cascada de problemas que van desde la reducción de la productividad y la moral, hasta la inconsistencia en las decisiones y la fragmentación del equipo. Paralelamente, una gestión arquitectónica y del conocimiento inadecuada conduce a una inestabilidad técnica intrínseca en el producto, manifestada en errores recurrentes y dificultades de escalabilidad, lo cual, a su vez, puede generar mayor frustración y presión en el equipo. Finalmente, los conflictos internos, la resistencia organizacional al cambio y las dinámicas individuales desfavorables no solo limitan la innovación y el progreso del proyecto, sino que también culminan en una entrega de software de baja calidad, cerrando un ciclo vicioso donde los síntomas de la disfunción se perpetúan si las causas raíz no son abordadas de manera estratégica.

Según Caballero *et al.* [6] también se describe una serie de fases que permiten observar cómo surgen los "olores comunitarios" las cuales son: *Fase 1, Inicio*: los líderes toman decisiones sociotécnicas que generan un ambiente tenso, lo que puede llevar a malestar entre los miembros del equipo. A pesar de que este estrés comienza a manifestarse, no es fácilmente detectado por los gerentes. *Fase 2, Impacto directo de los problemas comunitarios*: el equipo experimenta más episodios de emociones negativas, lo que lleva a ignorar procesos importantes y reducir la calidad del trabajo. Aunque los efectos son más notorios, aún pueden pasar desapercibidos por los líderes. *Fase 3, Difusión del equipo*: los miembros afectados por estrés comienzan a influir negativamente en el resto del equipo, lo que provoca baja productividad y la acumulación de deuda social. Aunque los gerentes pueden detectar los problemas, la solución dependerá de sus habilidades y estrategias. *Fase 4, Impacto en la organización*: el deterioro de la calidad del software afecta a más equipos, y los líderes pueden identificar fallos técnicos que requieren atención urgente para evitar que los problemas se extiendan. *Fase 5, Problemas avanzados*: los usuarios externos comienzan a sufrir las consecuencias de los problemas, lo que afecta la reputación de la empresa y provoca pérdidas económicas debido a la pérdida de clientes.

En particular durante la Fase 3, denominada: "Difusión del Equipo", se destaca que los miembros del equipo que sufren episodios recurrentes de emociones negativas comienzan a manifestar sus estados psicológicos internos. Esto tiene un impacto directo en el resto del equipo, derivado de decisiones sociotécnicas que afectan a la cohesión grupal y contribuyen al aumento de la deuda social. Sin embargo, los líderes de los proyectos pueden detectar la presencia de estos efectos, pero el desarrollo de un método preciso para diagnosticar y solucionar los problemas depende de la experiencia del líder y los enfoques de gestión disponibles. Estos líderes tienen una oportunidad significativa para prevenir posibles daños organizativos al supervisar y mitigar periódicamente los efectos de los olores comunitarios dentro de los equipos de desarrollo de software.

En este sentido, se presentan oportunidades para investigar nuevas tecnologías y herramientas, explorar la relación entre la deuda social y el rendimiento del equipo, estudiar el papel de la cultura organizacional y evaluar el impacto a largo plazo de la deuda social en la dinámica del equipo y la calidad del software. Estas oportunidades pueden conducir a una comprensión más

completa de la deuda social en entornos ágiles y al desarrollo de estrategias más efectivas para su gestión y mitigación.

C. PI3: ¿Cuáles son las practicas o enfoques que abordan la gestión de la deuda social en el desarrollo de software ágil?

Dressen *et al.* [13] sugieren una estrategia para abordar la deuda social, fundamentada en el modelo de las 3C's. Este modelo enfatiza las deficiencias en comunicación, colaboración y coordinación mediante una tabla mencionada en el artículo. El enfoque de mitigación sugerido se concentra particularmente en mejorar la comunicación y la colaboración, destacando los siguientes aspectos:

a) *Comunicación honesta y abierta*, la cual es fundamental en equipos de desarrollo de software ágil distribuido para mitigar la deuda social. La comunicación honesta y abierta se define por su franqueza y transparencia, lo que ayuda a construir confianza mutua entre los miembros del equipo. Esta confianza facilita la resolución efectiva de conflictos sociales, ya que los miembros del equipo pueden abordar los problemas directamente. En última instancia, esta comunicación directa contribuye a crear un ambiente de trabajo constructivo y ayuda a reducir la acumulación de deuda social en el equipo.

b) *Intensidad de la comunicación relacionada con las tareas*, se refiere a la naturaleza altamente enfocada y orientada a objetivos de la interacción en el trabajo remoto. Las interacciones y la comunicación no son aleatorias, sino que están determinadas o preparadas de antemano para completar tareas específicas, reduciendo las distracciones. Este enfoque intensificado en la comunicación relacionada con las tareas puede ayudar a mitigar el efecto de los factores que contribuyen a la deuda social en equipos de desarrollo de software ágil distribuido (ASD).

c) *Intensidad de la colaboración relacionada con las tareas*, se refiere a cómo las interacciones durante el trabajo remoto están altamente enfocadas en tareas y objetivos específicos. Esto significa que todas las acciones colaborativas están orientadas a completar la tarea en cuestión, reduciendo las distracciones. Este enfoque intensificado en la colaboración relacionada con las tareas puede tener un efecto tanto en promover como en mitigar DS en equipos de desarrollo de software ágil distribuidos. Por un lado, puede conducir a una comunicación más directa y eficiente, pero por otro, puede resultar en una falta de interacción informal que es importante para la construcción de relaciones y la resolución de conflictos.

d) *Costos iniciales reducidos para iniciar interacciones o comunicaciones*, se refieren a la capacidad de establecer rápidamente un entorno para la colaboración y comunicación uno a uno sin restricciones. Esto es beneficioso en un contexto de trabajo remoto, ya que las salas de reuniones libres ya no son un recurso escaso y se puede configurar la comunicación sin casi ninguna limitación. Este factor ayuda a mitigar la deuda social en equipos de desarrollo de software ágil distribuido al facilitar la resolución de conflictos y promover una comunicación más directa y abierta entre los miembros del equipo.

IV. DISCUSIÓN

La deuda social en el desarrollo de software ágil se ha consolidado como un factor crítico que incide directamente en

la productividad operativa, la calidad del software resultante y la cohesión intrínseca de los equipos de desarrollo. Esta deuda se manifiesta de diversas formas, a menudo a través de lo que se ha denominado “olores comunitarios” Caballero *et al.* [6]. Estos “olores” actúan como señales tempranas, indicando disfunciones sociotécnicas subyacentes que pueden surgir de problemas persistentes en la comunicación, la coordinación, la gestión del conocimiento compartido, las dinámicas internas del equipo y la propia cultura organizacional. Por ejemplo, la presencia de conflictos no resueltos entre miembros del equipo o la ausencia de estándares claros para el trabajo colaborativo pueden ser precursores directos de la acumulación de deuda social.

A través del análisis de la literatura, es posible observar que, para abordar eficazmente esta deuda, se requiere la implementación de un enfoque holístico e integral, y a través de las estrategias de mitigación propuestas por diversos estudios, entre ellos: Dressen *et al.* [13] y Saeeda *et al.* [11], donde consistentemente enfatizan la necesidad de promover una comunicación honesta y abierta, fomentar una colaboración estrechamente orientada a tareas específicas y, crucialmente, reducir las barreras o fricciones que dificultan la interacción fluida y eficiente entre los miembros del equipo. Un modelo como el de las “3C” (comunicación, colaboración y coordinación) se presenta como una herramienta útil para clasificar los olores comunitarios y diseñar estrategias de intervención adecuadas. Además, se han identificado y propuesto diversas estrategias y herramientas específicas, desde el análisis de redes sociales para comprender las dinámicas internas hasta la implementación de marcos de trabajo sociotécnicos y el uso de herramientas que monitorean el estado de ánimo del equipo o detectan problemas organizacionales.

A pesar de los avances en la identificación y caracterización de la deuda social, existen desafíos significativos que limitan una gestión óptima, persistiendo una falta de claridad sobre las maneras de identificar, medir y cuantificar la deuda social a través de las causas exactas que provocan su aumento, así como la determinación de cuáles son los mecanismos más efectivos para mitigar los efectos adversos que produce. La complejidad inherente a la interconexión de los factores sociales y técnicos, la urgente necesidad de desarrollar métricas efectivas y estandarizadas para su evaluación cuantitativa y cualitativa, y la importancia de integrar enfoques interdisciplinarios que combinen conocimientos de ingeniería de software, psicología organizacional y ciencias sociales, representan barreras significativas. No obstante, estos desafíos, a su vez, abren un amplio y prometedor campo de oportunidades para la investigación futura. La exploración de nuevas tecnologías para la detección temprana, el estudio del impacto a largo plazo en la resiliencia organizacional y el desarrollo de intervenciones basadas en evidencia podrían llevar a una comprensión más profunda y, en última instancia, a soluciones más robustas y sostenibles para mantener la salud social y la eficiencia técnica en los equipos de desarrollo de software ágil.

V. CONCLUSIONES Y TRABAJO FUTURO

Los hallazgos derivados de esta revisión sistemática de la literatura sugieren que la deuda social en el desarrollo de software ágil es un fenómeno crítico y multifacético. Asimismo,

se ha podido evidenciar que esta deuda no es un inconveniente de menor importancia, sino una acumulación de tensiones no técnicas, como el agotamiento emocional, la falta de reconocimiento profesional y el desequilibrio entre vida laboral y personal, que afectan directamente el bienestar del equipo. Además, que su manifestación se da a través de “olores comunitarios”, señales tempranas de disfunciones sociotécnicas subyacentes, que abarcan desde conflictos interpersonales y la falta de transparencia hasta una distribución desigual de las cargas de trabajo. La persistencia de estos “olores” tiene un costo operativo tangible, impactando la productividad, la eficiencia, la moral, la calidad del software y la capacidad de innovación.

A pesar de la creciente conciencia sobre el impacto de la deuda social, esta revisión subraya la persistencia de vacíos significativos en la literatura, específicamente, en que existe una falta de consenso sobre métricas estandarizadas que permitan cuantificarla de manera efectiva, y escasean las estrategias validadas empíricamente para su mitigación. Esta dispersión teórica y metodológica limita la capacidad de las organizaciones para implementar soluciones efectivas y oportunas. La complejidad inherente a la interconexión de factores sociales y técnicos, como la interacción entre una comunicación deficiente y una gestión arquitectónica inadecuada, requiere un enfoque integral que vaya más allá de la mera identificación de síntomas para abordar las causas profundas y las relaciones subyacentes entre los diferentes aspectos que la caracterizan.

En este contexto, la investigación futura se presenta con oportunidades valiosas para avanzar en el campo, entre ellas: (i) Es importante desarrollar herramientas de diagnóstico precisas que permitan a los líderes de proyecto no solo detectar la presencia de estos “olores” en fases tempranas, como la “difusión del equipo”, sino también medir su alcance y profundidad, superando la dependencia de la experiencia individual. Además, la (ii) validación empírica de estrategias de mitigación replicables es fundamental para establecer buenas prácticas que puedan ser adoptadas en diversos entornos de desarrollo ágil, esto incluye la evaluación de modelos como el de las “3C” (comunicación, colaboración y coordinación) y la eficacia de herramientas específicas que monitoreen el estado de ánimo o detecten problemas organizacionales. Finalmente, se identifica una valiosa oportunidad para explorar cómo las (iii) nuevas tecnologías, entre ellas la Inteligencia Artificial, pueden facilitar la detección temprana de los “olores comunitarios” y para investigar el (iv) impacto a largo plazo de la deuda social en la resiliencia organizacional. Un enfoque interdisciplinario, que combine conocimientos de ingeniería de software, psicología organizacional y ciencias sociales, será crucial para fomentar un ambiente de trabajo saludable y productivo, que no solo sea sostenible sino también propicio para la innovación y la creatividad.

AGRADECIMIENTOS

El profesor César Pardo agradece a la Universidad del Cauca el apoyo institucional brindado como profesor titular. El profesor Vladimir Villarreal es miembro del SNI en Panamá. La Profesora Eydy Suárez Brieva, profesora asociada de la Universidad Popular del Cesar, expresa su agradecimiento por el apoyo recibido a lo largo del desarrollo de este trabajo.

REFERENCIAS

- [1] D. A. Tamburri, P. Kruchten, P. Lago, y H. Van Vliet, “What is social debt in software engineering?”, en *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2013 - Proceedings*, 2013, pp. 93-96. doi: 10.1109/CHASE.2013.6614739.
- [2] S. Jalali y C. Wohlin, “Systematic literature studies: Database searches vs. backward snowballing,” en *Proceedings of the 2012 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, 2012, pp. 29-38. doi: 10.1145/2372251.2372257.
- [3] T. Dybå y T. Dingsøyr, “Strength of Evidence in Systematic Reviews in Software Engineering,” en *ESEM’08: Proceedings of the 2008 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, junio de 2008, pp. 178-187. doi: 10.1145/1414004.1414034.
- [4] L. Yang *et al.*, “Quality Assessment in Systematic Literature Reviews: A Software Engineering Perspective,” febrero 01, 2021, *Elsevier B.V.* doi: 10.1016/j.infsof.2020.106397.
- [5] M. Ivarsson y T. Gorschek, “A method for evaluating rigor and industrial relevance of technology evaluations,” *Empirical Software Engineering*, vol. 16, no. 3, pp. 365-395, octubre de 2010, doi: 10.1007/s10664-010-9146-4.
- [6] E. Caballero-Espinosa, J. C. Carver, y K. Stowers, “Community smells—The sources of social debt: A systematic literature review,” *Information and Software Technology*, vol. 153, enero de 2023, doi: 10.1016/j.infsof.2022.107078.
- [7] E. S. Brieva, C. Pardo, y V. Villarreal, “Deuda Social y Riesgos psicosociales en entornos de desarrollo de software ágil: Análisis preliminar de una Revisión Sistemática de la Literatura,” en *2024 IEEE VII Congreso Internacional en Inteligencia Ambiental, Ingeniería de Software y Salud Electrónica y Móvil (AmITIC)*, 2024, pp. 1-8. doi: 10.1109/AmITIC62658.2024.10747594.
- [8] F. Palomba, D. A. Tamburri, A. Serebrenik, A. Zaidman, F. A. Fontana, y R. Oliveto, “How do community smells influence code smells?,” en *Proceedings - International Conference on Software Engineering*, 2018, pp. 240-241. doi: 10.1145/3183440.3194950.
- [9] D. A. Tamburri, P. Kruchten, P. Lago, y H. van Vliet, “Social debt in software engineering: insights from industry,” *Journal of Internet Services and Applications*, vol. 6, no. 1, diciembre. 2015, doi: 10.1186/s13174-015-0024-6.
- [10] A. Martini, V. Stray, y N. B. Moe, “Technical-, Social- and Process Debt in Large-Scale Agile: An Exploratory Case-Study,” en *Lecture Notes in Business Information Processing*, Springer Verlag, 2019, pp. 112-119. doi: 10.1007/978-3-030-30126-2_14.
- [11] H. Saeeda, M. Ovais Ahamd, y T. Gustavsson, “A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt,” *e-Informatica Software Engineering Journal*, vol. 18, no. 1, p. 240101, 2024, doi: 10.37190/e-inf240101.
- [12] E. A. C. Espinosa, “Understanding Social Debt in Software Engineering,” University of Alabama, Tuscaloosa, Alabama, USA, 2021. [Online]. Available: <http://ir.ua.edu/handle/123456789/8278>
- [13] T. Dreesen, P. Hennel, C. Rosenkranz, y T. Kude, “The second vice is lying, the first is running into debt. Antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams,” en *Hawaii International Conference on System Sciences*, 2021, pp. 6826-6835. doi: 10.24251/hicss.2021.818.
- [14] D. A. Tamburri, P. Kruchten, P. Lago, y H. van Vliet, “Social debt in software engineering: insights from industry,” *Journal of Internet Services and Applications*, vol. 6, no. 1, p. 10, 2015, doi: 10.1186/s13174-015-0024-6.
- [15] E. S. Brieva, J. E. Bejarano Guar, N. D. Urbano Palechor, y C. J. Pardo Calvache, “Olores comunitarios en comunidades de desarrollo de software: Revisión Sistemática de la Literatura,” *Ingeniería y Competitividad*, vol. 27, no. 2, junio de 2025, doi: 10.25100/iyec.v27i2.14826.
- [16] E. S. Brieva, C. de J. Pardo Calvache, y H. A. Ordoñez Eraso, “Instrumento para medir las percepciones de las comunidades de desarrollo de software respecto a la comunicación, coordinación y cooperación: Resultado de la validación mediante juicio de expertos,” *Inge CuC*, vol. 20, no. 1, pp. 116-134, mayo de 2024, doi: 10.17981/ingecuc.20.1.2024.10.
- [17] T. Dreesen, P. Hennel, C. Rosenkranz, y T. Kude, “The second vice is lying, the first is running into debt. Antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams,” *Proceedings of the Annual Hawaii International Conference on System Sciences*, vol. 2020-Janua, pp. 6826-6835, enero de 2021, doi: 10.24251/hicss.2021.818.
- [18] B. Kitchenham y S. Charters, “Guidelines for performing Systematic Literature Reviews in Software Engineering Version 2.3,” Durham, UK, julio de 2007. doi: 10.1145/1134285.1134500.
- [19] D. Budgen, M. Turner, P. Brereton, y B. Kitchenham, “Using Mapping Studies in Software Engineering,” vol. 2, 2007.
- [20] G. T. Doran, “There’s a S.M.A.R.T. Way to Write Management’s Goals and Objectives,” *Management Review*, vol. 70, pp. 35-36, 1981.
- [21] R. Van Solingen, V. Basili, G. Caldiera, y H. D. Rombach, “Goal Question Metric (GQM) Approach,” 2002. doi: 10.1002/0471028959.sof142.
- [22] S. McCombes, “Writing strong research questions | Criteria & examples,” agosto de 2023. [En línea]. Disponible en: <https://www.scribbr.com/research-process/research-questions/>
- [23] A. B. M. N. A. Talukder, “Understanding Social Debt in Software Engineering,” no. septiembre de 2022.

ANEXO C: ACUERDO DE CONFIDENCIALIDAD DEL SOCIAL GUIDE

C.1 Declaración de Confidencialidad y Compromiso Ético

Yo, _____ (nombre completo del Social Guide), en mi rol de **Social Guide** del equipo _____ (nombre del equipo), me comprometo formalmente a cumplir con los principios éticos y salvaguardas de confidencialidad descritos a continuación, en el marco de la implementación de SocialScrum.

C.1.1 1. Confidencialidad absoluta de información individual

- **No compartiré información que permita identificar a personas específicas** fuera del equipo, incluyendo, entre otros:
 - Comentarios realizados durante eventos sociales (Social Daily Standup, Social Sprint Retrospective, sesiones de mediación).
 - Conflictos interpersonales descritos por miembros del equipo.
 - Información relacionada con estado emocional, burnout, desmotivación o vulnerabilidades personales.
 - Cualquier dato que pueda revelar directa o indirectamente la identidad de un miembro del equipo.
- **Solo reportaré información agregada y anonimizada**, como:
 - El Health Score promedio del equipo (escala 1–10).
 - *Community smells* identificados a nivel de equipo, sin atribuir responsabilidades individuales.
 - Tendencias de salud social a lo largo del tiempo.
 - Necesidades de intervención organizacional sin mencionar casos particulares.

C.1.2 2. Estructura de reporte y rendición de cuentas

Regla fundamental: El Social Guide nunca reporta a Recursos Humanos. Reporta al CTO/VP Engineering a través del Scrum Master.

C.1.2.1 Tipos de reportes

Tipo	Frecuencia	Contenido	Audiencia
Interno al equipo	Cada Sprint	Health Score del equipo, <i>community smells</i> activos y acciones sociales completadas durante el Sprint.	Equipo
Ejecutivo	Mensual	Health Score promedio, <i>community smells</i> críticos y solicitudes de recursos o apoyo organizacional.	CTO, Scrum Master

Tabla C.2: Reportes del Social Guide

Todos los reportes son **agregados y anónimos**. No identifican personas.

C.1.2.2 Protocolo si HR solicita información

1. Social Guide rechaza inmediatamente sin proporcionar información.
2. Informa al SM y CTO de la solicitud.
3. CTO evalúa si hay justificación legal (ej: investigación formal de acoso).
4. Si hay justificación legal: información estrictamente relevante, con consentimiento documentado.
5. Si no hay justificación: CTO responde que la información está protegida por confidencialidad.

C.1.2.3 Rendición de cuentas

- **Auditoría semestral:** Auditor externo entrevista al equipo, revisa reportes, emite informe público.
- **Feedback continuo:** Encuesta anónima cada Sprint (confidencialidad, seguridad, neutralidad). Si <60 % aprobación por 2 Sprints consecutivos, revisión formal.

C.1.2.4 Dedicación del rol

Contexto	Dedicación	Modelo
Equipo de 5–7 personas con Health Score ≥ 7	10–20 %	Rol rotativo
Equipo de 5–7 personas con Health Score < 7	30–40 %	Rol parcial
Equipo de 8–15 personas	30–70 %	Rol parcial o casi completo
Equipo de 16 o más personas	80–100 %	Tiempo completo
Múltiples equipos	100 %	Dedicado a 2–3 equipos

Tabla C.4: Dedicación del Social Guide según contexto

C.1.3 3. Separación total entre salud social y evaluación de desempeño

- **No utilizaré** la información que recopile como Social Guide para:
 - Evaluaciones de desempeño individuales.
 - Decisiones de contratación, promoción, reasignación o despido.
 - Justificar sanciones disciplinarias.
 - Influir en decisiones salariales o de compensación.
- **Rechazaré cualquier solicitud de información individual**, incluso si proviene de:
 - Recursos Humanos (HR), salvo los casos excepcionales previstos en la Sección 4.
 - Gerencia o liderazgo técnico, cuando dicha solicitud viole la confidencialidad acordada.
 - Entidades externas a la organización.

C.1.4 4. Estructura de reporte y rendición de cuentas

- **Reportaré exclusivamente a:**
 - El Scrum Master del equipo.
 - El Chief Technology Officer (CTO) o Vicepresidente de Ingeniería (VP Engineering).
- **NO reportaré directamente a:**
 - Recursos Humanos (HR), excepto en las excepciones detalladas en la Sección 4.
 - Gerencia de producto o partes interesadas (*stakeholders*) externos al área técnica.

- **Los reportes serán completamente transparentes para el equipo:**
 - Presentaré cualquier reporte al equipo antes de enviarlo a liderazgo técnico.
 - El equipo podrá solicitar ajustes si considera que se está violando la confidencialidad.
 - No generaré reportes paralelos ni comunicaciones ocultas.

C.1.5 5. Excepciones legalmente justificadas

Reconozco que existen situaciones excepcionales que requieren suspender la confidencialidad:

- **Riesgo inminente de daño:** si un miembro expresa intención de autolesión, daño a terceros o conducta ilegal, debo reportarlo a las autoridades correspondientes (HR, seguridad, etc.).
- **Investigaciones formales por acoso o discriminación:** en estos casos:
 - Solo compartiré información con consentimiento explícito de la(s) persona(s) afectada(s), cuando sea posible.
 - Me limitaré estrictamente a los hechos pertinentes.
 - Documentaré la excepción y notificaré al equipo siempre que sea legalmente permitido.

Siempre que la ley lo permita, **informaré al equipo** cuando deba suspender la confidencialidad y explicaré las razones.

C.1.6 6. Protección de la seguridad psicológica del equipo

- **Mi prioridad principal** es proteger la seguridad psicológica del equipo: que cada persona pueda expresar vulnerabilidades sin temor a represalias.
- **Actuaré con integridad** en todo momento:
 - No participaré en rumores o conversaciones que expongan vulnerabilidades.
 - No utilizaré información confidencial para influir en decisiones que no sean estrictamente sociales.
 - Seré coherente entre lo que prometo y lo que hago.
- **Facilitaré sin imponer:** mi rol no es “arreglar” al equipo, sino acompañarlo para que encuentre sus propias soluciones.

C.1.7 7. Derecho de auditoría del equipo

- El equipo tiene derecho a:
 - Revisar cualquier reporte que genere antes de compartirlo fuera del equipo.
 - Solicitar copias de todas mis comunicaciones relacionadas con mi rol.
 - Auditar mi cumplimiento de este acuerdo en cualquier momento.
- Si el equipo detecta una violación:
 - Puede solicitar mi remoción inmediata del rol.
 - Puede escalar la situación al CTO o a un auditor externo.
 - Puede suspender SocialScrum hasta restablecer la confianza.

C.1.8 8. Compromiso de mejora continua

- Me comprometo a:
 - Participar en auditorías semestrales realizadas por un auditor independiente.
 - Recibir retroalimentación del equipo sobre mi desempeño.
 - Mantener formación continua en ética, confidencialidad y facilitación.

C.1.9 9. Consecuencias de la violación de este acuerdo

Reconozco que, si incumplo alguno de los principios establecidos:

- Seré **removido de inmediato** del rol de Social Guide.
- **No podré ejercer este rol nuevamente** dentro de la organización.
- La organización ofrecerá una **disculpa formal** al equipo.
- Se realizará una **investigación formal** y se tomarán medidas correctivas si hubo daño.

Firma del Social Guide: _____

Nombre completo: _____

Fecha: _____

Testigo (Scrum Master): _____

Nombre completo: _____

Fecha: _____

Testigo (CTO/VP Ingeniería): _____

Nombre completo: _____

Fecha: _____

ANEXO D: CONSENTIMIENTO INFORMADO DEL EQUIPO PARA SOCIALSCRUM

D.1 Consentimiento Informado para la Implementación de SocialScrum

Yo, _____ (nombre completo del miembro del equipo), en mi rol de _____ (rol en el equipo: Developer, QA, etc.), declaro que he recibido una explicación clara y completa sobre la implementación del marco SocialScrum en el equipo _____ (nombre del equipo).

D.1.1 1. Comprensión del propósito de SocialScrum

Entiendo que SocialScrum es un proceso ágil orientado a:

- **Identificar, priorizar y mitigar la deuda social** del equipo, entendida como decisiones socio-técnicas que afectan nuestra productividad, calidad del software y cohesión interna.
- **Gestionar *community smells*** (patrones de comportamiento disfuncionales) mediante ciclos continuos de transparencia, inspección y adaptación.
- **Promover un entorno de trabajo sano y sostenible**, donde el bienestar del equipo y la evolución del producto avanzan de forma equilibrada.

También entiendo que SocialScrum NO es:

- Un sistema de evaluación de desempeño individual.
- Un mecanismo de vigilancia o control organizacional.
- Terapia grupal ni un proceso de intervención psicológica.
- Un requisito obligatorio para mantener mi empleo. La participación es voluntaria.

D.1.2 2. Información que será recopilada

Entiendo que, durante SocialScrum, se recopilará lo siguiente:

D.1.2.1 2.1. Información cuantitativa

- **Health Score:** Medición periódica (por Sprint) de los cinco valores de Scrum —Apertura, Compromiso, Respeto, Foco y Coraje— con una escala de 1 a 10.
- **Métricas sociales agregadas:** patrones de comunicación, distribución del conocimiento (Análisis de Redes Sociales (SNA, Social Network Analysis)), señales de burnout (horas extra, cambios abruptos en la velocidad), etc.
- **Community smells:** patrones disfuncionales detectados a nivel de equipo (no individual), como Radio Silence (Silencio Radiofónico), Lone Wolf (Lobo Solitario), Black Cloud (Nube Negra), entre otros.

D.1.2.2 2.2. Información cualitativa

- **Comentarios en eventos sociales:** Aportes realizados durante Social Daily Standup, Social Sprint Retrospective o sesiones de mediación.
- **Entrevistas 1:1 opcionales:** Conversaciones privadas con el Social Guide para profundizar en temas relevantes.
- **Observaciones contextuales:** Patrones de comportamiento observados en la dinámica diaria del equipo.

D.1.3 3. Cómo será utilizada la información

Comprendo que la información recopilada será usada **únicamente** para:

- **Mejoras internas:** Identificar *community smells*, diseñar intervenciones y medir su impacto.
- **Reportes agregados:** El Social Guide puede compartir únicamente datos anonimizados con el CTO o Scrum Master (por ejemplo: “El equipo tiene un Health Score de 5.8/10”), sin revelar identidades individuales.
- **Seguimiento de tendencias:** Revisar cómo evoluciona la salud social del equipo a lo largo del tiempo.

La información **NO** será utilizada para:

- Evaluaciones de desempeño.
- Decisiones de contratación, promoción, rotación o despido.
- Acciones disciplinarias o llamados de atención.
- Determinar compensación salarial.

D.1.4 4. Quién tendrá acceso a la información

Entiendo que:

- **El Social Guide** gestiona y protege toda la información recopilada.
- **El equipo completo** tendrá acceso a reportes agregados (Health Score, *community smells*, acciones de mitigación).
- **El Scrum Master y el CTO/VP de Ingeniería** recibirán solo información agregada y anonimizada.
- **Recursos Humanos (HR)** no tendrá acceso a información individual, salvo en los casos excepcionales explicados en la Sección 6.

D.1.5 5. Salvaguardas de confidencialidad

Entiendo que mi privacidad está protegida mediante:

- **Confidencialidad absoluta:** El Social Guide no compartirá información individual fuera del equipo.
- **Anonimización total:** Todos los reportes hacia liderazgo serán agregados.
- **Separación estricta entre salud social y desempeño:** La información de SocialScrum no influirá en mi evaluación laboral.
- **Derecho de auditoría:** Puedo revisar cualquier reporte antes de que sea compartido fuera del equipo.

D.1.6 6. Excepciones a la confidencialidad

Comprendo que la confidencialidad puede suspenderse solo en dos escenarios:

- **Riesgo inminente de daño:** autolesión, daño a terceros o conducta ilegal.
- **Investigaciones formales por acoso o discriminación:** el Social Guide podrá compartir información relevante, idealmente con mi consentimiento, y solo aquella estrictamente relacionada con la investigación.

En estos casos, el Social Guide me informará siempre que sea legalmente posible.

D.1.7 7. Mis derechos como participante

Entiendo que tengo derecho a:

- **Retirarme en cualquier momento**, sin consecuencias laborales.
- **Opt-out parcial**: elegir no participar en ciertos eventos sin ser penalizado(a).
- **Auditar reportes**: revisar reportes antes de ser compartidos.
- **Denunciar violaciones de confidencialidad** al Scrum Master, CTO o un auditor independiente.
- **Solicitar explicaciones** sobre cómo se está utilizando la información.

D.1.8 8. Duración del consentimiento

Este consentimiento será válido durante:

- El piloto inicial de SocialScrum (mínimo 3 Sprints).
- Una duración indefinida, siempre que:
 - Las salvaguardas éticas se mantengan.
 - Yo no haya solicitado retirarme.
 - Al menos el 70 % del equipo continúe apoyando SocialScrum.

Si deseo retirarme, puedo hacerlo por escrito ante el Social Guide o el Scrum Master. A partir de ese momento, mi información dejará de ser recopilada.

D.1.9 9. Declaración de consentimiento voluntario

Declaro que:

- He leído y entendido este documento.
- He tenido oportunidad de hacer preguntas y todas fueron respondidas.
- Sé que mi participación es voluntaria y reversible.
- Acepto participar en SocialScrum bajo las condiciones descritas.

Firma del participante: _____

Nombre completo: _____

Rol en el equipo: _____

Fecha: _____

Testigo (Social Guide): _____

Nombre completo: _____

Fecha: _____

Testigo (Scrum Master): _____

Nombre completo: _____

Fecha: _____

D.1.10 Nota para el equipo

Si menos del 70 % de los miembros del equipo firma este consentimiento, SocialScrum NO debe implementarse. Esto asegura apoyo mayoritario y evita que la adopción sea impuesta a quienes no están convencidos.

ANEXO E: HERRAMIENTAS Y TECNOLOGÍA DE SOPORTE PARA SOCIALSCRUM

Este anexo presenta las adaptaciones contextuales y herramientas tecnológicas recomendadas para implementar SocialScrum en diversos entornos. Incluye guías para ajustar el marco según el tamaño del equipo y la modalidad de trabajo, así como configuraciones específicas para Jira y Azure DevOps. Además, se proporcionan plantillas operacionales estandarizadas y ejemplos de dashboards automatizados para monitorear la salud social del equipo.

E.1 Adaptación contextual y escalabilidad

SocialScrum no es un marco monolítico; debe adaptarse al contexto específico de cada equipo. Un equipo de 5 personas co-localizadas en una startup tiene necesidades diferentes a un equipo de 20 personas distribuidas en 4 zonas horarias dentro de una corporación. Esta sección proporciona guías de adaptación según tamaño del equipo y modalidad de trabajo, asegurando que SocialScrum sea viable independientemente del contexto. El principio general es proporcionalidad: equipos pequeños requieren implementación ligera; equipos grandes requieren estructura más formal. La meta es mantener los beneficios de SocialScrum sin que la sobrecarga supere lo que el contexto puede absorber.

E.1.1 Configuración por tamaño de equipo

La tabla E.2 resume la configuración recomendada de SocialScrum según el tamaño del equipo:

Aspecto	Pequeño (5–7)	Mediano (8–15)	Grande (16+)
Social Guide	Rol rotativo (10 %)	Dedicación 50 %	Dedicación 100 %
Health Score	3 dimensiones	5 dimensiones	5 dimensiones + agregado organizacional
Social Retrospective	15 min integrada	30–45 min	Por subgrupo + general
Sobrecarga por Sprint	2–2.5 horas	21–29 horas	40+ horas
Auditoría	Scrum Master informal	Auditor externo semestral	Auditor externo semestral
Escalamiento a dedicado	Si HS < 4 por 3 Sprints	Si HS < 4 por 2 Sprints	Siempre dedicado

Tabla E.2: Configuración de SocialScrum por tamaño de equipo

Para **equipos pequeños**, el rol rotativo funciona porque: (1) todos los miembros desarrollan sensibilidad social, (2) nadie carga permanentemente con la responsabilidad del rol, y (3) diferentes perspectivas enriquecen la observación del equipo. Los requisitos para esta modalidad son una capacitación básica de dos horas para todos los miembros, un handover estructurado de treinta minutos entre Social Guides, y la disponibilidad del Scrum Master como respaldo.

Para **equipos medianos**, una dedicación aproximada del 50 % permite que el Social Guide permanezca conectado con la realidad técnica del equipo, preservando su credibilidad y evitando una separación excesiva entre el rol social y el trabajo diario.

Para **equipos grandes o múltiples equipos**, se contemplan tres configuraciones posibles: (a) un Social Guide por equipo con dedicación del 50 %, (b) un Social Guide compartido con dedicación completa para dos o tres equipos, o (c) un Social Guide Lead con representantes por equipo en organizaciones de 50 o más personas.

E.1.2 Adaptación por modalidad de trabajo

La modalidad de trabajo afecta significativamente cómo se implementa SocialScrum. El principio para equipos híbridos es “remote-first”: todas las prácticas deben diseñarse como si todos fueran remotos.

Aspecto	Co-localizado	Distribuido	Híbrido
Social Daily Standup	Presencial, 15–18 min	Asíncrono (Slack) o videollamada	Remote-first (todos conectados desde dispositivo)
Social Sprint Retrospective	Presencial con <i>post-its</i>	Digital (Miro u otras herramientas) + videollamada	Digital para todos los participantes
Observación	Directa y contextual (interacciones informales)	Patrones en Slack y sesiones 1:1 virtuales	Combinada, con énfasis en señales digitales
Cohesión	Almuerzos y eventos presenciales	Charlas informales (<i>coffee chats</i>) virtuales y canales sociales	Ambas modalidades, priorizando actividades inclusivas
Mediación	Preferentemente presencial	Videollamada; presencial puntual si es necesario	Según disponibilidad y contexto de los participantes
Health Score	Recopilación durante la retrospectiva	Formulario asíncrono con discusión poste-	Formulario asíncrono para todos

virtuales para detectar señales no verbales. Para equipos híbridos, la regla es: si 4 personas están en oficina y 3 remotas, cada persona se conecta desde su laptop para igualar la experiencia.

La adaptación contextual asegura que SocialScrum sea viable en cualquier configuración. La sobrecarga debe ser proporcional a la complejidad social: un equipo de 5 personas co-localizadas no necesita la misma infraestructura que una organización de 50 personas distribuidas en 4 continentes.

E.2 Herramientas y tecnología

E.2.1 Integración con Jira

E.2.1.1 Campos personalizados

Jira permite crear campos personalizados que capturan la información específica de SocialScrum:

Campo	Tipo	Descripción
Item Type	Select	TECHNICAL / SOCIAL
Social Points (SSP)	Number	Escala 1-13, solo para ítems sociales
Community Smell	Select	<i>Radio Silence, Conflict, Burnout</i> , etc.
Health Dimension	Multi-select	Apertura, Respeto, Coraje, Compromiso, Foco
Social Guide Assigned	User picker	Responsable del ítem social
Severity	Select	Leve, Moderada, Severa, Crítica

Tabla E.5: Campos personalizados para SocialScrum en Jira

E.2.1.2 Configuración de tipos de incidencias (*Issue Types*)

=== CONFIGURACIÓN DE ISSUE TYPES ===

Opción A: Etiqueta [SOCIAL]

- Usar Issue Type: Story
- Agregar label: "SOCIAL"
- Filtrar en boards con JQL: labels = "SOCIAL"

Opción B: Issue Type dedicado (requiere admin)

- Crear Issue Type: "Social Item"
- Icono distintivo (ej: corazón o personas)
- Workflow propio si se requiere

Recomendación: Empezar con Opción A (sin fricción), migrar a Opción B cuando el proceso esté maduro.

E.2.1.3 Workflow para ítems sociales

=== WORKFLOW PARA ÍTEMS SOCIALES ===

Estados:

1. Backlog Social -> Ítem identificado, no priorizado
2. Ready -> Priorizado para próximo Sprint
3. In Progress -> Social Guide trabajando activamente
4. Monitoring -> Intervención completa, en observación
5. Done -> Mejoría confirmada en Health Score

Transiciones:

- Backlog -> Ready: Durante Sprint Planning
- Ready -> In Progress: Al comenzar el Sprint
- In Progress -> Monitoring: Cuando se completa la intervención
- Monitoring -> Done: Cuando Health Score dimensión mejora ≥ 1 punto
- Monitoring -> In Progress: Si la situación empeora

Automatizaciones:

- Agregar comentario automático al cambiar a Monitoring:
"Recordar revisar en próxima retrospectiva"
- Notificar al Social Guide si ítem en "In Progress" > 5 días

E.2.1.4 Filtros JQL para SocialScrum

=== FILTROS JQL ÚTILES ===

1. Ítems sociales en Sprint actual:
labels = "SOCIAL" AND Sprint = currentSprint()
2. Ítems sociales críticos sin asignar:
labels = "SOCIAL" AND "Severity" = "Crítica"
AND "Social Guide Assigned" IS EMPTY
3. Ítems sociales completados último mes:
labels = "SOCIAL" AND status = Done

AND resolved >= -30d

4. Capacidad social consumida en Sprint:

labels = "SOCIAL" AND Sprint = currentSprint()
AND status != Backlog

5. Smells de comunicación activos:

labels = "SOCIAL" AND "Community Smell" IN
("Radio Silence", "Lone Wolf", "Black Cloud")
AND status != Done

6. Health Score en dimensión en riesgo:

labels = "SOCIAL" AND "Health Dimension" = "Respeto"
AND "Severity" IN ("Severa", "Crítica")

E.2.2 Integración con Azure DevOps

Azure DevOps permite personalización similar a través de su sistema de Work Item Types y campos personalizados.

E.2.2.1 Campos personalizados en Azure DevOps

=== CAMPOS PERSONALIZADOS EN AZURE DEVOPS ===

Navegación: Organization Settings > Process > [Tu proceso] >
User Story > New Field

Campos a crear:

1. Campo: "Social Points"

- Tipo: Integer
- Descripción: Estimación en SSP (1-13)
- Layout: Agregar a pestaña "Details"

2. Campo: "Community Smell"

- Tipo: Picklist
- Valores: Radio Silence, Truck Factor, Conflict,
Meeting Hell, Burnout, Black Cloud, Lone Wolf,
Knowledge Islands, Bottleneck, Invisible Output
- Layout: Agregar a tab "Classification"

3. Campo: "Health Dimension"
 - Tipo: Picklist (multiselect)
 - Valores: Apertura, Compromiso, Respeto, Foco, Coraje
 - Layout: Agregar a tab "Classification"
4. Campo: "Smell Severity"
 - Tipo: Picklist
 - Valores: Leve, Moderada, Severa, Crítica
 - Regla: Requerido si Community Smell no está vacío
5. Campo: "Social Guide"
 - Tipo: Identity
 - Descripción: Persona responsable del ítem social
 - Regla: Requerido para ítems con tag SOCIAL

E.2.2.2 Queries para Azure DevOps

=== QUERIES EQUIVALENTES EN AZURE DEVOPS ===

1. Ítems sociales en iteración actual:
 - Work Item Type = User Story
 - AND Tags Contains "SOCIAL"
 - AND Iteration Path = @CurrentIteration
2. Ítems sociales críticos sin asignar:
 - Tags Contains "SOCIAL"
 - AND Smell Severity = "Crítica"
 - AND Social Guide = ""
3. Dashboard de capacidad social:
 - Tags Contains "SOCIAL"
 - AND State <> "New"
 - AND Iteration Path = @CurrentIteration
 - Group by: State
 - Sum: Social Points

E.2.2.3 Integración de Health Score

=== INTEGRACIÓN DE HEALTH SCORE ===

Opción 1: Extensión de Azure DevOps

- Crear extensión personalizada que agregue widget de Health Score
- Conectar con formulario de encuesta (Microsoft Forms)
- Mostrar en dashboard del equipo

Opción 2: Wiki integrado

- Crear página de Wiki: "Health Score - [Equipo]"
- Actualizar manualmente después de cada retrospectiva
- Vincular desde Work Items sociales

Opción 3: Power BI embebido

- Crear reporte de Power BI con datos de Health Score
- Embeber en Azure DevOps dashboard
- Refresh automático desde Excel/SharePoint

E.2.3 Plantillas operacionales

Las siguientes plantillas estandarizan la captura de información y facilitan la adopción de SocialScrum.

E.2.3.1 Plantilla de User Story Social

=== USER STORY SOCIAL - PLANTILLA ===

TITULO: [SOCIAL-XXX] [Descripción breve del problema]

CONTEXTO DEL COMMUNITY SMELL

Tipo de smell: [Seleccionar de catálogo]

Severidad: [Leve | Moderada | Severa | Crítica]

Detectado por: [Social Guide / Equipo / Anónimo]

Detectado en: [Evento o momento]

Sprint de detección: [N]

Manifestación observable:

[Describir comportamientos específicos observados,

sin juicios de valor ni atribuciones de intención]

Personas afectadas:

- Número de personas: [N]
- Roles afectados: [Developer, PO, SM, etc.]

Impacto en Health Score:

- Dimensión afectada: [Apertura/Compromiso/Respeto/Foco/Coraje]
- Score actual: [X]/10
- Score objetivo: [Y]/10

CRITERIOS DE ACEPTACIÓN

- [] Health Score de la dimensión [X] mejora de [actual] a [objetivo]
- [] Comportamiento observable [Y] ya no se manifiesta
- [] Encuesta anónima post-intervención muestra satisfacción $\geq 7/10$
- [] [Criterio específico según el contexto]

HIPÓTESIS DE CAUSA RAÍZ

Usando técnica de \gls{five-whys}:

1. Por qué ocurre [manifestación]?

Respuesta: _____

2. Por qué [respuesta 1]?

Respuesta: _____

3. Por qué [respuesta 2]?

Respuesta: _____

4. Por qué [respuesta 3]?

Respuesta: _____

5. Por qué [respuesta 4]?

Respuesta: _____ (Causa raíz probable)

ESTRATEGIA DE INTERVENCIÓN

Tipo de intervención: [Facilitación / Mediación / Coaching /
Estructural / Escalación]

Pasos propuestos:

1. -----
2. -----
3. -----

Métricas de seguimiento:

- Frecuencia de medición: [Diaria / Semanal / Por Sprint]
- Indicadores: -----

CONFIDENCIALIDAD

Nivel: [Público / Equipo / Confidencial]

Personas con acceso: -----

Información que NO se debe compartir: -----

ESTIMACIÓN

Story Points Sociales (SSP): [1-13]

Desglose:

- Investigación/diagnóstico: ___ horas
- Sesiones 1:1: ___ horas x ___ personas = ___ horas
- Facilitación grupal: ___ horas
- Seguimiento: ___ horas
- TOTAL: ___ horas = ___ SSP (1 SSP = 2 horas)

E.2.3.2 Plantilla para mediación de conflictos

=== MEDIACIÓN DE CONFLICTO - PLANTILLA ===

Código: [SOCIAL-XXX]

Tipo: Mediación

Confidencialidad: ALTA - Solo Social Guide + partes

PARTES INVOLUCRADAS

Persona A: [Iniciales o código]

Persona B: [Iniciales o código]

Observadores afectados: [N personas]

CONTEXTO

Origen del conflicto (percepción inicial):

Fecha de la primera manifestación: -----

Eventos que lo agravaron: -----

Impacto observable en el equipo:

- Comunicación: -----

- Colaboración: -----

- Entregas: -----

SESIONES 1:1

[] Sesión con persona A completada

Fecha: ----- | Duración: ----- | Notas: -----

[] Sesión con persona B completada

Fecha: _____ | Duración: _____ | Notas: _____

[] Sesión conjunta (mediación) completada

Fecha: _____ | Acuerdos documentados: _____

[] Acuerdos explícitos documentados y firmados

[] Seguimiento programado en próxima retrospectiva

Fecha: _____ | Preguntas de seguimiento: _____

DEFINITION OF DONE

[] Social Guide documentó el proceso

[] Ambas partes consintieron los acuerdos

[] Acuerdos comunicados al equipo (sin detalles privados)

[] Seguimiento programado

Métricas de éxito:

[] Encuesta a persona A: relación mejoró de __/10 a 6+/10

[] Encuesta a persona B: relación mejoró de __/10 a 6+/10

[] Health Score en la dimensión afectada: subió de __ a __

ESTIMACIÓN

Story Points Sociales (SSP): ____

Desglose:

- Sesión 1:1 persona A: ____ horas

- Sesión 1:1 persona B: ____ horas

- Sesión conjunta: ____ horas

- Documentación + seguimiento: ____ horas

- TOTAL: ____ horas = ____ SSP

E.2.3.3 Plantilla de Health Score Dashboard semanal

=== HEALTH SCORE DASHBOARD - SEMANA X ===

Equipo: [Nombre]

Período: [Fecha inicio] - [Fecha fin]

Actualizado por: [Social Guide]

VALORES DE SCRUM - EVALUACIÓN

Apertura/Openness: [__|__|__|__|__|__|__|__|__] __/10

Tendencia: [+/-/=] | Smell activo: [Sí/No - cuál]

Compromiso: [__|__|__|__|__|__|__|__|__] __/10

Tendencia: [+/-/=] | Smell activo: [Sí/No - cuál]

Respeto: [__|__|__|__|__|__|__|__|__] __/10

Tendencia: [+/-/=] | Smell activo: [Sí/No - cuál]

Foco: [__|__|__|__|__|__|__|__|__] __/10

Tendencia: [+/-/=] | Smell activo: [Sí/No - cuál]

Coraje: [__|__|__|__|__|__|__|__|__] __/10

Tendencia: [+/-/=] | Smell activo: [Sí/No - cuál]

RESUMEN

HEALTH SCORE PROMEDIO: __/10

Semana anterior: __/10

Tendencia general: [MEJORANDO / ESTABLE / EMPEORANDO]

Capacidad estimada: __% de normal

Dimensiones críticas (<4): _____

Dimensiones en riesgo (4-5): _____

Dimensiones saludables (6+): _____

COMMUNITY SMELLS ACTIVOS

Severidad	Smell	Manifestación	Ítem Social	ETA
-----	-----	-----	-----	-----

ACCIONES ESTA SEMANA

[X] Acción completada 1
[X] Acción completada 2
[->] Acción en progreso
[] Acción pendiente

CORRELACIÓN CON MÉTRICAS TÉCNICAS

Health Score: __/10
Velocity actual: __ SP (vs __ SP normal)
Defectos: __ (vs __ promedio)
Observaciones: _____

E.2.3.4 Plantilla de notas de retrospectiva social

=== RETROSPECTIVA SOCIAL - SPRINT N ===

Fecha: [Fecha]
Facilitador: [Social Guide]
Participantes: [Lista]
Duración sección social: __ minutos

PRE-RETROSPECTIVA (Preparación)

Health Score actual: __/10

Dimensiones críticas: -----
Ítems sociales completados: -----
Ítems sociales en progreso: -----

Preguntas preparadas:

1. -----
2. -----
3. -----

DURANTE - PARTE SOCIAL

Dashboard presentado: [Sí/No]

Análisis de ítems completados:

- Ítem 1: [Resultado, feedback del equipo]
- Ítem 2: [Resultado, feedback del equipo]

Análisis de community smells:

- Smell 1: [Status: Resuelto/Mejorando/Igual/Peor]
- Smell 2: [Status]

Nuevos temas identificados:

1. -----
2. -----

Hipótesis de causa raíz (si aplica):

- Problema: -----
- Causa probable: -----

ACCIONES PROPUESTAS

Ítems sociales para próximo Sprint:

- [] [SOCIAL-XXX] [Descripción] - SSP: __ - Prioridad: __
- [] [SOCIAL-XXX] [Descripción] - SSP: __ - Prioridad: __

HEALTH SCORE ESPERADO SPRINT N+1

Apertura: __/10 -> __/10

Compromiso: __/10 -> __/10

Respeto: __/10 -> __/10

Foco: __/10 -> __/10

Coraje: __/10 -> __/10

PROMEDIO ESPERADO: __/10 -> __/10

SEGUIMIENTO PRÓXIMA RETRO

Preguntas de seguimiento:

1. -----

2. -----

NOTAS DEL FACILITADOR (Confidencial)

Observaciones no compartidas con el equipo:

Preocupaciones a monitorear:

Escalaciones necesarias:

E.2.3.5 Plantilla de reporte mensual para las partes interesadas (*stakeholders*)

=== REPORTE DE SALUD SOCIAL - MES [X] ===

Equipo: [Nombre]

Período: [Mes/Año]

Para: CTO / VP Engineering / Stakeholders

RESUMEN EJECUTIVO

El equipo [Nombre] tiene Health Score de __/10 (mes anterior: __/10).

Tendencia: [MEJORANDO / ESTABLE / REQUIERE ATENCIÓN]

Velocity técnica:

- Mes anterior: __ SP

- Mes actual: __ SP

- Correlación: Health Score +1 punto = Velocity +__% aproximadamente

HEALTH SCORE - TENDENCIA 3 MESES

	M-2	M-1	M0
10 -			
9 -			
8 -			
7 -		X	X
6 -	X		
5 -	X		
4 -			

Valores actuales por dimensión:

- Apertura: __/10 [+/-/=]

- Compromiso: __/10 [+/-/=]

- Respeto: __/10 [+/-/=]

- Foco: __/10 [+/-/=]

- Coraje: __/10 [+/-/=]

ÍTEMS SOCIALES COMPLETADOS

[X] [Descripción ítem 1]
Resultado: _____
Impacto: _____

[X] [Descripción ítem 2]
Resultado: _____
Impacto: _____

PRESUPUESTO SOCIAL (mes):

- Ítems sociales: __ SSP (~__ horas)
- Sesiones adicionales: __ horas
- TOTAL: __ horas (~__% de capacidad)

PRÓXIMAS ACCIONES

CRÍTICO (inmediato):

[] _____

IMPORTANTE (semanas 2-3):

[] _____

DESEABLE (mes siguiente):

[] _____

RIESGOS Y MITIGACIONES

Riesgo 1: _____

Mitigación: _____

Riesgo 2: _____

Mitigación: _____

RECOMENDACIÓN

[CONTINUAR / AJUSTAR / ESCALAR]

Justificación: -----

Próxima revisión: [Fecha]

E.2.3.6 Plantilla de registro de *community smells*

=== COMMUNITY SMELL REGISTRY ===

Equipo: [Nombre]

Propósito: Histórico de smells y cómo fueron abordados

SMELL #[NNN]: [NOMBRE DEL SMELL]

Identificado en: Sprint __, [Evento donde se detectó]

Identificado por: [Social Guide / Equipo / Automático]

Definición:

- Manifestación: -----
- Causa raíz (\gls{five-whys}): -----
- Tipo de smell: [Comunicación / Conocimiento / Relacional / Organizacional]

Severidad: [LEVE / MODERADA / SEVERA / CRÍTICA]

- Personas afectadas: __
- Impacto en velocity/calidad: -----
- Riesgo si no se resuelve: -----

Dimensión de Health Score afectada: -----

Health Score antes: __/10

Acción tomada:

- Ítem social: [SOCIAL-XXX]
- SSP estimado: __
- Estrategia: -----
- Facilitador: [Social Guide]

Resolución:

- Estado: [Resuelto / En progreso / Escalado / Aceptado como restricción]
- Fecha resolución: _____
- Health Score después: __/10

Post-mortem:

- Qué funcionó: _____
- Qué no funcionó: _____
- Cómo prevenir recurrencia: _____

E.2.4 Dashboards automatizados

Para equipos que requieren visualización más sofisticada del Health Score y su correlación con métricas técnicas, es posible construir dashboards automatizados usando herramientas de Business Intelligence.

E.2.4.1 Arquitectura de datos

=== ARQUITECTURA DE DATOS PARA DASHBOARD ===

Fuentes de datos:

1. Jira/Azure DevOps API

- Ítems técnicos: Story Points, status, Sprint
- Ítems sociales: SSP, community smell, severity
- Velocity por Sprint

2. Formulario de Health Score (Google Forms / Microsoft Forms)

- Respuestas individuales (anónimas)
- Promedios por dimensión
- Timestamp para histórico

3. Hoja de cálculo intermedia (Google Sheets / Excel)

- Consolidación de datos de múltiples fuentes
- Cálculos derivados (promedios, tendencias)
- Exportación a herramienta BI

Pipeline:

```
[Jira API] ----+
                |
[Forms]  -----+----> [Google Sheets] ---> [Dashboard BI]
                |
```

[Manual] -----+

Frecuencia de actualización:

- Velocity: Automática al cerrar Sprint
- Health Score: Semanal (después de retrospectiva)
- Community Smells: Tiempo real (cuando se crean/cierran ítems)

E.2.4.2 Dashboard en Google Sheets

Para equipos sin acceso a herramientas BI, Google Sheets proporciona visualización básica:

=== DASHBOARD EN GOOGLE SHEETS ===

Hoja 1: "Datos"

- Columnas: Sprint, Fecha, Apertura, Compromiso, Respeto, Foco, Coraje, Promedio, Velocity, Defectos
- Una fila por Sprint

Hoja 2: "Dashboard"

- Gráfico de líneas: Health Score Promedio vs Sprint
- Gráfico de radar: 5 dimensiones del Sprint actual
- Gráfico de barras: Velocity vs Health Score (correlación)
- Tabla: Community Smells activos

Fórmulas útiles:

- Promedio: =AVERAGE(B2:F2)
- Tendencia: =IF(G2>G1,"+",IF(G2<G1,"-", "="))
- Correlación: =CORREL(G:G, H:H)

Automatización:

- Google Apps Script para importar de Jira API
- Trigger semanal para actualizar datos
- Notificación si Health Score < 4

E.2.4.3 Dashboard en Grafana

Grafana es una herramienta de visualización open-source para dashboards sofisticados:

=== CONFIGURACIÓN DE GRAFANA ===

Data Source:

- PostgreSQL / MySQL con datos exportados
- O plugin de Jira para conexión directa
- O JSON API para Google Sheets

Paneles recomendados:

1. Panel: "Health Score - Tendencia"
 - Tipo: Time series
 - Query: SELECT date, avg_health_score FROM health_scores
 - Visualización: Línea con puntos
2. Panel: "Dimensiones Actuales"
 - Tipo: Gauge o Stat
 - 5 gauges, uno por dimensión
 - Colores: Verde (7+), Amarillo (4-6), Rojo (<4)
3. Panel: "Correlación Health-Velocity"
 - Tipo: Scatter plot
 - X: Health Score
 - Y: Velocity
 - Anotación: Línea de tendencia
4. Panel: "Community Smells Activos"
 - Tipo: Table
 - Columnas: Smell, Severity, Sprint detectado, Status
 - Filtro: Status != "Resolved"
5. Panel: "Alertas"
 - Tipo: Alert list
 - Condición: Health Score < 4 OR Severity = "Critical"
 - Notificación: Slack / Email

Refresh: Cada 1 hora o manual

E.2.4.4 Dashboard en Power BI / Tableau

=== DASHBOARD EN POWER BI ===

Conexión de datos:

- Jira: Usar conector nativo o API REST

- Azure DevOps: Conector nativo disponible
- Google Sheets: Conector web

Modelo de datos:

- Tabla "Sprints": Sprint_ID, Start_Date, End_Date, Velocity
- Tabla "Health_Scores": Sprint_ID, Date, Dimension, Score
- Tabla "Social_Items": Item_ID, Sprint_ID, Smell, SSP, Status
- Relaciones: Sprint_ID como clave foránea

Visualizaciones:

Página 1: "Resumen Ejecutivo"

- KPI Card: Health Score actual con tendencia
- Line Chart: Health Score últimos 6 Sprints
- Donut: Distribución de smells por categoría
- Table: Top 3 smells críticos

Página 2: "Detalle por Dimensión"

- 5 Gauge Charts: Una por dimensión
- Heatmap: Dimensión x Sprint (colores por score)
- Insights: Dimensión con mayor variabilidad

Página 3: "Correlación"

- Scatter: Health Score vs Velocity
- Trendline: Regresión lineal
- R-squared: Mostrar fuerza de correlación
- Filtros: Por equipo, por período

Página 4: "Ítems Sociales"

- Kanban: Ítems por status
- Burndown: SSP restantes en Sprint
- Table: Histórico de ítems completados

Refresh automático:

- Scheduled refresh cada 6 horas
- 0 refresh manual post-retrospectiva

E.2.5 Alertas automatizadas

Independientemente de la herramienta, configurar alertas para situaciones críticas:

=== CONFIGURACIÓN DE ALERTAS ===

Alerta 1: Health Score Crítico

- Condición: Health Score Promedio < 4.0
- Canal: Slack #team-alerts + Email a CTO
- Mensaje: "ALERTA: Health Score del equipo [X] es [Y]/10.
Requiere atención inmediata."

Alerta 2: Dimensión en Caída

- Condición: Cualquier dimensión baja > 2 puntos en 1 Sprint
- Canal: Slack #social-guide
- Mensaje: "Dimensión [X] cayó de [Y] a [Z]. Investigar causa."

Alerta 3: Smell Crítico Sin Asignar

- Condición: Ítem social con Severity = "Crítica"
AND Status = "Backlog" por > 48 horas
- Canal: Email a Social Guide
- Mensaje: "Ítem social crítico sin asignar: [Link]"

Alerta 4: Capacidad Social Excedida

- Condición: SSP en Sprint > 20% de capacidad total
- Canal: Slack #scrum-master
- Mensaje: "Ítems sociales exceden 20% de capacidad.
Revisar priorización o escalar."

Implementación:

- Grafana: Alerting rules nativas
- Power BI: Power Automate para triggers
- Google Sheets: Apps Script con triggers condicionales
- Jira: Automation rules nativas

E.2.6 Consideraciones de implementación

Las herramientas de soporte no son requisito para implementar SocialScrum, pero reducen la fricción operacional. La integración con Jira o Azure DevOps permite que los ítems sociales vivan junto a los técnicos, visibles para todo el equipo. Las plantillas estanda-

rizan la captura de información y facilitan la adopción. Los dashboards automatizados proporcionan visibilidad continua sin esfuerzo manual.

El principio rector es que la tecnología debe servir al proceso, no al revés: si una hoja de cálculo simple funciona para el equipo, no hay necesidad de implementar infraestructura compleja. La sofisticación debe ser proporcional al valor que genera.

ANEXO F: PROTOCOLOS OPERATIVOS DE GOBERNANZA Y ÉTICA DEL SOCIAL GUIDE

Este anexo presenta los protocolos operativos detallados para la gobernanza ética del Social Guide en SocialScrum. Estos protocolos complementan los fundamentos éticos presentados en el Capítulo 3, Sección 3.4 y las directrices generales del Capítulo 4.

F.1 1 Protocolo técnico de protección de datos

El Social Guide tiene acceso a información sensible sobre dinámicas interpersonales, conflictos y vulnerabilidades del equipo. Sin salvaguardas explícitas, existe riesgo de que esta información sea utilizada para evaluaciones de desempeño punitivas, decisiones de despido, o manipulación organizacional. Edmondson [34] describe cómo el miedo a represalias silencia a las organizaciones y destruye la seguridad psicológica [32]. Esta sección establece los principios éticos obligatorios para la implementación de SocialScrum, asegurando que el marco construya seguridad psicológica en lugar de destruirla.

Los principios aquí descritos no son sugerencias; son condiciones necesarias. Una organización que implementa SocialScrum sin estas salvaguardas convierte el marco en una herramienta de vigilancia, no de salud social.

F.1.1 Confidencialidad absoluta: Principio no negociable

La confidencialidad es la base sobre la cual se construye la confianza del equipo en SocialScrum. Mayer et al. [31] establecen que la confianza organizacional requiere tres componentes: habilidad, benevolencia e integridad. La confidencialidad protege la *integridad* del Social Guide y demuestra *benevolencia* hacia el equipo.

F.1.1.1 Definición del principio

Toda información compartida en eventos sociales (Social Daily Standup, Social Sprint Retrospective, sesiones de mediación) es **confidencial** y **no puede ser utilizada** para evaluaciones de desempeño, decisiones de contratación o despido, o cualquier acción que afecte el estatus laboral de un individuo.

Este principio es absoluto. No existen excepciones basadas en “interés de la empresa” o “necesidad de negocio”. La única excepción es una obligación legal (por ejemplo, investigación de acoso documentado), y aún en ese caso, se requiere consentimiento explícito del afectado y documentación formal.

F.1.1.2 Qué es confidencial vs. qué es reportable

La distinción entre información confidencial e información reportable es clara:

Confidencial (NO reportable)	Reportable (datos agregados)
Declaraciones individuales en retrospectivas	Health Score promedio del equipo
Conflictos interpersonales con nombres	“El equipo presenta aislamiento social” (sin nombres)
Estado emocional de personas específicas	“Se detectó burnout en el equipo” (sin identificar personas)
Intención de renuncia de alguien	“Riesgo de rotación elevado” (dato agregado)
Críticas a gerencia por persona	“El equipo reporta fricción con <i>stakeholders</i> ”
Detalles de mediaciones 1:1	“Se realizó mediación; conflicto en proceso de resolución”

Tabla F.2: Información confidencial vs. reportable

Regla práctica Si la información permite identificar a una persona específica y podría afectarla negativamente, es confidencial. Si es un dato agregado que describe al equipo sin señalar individuos, es reportable.

F.1.1.3 Ejemplos de aplicación

La diferencia entre información confidencial y reportable se ilustra con un caso: si un miembro menciona en retrospectiva que “está considerando renunciar porque se siente excluido”, el Social Guide **nunca** comparte esto con HR. En cambio, reporta de forma agregada: “El Health Score es 4.2/10; existe *community smell* de aislamiento social; se requiere intervención para restaurar cohesión.” El Anexo C presenta el Acuerdo de Confidencialidad completo que formaliza estas obligaciones.

F.1.1.4 Acuerdos formales de confidencialidad

La confidencialidad debe formalizarse en documentos firmados por el Social Guide y aceptados por el equipo. El Anexo C contiene la plantilla completa del Acuerdo de Confidencialidad del Social Guide, que incluye:

- Compromiso de no compartir información individual identificable.
- Obligación de reportar solo datos agregados.

- Prohibición de uso para evaluaciones de desempeño.
- Rechazo de solicitudes de información confidencial, incluso de gerencia.
- Consecuencias por violación (remoción inmediata del rol).

F.1.1.5 Protocolo técnico de protección de datos

La confidencialidad requiere no solo políticas organizacionales, sino también implementación técnica que proteja la información sensible.

Requisito	Implementación
Cifrado en reposo	Datos almacenados con cifrado AES-256 o equivalente. Bases de datos con Transparent Data Encryption (TDE).
Cifrado en tránsito	HTTPS obligatorio; TLS 1.2 o superior para todas las conexiones.
Ubicación física	Servidores en jurisdicciones con protección de datos adecuada (GDPR si aplica). Nunca en dispositivos personales sin cifrado.
Separación lógica	Datos de SocialScrum en una base de datos separada de HR y de evaluaciones de desempeño, sin acceso cruzado.

Tabla F.4: Requisitos técnicos de almacenamiento

Almacenamiento seguro

Rol	Acceso permitido
Social Guide	Acceso completo (lectura y escritura)
Scrum Master	Métricas agregadas (solo lectura)
CTO	Métricas agregadas (solo lectura)
HR	Ninguno (acceso prohibido)
Managers	Ninguno (acceso prohibido)
Auditor externo	Acceso completo en modo solo lectura, limitado al periodo de auditoría

Tabla F.6: Matriz de acceso a datos de SocialScrum

Control de acceso **Nota:** Los accesos se implementan mediante roles técnicos en el sistema, no mediante confianza implícita en los usuarios.

Política de retención y eliminación

- **Retención máxima:** Datos individuales se retienen por 12 meses desde su creación. Después, solo se conservan agregados anonimizados.
- **Derecho de eliminación:** Cualquier miembro puede solicitar eliminación de sus datos individuales en cualquier momento. El Social Guide tiene 5 días hábiles para ejecutar la solicitud.

- **Salida del equipo:** Cuando un miembro deja el equipo (renuncia, despido, transferencia), sus datos individuales se eliminan en 30 días. Datos agregados que lo incluían permanecen sin modificar.
- **Fin de SocialScrum:** Si la organización discontinúa SocialScrum, todos los datos se eliminan en 60 días, con notificación previa al equipo.

Auditoría de accesos Toda consulta a datos de SocialScrum debe generar un registro con: usuario, timestamp, datos consultados y origen. Los logs se retienen 24 meses y el Social Guide los revisa mensualmente. Accesos anómalos (HR intentando acceder, consultas masivas, horarios inusuales) generan alerta automática.

F.1.2 Estructura organizacional y rendición de cuentas

La ubicación del Social Guide en la estructura organizacional determina a quién rinde cuentas y qué incentivos tiene. Una ubicación incorrecta puede convertir el rol en un instrumento de control en lugar de apoyo.

F.1.2.1 Principio de reporte

El Social Guide reporta al Scrum Master o CTO/VP de Ingeniería, **nunca a Recursos Humanos (HR)**. Esto evita conflictos de interés, dado que HR gestiona evaluaciones de desempeño y decisiones de empleo.

F.1.2.2 Estructura organizacional recomendada

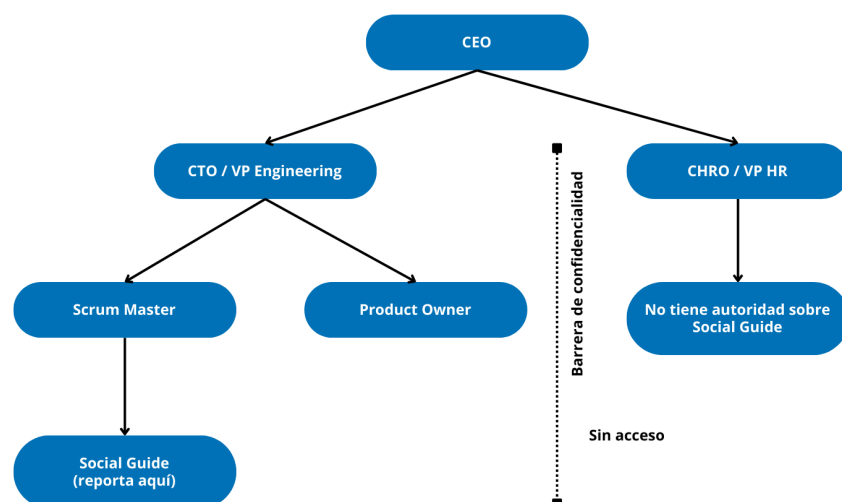


Figura F.1: Estructura organizacional del Social Guide: reporta a CTO/SM, nunca a HR

F.1.2.3 Justificación de la estructura

Existen tres razones para esta estructura:

Razón 1: Conflicto de interés HR tiene como función evaluar desempeño y gestionar decisiones de empleo (contratación, despido, promociones). Si el Social Guide reporta a HR, la información social se convierte en “evidencia” para esas decisiones. El resultado: el equipo deja de ser auténtico porque sabe que lo que diga puede usarse en su evaluación.

Razón 2: Alineación de objetivos El CTO y el Scrum Master tienen como objetivo equipos sanos y productivos. El Social Guide tiene como objetivo mitigar deuda social. Estos objetivos están naturalmente alineados. HR, en contraste, tiene objetivos adicionales (cumplimiento legal, gestión de nómina, políticas organizacionales) que pueden entrar en conflicto.

Razón 3: Protección de datos En algunas jurisdicciones, la información sensible de empleados requiere protección especial. Si el Social Guide es parte de Engineering (no HR), la información se trata como “datos operacionales del equipo”, no como “datos de personal”, lo que puede simplificar el cumplimiento legal.

F.1.2.4 Protocolo cuando HR solicita información

Cuando HR solicita información individual, el Social Guide: (1) rechaza inmediatamente, (2) informa al SM/CTO, (3) documenta el intento. La única excepción es una obligación legal documentada (investigación de acoso), donde se proporciona solo información relevante, con consentimiento explícito del afectado. HR recibe respuesta estándar: “La información sobre conflictos específicos es confidencial. Puedo compartir el Health Score agregado (X/10) y que estamos trabajando en la dimensión Y.”

F.1.2.5 Separación entre salud social y evaluación de desempeño

La organización debe establecer una política explícita firmada por CEO, CTO y CHRO que establezca: (1) la información de SocialScrum no se usa en evaluaciones de desempeño; (2) reportar burnout o conflictos es señal de confianza, no de bajo desempeño; (3) la participación es voluntaria y no afecta promociones; (4) las violaciones son falta grave.

F.1.3 Protocolo de conflicto irreconciliable PO-Social Guide

En ocasiones, el Product Owner y el Social Guide pueden tener visiones incompatibles sobre la priorización del Sprint. El PO puede insistir en entregar funcionalidades técnicas; el Social Guide puede argumentar que el equipo necesita recuperación social primero. Cuando el diálogo no produce acuerdo, se requiere un mecanismo de resolución.

F.1.3.1 Principio rector

Ni el PO ni el Social Guide tienen autoridad unilateral para imponer su visión. El conflicto irreconciliable se escala a un tercero neutral (Scrum Master o CTO) para decisión final.

F.1.3.2 Protocolo de escalamiento

El conflicto se resuelve en tres niveles progresivos:

1. **Nivel 1 - Diálogo directo (30 min)**: PO y SG exponen posiciones con datos y buscan compromiso (ej: 60 % técnico, 40 % social).
2. **Nivel 2 - Mediación del SM (1 hora)**: SM facilita conversación estructurada, cada parte tiene 10 min sin interrupción, SM propone opciones.
3. **Nivel 3 - Decision del CTO (24-48h)**: SM presenta caso por escrito. CTO decide considerando riesgo de negocio vs. riesgo de burnout. Decision final para ese Sprint.

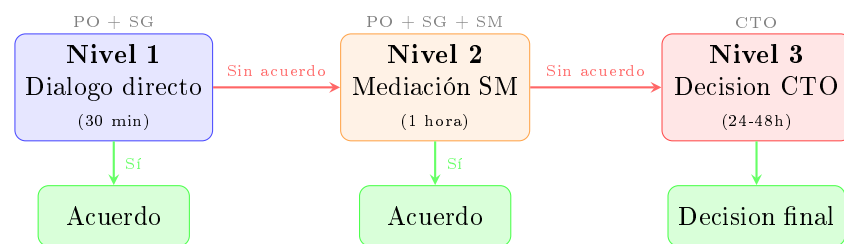


Figura F.2: Protocolo de escalamiento en conflicto PO-Social Guide

F.1.3.3 Criterios de decisión para el CTO

Escenario	Recomendación	Justificación
Health Score > 7, dead-line contractual	Priorizar técnico	Equipo sano puede absorber presión temporal
Health Score < 4, dead-line flexible	Priorizar social	Equipo en crisis no entrega calidad
Health Score 4–6, dead-line crítico	Balance 50/50 o decisión caso por caso	Zona gris; evaluar riesgo específico
Burnout activo documentado	Siempre priorizar social	Ignorar burnout garantiza rotación

Tabla F.8: Criterios de decisión en conflicto PO–SG

F.1.3.4 Prevención de conflictos recurrentes

Si el mismo tipo de conflicto ocurre 3+ veces en 6 meses, indica un problema sistémico. Se recomienda una revisión profunda de la dinámica PO-SG, incluyendo:

- **PO siempre gana:** El Social Guide no tiene suficiente autoridad formal; revisar estructura organizacional.
- **SG siempre gana:** El PO puede estar ignorando legítimas presiones de negocio; revisar comunicación con *stakeholders*.
- **Siempre escala a CTO:** PO y SG no han construido relación de trabajo colaborativa; sesión de alineación de roles.

F.1.4 Protocolo de continuidad del Social Guide

El Social Guide acumula conocimiento sensible sobre dinámicas del equipo. Su salida (renuncia, transferencia, despido, incapacidad prolongada) requiere un protocolo que proteja tanto la continuidad del programa como la confidencialidad de la información.

F.1.4.1 Escenarios de salida

Escenario 1: Renuncia voluntaria con aviso

Escenario aplicado

Renuncia voluntaria del Social Guide (aviso ≥ 2 semanas)

Semana 1

- Notificación formal al Scrum Master.
- Inicio del proceso de reemplazo.
- Documentación del estado actual del equipo.

Semana 2

- Entrenamiento del reemplazo (shadowing).
- Traspaso de documentación no individual.
- Evento de cierre ritual con el equipo.

Día final

- Eliminación de accesos del SG saliente.
- Destrucción o transferencia consentida de datos individuales.

- Firma de compromiso de confidencialidad post-salida.

Escenario 2: Salida abrupta (despido, emergencia, incapacidad)

Escenario aplicado

Salida abrupta del Social Guide

Acciones inmediatas

- El Scrum Master asume de forma temporal las funciones mínimas del Social Guide.
- Los accesos del Social Guide saliente se revocan en un plazo máximo de 24 horas.
- El CTO autoriza acceso temporal del Scrum Master a la documentación necesaria para garantizar continuidad.

Semana 1

- El Scrum Master revisa la documentación existente e identifica intervenciones sociales urgentes en progreso.
- Se comunica al equipo la situación de forma transparente: el Social Guide ya no está; el Scrum Master asume temporalmente mientras se busca reemplazo.
- El Scrum Master no tiene acceso a notas individuales de sesiones 1:1; dichas notas se destruyen si el Social Guide no puede transferirlas con consentimiento explícito.

Semanas 2 a 4

- Se ejecuta un proceso acelerado de selección de un nuevo Social Guide.
- El Scrum Master facilita únicamente eventos sociales básicos (por ejemplo, retrospectivas), evitando profundizar en temas sensibles en curso.

Ingreso del nuevo Social Guide

- El nuevo Social Guide inicia las relaciones 1:1 desde cero.
- Se transfiere únicamente documentación agregada sobre *community smells* y Health Score.
- No se transfieren notas personales ni registros individuales del Social Guide anterior.

F.1.4.2 Compromiso de confidencialidad post-salida

El Social Guide saliente firma un acuerdo de confidencialidad permanente que incluye: mantener confidencialidad sobre toda información individual, destruir notas personales de sesiones 1:1, y aceptar consecuencias legales por violaciones. El CTO actúa como testigo.

F.1.5 Protocolo de auditoría independiente

La confianza en SocialScrum no puede depender únicamente de la buena voluntad del Social Guide. Se requiere verificación externa periódica para asegurar que los principios éticos se cumplen en la práctica.

F.1.5.1 Definición del principio

Cada 6 meses, un auditor independiente (sin conflicto de interés) evalúa si el Social Guide está cumpliendo los principios éticos de SocialScrum.

F.1.5.2 Selección del auditor

El auditor debe ser:

- Externo a la organización (consultor, coach ágil certificado)
- Sin relación comercial con la empresa (excepto el contrato de auditoría)
- Con experiencia en dinámicas de equipo y ética organizacional

F.1.5.3 Proceso de auditoría

Paso 1: Entrevistas confidenciales El auditor entrevista a cada miembro del equipo de forma individual (30 minutos), sin presencia del Social Guide.

Preguntas clave de la entrevista El auditor pregunta (escala 1–10 + comentarios):

1. ¿Sientes que la información es confidencial?
2. ¿El Social Guide ha usado información en tu contra?
3. ¿Te sientes seguro para ser auténtico?
4. ¿Has sido presionado a compartir?
5. ¿Conoces tus derechos (opt-out, auditoría)?
6. ¿Has observado violaciones éticas?
7. ¿Recomendarías SocialScrum?

Paso 2: Revisión de documentación El auditor revisa:

- Todos los reportes generados por el Social Guide en el período.
- Comunicaciones entre Social Guide y gerencia/HR (si las hay).
- Acuerdos de confidencialidad firmados.
- Evidencia de reportes “secretos” (no deberían existir).

Paso 3: Reporte de auditoría El auditor emite un reporte con:

- Cumplimiento de principios éticos: Sí / No / Parcial
- Hallazgos específicos (violaciones, si existen).
- Fortalezas observadas.
- Recomendaciones de mejora.

El reporte es **público para el equipo**. No existen reportes de auditoría secretos.

F.1.5.4 Consecuencias de hallazgos negativos

Protocolo operativo

Auditoría negativa (hallazgo ético)

Condición de activación

- El auditor identifica una posible violación ética.

Acción inmediata

- Se inicia una investigación formal liderada por el CTO y el auditor (duración estimada: 1-2 semanas).

Resultado A: violación confirmada

- El Social Guide es removido inmediatamente del rol.
- El Social Guide no puede volver a ocupar el rol en la organización.
- La organización emite una disculpa formal al equipo.
- Se nombra un nuevo Social Guide, con reentrenamiento ético completo.
- Se programa una auditoría de seguimiento a los 3 meses.

Resultado B: no confirmada, pero con preocupaciones

- El Social Guide recibe feedback detallado y un plan de mejora.
- Se realiza una reauditora a los 3 meses (en lugar de 6).
- Se incrementa el monitoreo por parte del Scrum Master, limitado a indicadores agregados y cumplimiento del protocolo.

F.1.5.5 Canal de denuncia (whistleblowing)

Además de la auditoría periódica, debe existir un canal para denuncias inmediatas:

Opciones de denuncia

1. **Canal interno:** Denuncia al Scrum Master, quien escala a CTO.
2. **Canal externo:** Denuncia directa al auditor independiente (contacto proporcionado al inicio de SocialScrum).
3. **Canal anónimo:** Formulario web anónimo revisado por CTO + auditor.

Protección contra represalias Política de tolerancia cero: si alguien denuncia violación ética y sufre represalias, se investiga inmediatamente, la persona que tomó represalias es sancionada (hasta despido), y el denunciante recibe protección formal (no puede ser despedido sin aprobación de CTO + auditor durante 12 meses; evaluaciones negativas son revisadas por auditor).

Monitoreo de represalias silenciosas Las represalias no siempre son obvias. Un manager puede evitar despedir a alguien pero tomar acciones sutiles que degradan la experiencia laboral de la persona. El Social Guide debe monitorear indicadores de represalias indirectas:

Indicador	Patrón sospechoso	Acción
Asignación de tareas	Persona recibe solo tareas triviales o indeseables después de reportar un problema	Entrevista privada; comparación con asignaciones previas
Exclusión de reuniones	La persona deja de ser invitada a reuniones relevantes	Verificación en calendario; conversación con la persona afectada
Aislamiento social	Colegas evitan la interacción; la persona almuerza sola	Observación directa; análisis de redes sociales
Evaluación de desempeño	Calificación baja inesperada después de una denuncia	Comparación con evaluaciones anteriores; revisión por auditor
Oportunidades negadas	Promoción, capacitación o proyectos relevantes asignados a otros	Revisión del historial de asignaciones; análisis del patrón temporal
Micromanagement repentino	Supervisión excesiva que no existía previamente	Comparación con el trato hacia otros miembros del equipo

Tabla F.10: Indicadores de represalias silenciosas

Si el Social Guide detecta un patrón sospechoso, debe: (1) documentar observaciones con fechas, (2) entrevistar confidencialmente al afectado, (3) si confirma percepción de represalia, escalar a CTO para investigación, (4) si no percibe represalia, documentar para monitoreo y revisar en 30 días. El Social Guide detecta y escala; la investigación formal corresponde a CTO, auditor o asesoría legal.

F.1.6 Prevención del “Teatro Social”

El “Teatro Social” ocurre cuando el equipo simula bienestar para satisfacer las expectativas de SocialScrum, mientras los problemas reales permanecen ocultos. Es la antítesis del objetivo del marco: en lugar de transparencia, genera una fachada.

F.1.6.1 Síntomas del “Teatro Social”

Síntoma	Descripción
Health Score artificialmente alto	Todos los valores consistentemente entre 8 y 10, sin variación, incluso cuando existen problemas visibles.
Retrospectivas sin sustancia	“Todo bien”, “sin comentarios”, “seguir igual”; ausencia de crítica constructiva.
Discrepancia con la realidad	Health Score de 9/10 mientras la velocidad cae, hay rotación elevada o conflictos visibles.
Participación forzada	Respuestas rápidas solo para cumplir, sin reflexión real.
Miedo a ser diferentes	Nadie puntúa bajo porque “todos los demás puntúan alto”.

Tabla F.12: Síntomas del “Teatro Social”

F.1.6.2 Causas del “Teatro Social”

El “Teatro Social” no ocurre espontáneamente¹⁸⁴; es respuesta a incentivos perversos:

Causa 1: Miedo a represalias Si el equipo percibe (correcta o incorrectamente) que

Causa 3: Fatiga de retrospectiva Si las retrospectivas son percibidas como burocracia sin impacto (“hablamos pero nada cambia”), el equipo minimiza esfuerzo: respuestas rápidas, sin profundidad, para terminar pronto.

Causa 4: Social Guide inefectivo Un Social Guide que no genera confianza, no facilita bien, o es percibido como “espía de gerencia” produce “Teatro Social” como mecanismo de defensa del equipo.

F.1.6.3 Estrategias de prevención

Estrategia 1: Nunca premiar o castigar por Health Score

Protocolo operativo

Uso correcto del Health Score

El Health Score es un instrumento diagnóstico orientado a la mejora continua y al cuidado del equipo. No debe utilizarse como indicador de desempeño ni como métrica de evaluación.

Prácticas prohibidas

- Celebrar públicamente a un equipo por tener un Health Score alto.
- Señalar o estigmatizar a un equipo por tener un Health Score bajo.
- Asociar bonos, incentivos o reconocimientos a un Health Score elevado.
- Presionar explícita o implícitamente para “mejorar el número”.

Prácticas permitidas

- Utilizar el Health Score como señal para identificar áreas de mejora (por ejemplo, comunicación, foco o seguridad psicológica).
- Interpretar un Health Score bajo como indicio de necesidad de apoyo, no como falla del equipo.
- Usar el Health Score para asignar recursos de acompañamiento, facilitación o intervención social.

Estrategia 2: Validación cruzada de datos

El Social Guide debe comparar el Health Score con indicadores objetivos:

Health Score	Indicador objetivo	Interpretación
Alto (8+)	Velocidad estable o creciente	Consistente: probablemente real
Alto (8+)	Velocidad cayendo	Inconsistente: posible “Teatro Social”
Alto (8+)	Rotación reciente	Inconsistente: requiere investigación
Bajo (4-)	Velocidad baja	Consistente: el equipo está siendo honesto
Bajo (4-)	Velocidad alta	Inusual: equipo muy crítico o presencia de problema oculto

Tabla F.14: Validación cruzada del Health Score

Estrategia 3: Encuestas anónimas periódicas

Además de las retrospectivas facilitadas, realizar encuestas completamente anónimas cada 2-3 Sprints:

Encuesta anónima de validación

Protocolo operativo

Instrumento de validación del Health Score

Instrucciones. Esta encuesta es completamente anónima. Ni el Social Guide ni ninguna otra persona puede identificar tus respuestas. El objetivo es validar la fidelidad del Health Score y la seguridad percibida en el proceso.

1. **¿El Health Score del equipo refleja la realidad?**
 - Sí, es preciso.
 - Parcialmente, algunos aspectos no se capturan.
 - No, existen problemas que no se reportan.
2. **¿Te sientes seguro para ser honesto en retrospectivas?**
 - Sí, completamente.
 - Parcialmente, algunas cosas prefiero no decir.
 - No, temo consecuencias.
3. **¿Hay algo que el equipo no dice en retrospectivas, pero debería decir?**
(Respuesta abierta, opcional)
4. **¿Confías en que el Social Guide mantiene la confidencialidad?**
 - Sí.
 - No estoy seguro.

- No.

Nota. Las respuestas se analizan únicamente de forma agregada y se utilizan para contrastar el Health Score con indicadores objetivos y señales de “Teatro Social”.

Si la encuesta anónima revela discrepancias significativas con el Health Score reportado, el Social Guide debe investigar las causas y, potencialmente, reiniciar la construcción de confianza.

Estrategia 4: Rotación del facilitador de retrospectiva Si el Social Guide siempre facilita, el equipo puede sentir que “le reportan a él”. Rotar la facilitación ocasionalmente (Scrum Master, miembro del equipo voluntario) puede revelar dinámicas diferentes.

Estrategia 5: Celebrar la honestidad, no los números altos

Ejemplo de comunicación correcta e incorrecta

Escenario aplicado

Comunicación correcta (enfoque diagnóstico)

Social Guide al equipo:

“En esta retrospectiva identificamos tres *community smells* que no habíamos visto antes. Eso no es algo negativo; es una señal de que estamos siendo honestos. Prefiero un Health Score de 5 real que un 9 ficticio.”

Comunicación incorrecta (enfoque tipo KPI)

Manager al equipo:

“Felicitaciones, su Health Score subió de 6 a 8. ¡Sigan así!”

Nota. Este tipo de mensaje incentiva la optimización del número en lugar de la representación fiel de la realidad del equipo.

F.1.6.4 Qué hacer si se detecta “Teatro Social”

Protocolo operativo

“Teatro Social” detectado

Este protocolo se activa cuando existen señales de que el Health Score no refleja la realidad del equipo y se está produciendo optimización del número en lugar de honestidad.

Indicadores de activación

- La encuesta anónima revela desconfianza (más del 30 % de respuestas del tipo “No me siento seguro”).
- Health Score alto combinado con velocity, rotación o conflictos inconsistentes.
- Retrospectivas sin sustancia durante tres o más Sprints consecutivos.

Acciones

1. No confrontar al equipo

- Evitar mensajes del tipo “están mintiendo” o “están maquillando los datos”.
- La confrontación directa destruye la confianza restante.

2. Reflexión del Social Guide

- ¿Qué comportamientos propios pueden estar generando desconfianza?
- ¿Existe algún incentivo estructural que fomente el “Teatro Social”?

3. Sesión de reinicio (reset)

- La sesión debe ser facilitada por un actor externo (no el Social Guide).
- Reconocer explícitamente que algo no está funcionando.
- Preguntar de forma abierta: “¿Qué necesitan para poder ser honestos?”
- Escuchar sin justificar ni defender decisiones previas.

4. Ajustes estructurales

- Reforzar explícitamente las garantías de confidencialidad.
- Considerar el cambio de Social Guide si la pérdida de confianza es irreparable.
- Revisar si la gerencia ha ejercido presión directa o indirecta por “buenos números”.

5. Período de reconstrucción

- Duración estimada: entre tres y seis Sprints.
- Aceptar Health Scores bajos pero realistas durante el proceso.
- Enfatizar acciones concretas sobre métricas.
- Reconocer y reforzar pequeñas señales de honestidad emergente.

Las salvaguardas éticas no son opcionales; son condición necesaria para que SocialScrum funcione. Sin confidencialidad, el equipo no será auténtico. Sin estructura organizacional correcta, la información se usará mal. Sin auditoría, no hay verificación. Sin prevención de “Teatro Social”, el Health Score será una métrica de vanidad. Implementar SocialScrum sin estas salvaguardas es peor que no implementar nada: destruye la confianza que el marco pretende construir. La regla de oro es simple: si hay duda sobre si algo es ético, no se hace.

ANEXO G: GUÍA DE FACILITACIÓN Y PROTOCOLOS OPERATIVOS

G.1 1. Guía para la presentación inicial de SocialScrum

G.1.1 Estructura de la presentación (45 minutos).

La presentación sigue una estructura sencilla y directa que permite contextualizar, alinear expectativas y asegurar el apoyo necesario.

1. Contexto y justificación (10 min) En este primer bloque, se comparte evidencia clara sobre por qué SocialScrum es pertinente en este momento. La idea es mostrar que no se trata de una moda ni de una iniciativa improvisada, sino de una respuesta informada a problemas reales.

- **Datos propios del equipo (si existen):** cifras de rotación, fluctuaciones notorias en la velocidad, picos de defectos que coinciden con periodos de tensión, o cualquier patrón que muestre impacto de la dinámica social en el rendimiento técnico.
- **Evidencia investigativa:** estudios que han documentado cómo la salud social influye en la productividad y la calidad del software [3], [22]. Estas referencias ayudan a mostrar que la deuda social no es un concepto abstracto, sino un fenómeno medido y reconocido.

Este segmento busca que todos entiendan el “porque” antes del “cómo”: sin un motivo claro, cualquier cambio de proceso se siente impuesto; con la evidencia a la vista, se entiende como una necesidad real y oportuna.

2. Explicación del modelo (15 min). En este segmento se presenta SocialScrum de manera práctica, mostrando cómo funciona en el día a día. El objetivo es que los *stakeholders* entiendan el modelo sin tecnicismos innecesarios, visualizando claramente qué cambia y qué se mantiene igual.

- **Roles:** se explica qué hace el Social Guide en el trabajo cotidiano, cómo se coordina con el Scrum Master y cuál es el papel del Product Owner dentro del proceso. La idea es mostrar que cada rol aporta algo distinto y complementario.

- **Eventos adaptados:** se describe cómo se integran los elementos sociales en los eventos ya existentes —Daily, Planning, Review y Retrospective— sin alterar su esencia. El enfoque está en cómo estos espacios se vuelven más sensibles a la dinámica humana del equipo.
- **Artefactos:** se introducen los elementos nuevos del proceso, como el Social Product Backlog, el Health Score y el Registro de Smells, mostrando cómo cada uno ayuda a documentar y gestionar la deuda social con rigor y trazabilidad.

Un punto crucial de esta parte es enfatizar que **SocialScrum NO agrega reuniones nuevas**. Todo se integra a los eventos que ya existen. Este es el corazón del Principio de No-Sobrecarga: mejorar la salud social sin aumentar el peso operativo del equipo.

3. Salvaguardas éticas (10 min). En esta parte de la presentación se explican, de forma clara y sin ambigüedades, las protecciones que hacen que SocialScrum sea un proceso seguro y no una herramienta de vigilancia. El propósito es desactivar cualquier temor o malentendido antes de que aparezca.

- **Confidencialidad absoluta de información individual:** nada de lo que diga una persona será compartido fuera del equipo si puede identificarla directa o indirectamente.
- **Separación total entre el Health Score y las evaluaciones de desempeño:** los datos sociales jamás se usarán para medir rendimiento individual, decidir ascensos, despidos o salarios.
- **Estructura de reporte clara:** el Social Guide reporta únicamente al CTO o al Scrum Master. Recursos Humanos no tiene acceso a información social, salvo en las excepciones legales previamente establecidas.
- **Derecho de auditoría del equipo:** cualquier reporte que vaya a salir del equipo debe ser revisado primero por sus integrantes. Si algo vulnera la confidencialidad, pueden pedir ajustes o bloquear el envío.

Esta parte de la presentación es esencial. Muchos *stakeholders* podrían ver SocialScrum como algo invasivo si no comprenden bien estas salvaguardas. Aclararlas con calma y transparencia ayuda a construir confianza desde el primer día.

4. Plan de implementación y métricas de éxito (10 min). Aquí se comparte el plan de trabajo para los próximos tres Sprints, mostrando cómo será la adopción en la práctica y qué espera ver la organización durante este periodo inicial.

- **Sprint 1:** puesta en marcha del modelo, trabajando sobre un smell de baja complejidad para aprender el proceso sin generar sobrecarga.
- **Sprint 2:** ajustes y mejoras basadas en lo aprendido durante el Sprint 1; aquí el equipo refina su manera de aplicar SocialScrum.
- **Sprint 3:** evaluación formal de efectividad, comparando el Health Score inicial con el actual y revisando avances en los smells priorizados.

También se definen las métricas de éxito que permitirán evaluar si la implementación va por buen camino:

- El Health Score aumenta al menos en 1 punto después de tres Sprints.
- Se logra mitigar parcialmente al menos un *community smell* crítico.
- La velocity no cae más del 20 % en el proceso de adopción y muestra señales de recuperación en el Sprint 3.

Nota sobre pre-requisitos de tiempo: El tiempo de 45 minutos supone que el equipo ha revisado previamente las secciones 3.2-3.3 de este documento (Modelo conceptual y Fundamentos de SocialScrum). Si eso no es viable, se recomienda:

- **Opción A:** Expandir a 90 minutos
- **Opción B:** Dos sesiones de 45 min (con 1 semana diferencia)

De lo contrario, el equipo no tendrá comprensión real del modelo, y el consentimiento será formal pero no genuino.

G.1.2 Documentación post-presentación

Después de la presentación, el Social Guide publica un documento formal que contiene:

- Un resumen ejecutivo de SocialScrum (máximo una página, claro y directo).
- La política de confidencialidad firmada por el CTO.
- El plan de implementación para los próximos tres Sprints.
- Los datos de contacto del Social Guide para dudas, inquietudes o comentarios —así, cualquier persona del equipo puede escribirle sin rodeos.

Este documento queda disponible en la herramienta que use la organización para que todas las personas del equipo y los *stakeholders* lo puedan consultar cuando lo necesiten, es decir, nada escondido, nada de puertas cerradas: todo transparente, como debe ser.

La adopción inicial de SocialScrum no es un evento de una sola vez. Es un proceso que se construye paso a paso, con intención y cuidado. Cuando se hace sin la preparación adecuada, el modelo puede sentirse como una imposición o, peor aún, como una amenaza. Pero cuando se implementa con claridad, respeto y buena comunicación, SocialScrum se integra de manera natural en la cultura del equipo. Desde el primer día se empieza a sembrar confianza, y eso abre el camino para una gestión sostenible de la salud social a largo plazo —esa que, a la larga, marca la diferencia en cómo trabaja y se siente un equipo.

Sin embargo, el proceso de adopción, por bien diseñado que esté, requiere alguien que lo lidere. Los pasos descritos —el consentimiento informado, el diagnóstico inicial, la configuración de herramientas— no se ejecutan solos. Aquí entra la figura que distingue a SocialScrum de otros marcos: el Social Guide, cuyo perfil, competencias y gobernanza se detallan a continuación en la siguiente sección.

G.2 2. Programa de capacitación del Social Guide

El programa consta de cuatro fases secuenciales con criterios de aprobación en cada etapa y una evaluación final post-capacitación.

G.2.1 Fase 1: Fundamentos teóricos (8 horas)

- Semanas 1-2 (antes de Sprint 0)
- 4 sesiones de 2h sincrónicas con tutor externo
- **Contenido:**
 - **Sesión 1: Teoría de la Autodeterminación (TAD) (2h)**
 - Qué es, cómo opera en equipos, checklist de autonomía
 - **Sesión 2: Confianza Mayer (2h)**
 - Qué es, diagnóstico de baja confianza, entrevista rápida
 - **Sesión 3: *Community Smells* (2h)**
 - Catálogo, identificación, registro (Cap. 4.5.3)
 - **Sesión 4: Valores Scrum aplicados (2h)**
 - Matriz valores × smells (Cap. 3.7)
- **Evaluación:** Quiz 30 min, $\geq 80\%$ para pasar

G.2.2 Fase 2: Facilitación y mediación (12 horas)

- Semanas 3-4 + Sprints 1-2
- 6 sesiones de 2h (sincrónica + práctica en vivo)
- **Contenido:**
 - **Sesiones 1-2: Retrospectivas sociales (4h)**
 - Protocolo 4 fases, facilitación sin juzgar, generación acciones
 - **Sesiones 3-4: CNV - Comunicación no violenta (4h)**
 - Observación, sentimientos, necesidades, peticiones
 - **Sesiones 5-6: Conversaciones difíciles (4h)**
 - Validación, legitimación, expansión de opciones
- **Evaluación:** Facilita retrospectiva observada por SM + evaluador
 - Rúbrica: Seguridad psicológica ($\geq 3/5$), preguntas generativas ($\geq 3/5$), síntesis clara ($\geq 3/5$). Si < 3 en alguna: sesión coaching adicional

G.2.3 Fase 3: Medición y análisis (8 horas)

- Semanas 5-6
- 4 sesiones de 2h
- **Contenido:**
 - **Sesión 1:** Health Score (definición, dimensiones, cálculo).
 - **Sesión 2:** Instrumentos (encuestas Likert, entrevistas, SNA).
 - **Sesión 3:** Análisis cualitativo (codificación, síntesis).
 - **Sesión 4:** Generación de reporte (qué comunicar, a quién, formato).
- **Evaluación:** Analizar dataset pequeño (10 encuestas); producir Health Score + 3 smells detectados. Si hay sesgos: revisión

G.2.4 Fase 4: Ética y salvaguardas (12 horas)

- Semanas 2-4 (intercalado).
- 6 sesiones de 2h.
- **Contenido:**

- **Sesión 1:** Confidencialidad (qué es confidencial vs reportable)
 - **Sesión 2:** Límites profesionales (cuándo derivar a psicólogo)
 - **Sesión 3:** Sesgos y conflictos de interés
 - **Sesión 4:** “Teatro Social” (detección y prevención)
 - **Sesión 5:** Casos éticos complejos (role-plays)
 - Ejemplo: “Dev reporta ideación suicida” → respuesta correcta
 - **Sesión 6:** Auditoría y responsabilidad
- **Evaluación:** Cuestionario ético (50 preguntas); role-play observado

G.2.5 Evaluación final post-capacitación (semana 7)

- Quiz teórico: 60 min, $\geq 80\%$ para pasar
- Facilita retrospectiva simulada + observada
- Caso de análisis: Produce Health Score + diagnóstico + plan
- **Umbral de aprobación:** Todo debe estar $\geq 3/5$

G.2.6 Seguimiento (en primer año)

- **Mes 1-2:** Supervisión semanal (SM/tutor externo)
- **Mes 3-6:** Supervisión quincenal
- **Mes 6-12:** Supervisión mensual + sesión de desarrollo anual
- **Año 2+:** Supervisión según necesidad + educación continua 10h/año

G.3 3. Protocolos operativos para eventos clave

G.3.1 Preguntas sociales recomendadas

Tipo	Ejemplo de pregunta
Estado general	“En una palabra, ¿cómo te sientes respecto al trabajo en equipo?”
Colaboración	“¿Hay algo que necesites de alguien y no estés recibiendo?”
Reconocimiento	“¿Alguien te ayudó de forma que quieras reconocer?”
Tensión	“¿Hay algo que te genere tensión y quieras mencionar?”

Tabla G.2: Tipos de preguntas sociales para el Daily

G.3.2 Preguntas de evaluación de riesgos sociales

Pregunta	Propósito
¿Existen conflictos activos?	Identificar conflictos entre miembros que deban colaborar durante el Sprint.
¿Hay concentración de conocimiento?	Detectar dependencias riesgosas en una sola persona o subgrupo.
¿Se observan señales de burnout?	Evaluar si el compromiso técnico es realista dadas las condiciones actuales del Health Score.
¿Qué ítems sociales deberían incluirse?	Decidir qué ítems del Social Product Backlog deben abordarse en el Sprint.

Tabla G.4: Preguntas de evaluación de riesgos sociales

G.3.3 Documentación de riesgos sociales

Riesgo identificado	Probabilidad	Impacto potencial	Mitigación acordada
Conflicto entre Ana y Carlos en el módulo de pagos	Alta	Bloqueo de una funcionalidad crítica	Sesión de alineación previa al inicio del trabajo conjunto.
Dependencia única de Miguel en el backend	Media	Retraso significativo en caso de ausencia	Pair programming dos horas al día con Laura para transferencia de conocimiento.
Fatiga generalizada (Health Score 5.8)	Alta	Reducción estimada de la velocidad entre 20 y 30 %	Reducción del compromiso del Sprint a 38 puntos.

Tabla G.6: Ejemplo de registro de riesgos sociales del Sprint

Este registro se revisa en la Social Sprint Retrospective para evaluar si los riesgos se materializaron y si las mitigaciones fueron efectivas.

G.3.4 Contenido de la evaluación de salud social

Elemento	Tiempo	Descripción
Health Score actual vs. anterior	2 min	Comparación y explicación de cambios significativos entre mediciones consecutivas.
<i>Community smells</i> abordados	3 min	Revisión de los ítems sociales completados y del impacto observable generado.
Riesgos materializados vs. mitigados	3 min	Análisis del registro de riesgos definido durante el Sprint Planning.
Tendencias y proyección	2 min	Visión de mediano plazo: ¿la salud social está mejorando, empeorando o se mantiene estable?

Tabla G.8: Elementos de la evaluación de salud social

G.3.4.1 Técnicas por fase

Fase 1: Preparación del espacio Registro emocional (5 min): cada persona responde una pregunta breve (“En una palabra, ¿cómo te sientes?”). Recordatorio de acuerdos (3 min): confidencialidad, no culpa, presunción de buena fe, derecho a no participar. Verificación de seguridad (2 min): confirmar que todos pueden hablar con honestidad.

Fase 2: Recolección de datos Técnica principal: Se identifican cuatro cuadrantes sociales tales como la colaboración positiva/problemática y comunicación efectiva/deficiente. Cada persona escribe notas específicas basadas en eventos concretos. Técnica alternativa: línea de tiempo emocional del Sprint.

Fase 3: Análisis de patrones Selección de 2-3 temas prioritarios mediante la votación por puntos. Análisis de causa raíz con 5 Whys (5 Porqués) adaptados a contexto social. El Social Guide redirige señalamientos de culpables hacia condiciones sistémicas. Mapeo a *community smells* del catálogo cuando corresponda.

Fase 4: Generación de compromisos Propuesta de acciones SMART: específicas, medibles, alcanzables, relevantes, temporales. Estimación en SSP y priorización por severidad/esfuerzo. Formalización como ítems del Social Product Backlog con título, smell abordado, acción, responsable, métrica de éxito y estimación.

G.3.4.2 Cierre y documentación

Resumen de compromisos (2 min), registro de salida (*check-out*) emocional (3 min), agradecimiento (1 min). En 24 horas, el Social Guide publica resumen interno: temas discutidos, causas raíz, acciones comprometidas. Los *stakeholders* solo reciben información de alto nivel.

G.3.4.3 Frecuencia

Dos modelos: (1) **integrado** —cada Retrospective incluye componentes técnicos y sociales (90-120 min); (2) **alternado** —Sprints pares técnicos, impares sociales. Equipos con Health Score < 6 se benefician del modelo integrado.

G.3.4.4 Manejo de situaciones difíciles

Situación	Respuesta del Social Guide
Conflicto abierto entre dos personas	Intervenir para contener, proponer una conversación privada posterior y continuar con el resto de los temas.
Silencio prolongado	Normalizar la situación, ofrecer mecanismos de aporte anónimo; si persiste, cerrar antes la sesión y programar encuentros 1:1.
Expresión emocional intensa	Validar la emoción expresada, evitar presionar, ofrecer seguimiento posterior y continuar sin dramatizar.
Señalamiento de culpables	Redirigir la conversación hacia condiciones sistémicas: “¿Qué condiciones permitieron que esto ocurriera?”.

Tabla G.10: Manejo de situaciones difíciles en la Social Sprint Retrospective

La Social Retrospective es el motor de mejora continua de SocialScrum. Con ella, el equipo desarrolla capacidad de gestionar su propia salud social con autonomía creciente.

ANEXO H: FRAMEWORK DE PRIORIZACIÓN EXTENDIDO

H.1 Guía del Framework de Priorización Extendido

H.1.1 Matriz de 18 escenarios de priorización

Aunque no existe una fórmula, sí existen patrones reconocibles. La matriz de escenarios presenta las 18 combinaciones más comunes de valores alto (7-10), medio (4-6) y bajo (1-3) en cada dimensión, junto con la decisión típica para cada caso.

#	Valor	Técnica	Social	Prioridad	Decisión típica
1	Alto	Alto	Bajo	Máxima	Entregar inmediatamente. Equipo sano puede absorber complejidad.
2	Alto	Bajo	Alto	Alta	Mitigar lo social primero. Un equipo en crisis no puede entregar calidad.
3	Bajo	Alto	Alto	Alta	Resolver ambos en paralelo. La urgencia técnica y social no compiten.
4	Alto	Alto	Alto	Escalar	Crisis total. Escalar a CTO para una decisión organizacional.
5	Alto	Medio	Medio	Alta	Entregar con precaución. Monitorear la salud durante la implementación.
6	Medio	Alto	Medio	Alta	Resolver lo técnico primero. La deuda técnica bloquea el progreso.
7	Medio	Medio	Alto	Alta	Resolver lo social primero. Un equipo en crisis contamina todo lo demás.
8	Alto	Bajo	Alto	Media-Alta	Balancear: MVP del feature más intervención social.
9	Bajo	Alto	Bajo	Media	Resolver lo técnico. Importante aunque no genere valor inmediato.
10	Bajo	Bajo	Alto	Media	Resolver lo social. La salud del equipo es un prerequisite.
11	Alto	Alto	Bajo	Media-Alta	Entregar. Equipo sano, aprovechar la capacidad disponible.
12	Bajo	Bajo	Bajo	Baja	Posponer. No hay urgencias ni impacto inmediato.
13	Alto	Bajo	Bajo	Alta	Entregar. Feature valiosa sin impedimentos relevantes.
14	Bajo	Alto	Bajo	Media	Resolver lo técnico. Deuda urgente aunque el valor sea bajo.
15	Bajo	Bajo	Alto	Media	Resolver lo social. El equipo necesita atención.
16	Medio	Medio	Medio	Media	Caso por caso. Todo es moderado; decidir según el contexto.
17	Alto	Bajo	Alto	Alta	Feature y social en paralelo. MVP mientras se atiende al equipo.
18	Alto	Alto	Bajo	Alta	Resolver lo técnico primero. Bug crítico bloquea, equipo sano.

Tabla H.2: Matriz de 18 escenarios de priorización

La matriz de la tabla H.2 sirve como guía inicial, no como regla definitiva. Cada equipo desarrolla su propia interpretación basada en experiencia acumulada.

H.1.2 Protocolo de decisión colaborativa

La priorización en SocialScrum es un proceso colaborativo que ocurre durante el Social Sprint Planning. Esta sección describe el protocolo paso a paso, los roles de cada participante y el mecanismo de resolución cuando no hay consenso.

H.1.2.1 Participantes y autoridad

Cada participante tiene autoridad sobre una dimensión específica:

Rol	Dimensión	Autoridad
Product Owner	Valor de Negocio	Decide qué entregar al cliente. Prioriza <i>features</i> por valor.
Scrum Master	Proceso	Decide cómo trabajar. Protege el marco de Scrum.
Social Guide	Salud Social	Decide cuándo el equipo puede comprometerse. Protege la capacidad del equipo.
Dev Team	Urgencia Técnica	Decide cuánto es factible. Estima y advierte sobre deudas técnicas.

Indicador	Estado
Health Score actual	6.2/10 (zona: en riesgo)
Dimensiones bajas	Coraje (4.5), Foco (5.1)
<i>Community smells</i> activos	<i>Radio Silence</i> (moderado)
Tendencia	Mejorando (5.1 en el Sprint anterior)
Capacidad estimada	Aproximadamente 80 % de la normal (32 SP frente a 40 SP típicos)

Tabla H.6: Diagnóstico del Social Guide

Este paso establece el contexto. Si el Health Score es crítico, los pasos siguientes se ajustan automáticamente.

Paso 2: Product Owner presenta *features* priorizadas (5 minutos) El Product Owner comparte los items de mayor valor que desea incluir en el Sprint:

#	Feature	Valor
1	Integración con API de Pagos	9/10
2	Dashboard de Analytics	7/10
3	Notificaciones push	8/10
4	Mejora de UX en login	5/10
5	Exportar reportes a PDF	6/10

Tabla H.8: *Features* priorizadas por valor de negocio (Product Owner)

El Product Owner presenta más items de los que caben en un Sprint para permitir flexibilidad en la selección final.

Paso 3: Social Guide propone items sociales (5 minutos) Si el Health Score indica necesidad, el Social Guide propone items sociales con su estimación en Social Story Points (SSP):

#	Intervención propuesta	Esfuerzo (SSP)
1	Mediación Dev-QA para resolver <i>Radio Silence</i>	5
2	Establecer protocolo de daily asíncrono	3

Tabla H.10: Propuesta de intervención del Social Guide (Health Score 6.2)

Proyección: Si estas intervenciones se abordan, se espera una mejora del Health Score hasta un rango de 7.0–7.5 al final del Sprint.

Si el Health Score es saludable (≥ 8), el Social Guide indica que no se requieren items sociales.

Paso 4: Discusión y priorización colaborativa (5-10 minutos) Los cuatro roles dialogan para llegar a un Sprint Backlog consensuado:

Ejemplo de diálogo de decisión en Sprint Planning

Escenario aplicado

Sprint Planning — Diálogo de priorización colaborativa

Es factible, pero consume gran parte de la capacidad.

Product Owner: Es la feature de mayor valor. ¿Podemos comprometernos?

Social Guide: Con un Health Score de 6.2 estamos en el límite. Si incluimos la mediación Dev-QA (5 SSP), se mejora la comunicación y se reduce el riesgo de la feature compleja. Recomendando incluir ambas.

Dev Team: Con $13 \text{ SP} + 5 \text{ SSP} = 18$ puntos, quedan 14 SP disponibles. El Dashboard de Analytics (8 SP) y el daily asíncrono (3 SSP) son viables.

Product Owner: Entonces las notificaciones push (Valor 8) quedan fuera.

Social Guide: Correcto. Con un Health Score mejorado, el siguiente Sprint tendrá capacidad completa.

Ítem comprometido	Tipo	Esfuerzo
Integración con API de Pagos	Técnico	13 SP
Mediación Dev-QA	Social	5 SSP
Dashboard de Analytics (MVP)	Técnico	8 SP
Daily asíncrono	Social	3 SSP
Total		29 puntos

Tabla H.12: Compromiso final del Sprint tras diálogo de priorización

Sprint Goal: Integrar pagos y mejorar la comunicación Dev-QA, manteniendo el compromiso dentro de la capacidad disponible (32 SP).

H.1.2.3 Conversión entre Social Story Points y Story Points

Durante la discusión, es necesario comparar items técnicos (estimados en SP) con items sociales (estimados en SSP). SocialScrum establece una equivalencia aproximada:

$$1 \text{ SSP} \approx 1 \text{ SP} \quad (\text{H.1})$$

La justificación es práctica: ambos tipos de items consumen tiempo y atención del equipo. Un item social de 5 SSP requiere aproximadamente el mismo esfuerzo que un item técnico de 5 SP.

Ejemplo de cálculo de capacidad:

Ítem	Tipo	Esfuerzo
Feature A	Técnico	13 SP
Feature B	Técnico	8 SP
Mediación	Social	5 SSP
Workshop	Social	3 SSP
Total comprometido		29 puntos
Capacidad del equipo		40 SP
Margen restante		11 puntos

Tabla H.14: Capacidad del equipo y compromiso del Sprint

Esta conversión es una aproximación que cada equipo debe calibrar según su contexto. Algunos equipos descubren que sus ítems sociales requieren más o menos esfuerzo relativo, y ajustan la equivalencia.

H.1.2.4 Protocolo de discrepancia

Cuando Product Owner, Social Guide y Scrum Master no logran consenso, se activa un protocolo de resolución estructurado:

Fase 1: Argumentación (9 minutos) Cada rol dispone de 3 minutos para presentar su posición con evidencia:

Escenario aplicado

Product Owner (3 min): Esta feature es crítica porque el cliente principal amenazó con cancelar el contrato si no entregamos en dos semanas. Valor de negocio: 10/10.

Social Guide (3 min): El Health Score es 4.2. Dos personas reportaron síntomas de burnout. Si se fuerza esta entrega, existe un riesgo real de renuncia. Salud social: 9/10.

Scrum Master (3 min): Desde la perspectiva de proceso, forzar un Sprint con el equipo en crisis viola el principio de sostenibilidad. Sin embargo, también existe una presión de negocio significativa que debe ser considerada.

Fase 2: Input del Development Team (5 minutos) El equipo de desarrollo expresa su perspectiva:

Escenario aplicado

Dev Team: Honestamente, no nos sentimos capaces de entregar calidad en este estado. Si se nos fuerza a hacerlo, entregaremos algo, pero con alta probabilidad de errores y re-trabajo posterior.

Fase 3: Votación informal Se presenta una votación con tres opciones:

- **Opción A:** Priorizar feature (aceptar riesgo social).
- **Opción B:** Priorizar salud social (posponer feature).
- **Opción C:** Híbrido (MVP de feature + mitigación social).

Fase 4: Decisión o escalamiento Si hay mayoría clara, se implementa esa opción. Si no hay mayoría, se escala a CTO o VP Engineering con un resumen del dilema. El Social Sprint Planning continúa con items de consenso mientras se espera la decisión (máximo 24 horas).

H.1.2.5 Ejemplo completo de priorización

El siguiente ejemplo ilustra el proceso completo para un equipo en situación típica:

Escenario aplicado

Sprint Planning — Equipo Alpha (Sprint 14)

Fecha: 2024-03-18

Contexto

El Social Guide reporta un Health Score de 6.8/10, ubicado en la zona de riesgo (límite superior). Se identifican dimensiones con valores inferiores al promedio, particularmente Foco (5.8), asociadas a *Context Switching* moderado. La capacidad estimada del equipo es del 90 %, equivalente a 36 SP.

Backlog inicial

El Product Owner presenta un conjunto de *features* priorizadas por valor de negocio y urgencia técnica (ver Tabla H.18).

Propuesta social

El Social Guide propone una intervención ligera orientada a mejorar el foco del equipo mediante la iniciativa *Focus Fridays* (ver Tabla H.20).

Discusión

Dev Team: Migración Auth y API Partners suman 21 SP. Con *Focus Fridays* (3 SSP) el total asciende a 24 puntos, quedando 12 SP disponibles.

Product Owner: Consulta por el refactor del módulo de reportes.

Dev Team: Se propone para el Sprint 15; no es urgente.

Social Guide: La combinación es balanceada; el Health Score debería mantenerse o mejorar levemente.

Scrum Master: Consenso alcanzado.

Resultado

El Sprint se compromete con un backlog balanceado entre valor técnico y salud social (ver Tabla H.22).

Sprint Goal: Habilitar microservicios y mejorar el foco del equipo.

Indicador	Estado
Health Score	6.8/10 (zona en riesgo, límite superior)
Dimensiones	Apertura 7.2, Compromiso 7.0, Respeto 7.5, Foco 5.8, Coraje 6.5
Community smells	Context Switching (moderado)
Tendencia	Estable (6.7 en el Sprint anterior)
Capacidad estimada	90 % → 36 SP de 40 típicos

Tabla H.16: Diagnóstico social inicial del equipo Alpha

#	Ítem	Valor / Urgencia	Esfuerzo
1	Migración a microservicios — Módulo Auth	Valor 8	13 SP
2	API para partners externos	Valor 9	8 SP
3	Mejora de dashboard admin	Valor 5	5 SP
4	Optimización de queries lentas	Urgencia 7	5 SP
5	Refactor del módulo de reportes	Urgencia 5	8 SP

Tabla H.18: Backlog priorizado por valor y urgencia

Intervención social	Esfuerzo	Beneficio esperado
Focus Fridays (sin reuniones)	3 SSP	Incrementar el foco a un rango de 6.5–7.0

Tabla H.20: Intervención social propuesta por el Social Guide

Ítem	Tipo	Puntos
Migración Auth	Técnico	13 SP
API Partners	Técnico	8 SP
Optimización de queries	Técnico	5 SP
Mejora dashboard admin	Técnico	5 SP
Focus Fridays	Social	3 SSP
Total comprometido		34 pts
Capacidad		36 SP
Margen		2 SP

Tabla H.22: Sprint Backlog final — Equipo Alpha

ANEXO I: SISTEMA DE ESTIMACIÓN SOCIAL (SSP)

I.1 Sistema de estimación de items sociales (Social Story Points)

En Scrum tradicional, el Development Team estima el esfuerzo de los items técnicos usando Story Points (SP) con la escala Fibonacci (1, 2, 3, 5, 8, 13, 21). SocialScrum enfrenta un problema análogo: los items de deuda social también consumen tiempo y atención del equipo, pero no tienen una unidad de medida comparable. Esta sección introduce los Social Story Points (SSP), una escala de estimación donde $1 \text{ SSP} \approx 1 \text{ SP}$, permitiendo integrar items técnicos y sociales en un mismo Social Sprint Backlog.

I.1.1 Escala de estimación SSP

La escala SSP utiliza los mismos valores Fibonacci: 1, 2, 3, 5, 8 y 13. Valores superiores a 13 indican que el item debe descomponerse.

SSP	Duración	Tipo de intervención	Ejemplo
1–2	1–4 horas	Conversación facilitada, ajuste menor	Feedback rápido, norma simple
3–5	4 h–2 días	Workshop, mediación, retrospectiva profunda	Código de conducta, mediación bilateral
8–13	2 días–1 semana	Intervención organizacional, crisis	Cambio de liderazgo, burnout severo

Tabla I.2: Escala de Social Story Points (SSP)

I.1.2 Metodología de estimación en 3 pasos

La estimación SSP sigue un proceso de tres pasos: identificar el *community smell*, evaluar severidad, y evaluar complejidad de intervención.

Paso	Criterio	Entrada	Salida
1	Identificar smell	Registro de Community Smells	Tipo específico confirmado
2	Evaluar severidad	Health Score y observación	Leve / Moderada / Severa / Crítica
3	Evaluar complejidad	Alcance, facilitación y sesiones	Baja (+0) / Media (+1) / Alta (+2)

Tabla I.4: Proceso de estimación de SSP

I.1.2.1 Matriz de severidad y SSP base

Severidad	Indicadores	SSP base
Leve	Afecta 1-2 personas ocasionalmente; no impacta la entrega	1-2
Moderada	Afecta 3-5 personas; retrasos menores; tensión visible	3-5
Severa	Afecta al equipo entero; riesgo de rotación; calidad degradada	5-8
Crítica	Equipo disfuncional; rotación inminente; proyecto en riesgo	8-13

Tabla I.6: Niveles de severidad y SSP base

I.1.2.2 Cálculo del SSP final

$$\text{SSP Final} = \text{SSP Base (severidad)} + \text{Ajuste (complejidad)} \quad (\text{I.1})$$

Ejemplo: Conflicto Dev A-B (Severidad Moderada = 3 SSP base) + Mediación especializada requerida (Complejidad Alta = +2) → SSP Final = 5.

I.1.3 Tabla de referencia rápida

Community smell	Severidad típica	Intervención típica	SSP
<i>Radio Silence</i>	Moderada	Workshop y ajuste de proceso	3-5
<i>Lone Wolf</i>	Leve-Moderada	Mediación y reintegración	3-5
Conflicto interpersonal leve	Leve	Mediación bilateral simple	3
Conflicto interpersonal severo	Severa	Mediación intensiva y seguimiento	8
<i>Burnout</i> leve / moderado	Leve-Moderada	Conversación y ajuste de carga	2-5
<i>Burnout</i> severo	Severa-Crítica	Intervención de crisis	13
Inseguridad psicológica	Moderada-Severa	Workshop y retrospectiva facilitada	5-8
<i>Knowledge hoarding</i>	Moderada	Documentación y transferencia de conocimiento	3-5

Tabla I.8: Referencia rápida de estimación SSP por *community smell*

I.1.4 Conversión SSP ↔ SP y gestión de capacidad

El principio fundamental es: **1 SSP ≈ 1 SP**. Ambos tipos de items pueden sumarse para calcular la capacidad total del Sprint.

Tipo	Ítems ejemplo 207	Puntos	Total
Técnico	Feature A (13 SP) + Feature B (8 SP) + Bug fix (5 SP)	26 SP	
Social	Mediación de conflicto (5 SSP) + Daily	8 SSP	

I.1.4.1 Mejores prácticas de estimación

- **Estimar conservadoramente:** Añadir 20 % de buffer en situaciones inciertas.
- **Descomponer items grandes:** Si un item supera 8 SSP, buscar cómo dividirlo.
- **Registrar real vs. estimado:** Mantener historial para calibración.
- **Un item = un community smell específico:** Evitar items vagos como “mejorar comunicación”.

ANEXO J: CATÁLOGO DE ESTRATEGIAS DE MITIGACIÓN

Este anexo presenta un catálogo de estrategias de mitigación para diferentes categorías de *community smells*. Cada estrategia incluye una descripción, pasos recomendados y consideraciones clave para su implementación efectiva.

J.1 Estrategias de mitigación de community smells

Detectar un *community smell* es solo el primer paso. La detección sin acción genera frustración: el equipo sabe que hay un problema pero no ve mejora. Esta sección proporciona un catálogo de estrategias de mitigación basadas en la taxonomía de *community smells* de Caballero et al. [2], organizadas por categoría de smell. Herramientas como csDetector [38] permiten detección automática de algunos de estos patrones a partir de metadatos de repositorios.

El objetivo no es eliminar todos los smells —algunos son síntomas de tensiones organizacionales que el equipo no controla— sino reducir su impacto en la capacidad del equipo de entregar valor.

J.1.1 Catálogo de estrategias por categoría

Para cada categoría de *community smell*, se describen problemas comunes, ejemplos representativos y tipos de intervención recomendados (ver Tabla J.2).

Categoría	Problema	Smells representativos	Tipo de intervención
Comunicación	Información no fluye entre miembros o hacia afuera	Radio Silence (Silencio Radiofónico), Information Hoarding (Acaparamiento de Información), Broadcast Storm (Tormenta de Difusión), Missing Link (Eslabón Perdido), Synchronous Overload (Sobrecarga Sincrónica)	Rediseño de canales, protocolos y automatización
Conocimiento	Saber técnico concentrado o no compartido	Truck Factor (Key Person Dependency) = 1, Knowledge Islands (Islas de Conocimiento), Outdated Documentation (Documentación Desactualizada), Tribal Knowledge (Conocimiento Tribal), Expert Bottleneck (Cuello de Botella Experto)	<i>Pair programming</i> , documentación y rotación
Relacional	Conflictos, exclusión y dinámicas tóxicas	Lone Wolf (Lobo Solitario), Prima Donnas (Primas Donnas), Clique Formation (Formación de Cliques), Unresolved Conflict (Conflicto No Resuelto), Toxic Leadership (Liderazgo Tóxico)	Mediación, facilitación y feedback directo
Organizacional	Estructura o procesos disfuncionales	Organizational Silo (Silo Organizacional), Meeting Hell (Infierno de Reuniones), Unclear Ownership (Propiedad Poco Clara), Process Overhead (Sobrecarga de Procesos), Unstable Team (Equipo Inestable)	Escalamiento, negociación y reestructuración

Tabla J.2: Categorías de *community smells*, ejemplos e intervenciones

J.1.1.1 Estrategias para Community Smells (Olores Comunitarios) de comunicación

Estas estrategias abordan problemas de flujo de información dentro del equipo y con *stakeholders* externos.

Smell	Síntomas	Estrategia de mitigación	SSP
Radio Silence (Silencio Radiofónico)	Sorpresas frecuentes; “nadie me dijo”; la información llega tarde	1) Identificar nodos centrales mediante SNA. 2) Crear un canal de broadcast para actualizaciones críticas. 3) Establecer <i>office hours</i> donde cualquiera pueda preguntar. 4) Rotar la responsabilidad de comunicar en la Daily.	3–5
Information Hoarding (Acaparamiento de Información)	Personas guardan información “por si acaso”; duplicación de esfuerzo	1) Workshop sobre beneficios de compartir. 2) Crear una wiki o Confluence con expectativa explícita de documentar decisiones. 3) Reconocer públicamente cuando alguien comparte información útil.	3–5
Broadcast Storm (Tormenta de Difusión)	Canales saturados; nadie lee mensajes; información importante se pierde	1) Auditar y consolidar canales existentes. 2) Establecer convenciones (por ejemplo, @here solo para urgencias). 3) Crear resúmenes diarios o semanales. 4) Eliminar canales redundantes.	2–3
Missing Link (Eslabón Perdido)	Subgrupos no se comunican; integraciones tardías generan conflictos	1) Mapear la comunicación actual mediante SNA. 2) Crear rituales de sincronización entre subgrupos. 3) Asignar enlaces rotativos entre equipos. 4) Realizar retrospectivas conjuntas mensuales.	5–8
Synchronous Overload (Sobrecarga Sincrónica)	Calendarios saturados; trabajo fragmentado; fatiga de videollamadas	1) Auditar reuniones y eliminar las innecesarias. 2) Establecer <i>no-meeting</i> hours.	3–5

J.1.1.2 Estrategias para smells de conocimiento

Estas estrategias se enfocan en mejorar la gestión y transferencia del conocimiento dentro del equipo.

Smell	Síntomas	Estrategia de mitigación	SSP
<i>Truck Factor</i> = 1	“Solo Juan sabe cómo funciona X”; bloqueos cuando Juan no está	1) Identificar componentes críticos con un solo experto. 2) <i>Pair programming</i> forzado: Juan + otro dev en cada tarea del componente. 3) Juan documenta la arquitectura básica (2–3 páginas). 4) Otro dev realiza un cambio pequeño de forma autónoma; Juan revisa.	5–8
Knowledge Islands (Islas de Conocimiento)	Subgrupos especializados no comparten conocimiento; la integración es dolorosa	1) <i>Tech talks</i> semanales rotativos (30 min). 2) <i>Code reviews</i> cruzados entre subgrupos. 3) Rotación temporal: un Sprint en otro subgrupo. 4) Documentación explícita de interfaces entre componentes.	5–8
Outdated Documentation (Documentación Desactualizada)	La documentación existe pero es incorrecta; nadie confía en ella	1) Auditar documentación: marcar obsoleta o eliminar. 2) Establecer <i>docs as code</i> : la documentación vive junto al código. 3) Incluir actualización de documentación en la <i>Definition of Done</i> . 4) Asignar un responsable de cada documentación crítica.	3–5
Tribal Knowledge (Conocimiento Tribal)	Conocimiento principalmente	1) Grabar sesiones de	5–8

J.1.1.3 Estrategias para smells relacionales

Estas estrategias se enfocan en mejorar las interacciones y relaciones dentro del equipo.

Smell	Síntomas	Estrategia de mitigación	SSP
Lone Wolf (Lobo Solitario)	Trabaja solo; no pide ni ofrece ayuda; PRs sin contexto	1) 1:1 para entender la motivación (preferencia o desconfianza). 2) Pair programming gradual (1 hora al día). 3) Asignar tareas que requieran colaboración. 4) Reconocer públicamente cuando colabora.	3-5
Prima Donnas (Primas Donnas)	Cree que sus ideas son superiores; no acepta feedback; bloquea el consenso	1) 1:1 para dar feedback específico sobre el impacto. 2) Establecer la norma de decisiones por consenso, no por imposición. 3) Documentar criterios objetivos para decisiones técnicas. 4) Si persiste, escalar al Scrum Master.	5-8
Clique Formation (Formación de Cliques)	Subgrupo cerrado; exclusión en almuerzos o canales; la información no sale del clique	1) Workshop de cohesión con todo el equipo. 2) Rotar parejas de trabajo para romper cliques. 3) Crear rituales inclusivos (coffee chat aleatorio). 4) 1:1 con líderes informales del clique.	5-8
Unresolved Conflict (Conflicto No Resuelto)	Tensión visible; evitan trabajar juntos; comentarios pasivo-agresivos	1) 1:1 con cada parte para entender su perspectiva. 2) Sesión de mediación facilitada. 3) Documentar acuerdos de comportamiento. 4) Seguimiento en 2-4 semanas.	5-8
Toxic Leadership (Liderazgo Tóxico)	Micromanagement; culpabilización pública; generación de miedo	1) Documentar incidentes específicos con fechas. 2) Escalar a CTO o VP con evidencia. 3) Proteger al equipo mientras se resuelve la situación. 4) En casos severos, separar al líder del equipo.	8-13

Tabla J.8: Estrategias para smells relacionales

J.1.1.4 Estrategias para smells organizacionales

Estas estrategias abordan problemas estructurales y de proceso que afectan al equipo.

Smell	Síntomas	Estrategia de mitigación	SSP
Organizational Silo (Silo Organizacional)	Equipos no se comunican; duplicación de trabajo; integraciones tardías	1) Identificar puntos de contacto necesarios. 2) Establecer embajadores entre equipos. 3) Reuniones de sincronización bisemanales. 4) Escalar a liderazgo si la estructura impide la colaboración.	5-8
Meeting Hell (Infierno de Reuniones)	Calendarios saturados por demandas externas al equipo	1) Auditar reuniones: quién las convoca y si son necesarias. 2) Negociar con <i>stakeholders</i> un representante único. 3) Establecer <i>focus time</i> protegido. 4) Escalar al Scrum Master para proteger al equipo.	3-5
Unclear Ownership (Propiedad Poco Clara)	Preguntas frecuentes sobre responsables; decisiones postergadas	1) Mapear componentes y asignar responsables explícitos. 2) Documentar una matriz RACI para decisiones comunes. 3) Publicar el <i>ownership</i> en una wiki accesible. 4) Revisar y actualizar cada tres meses.	3-5
Process Overhead (Sobrecarga de Procesos)	Aprobaciones múltiples y burocracia que no agrega valor	1) Documentar tiempo perdido en procesos. 2) Proponer simplificación basada en evidencia. 3) Ejecutar un piloto con exención de un proceso durante dos Sprints. 4) Medir impacto y escalar resultados.	5-8
Unstable Team (Equipo Inestable)	Miembros prestados y rotación frecuente	1) Documentar el impacto en velocidad y calidad. 2) Escalar a liderazgo con datos. 3) Negociar un período mínimo de estabilidad. 4) Si no es posible cambiar, optimizar el onboarding.	5-8

Tabla J.10: Estrategias para smells organizacionales

J.1.1.5 Smells que requieren escalamiento

Algunos smells no pueden resolverse a nivel de equipo. El Social Guide debe reconocer estos casos y escalar apropiadamente:

Smell	Por qué requiere escalamiento	A quién escalar
Toxic Leadership (Liderazgo Tóxico)	El líder tiene poder formal sobre el equipo; la confrontación directa es riesgosa	CTO, VP Engineering, HR (con cuidado)
Organizational Silo (Silo Organizacional) (estructural)	Equipos separados por decisiones organizacionales	VP Engineering, Director de Producto
Unstable Team (Equipo Inestable) (política)	Las decisiones de staffing provienen de niveles superiores	CTO, Resource Manager
Budget Constraints (Restricciones Presupuestarias)	Problemas derivados de falta de recursos	CTO, VP con autoridad presupuestaria
Legal / Problemas de cumplimiento (<i>Compliance Issues</i>)	Procesos impuestos por regulación o cumplimiento normativo	Legal, Compliance Officer

Tabla J.12: Smells que requieren escalamiento

Para estos casos, el rol del Social Guide es:

1. Documentar el impacto con datos cuantitativos.
2. Preparar propuesta de solución (no solo el problema).
3. Escalar al *stakeholder* apropiado con Scrum Master.
4. Proteger al equipo mientras se resuelve externamente.

J.1.2 Diseño de experimentos de mejora

Las estrategias del catálogo son puntos de partida que pueden requerir adaptaciones al contexto. El diseño de experimentos permite probar intervenciones sistemáticamente.

Componente	Descripción
Hipótesis	Declaración falsificable que conecta una intervención con un resultado esperado. Formato: <i>Si [intervención], entonces [resultado], porque [mecanismo]</i> .
Métricas	Indicadores cuantitativos definidos con baseline y objetivo (<i>target</i>).
Diseño	Definición de qué se hará, cuándo, quiénes participan y recursos requeridos (SSP).
Criterios de evaluación	Éxito: métricas alcanzan el <i>target</i> → institucionalizar. Parcial: mejora $\geq 50\%$ → iterar. Fracaso: sin mejora → pivotar. Inconcluso: datos insuficientes → extender experimento.
Documentación	Registro estructurado de la hipótesis, resultados obtenidos, aprendizajes y decisión final.

Tabla J.14: Componentes de un experimento de mejora

No toda intervención requiere experimento formal. Se recomienda diseño experimental cuando: el contexto es único, la intervención es costosa (>8 SSP), el equipo es escéptico, o el impacto afecta múltiples equipos. Intervenciones simples y reversibles pueden implementarse directamente y ajustarse.

ANEXO K: PRINCIPIO DE NO-SOBRECARGA: INTEGRACIÓN SIN AGREGAR TIEMPO

K.1 1. Principio de No-Sobrecarga: Integración sin agregar tiempo

Una preocupación frecuente ante cualquier extensión metodológica es el tiempo adicional que consume. Esta sección demuestra que SocialScrum, implementado correctamente, agrega una sobrecarga mínima: aproximadamente 45 minutos cada dos semanas, equivalente al 0.6 % del tiempo total del Sprint.

El principio fundamental es **integración, no adición**: los componentes sociales de SocialScrum se incorporan a los eventos Scrum existentes, no crean reuniones nuevas.

K.1.1 Sobrecarga temporal por evento

Evento	Scrum estándar	SocialScrum	Sobrecarga (redistribución interna)	% Aumento
Social Sprint Planning	4 horas	4 h 15 min	+15 min	+6 %
Social Daily Standup (x10)	150 min total	170 min total	+20 min	+13 %
Social Sprint Review	2 horas	2 h 10 min	+10 min	+8 %
Social Sprint Retrospective	1.5 horas	1.5 horas	+0 min	0 %
Total por Sprint	10 horas	10 h 45 min	+45 min	+7.5 %

Tabla K.2: Sobrecarga temporal de SocialScrum por evento

K.1.1.1 Detalle por evento

Evento	Cambio en SocialScrum	Tiempo adicional
Social Sprint Planning	Tema 3: Riesgos sociales (Health Score, <i>community smells</i> activos e ítems sociales)	+15 min
Social Daily Standup (por día)	Pregunta final: “¿Hay algún impedimento social hoy?” (solo identificar, no resolver)	+2–3 min/día
Social Sprint Review	Sección dedicada a la evaluación de la dinámica social y la correlación entre Health Score y velocidad	+10 min
Social Sprint Retrospective	Redistribución del tiempo: inspección técnica (30 min), inspección social (45 min) y definición de acciones (15 min)	+0 min

Tabla K.4: Detalle de cambios por evento

K.1.2 Sesiones adicionales y presupuesto

Algunos ítems sociales requieren sesiones fuera de eventos estándar (mediaciones 1:1, talleres de cohesión). Estas sesiones sí representan tiempo adicional, pero están acotadas:

- **Máximo 20 % de capacidad del equipo** ²²⁰ para ítems sociales.
- **Máximo 2 horas de sesiones adicionales por semana** (fuera de eventos Scrum).

Optimización	Descripción	Ahorro
Daily asíncrono	Reemplazar el Daily síncrono por un formato escrito en Slack o Teams (campos: DONE, TODO, BLOCK, SOCIAL)	Elimina 15–18 min/día de reunión
Retros alternadas	Sprint impar: énfasis técnico (50/25 min). Sprint par: énfasis social (25/50 min)	Reduce la fatiga de retrospectiva
Social Guide rotativo	En equipos de 5–7 personas, rotar el rol cada Sprint. Dedicación aproximada: 10 % del tiempo cuando le corresponde	Distribuye la carga entre todos
Integración primero	Jerarquía de integración: Daily → Planning → Review → Retrospective. Solo crear una sesión adicional si no cabe en los eventos existentes	Minimiza sesiones adicionales

Tabla K.6: Optimizaciones para reducir sobrecarga

K.1.4 Gestión de expectativas

K.1.4.1 Comunicación al equipo

Mensaje clave: “SocialScrum agrega 45 min cada 2 semanas a nuestras reuniones Scrum (4.5 min/día). No creamos reuniones nuevas; nos integramos en las existentes. Beneficio esperado: equipos con mejor comunicación entregan más a largo plazo.”

K.1.4.2 Comunicación a stakeholders

Mensaje clave: “Implementamos SocialScrum para mejorar sostenibilidad del equipo. Verán un Health Score en cada Sprint Review. Sobrecarga: 0.6 % del tiempo. ROI: una renuncia evitada ahorra 3-6 meses de capacitación del reemplazo.”

K.1.5 Retorno de inversión (ROI) de SocialScrum

El retorno de inversión se estima en dólares y pesos colombianos con conversión de \$1 dólar es igual a \$4.000 COP, en este escenario se hace un ejemplo mostrado en las tablas K.8 y K.10.

Inversión (anual)	Horas	Detalle	Costo (USD)	Costo (COP)
Sobrecarga de reuniones	19.5 h	45 min × 26 Sprints	\$975	~3 900 000 COP
Sesiones adicionales	130 h	~5 h × 26 Sprints	\$6 500	~26 000 000 COP
Total	150 h	Costo anual estimado	\$7 500	~30 000 000 COP

Tabla K.8: Inversión anual estimada en sobrecarga de SocialScrum

Retorno (anual)	Valor estimado
1 renuncia evitada	\$50 000–\$100 000 dólares (50–200 % del salario anual)

ANEXO L: EJEMPLOS DETALLADOS DE INTERVENCIONES

Este anexo presenta ejemplos detallados de intervenciones para diferentes categorías de *community smells*. Cada ejemplo incluye diagnóstico, estrategia paso a paso, métricas de éxito y resultados observados.

L.1 1 Intervención: *Radio Silence* (Smell de Comunicación)

=== INTERVENCIÓN: RADIO SILENCE ===

Equipo: Alpha (7 personas)

Severidad detectada: Moderada (afecta velocidad, no bloquea)

DIAGNÓSTICO:

- SNA muestra que María centraliza 70% de comunicación
- 3 DEVs reportan "enterarse tarde" de cambios de API
- En Daily, solo María y Pedro hablan regularmente
- Health Score "Comunicación" 4.5/10

ESTRATEGIA (5 SSP):

Semana 1:

- Workshop "Comunicación distribuida" (2h)
- Crear canal #alpha-actualizaciones para cambios técnicos
- Norma: quien cambia API pública, postea en canal

Semana 2:

- Rotación de "facilitador de Daily" (cada día alguien diferente)
- Pregunta explícita en Daily: "¿Hay algo que otros deban saber?"

MÉTRICAS DE ÉXITO:

- Mensajes en #alpha-actualizaciones: ≥ 5 /semana
- Participación en Daily: todos hablan al menos 3x/semana
- Encuesta: "Me entero a tiempo" sube de 4/10 a 7/10

RESULTADO (Sprint +1):

- Mensajes en canal: 8/semana [OK]
- Participación Daily: 6 de 7 hablan regularmente [OK]
- Encuesta: 6.5/10 (mejora parcial, continuar)

L.2 2 Intervención: Truck Factor = 1 (Smell de Conocimiento)

=== INTERVENCIÓN: TRUCK FACTOR = 1 ===

Equipo: Beta (5 personas)

Componente crítico: Motor de reglas de negocio

Único experto: Carlos

DIAGNÓSTICO:

- Carlos escribió 95% del código del motor
- Cuando Carlos está de vacaciones, bugs en motor quedan sin resolver
- Otros devs "tienen miedo" de tocar ese código

ESTRATEGIA (8 SSP distribuidos en 3 Sprints):

Sprint 1 (3 SSP):

- Carlos documenta arquitectura en 3 páginas
- Carlos hace pair programming con Ana en 2 tareas del motor

Sprint 2 (3 SSP):

- Ana hace tarea sola en motor, Carlos solo revisa
- Carlos hace pair con Pedro en 1 tarea
- Sesión de 2h: "Cómo debuggear el motor"

Sprint 3 (2 SSP):

- Pedro hace tarea solo, Carlos revisa
- Documentar "troubleshooting guide" para problemas comunes
- Evaluar: ¿Ana y Pedro pueden resolver bugs nivel 1-2?

MÉTRICAS DE ÉXITO:

- Truck factor: de 1 a 3 (Carlos + Ana + Pedro)
- Tiempo de resolución de bugs cuando Carlos no está: < 48h

- Ana y Pedro reportan confianza $\geq 6/10$ para trabajar en motor

RESULTADO:

- Sprint 3: Ana resolvió bug de producción sola [OK]
- Confianza: Ana 7/10, Pedro 5/10 (continuar con Pedro)
- Truck factor efectivo: 2.5 (Ana casi independiente, Pedro parcial)

L.3 3 Intervención: Conflicto No Resuelto (Smell Relacional)

=== INTERVENCIÓN: UNRESOLVED CONFLICT ===

Equipo: Gamma (6 personas)

Partes: Dev A (frontend) vs Dev B (backend)

Origen: Desacuerdo sobre diseño de API hace 2 meses

DIAGNÓSTICO:

- Code reviews entre A y B son hostiles
- Otros miembros evitan tareas que requieren A+B
- Daily tiene tensión cuando ambos reportan

ESTRATEGIA (5 SSP):

Día 1 - Sesión 1:1 con Dev A (1h):

- Escuchar su perspectiva sin juzgar
- Identificar: ¿qué necesita para mejorar la situación?
- A dice: "B nunca acepta mis sugerencias, siempre impone"

Día 2 - Sesión 1:1 con Dev B (1h):

- Escuchar su perspectiva sin juzgar
- B dice: "A no entiende las restricciones de backend"

Día 3 - Preparación:

- Identificar intereses comunes (ambos quieren producto exitoso)
- Preparar estructura para sesión conjunta

Día 4 - Sesión conjunta facilitada (2h):

- Reglas: hablar en primera persona, no interrumpir
- Cada uno expresa su perspectiva (10 min c/u)

- Identificar puntos de acuerdo
- Negociar: ¿cómo manejar desacuerdos futuros?
- Documentar acuerdos firmados por ambos

Día 15 - Seguimiento (30 min):

- ¿Se están respetando acuerdos?
- Ajustar si es necesario

ACUERDOS DOCUMENTADOS:

1. Desacuerdos técnicos se discuten en 1:1, no en PR comments
2. Si no hay acuerdo en 30 min, escalar a Tech Lead
3. Ambos se comprometen a asumir buena intención del otro

MÉTRICAS DE ÉXITO:

- Code reviews sin comentarios hostiles: 4 semanas consecutivas
- A y B aceptan trabajar en tarea conjunta sin resistencia
- Health Score "Respeto" sube de 4/10 a 6/10

RESULTADO (Sprint +2):

- 0 comentarios hostiles en PRs [OK]
- Aceptaron tarea conjunta en Sprint 16 [OK]
- Respeto: 6.5/10 [OK]

L.4 4 Intervención: Meeting Hell (Smell Organizacional)

=== INTERVENCIÓN: MEETING HELL ===

Equipo: Delta (8 personas)

Problema: Promedio de 25h/semana en reuniones por persona

DIAGNÓSTICO:

- Auditoría de calendarios muestra:
 - 8h/semana: reuniones de Scrum (normal)
 - 10h/semana: reuniones con partes interesadas (\textit{stakeholders}) (excesivo)
 - 7h/semana: reuniones "informativas" donde equipo es pasivo
- Foco del equipo: 4.2/10 (crítico)
- Velocity cayó 30% en últimos 3 Sprints

ESTRATEGIA (5 SSP):

Semana 1 - Documentación:

- Crear spreadsheet: reunión, convocador, frecuencia, asistentes
- Calcular costo: horas $\times$ personas $\times$ tarifa hora $\approx$ $\$X$ /semana

Semana 2 - Propuesta:

- Clasificar reuniones: eliminar, reducir frecuencia, o delegar
- Propuesta: representante único para reuniones informativas
- Establecer "Focus Fridays": sin reuniones externas

Semana 3 - Negociación:

- Reunión con `\textit{stakeholders}` clave (SM lidera)
- Presentar datos de impacto en velocity
- Acordar: representante rota, resumen escrito después

Semana 4 - Implementación:

- Focus Fridays en efecto
- Representante único para 5 reuniones
- Monitorear calendarios

MÉTRICAS DE ÉXITO:

- Horas en reuniones: de 25h a 15h/semana
- Foco: de 4.2 a 6.0
- Velocity: recuperar al menos 50% de la caída

RESULTADO (Sprint +2):

- Reuniones: 16h/semana (36% reducción) [OK]
- Foco: 5.8/10 (mejora significativa) [OK]
- Velocity: +20% (recuperación parcial, continuar)

L.5 5 Experimento: Reducir Knowledge Islands

=== EXPERIMENTO: REDUCIR KNOWLEDGE ISLANDS ===

CONTEXTO:

Equipo Zeta tiene 3 subgrupos: Frontend, Backend, Data.

Casi no hay comunicación entre subgrupos.

Integración al final de Sprint genera conflictos.

Health Score "Compromiso": 5.5/10

HIPÓTESIS:

"Si implementamos code reviews cruzados obligatorios (1 PR/semana revisado por miembro de otro subgrupo), entonces el conocimiento compartido aumentará y los conflictos de integración disminuirán, porque la exposición temprana a código de otros reduce sorpresas y builds empathy."

MÉTRICAS:

Métrica	Linea base	Target
PRs con reviewer cruzado	0/semana	5/semana
Conflictos de integración	4/Sprint	2/Sprint
"Entiendo código otros"	3/10	5/10
Health Score Compromiso	5.5	6.5

INTERVENCIÓN:

Semana 1:

- Workshop: "Por qué code reviews cruzados" (1h)
- Configurar GitHub: reviewer aleatorio de otro subgrupo
- Norma: al menos 1 PR cruzado por persona/semana

Semanas 2-4:

- Monitorear cumplimiento
- 1:1 con personas que no participan
- Celebrar en Daily cuando review cruzado genera insight

DURACIÓN: 2 Sprints

CRITERIOS DE EVALUACIÓN:

- Éxito: Conflictos ≤ 2 , "Entiendo código" ≥ 5
- Parcial: Conflictos ≤ 3 , alguna mejora en percepción
- Fracaso: Sin mejora o resistencia activa del equipo

RESULTADOS (Sprint +2):

- PRs cruzados: 6/semana [OK] (superó target)
- Conflictos: 3/Sprint (mejora, no alcanzó target)

- "Entiendo código": 4.5/10 (mejora parcial)
- Compromiso: 6.0/10 (mejora)

EVALUACIÓN: Éxito parcial

APRENDIZAJES:

1. Reviews cruzados toman más tiempo (factor 1.5x)
2. Backend → Frontend funciona mejor que inverso
3. Data team necesita onboarding adicional para participar

DECISIÓN: Iterar

- Reducir a 1 PR cruzado cada 2 semanas (más sostenible)
- Sesión de 1h para Data team sobre arquitectura general
- Re-evaluar en Sprint 20

ANEXO M: CONTEXTUALIZACIÓN DEL MODELO SOCIALSCRUM

Este documento presenta una síntesis autocontenida del modelo SocialScrum, un proceso ágil que extiende Scrum para gestionar la **deuda social** en equipos de desarrollo de software. Está dirigido a expertos evaluadores, por lo que todos los términos se definen en el propio texto sin remitir a glosarios externos.

M.1 Conceptos clave

Deuda social. Acumulación de efectos adversos derivados de decisiones socio-técnicas subóptimas (comunicación deficiente, distribución desigual del conocimiento, conflictos no abordados) que impactan la productividad, la calidad del software y la cohesión del equipo.

Community smells (olores comunitarios). Antipatrones socio-técnicos observables que actúan como síntomas de deuda social. Ejemplos: *Radio Silence* (silencio comunicativo entre subgrupos), *Lone Wolf* (desarrollador que trabaja aislado ignorando al equipo), *Organisational Silo* (equipos que operan sin intercambio de información entre áreas).

Health Score (puntaje de salud social). Indicador agregado en escala 1–10 que evalúa cinco dimensiones de la salud del equipo, alineadas con los valores de Scrum: Apertura, Compromiso, Respeto, Foco y Coraje. Se calcula como el promedio de las cinco dimensiones, cada una medida mediante encuestas anónimas.

Social Guide (Guía Social). Nuevo rol introducido por SocialScrum. Persona con perfil híbrido (psicología organizacional + conocimiento técnico de desarrollo de software) que facilita la identificación, análisis y mitigación de la deuda social, sin ejercer autoridad jerárquica. No reemplaza al Scrum Master: este facilita el proceso ágil; el Social Guide vela por la salud social del equipo.

Social Story Points (SSP). Unidad de estimación del esfuerzo requerido para abordar un ítem de deuda social. Utiliza la misma escala Fibonacci que los Story Points

técnicos (1, 2, 3, 5, 8, 13; valores superiores a 13 indican que el ítem debe descomponerse) y se define con la equivalencia **1 SSP** $\approx$ **1 SP**, permitiendo sumar ítems técnicos y sociales en un único Sprint Backlog para calcular la capacidad total del Sprint. La estimación se realiza en tres pasos: (1) identificar el community smell específico, (2) evaluar su severidad —Leve (1–2 SSP base), Moderada (3–5), Severa (5–8) o Crítica (8–13)— y (3) ajustar por complejidad de intervención (+0 baja, +1 media, +2 alta). Por ejemplo, un conflicto interpersonal moderado (3 SSP base) que requiere mediación especializada (complejidad alta, +2) se estima en 5 SSP. Todo el equipo participa en la estimación durante el Social Sprint Planning, tal como ocurre con los Story Points técnicos; el Social Guide aporta contexto sobre la severidad y el tipo de intervención requerido, pero no impone la cifra.

SNA (Social Network Analysis — Análisis de Redes Sociales). Método para analizar relaciones y patrones de interacción dentro del equipo (e.g., quién se comunica con quién, cuellos de botella de información), usando metadatos de herramientas como Slack o GitHub.

Seguridad psicológica. Percepción compartida de que el equipo es un espacio seguro para expresar opiniones, errores o preocupaciones sin temor a represalias.

Burnout (agotamiento profesional). Síndrome de estrés laboral crónico caracterizado por agotamiento emocional, cinismo y sensación de ineficacia profesional; consecuencia frecuente de la deuda social acumulada.

M.2 Fundamentos teóricos de SocialScrum (síntesis del Capítulo 3)

SocialScrum se sustenta en tres marcos teóricos complementarios que abordan la motivación individual, las relaciones de confianza y los valores del equipo.

M.2.1 Teoría de la Autodeterminación (TAD)

Propuesta por Ryan y Deci, establece que la motivación intrínseca sostenible depende de la satisfacción de tres necesidades psicológicas básicas:

1. **Autonomía:** sentirse autor de las propias acciones y decisiones, no un mero ejecutor de órdenes externas.
2. **Competencia:** percibir que el esfuerzo propio genera progreso y resultados con sentido.

3. **Relacionalidad:** sentirse conectado con los demás, experimentar pertenencia y vínculos significativos dentro del grupo.

Cuando estas necesidades se frustran, emergen el agotamiento emocional (falta de autonomía), la sensación de ineficacia (falta de competencia) y el cinismo o aislamiento (falta de relacionalidad), configurando el burnout.

M.2.2 Modelo de confianza de Mayer, Davis y Schoorman

Define la confianza como la disposición a ser vulnerable frente a las acciones de otra persona, basada en tres dimensiones:

1. **Habilidad:** percepción de que el otro posee las competencias técnicas necesarias para cumplir con su rol.
2. **Benevolencia:** creencia de que el otro actúa con intención genuina de bienestar hacia el equipo, no solo por interés propio.
3. **Integridad:** percepción de coherencia entre lo que la persona dice y lo que hace; cumplimiento de compromisos.

La confianza genuina (no la cooperación forzada) es requisito para que los equipos aborden abiertamente sus community smells.

M.2.3 Valores de Scrum aplicados a la deuda social

SocialScrum reinterpreta los cinco valores de Scrum como mecanismos de protección frente a la deuda social:

- **Coraje:** base emocional para conversaciones difíciles y para mostrar vulnerabilidad.
- **Apertura:** transparencia sobre problemas sociales; hacer visible lo que normalmente se oculta.
- **Respeto:** antídoto frente a relaciones tóxicas; garantiza seguridad psicológica.
- **Compromiso:** responsabilidad compartida sobre el bienestar colectivo, no solo sobre entregables técnicos.
- **Foco:** equilibrio entre la entrega del producto y la atención a la salud del equipo.

M.2.4 Principios empíricos aplicados al ámbito social

SocialScrum aplica los tres pilares empíricos de Scrum al plano social:

- **Transparencia:** transformar las dinámicas sociales difusas en objetos observables y gestionables, mediante artefactos explícitos, métricas sociales y espacios seguros de revelación.
- **Inspección:** monitoreo continuo en múltiples cadencias (diaria, por Sprint, por Release y continua) porque los community smells evolucionan, escalan e interactúan entre sí.
- **Adaptación:** experimentación controlada, pequeña y reversible para mitigar la deuda social; cada intervención se formula como hipótesis con métricas de éxito verificables.

M.3 Modelo conceptual de SocialScrum

El modelo se estructura en tres componentes que se integran al marco Scrum: roles especializados, eventos adaptados y artefactos enriquecidos con dimensión social. Estos actúan coordinadamente para identificar, analizar y gestionar la deuda social.

M.3.1 Roles

- **Product Owner:** mantiene su rol de maximizar valor del producto. Recibe del Social Guide una evaluación del estado social del equipo antes de cada Sprint Planning. Si el Health Score es crítico, prioriza acciones de mitigación de deuda social por encima de features de alta complejidad.
- **Scrum Master:** facilita el proceso ágil y colabora con el Social Guide para identificar impedimentos sociales. No gestiona la deuda social directamente.
- **Developers:** asumen un rol activo en la detección de community smells y proponen acciones de mejora que se integran al Social Sprint Backlog.
- **Social Guide:** rol central de SocialScrum. Detecta community smells mediante herramientas como SNA, facilita estrategias de mitigación, educa sobre deuda social y colabora con Scrum Master y Product Owner. No evalúa desempeño individual, no sanciona, no toma decisiones sobre el producto.

M.3.2 Eventos adaptados

SocialScrum no añade nuevos eventos; adapta los existentes incorporando una dimensión social:

- **Social Sprint Planning:** se añade una revisión de riesgos sociales (15 min) donde el equipo analiza si su estado social le permite comprometerse con tareas de alta complejidad.
- **Social Daily Standup:** se añade una pregunta social breve (2–3 min): “¿Hay algún impedimento social?”. El Social Guide observa dinámicas (participación, tono, silencios) como señales tempranas.
- **Social Sprint Review:** se dedican 5–10 min a presentar la evolución del Health Score y los community smells abordados ante stakeholders (partes interesadas), siempre con datos agregados y anónimos.
- **Social Sprint Retrospective:** núcleo de la gestión social. Se reestructura en cuatro fases (preparación del espacio, recolección de datos, análisis de patrones y generación de compromisos) durante 90 min. No requiere tiempo adicional; redistribuye el foco de 90 % técnico a 50 %–50 % técnico-social.

Sobrecarga total estimada: ~45 min por Sprint de dos semanas (<1 % del tiempo total).

M.3.3 Artefactos sociales

Además de extender Product Backlog, Sprint Backlog e Increment con dimensión social, SocialScrum introduce cuatro artefactos complementarios:

1. **Social Product Backlog:** inventario priorizado de todos los ítems de deuda social identificados. Cada ítem incluye: community smell asociado, severidad (Alta/Media/Baja), dimensión del Health Score afectada, estimación en SSP y criterios de aceptación.
2. **Social Sprint Backlog:** ítems del Social Product Backlog comprometidos para el Sprint actual (típicamente 1–3 ítems). Se recomienda integrarlos en el mismo tablero que los ítems técnicos (modelo integrado).
3. **Registro de Community Smells:** historial centralizado de smells detectados, con fecha, severidad, estado, acciones tomadas y resultados. Transparente para todo el equipo.
4. **Dashboard de Métricas Sociales:** visualización del Health Score, distribución de conocimiento (SNA), señales de burnout y participación en decisiones.

M.4 Operacionalización de SocialScrum (síntesis del Capítulo 4)

M.4.1 Adopción inicial

La implementación de SocialScrum requiere verificar cuatro pre-requisitos organizacionales mínimos:

1. **Compromiso ejecutivo genuino:** el liderazgo técnico (CTO, VP de Ingeniería o equivalente) respalda el modelo con recursos, presupuesto y aceptación de reducción temporal (10–20 %) de la capacidad técnica.
2. **Cultura organizacional mínimamente saludable:** no existe desconfianza institucionalizada, abuso jerárquico normalizado, rotación extrema (>30 % anual) ni comunicación totalmente fracturada.
3. **Equipo con autonomía:** el equipo puede priorizar ítems sociales, ajustar su capacidad según el Health Score y decidir cómo abordar los community smells sin autorización externa.
4. **Social Guide calificado disponible:** perfil híbrido con formación en dinámicas de grupo/psicología organizacional y comprensión del contexto técnico de desarrollo de software.

Si estas condiciones no se cumplen, SocialScrum no debe implementarse: el modelo no arregla culturas disfuncionales, las requiere como base.

M.4.1.1 Sprint 0 (preparación)

Cuando los pre-requisitos están validados, se dedica un Sprint preparatorio (1–2 semanas) con cinco actividades:

1. Capacitación en fundamentos de SocialScrum (4 h).
2. Establecimiento de la línea base del Health Score (2 h): encuesta anónima sobre los cinco valores de Scrum, entrevistas 1:1 opcionales e identificación preliminar de community smells.
3. Firma de acuerdos de consentimiento informado (1 h): confidencialidad del Social Guide y consentimiento del equipo. Si <70 % del equipo firma, SocialScrum no se implementa.
4. Configuración de herramientas y artefactos (3 h): Social Product Backlog, Dashboard de Health Score, Registro de community smells.

5. Retrospectiva social simulada (2 h): práctica del protocolo sin presión de Sprint real.

M.4.2 El Social Guide: perfil y gobernanza

El Social Guide requiere:

- **Habilidades duras:** análisis SNA, medición cualitativa, facilitación de grupos, mediación de conflictos (Comunicación No Violenta), dominio de Scrum.
- **Habilidades blandas:** neutralidad emocional, empatía calibrada (conectar sin validar conductas destructivas), coraje profesional, integridad ética, humildad intelectual.

Su selección implica cinco fases: identificación de candidatos, evaluación técnica, entrevista ética, decisión consensuada (Scrum Master + Development Team + CTO) y período de prueba (3 Sprints; requiere aprobación $\geq 70\%$ del equipo).

El Social Guide reporta al Scrum Master o al liderazgo técnico, **nunca** a Recursos Humanos, para evitar conflictos de interés con evaluaciones de desempeño.

M.4.3 Sistema de medición: Health Score

M.4.3.1 Cálculo

$$\text{Health Score} = \frac{\text{Apertura} + \text{Compromiso} + \text{Respeto} + \text{Foco} + \text{Coraje}}{5}$$

Cada dimensión se mide en escala 1–10 mediante dos instrumentos:

- **Pulso semanal:** 5 preguntas (1 por dimensión), 2–3 min, tasa de respuesta mínima 70 %.
- **Encuesta de Sprint completa:** 20 preguntas + 3 abiertas, 5–8 min, tasa mínima 80 %.

M.4.3.2 Zonas de interpretación y respuesta

- **7.0–10.0 (Saludable):** mantenimiento preventivo, 5–10 % de capacidad social.
- **5.0–6.9 (En riesgo):** atención activa, 15–20 % de capacidad social, revisión semanal.
- **1.0–4.9 (Crítico):** intervención inmediata, 30–50 % de capacidad social, escalar a CTO, reducir alcance técnico.

M.4.3.3 Alertas tempranas

- Caída >1.5 puntos en una semana: investigar inmediatamente.
- 3 semanas consecutivas de caída: activar protocolo “En riesgo”.
- Cualquier dimensión <4.0 : priorizar esa dimensión.
- $<50\%$ de respuesta en encuestas por 2 semanas: revisar el proceso.

M.4.4 Framework de priorización social-técnica

SocialScrum introduce un “Triángulo de Decisión” con tres dimensiones evaluadas en escala 1–10:

1. **Valor de negocio** (responsable: Product Owner): beneficio para el cliente u organización.
2. **Urgencia técnica** (responsable: Development Team): necesidad desde la perspectiva de arquitectura o calidad.
3. **Salud social** (responsable: Social Guide): necesidad de restaurar la capacidad del equipo.

No se usa fórmula matemática; se estructura una conversación entre los responsables de cada vértice. Tres reglas de oro para casos extremos:

1. Health Score $< 5 \rightarrow$ no comprometer features de alta complejidad.
2. Valor de negocio 10 + Salud social 10 $\rightarrow$ escalar decisión al CTO.
3. Health Score $\geq 8 \rightarrow$ priorizar por valor + urgencia técnica (SocialScrum se comporta como Scrum estándar).

M.4.5 Sistema de estimación Social Story Points (SSP)

Los SSP permiten que el esfuerzo social sea visible y planificable en el Sprint, al igual que los Story Points técnicos. La escala, el proceso y las prácticas se resumen a continuación.

M.4.5.1 Escala y rangos de referencia

SSP	Duración típica	Tipo de intervención	Ejemplo
1–2	1–4 h	Conversación facilitada, ajuste menor	Feedback rápido, norma si
3–5	4 h–2 días	Workshop, mediación, retrospectiva profunda	Código de conducta, media
8–13	2 días–1 semana	Intervención organizacional, crisis	Burnout severo, cambio lic

M.4.5.2 Proceso de estimación

Se aplican tres pasos: (1) **Identificar el community smell** a partir del Registro de Community Smells; (2) **evaluar severidad** según el Health Score y la observación directa (Leve 1–2 SSP base, Moderada 3–5, Severa 5–8, Crítica 8–13); y (3) **evaluar complejidad de intervención** según alcance, facilitación necesaria y número de sesiones (Baja +0, Media +1, Alta +2). El SSP final se calcula como:

$$\text{SSP Final} = \text{SSP Base (severidad)} + \text{Ajuste (complejidad)}$$

M.4.5.3 Integración con Story Points técnicos

Dado que $1 \text{ SSP} \approx 1 \text{ SP}$, ambos tipos de ítems se suman para calcular la capacidad total del Sprint. Por ejemplo: si el equipo tiene velocidad de 50 puntos y asigna 20 % a deuda social, dispone de 40 SP técnicos + 10 SSP sociales. Esta integración permite que el Product Owner y el Social Guide negocien la distribución de capacidad de forma transparente durante el Social Sprint Planning, usando la misma unidad de medida.

Mejores prácticas: estimar conservadoramente (añadir 20 % de buffer en situaciones inciertas), descomponer ítems que superen 8 SSP, registrar el esfuerzo real vs. estimado para calibrar la escala, y asegurar que cada ítem corresponda a un community smell específico (evitar ítems vagos como “mejorar comunicación”).

M.4.6 Salvaguardas éticas

Las salvaguardas éticas son requisitos indispensables, no recomendaciones:

- **Confidencialidad absoluta:** ningún dato de eventos sociales puede usarse para evaluaciones de desempeño ni decisiones de empleo. Se comparten solo datos agregados y anónimos.
- **Consentimiento informado:** cada miembro entiende qué información se recopila, para qué, quién tiene acceso y cuáles son sus derechos. Umbral mínimo: 70 % del equipo.
- **Anonimización:** reportes a nivel organizacional nunca permiten identificar individuos.
- **Derecho de auditoría del equipo:** el equipo revisa cualquier reporte antes de su difusión externa y puede bloquear su publicación si vulnera las salvaguardas.
- **Prevención del “teatro social”:** garantizar que reconocer problemas no genera consecuencias negativas; si las salvaguardas fallan, el equipo fingirá bienestar.

M.5 Simulación de implementación: caso del equipo Phoenix

Para ilustrar la aplicación práctica, SocialScrum se simuló durante 3 Sprints en un equipo ficticio (basado en patrones reales):

Contexto: equipo de 7 personas, modalidad híbrida, 18 meses con Scrum estándar. Síntomas: velocidad inconsistente, conflicto no resuelto entre dos desarrolladores, concentración de conocimiento en una persona (60 % del código crítico), retrospectivas “ceremoniales”.

Diagnóstico inicial: Health Score = 5.2/10 (zona “en riesgo”). Community smells activos: Radio Silence (alta severidad), Organisational Silo (media), Lone Wolf (media).

Evolución en 3 Sprints:

Métrica	Inicial	Sprint 1	Sprint 2	Sprint 3
Health Score	5.2	5.8	6.4	7.1
Velocity (SP)	38	42	45	48
Smells activos	3	3	2	1
Capacidad social (%)	0	16	20	12

Intervenciones clave: sesión de alineación entre desarrolladores en conflicto, pair programming rotativo, transferencia de conocimiento de la desarrolladora senior al equipo (bus factor pasó de 1 a 3), creación de canal de comunicación social, acuerdo de feedback con reglas explícitas, ritual de sincronización semanal entre áreas Frontend y Backend.

Resultado: mejora del Health Score de +36 %, aumento de velocity de +26 %, reducción de smells activos de 3 a 1. Aunque la correlación no prueba causalidad, la consistencia de ambas tendencias sugiere que abordar la deuda social beneficia tanto el bienestar del equipo como su productividad.

M.6 Limitaciones reconocidas

- SocialScrum no ha sido validado empíricamente en múltiples organizaciones.
- No funciona en entornos con toxicidad organizacional severa (desconfianza sistémica, abuso jerárquico normalizado).
- La escalabilidad no es lineal: gestionar la deuda social de múltiples equipos requiere construir cultura, no solo contratar Social Guides.
- La sobrecarga de 15–20 % de capacidad puede ser prohibitiva en equipos bajo presión extrema de entrega.

- El perfil híbrido del Social Guide (psicología + técnica) es escaso en el mercado laboral actual.
- Tensiones inherentes (Product Owner vs. Social Guide; supervisión del Social Guide por quien puede ser parte del problema) no se eliminan, solo se hacen visibles.

BIBLIOGRAFÍA

- [1] D. E. Muir y E. A. Weinstein, «The social debt: An investigation of lower-class and middle-class norms of social obligation,» *American Sociological Review*, vol. 27, págs. 532-539, 1962, Accedido el 24 de septiembre de 2024. DOI: 10.2307/2090036 dirección: <https://doi.org/10.2307/2090036>
- [2] E. A. Caballero Espinosa, J. C. Carver y K. Stowers, «Community smells—The sources of social debt: A systematic literature review,» *Information and Software Technology*, vol. 153, pág. 107078, sep. de 2022, Accedido el 24 de septiembre de 2024. DOI: 10.1016/j.infsof.2022.107078 dirección: <https://doi.org/10.1016/j.infsof.2022.107078>
- [3] D. A. Tamburri, P. Kruchten, P. Lago y H. van Vliet, «What is social debt in software engineering?» En *2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*, Accedido el 24 de septiembre de 2024, San Francisco, CA, USA, 2013, págs. 93-96. DOI: 10.1109/chase.2013.6614739 dirección: <https://doi.org/10.1109/chase.2013.6614739>
- [4] E. A. Caballero Espinosa, «Understanding Social Debt in Software Engineering,» Tesis doctoral. Accedido el 24 de septiembre de 2024, Tesis doct., Universidad de Alabama, Tuscaloosa, Alabama, EE. UU, 2021. dirección: <http://ir.ua.edu/handle/123456789/8278>
- [5] T. Dreesen, P. Hennel, C. Rosenkranz y T. Kude, «The second vice is lying, the first is running into debt. Antecedents and mitigating practices of social debt: An exploratory study in distributed software development teams,» en *Hawaii International Conference on System Sciences*, Accedido el 24 de septiembre de 2024, 2021, págs. 6826-6835. DOI: 10.24251/hicss.2021.818 dirección: <https://doi.org/10.24251/hicss.2021.818>
- [6] B. S. Bloom, *Taxonomy of Educational Objectives: The Classification of Educational Goals, Parts 1-2*. Handbook I: Cognitive Domain, 1956, pág. 223.
- [7] R. L. Baskerville, «Investigating information systems with action research,» *Communications of the Association for Information Systems*, vol. 2, n.º 19, 1999, Accedido el 25 de septiembre de 2024. DOI: 10.17705/1cais.00219 dirección: <https://doi.org/10.17705/1cais.00219>

- [8] G. Adillah, A. Ridwan y W. Rahayu, «Content Validation through Expert Judgment of an Instrument on the Self-Assessment of Mathematics Education Student Competency,» *International Journal of Multicultural and Multireligious Understanding*, vol. 9, n.º 3, 2022, ISSN: 2364-5369. DOI: 10.18415/ijmmu.v9i3.3738 dirección: <https://ijmmu.com/index.php/ijmmu/article/view/3738>
- [9] D. A. Quintana Guzmán, «Método para definir procesos en organizaciones desarrolladoras de software,» Trabajo de Grado, Facultad de Ingeniería Electrónica y Telecomunicaciones, Departamento de Ingeniería de Sistemas, Grupo IDIS – Investigación y Desarrollo en Ingeniería de Software, Popayán, Colombia, mayo 2017, Tesis de mtría., Universidad del Cauca, 2017.
- [10] D. A. Tamburri, P. Kruchten, P. Lago y H. van Vliet, «Social debt in software engineering: Insights from industry,» *Journal of Internet Services and Applications*, vol. 6, n.º 1, pág. 10, mayo de 2015, Accedido el 24 de septiembre de 2024. DOI: 10.1186/s13174-015-0024-6 dirección: <https://doi.org/10.1186/s13174-015-0024-6>
- [11] A. Martini, V. Stray y N. B. Moe, «Technical-, social- and process debt in large-scale agile: An exploratory case-study,» en *Agile Processes in Software Engineering and Extreme Programming – Workshops*, M. Morciano y T. Stalhane, eds., Accedido el 24 de septiembre de 2024, Cham: Springer International Publishing, 2019, págs. 112-119. DOI: 10.1007/978-3-030-30126-2_14 dirección: https://doi.org/10.1007/978-3-030-30126-2_14
- [12] D. A. Tamburri, «Software architecture social debt: Managing the incommunicability factor,» *IEEE Transactions on Computational Social Systems*, vol. 6, n.º 1, págs. 20-37, 2019.
- [13] T. R. Tulili, A. Capiluppi y A. Rastogi, «Burnout in software engineering: A systematic mapping study,» *Information and Software Technology*, vol. 155, pág. 107116, nov. de 2022, Accedido el 24 de septiembre de 2024. DOI: 10.1016/j.infsof.2022.107116 dirección: <https://doi.org/10.1016/j.infsof.2022.107116>
- [14] K. Schwaber y J. Sutherland, *La guía de Scrum: Las Reglas del Juego*. 2020, Accedido el 4 de agosto de 2025. dirección: <https://repositorio.uvm.edu.ve/handle/123456789/59>
- [15] H. Takeuchi e I. Nonaka, «The New New Product Development Game,» *Harvard Business Review*, vol. 64, n.º 1, págs. 137-146, 1986, Accedido el 4 de agosto de 2025. dirección: <https://hbr.org/1986/01/the-new-new-product-development-game>

- [16] S. Jalali y C. Wohlin, «Systematic literature studies: database searches vs. backward snowballing,» en *Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, Lund, Sweden, 2012, págs. 29-39. DOI: 10.1145/2372251.2372257 dirección: <https://doi.org/10.1145/2372251.2372257>
- [17] F. Palomba, D. A. Tamburri, F. Arcelli Fontana, R. Oliveto, A. Zaidman y A. Serebrenik, «Beyond technical aspects: How do community smells influence the intensity of code smells?» *IEEE Transactions on Software Engineering*, vol. 47, n.º 1, págs. 108-129, ene. de 2021. DOI: 10.1109/tse.2018.2883603 dirección: <https://doi.org/10.1109/tse.2018.2883603>
- [18] V. R. B. Caldiera y H. D. Rombach, «The goal question metric approach,» *Encyclopedia of Software Engineering*, págs. 528-532, 1994.
- [19] G. T. Doran, «There's a S.M.A.R.T. Way to Write Management's Goals and Objectives,» *Management Review*, vol. 70, págs. 35-36, 1981.
- [20] OpenAI. «ChatGPT.» Accedido el 24 de septiembre de 2024. dirección: <https://chat.openai.com/chat>
- [21] Microsoft. «Microsoft Copilot.» Accedido el 24 de septiembre de 2024. dirección: <https://www.microsoft.com/copilot>
- [22] H. Saeeda, M. O. Ahamd y T. Gustavsson, «A multivocal literature review on non-technical debt in software development: an insight into process, social, people, organizational, and culture debt,» *e-Informatica Software Engineering Journal*, vol. 18, n.º 1, pág. 240 101, 2024, Accedido el 24 de septiembre de 2024. DOI: 10.37190/e-inf240101 dirección: <https://doi.org/10.37190/e-inf240101>
- [23] R. M. Ryan y E. L. Deci, «Self-Determination Theory and the Facilitation of Intrinsic Motivation, Social Development, and Well-Being,» *American Psychologist*, vol. 55, n.º 1, págs. 68-78, 2000. DOI: 10.1037/0003-066X.55.1.68 dirección: <https://www.simplypsychology.org/self-determination-theory.html>
- [24] R. C. Mayer, J. H. Davis y F. D. Schoorman, «An Integrative Model of Organizational Trust,» *Academy of Management Review*, vol. 20, n.º 3, págs. 709-734, 1995. DOI: 10.2307/258792 dirección: <https://journals.aom.org/doi/abs/10.5465/amr.1995.9508080335>
- [25] J. M. Wilson, S. G. Straus y B. McEvily, «All in Due Time: The Development of Trust in Computer-Mediated and Face-to-Face Teams,» *Organizational Behavior and Human Decision Processes*, vol. 99, n.º 1, págs. 16-33, 2006. DOI: 10.1016/j.obhdp.2005.08.001 dirección: <https://doi.org/10.1016/j.obhdp.2005.08.001>
- [26] A. Bandura, *Self-Efficacy: The Exercise of Control*. New York: W.H. Freeman, 1997, pág. 604, ISBN: 978-0-7167-2850-4.

- [27] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. New York: Harper & Row, 1990, pág. 320, ISBN: 978-0-06-016253-5.
- [28] V. H. Vroom, *Work and Motivation*. New York: John Wiley & Sons, 1964, ISBN: 978-0471963078.
- [29] L. P. Robert, S. Kim, A. R. Dennis e Y.-T. C. Hung, «Individual Swift Trust and Knowledge-Based Trust in Face-to-Face and Virtual Team Members,» *Journal of Management Information Systems*, vol. 26, n.º 2, págs. 241-279, 2009. DOI: 10 . 2753 /MIS0742 - 1222260209 dirección: <https://doi.org/10.2753/MIS0742-1222260209>
- [30] A. Bandura, *Self-Efficacy: The Exercise of Control*. New York: W.H. Freeman y Company, 1997, Accedido el 6 de agosto de 2025, ISBN: 0716728508. dirección: <https://www.worldcat.org/title/self-efficacy-the-exercise-of-control/oclc/36074515>
- [31] R. C. Mayer, J. H. Davis y F. D. Schoorman, «An Integrative Model of Organizational Trust,» *Academy of Management Review*, vol. 20, n.º 3, págs. 709-734, 1995. DOI: 10.5465/amr.1995.9508080335
- [32] A. C. Edmondson, «Psychological Safety and Learning Behavior in Work Teams,» *Administrative Science Quarterly*, vol. 44, n.º 2, págs. 350-383, 1999. DOI: 10.2307/2667054
- [33] R. M. Ryan y E. L. Deci, «Self-Determination Theory and the Facilitation of Intrinsic Motivation, Social Development, and Well-Being,» *American Psychologist*, vol. 55, n.º 1, págs. 68-78, 2000. DOI: 10.1037/0003-066X.55.1.68
- [34] A. C. Edmondson, *The Fearless Organization: Creating Psychological Safety in the Workplace for Learning, Innovation, and Growth*. John Wiley & Sons, 2018, ISBN: 9781119477242.
- [35] W. Ortega y C. Pardo, «Master Thesis Supervision: Framework para soportar la evaluación de la agilidad de los procesos software de las organizaciones,» Tesis doct., Universidad del Cauca, nov. de 2019. DOI: 10.13140/RG.2.2.30624.20487
- [36] J. M. Wilson, S. G. Straus y B. McEvily, «All in Due Time: The Development of Trust in Computer-Mediated and Face-to-Face Teams,» *Organizational Behavior and Human Decision Processes*, vol. 99, n.º 1, págs. 16-33, 2006. DOI: 10.1016/j.obhdp.2005.08.001
- [37] J. Noll, «Motivation and Autonomy in Global Software Development,» Tesis doct., University of Glasgow, 2020.

- [38] N. Almarimi, A. Ouni, M. Chouchen y M. W. Mkaouer, «csDetector: an open source tool for community smells detection,» en *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ép. ESEC/FSE 2021, Athens, Greece: Association for Computing Machinery, 2021, págs. 1560-1564, ISBN: 9781450385626. DOI: 10.1145/3468264.3473121 dirección: <https://doi.org/10.1145/3468264.3473121>

ANEXO N: ACRÓNIMOS

CTO Chief Technology Officer (Director de Tecnología). 60

KPI Indicadores Clave de Desempeño (Key Performance Indicators). 28

RSL Revisión Sistemática de la Literatura. 11

TAD Teoría de la Autodeterminación. 27, 28, 30, 33, 35, 41, 51, 54, 56, 193

VP Vicepresidente. 60

ANEXO Ñ: GLOSARIO

5 Whys (5 Porqués)

Técnica iterativa de análisis de causa raíz que consiste en preguntar “¿por qué?” de forma sucesiva (generalmente cinco veces) ante un problema, con el fin de identificar la causa fundamental que lo origina. Es ampliamente utilizada en metodologías ágiles y Lean para abordar problemas de forma estructurada, evitando soluciones superficiales o paliativas.. 22, 197

Absent Stakeholder (Parte Interesada Ausente)

Community smell que ocurre cuando las partes interesadas clave (stakeholders) no participan activamente en los procesos del equipo, lo que genera falta de dirección, decisiones sin contexto de negocio, retrabajo y desalineación entre las expectativas del producto y el trabajo del equipo.. 36

Análisis de Redes Sociales (SNA, Social Network Analysis)

Método para analizar las relaciones y estructuras sociales dentro de un equipo u organización, identificando patrones de interacción, comunicación y colaboración. Facilita la detección de cuellos de botella, silos y oportunidades para mejorar la cohesión social.. 18, 19, 25, 26, 145

Black Cloud (Nube Negra)

Ausencia de boundary spanners (conectores entre grupos) y protocolos formales de intercambio de información genera desconfianza e información ofuscada. Impide el intercambio efectivo de conocimiento, experiencia profesional y contexto de proyecto entre miembros del equipo. Resulta en iniciativas no sancionadas, comportamiento egotista y fragmentación.. 25, 36, 145

Blame Culture (Cultura de la Culpa)

Community smell que se caracteriza por un ambiente donde los errores se atribuyen a individuos específicos en lugar de abordarse como oportunidades de aprendizaje colectivo. Esta cultura genera miedo a asumir riesgos, inhibe la innovación y deteriora la confianza dentro del equipo.. 83

Bottleneck (Cuello de Botella)

Toda comunicación transita por nodos saturados (managers, arquitectos), causando compresión severa y retrasos comunicacionales.. 8, 36

Broadcast Storm (Tormenta de Difusión)

Exceso de comunicación no estructurada y sin filtros adecuados entre miembros del equipo. Genera saturación informacional, pérdida de mensajes críticos, distracciones constantes y disminución de la productividad.. 210, 212

Budget Constraints (Restricciones Presupuestarias)

Community smell organizacional que surge cuando las restricciones presupuestarias impiden al equipo acceder a los recursos necesarios —herramientas, capacitación, personal— para gestionar adecuadamente su deuda social y técnica, generando frustración y deterioro progresivo.. 217

Burnout (Síndrome de Agotamiento)

Estado de agotamiento físico, emocional y mental causado por estrés prolongado y excesivo en el entorno laboral. Se caracteriza por sentimientos de fatiga, cinismo, despersonalización y disminución de la eficacia profesional. En equipos de desarrollo de software, el burnout puede afectar negativamente la productividad, la calidad del trabajo y la salud general del equipo.. 25, 26

Clique Formation (Formación de Cliques)

Community smell que se manifiesta cuando se forman subgrupos cerrados dentro del equipo que excluyen a otros miembros, limitando el flujo de información, generando dinámicas de favoritismo y erosionando la cohesión y la confianza del equipo en su conjunto.. 210, 215

Cliques (Cliques)

Community smell que se presenta cuando se forman grupos cerrados dentro del equipo que excluyen a otros miembros, limitando el flujo de información, generando dinámicas de favoritismo y erosionando la cohesión y la confianza del equipo en su conjunto.. 83

Cognitive Distance (Distancia Cognitiva)

Divergencia perceptible entre desarrolladores en niveles físico, técnico, social y cultural. Diferencias de experiencia generan “mundos de pensamiento” incompatibles que causan malentendidos, conflictos interpersonales, desperdicio de recursos, generación de code smells y erosión progresiva de confianza entre pares.. 36, 37

Community Smells (Olores Comunitarios)

Antipatrones socio-técnicos que representan síntomas de problemas organizacionales y sociales en comunidades de desarrollo de software. Son fuentes de deuda social que afectan la comunicación, coordinación, confianza y desempeño del equipo.. 8, 22, 23, 25, 26, 50

Context Switching (Cambio de Contexto)

Community smell que ocurre cuando los miembros del equipo deben cambiar frecuentemente entre tareas, proyectos o dominios de conocimiento, lo que genera pérdida de foco, aumento de errores y reducción de la eficiencia operativa.. 83

Decision Incommunicability (Incomunicabilidad de Decisiones)

Community smell que se manifiesta cuando las decisiones importantes —técnicas u organizacionales— no se comunican de manera adecuada al equipo, generando desinformación, duplicación de esfuerzos y erosión de la confianza en la transparencia del grupo.. 36

Definition of Done (Definición de Terminado)

Descripción formal del estado del Incremento cuando cumple las medidas de calidad requeridas para el producto. En SocialScrum, se amplía para incluir criterios de sostenibilidad social, evaluando si el trabajo se completó sin generar burnout, sin crear nuevos community smells y si mejoraron las dinámicas de colaboración.. 25

Deuda Social

Decisiones socio-técnicas subóptimas que generan costos no previstos en equipos de desarrollo de software. Acumulación de efectos adversos derivados de problemas organizacionales y sociales que impactan el desempeño del equipo y la calidad del software. 15

Disengagement (Desenganche)

Desarrolladores perciben el producto como suficientemente maduro y lo envían a producción prematuramente sin completar funcionalidades. Refleja baja curiosidad e involucramiento del equipo, supuestos salvajes sin validación, falta de contexto compartido y genera software con deficiencias funcionales.. 36, 82

Dismissiveness (Desdén)

Community smell caracterizado por la actitud de descarte o menosprecio hacia las ideas, opiniones o contribuciones de ciertos miembros del equipo. Esta conducta erosiona la seguridad psicológica, desincentiva la participación y genera resentimiento que se acumula como deuda social.. 36

Dissensus (Disenso)

Incapacidad persistente de alcanzar consenso pese a múltiples intentos deliberados de alcanzar acuerdos. Las decisiones técnicas se quedan sin resolución, code smells permanecen sin ajustes aplicados, hay paralización en procesos de toma de decisiones y acumulación de problemas pospuestos.. 36

Documentation Silo (Silo de Documentación)

Community smell que aparece cuando la documentación del proyecto se encuentra fragmentada, dispersa o inaccesible para ciertos miembros del equipo, lo que dificulta la transferencia de conocimiento, aumenta la dependencia de individuos clave y contribuye a la opacidad organizacional.. 36

Expert Bottleneck (Cuello de Botella Experto)

Community smell que surge cuando una sola persona concentra la responsabilidad de revisión, aprobación o validación técnica, generando cuellos de botella en el flujo de trabajo, retrasos en las entregas y dependencia crítica de un individuo.. 210, 214

Fear of Speaking Up (Miedo a Expresarse)

Community smell que se presenta cuando los miembros del equipo temen expresar sus opiniones, preocupaciones o ideas debido a posibles represalias, juicios negativos o exclusión social, lo que limita la comunicación abierta y la innovación colectiva.. 83

Feedback Loops (Ciclos de Retroalimentación)

Ciclos de retroalimentación continua donde la información sobre el desempeño y los resultados se utiliza para ajustar y mejorar procesos, comportamientos y decisiones en el equipo. Facilitan la adaptación y el aprendizaje organizacional.. 52

Free-Riding (Polizones)

Community smell que se manifiesta cuando algunos miembros del equipo contribuyen significativamente menos que sus pares, aprovechándose del esfuerzo colectivo sin asumir responsabilidades equitativas, lo que genera resentimiento, desmotivación y desequilibrios en la dinámica grupal.. 82

Health Score

Indicador compuesto que evalúa la salud social de un equipo de desarrollo mediante la medición de los cinco valores fundamentales de Scrum (Apertura, Compromiso, Respeto, Foco y Coraje). Cada valor se califica en una escala del 1 al 10, generando un puntaje promedio que representa la salud social general del equipo. Un puntaje

inferior a 4/10 indica una situación crítica que requiere intervención inmediata.. 21, 25, 26

Inefficacy (Ineficacia)

Community smell que se manifiesta cuando los miembros del equipo perciben que sus esfuerzos no generan resultados significativos o que su trabajo carece de impacto real. Esta sensación de ineficacia profesional erosiona la motivación intrínseca, la competencia percibida y contribuye al agotamiento emocional.. 37

Information Hoarding (Acaparamiento de Información)

Retención deliberada de información crítica del proyecto por parte de individuos o subgrupos. Impide flujo efectivo de conocimiento, genera silos informacionales, dificulta toma de decisiones informadas y erosiona confianza entre miembros del equipo.. 82, 210, 212

Information Island (Isla de Información)

Community smell que se produce cuando la información relevante queda aislada en personas, equipos o herramientas específicas, sin mecanismos efectivos para compartirla, lo que genera dependencias ocultas, retrasos y pérdida de contexto organizacional.. 36

Knowledge Islands (Islas de Conocimiento)

Community smell que se produce cuando subgrupos especializados dentro del equipo acumulan conocimiento exclusivo sobre componentes o dominios específicos, sin mecanismos de transferencia, generando dependencias ocultas e integraciones dolorosas.. 210, 214

Lack of Conflict Resolution (Falta de Resolución de Conflictos)

Community smell que surge cuando los conflictos interpersonales dentro de un equipo de desarrollo no se abordan ni resuelven de manera oportuna, lo que genera resentimiento acumulado, comunicación deteriorada y un ambiente de trabajo tóxico que alimenta la deuda social.. 36

Lone Wolf (Lobo Solitario)

Desarrollador que trabaja independientemente sin considerar genuinamente a sus pares e ignora sistemáticamente necesidades explícitas del proyecto. Toma decisiones no sancionadas, genera duplicación de trabajo, causa churn de código frecuente, retrasos de proyecto y reduce confianza del equipo.. 8, 30, 36, 82, 145, 210, 215

Lunch & Learn (Almuerzo y Aprendizaje)

Práctica informal de intercambio de conocimiento en la que los miembros de un equipo se reúnen durante el almuerzo o un espacio breve para compartir aprendizajes, presentar temas técnicos o discutir experiencias. En SocialScrum, se utiliza como estrategia de mitigación para reducir silos de conocimiento y fortalecer la distribución de competencias dentro del equipo.. 25

Meeting Hell (Infierno de Reuniones)

Community smell que se presenta cuando los calendarios del equipo están saturados de reuniones, muchas de ellas innecesarias o mal estructuradas, dejando poco tiempo para el trabajo profundo y generando frustración, fatiga y pérdida de productividad.. 83, 210, 216

Missing Link (Eslabón Perdido)

Falta de conectores, mediadores o boundary spanners clave entre subgrupos del equipo que faciliten comunicación efectiva. Resulta en aislamiento progresivo de grupos, información fragmentada sin integración, pérdida de contexto compartido y ruptura de redes colaborativas funcionales.. 36, 210, 212

Newbie Ignored (Newbie Ignorado)

Community smell que ocurre cuando los nuevos integrantes de un equipo son ignorados o marginados por los miembros más experimentados, dificultando su integración, reduciendo su motivación y generando aislamiento social que contribuye a la deuda social del equipo.. 36

Organizational Rigidity (Rigidez Organizacional)

Estructuras organizacionales inflexibles y procesos burocráticos que dificultan la adaptación a cambios tecnológicos y de mercado. Resulta en resistencia al cambio, innovación limitada, baja agilidad operativa y pérdida de competitividad.. 29, 36

Organizational Silo (Silo Organizacional)

Alto desacople lógico de tareas con baja comunicación y colaboración entre equipos o departamentos. Dificulta verificación de dependencias entre componentes, impide comunicación efectiva entre equipos que deben colaborar, genera decisiones locales óptimas pero globalmente subóptimas y pone en riesgo la congruencia socio-técnica general.. 8, 33, 36, 37, 210, 216, 217

Outdated Documentation (Documentación Desactualizada)

Community smell que aparece cuando la documentación existente del proyecto es incorrecta, desactualizada o no refleja el estado real del sistema. Genera desconfianza

en las fuentes escritas, aumenta la dependencia del conocimiento oral y dificulta el onboarding de nuevos miembros.. 210, 214

Overload (Sobrecarga)

Community smell que ocurre cuando uno o más miembros del equipo asumen una cantidad excesiva de tareas, responsabilidades o roles simultáneos, superando su capacidad sostenible. Genera estrés crónico, errores por fatiga y desequilibrio en la distribución del trabajo.. 37

Pair Programming (Programación en Pareja)

Técnica de desarrollo ágil donde dos desarrolladores trabajan juntos en una misma estación de trabajo. Uno escribe el código mientras el otro revisa y proporciona feedback en tiempo real, fomentando la colaboración y la calidad del software.. 33

Power Holder (Concentrador de Poder)

Community smell que se manifiesta cuando uno o más individuos concentran poder de decisión desproporcionado dentro del equipo, limitando la participación democrática, inhibiendo la autonomía de los demás y generando dinámicas jerárquicas que erosionan la colaboración horizontal.. 36

Priggish Members (Miembros Remilgados)

Miembros del equipo con actitudes presuntuosas y exigentes que demandan conformidad excesiva e inapropiada de otros miembros. Genera frustración recurrente en el equipo, ralentización de procesos por imposición de criterios puntillosos, degradación del clima laboral y fricción interpersonal.. 25, 31, 36

Prima Donnas (Primas Donnas)

Miembros que trabajan en aislamiento deliberado, rechazan cambios evolutivos en productos legacy y rehúsan proporcionar soporte genuino a compañeros. Bloquean activamente innovación organizacional, impiden comunicación y colaboración efectiva, generan dependencias negativas y erosionan cohesión.. 36, 83, 210, 215

Process Overhead (Sobrecarga de Procesos)

Community smell que se manifiesta cuando los procesos organizacionales —aprobaciones múltiples, burocracia excesiva, flujos de trabajo rígidos— consumen tiempo y energía sin agregar valor significativo, reduciendo la agilidad y la motivación del equipo.. 210, 216

Radio Silence (Silencio Radiofónico)

Comunicación insuficiente o nula entre equipos. Información crítica no circula, decisiones no se comparten.. 8, 32, 36, 82, 145, 210, 212

Risk Taking in Relationship (RTR, Toma de Riesgos en las Relaciones)

Disposición de los miembros del equipo para asumir riesgos interpersonales al confiar en otros. Implica vulnerabilidad y apertura en las relaciones laborales, facilitando la colaboración y el apoyo mutuo.. 34

Salud Social

Estado general de las dinámicas humanas, relacionales y organizacionales dentro de un equipo de desarrollo de software. Abarca aspectos como la comunicación, la confianza, la colaboración, el bienestar emocional y la cohesión del grupo. En SocialScrum, la salud social se mide a través del Health Score y se gestiona de forma sistemática mediante eventos, artefactos y roles especializados.. 25

Scattered Development (Desarrollo Disperso)

Community smell que describe una situación en la que el esfuerzo de desarrollo se dispersa entre demasiadas tareas, proyectos o prioridades simultáneas, impidiendo que el equipo logre profundidad y calidad en su trabajo. Este patrón fragmenta la atención y reduce la eficacia colectiva.. 37

seguridad psicológica

Creencia compartida por los miembros de un equipo de que el entorno es seguro para asumir riesgos interpersonales, expresar ideas, admitir errores y plantear preocupaciones sin temor a represalias. Es un requisito fundamental para la transparencia en SocialScrum.. 19

Sharing Villainy (Villanía del Compartir)

Retención deliberada u ofuscación intencional de conocimiento técnico e información valiosa por parte de miembros del equipo. Daña la cooperación organizacional, reduce coordinación efectiva, obstaculiza el cumplimiento de objetivos compartidos y genera ciclos de reinversión costosos.. 36

Social Guide

Rol especializado introducido por SocialScrum cuya misión es cuidar la dimensión social del equipo. Actúa como facilitador y consultor en temas humanos y organizacionales, sin ejercer autoridad jerárquica. Su objetivo principal es mantener el equilibrio emocional y relacional del grupo, garantizando la confianza, la comunicación y la cooperación como pilares constantes del trabajo.. 17, 25, 26, 58, 60

Social Isolation (Aislamiento Social)

Miembros del equipo experimentan desconexión social significativa debido a barreras geográficas, culturales o de comunicación. Genera sentimientos de exclusión,

disminución de la colaboración efectiva, pérdida de cohesión grupal y reducción del intercambio genuino de conocimiento.. 30, 36

SocialScrum

Proceso ágil que extiende el Scrum Framework para gestionar explícitamente la deuda social en equipos de desarrollo. Incorpora eventos, roles y artefactos dedicados a identificar y mitigar community smells.. 15

Solution Defiance (Desafío a la Solución)

Resistencia activa y persistente de grupos con factores profundos similares (background técnico, valores organizacionales) a decisiones y soluciones acordadas formalmente. Mina alineación técnica y social, genera conflictos recurrentes en reuniones de decisión, reduce cohesión organizacional y paraliza avance.. 29, 36

Stakeholders (Partes Interesadas)

Individuos o grupos que tienen interés o están afectados por el desarrollo del software, incluyendo clientes, usuarios finales, desarrolladores, gerentes y otros actores relevantes. Su participación es crucial para el éxito del proyecto.. 60

Synchronous Overload (Sobrecarga Sincrónica)

Community smell que ocurre cuando el equipo depende excesivamente de comunicación sincrónica (reuniones, videollamadas), saturando los calendarios, fragmentando el tiempo de trabajo profundo y generando fatiga comunicacional que reduce la productividad y el bienestar.. 210, 212

Time Wasted (Tiempo Perdido)

Community smell que se refiere a la pérdida sistemática de tiempo productivo debido a reuniones innecesarias, procesos burocráticos excesivos, interrupciones frecuentes o falta de priorización clara. Afecta el foco del equipo y genera frustración acumulada.. 37

Toxic Leadership (Liderazgo Tóxico)

Community smell que describe un estilo de liderazgo basado en el micromanagement, la culpabilización pública, la generación de miedo o la manipulación. Este patrón destruye la seguridad psicológica, inhibe la autonomía del equipo y genera un deterioro profundo de la confianza y el bienestar colectivo.. 83, 210, 215, 217

Toxic Relationships (Relaciones Tóxicas)

Relaciones interpersonales disfuncionales y conflictivas entre miembros del equipo que generan un ambiente laboral negativo. Causa estrés elevado, disminución de la

productividad, alta rotación de personal y erosión de la confianza mutua.. 30, 31, 36

Tribal Knowledge (Conocimiento Tribal)

Información crítica del sistema, decisiones arquitectónicas previas y racionales de diseño existen únicamente en mentes de desarrolladores específicos sin documentación formal. Causa vulnerabilidad extrema ante rotación de personal, dificulta significativamente el onboarding de nuevos miembros y genera riesgo de pérdida irreversible de información.. 210, 214

Truck Factor (Key Person Dependency)

Conocimiento técnico crítico y habilidades esenciales están concentradas en pocas personas cuyos ausencias, desvinculación laboral o sobrecarga amenazan directamente la continuidad operativa del proyecto. Genera fragilidad organizacional extrema, riesgo sistémico de pérdida de capacidad técnica y vulnerabilidad ante contingencias.. 36, 210

Unclear Ownership (Propiedad Poco Clara)

Community smell que aparece cuando no hay claridad sobre quién es responsable de qué componentes, decisiones o procesos dentro del equipo. Genera preguntas frecuentes, decisiones postergadas, duplicación de esfuerzos y ambigüedad operativa.. 210, 216

Unfriendly Environment (Ambiente Hostil)

Community smell que describe un ambiente de trabajo hostil, poco acogedor o emocionalmente inseguro, donde los miembros del equipo no se sienten cómodos para expresarse, colaborar o reportar problemas. Este patrón deteriora la confianza, la cohesión y el bienestar colectivo.. 36

Unhealthy Contribution Patterns (Patrones de Contribución No Saludables)

Community smell que se presenta cuando la distribución del trabajo y las contribuciones dentro del equipo es desigual o desequilibrada, generando sobrecarga en algunos miembros y subutilización en otros, lo que afecta la equidad, la motivación y la sostenibilidad del equipo.. 36

Unresolved Conflict (Conflicto No Resuelto)

Community smell que ocurre cuando los conflictos interpersonales dentro del equipo no se abordan ni resuelven, generando tensión sostenida, evitación mutua, comentarios pasivo-agresivos y un ambiente de trabajo deteriorado que alimenta la deuda social.. 210, 215

Unstable Team (Equipo Inestable)

Community smell que ocurre cuando la composición del equipo cambia frecuentemente debido a rotación de personal, préstamos de recursos o decisiones organizacionales, impidiendo la construcción de confianza, cohesión y conocimiento compartido necesarios para un desempeño sostenible.. 210, 216, 217

Work Exhaustion (Agotamiento Laboral)

Community smell relacionado con el agotamiento crónico derivado de cargas de trabajo excesivas o mal distribuidas. Se manifiesta en fatiga persistente, reducción del rendimiento y deterioro del bienestar emocional, siendo un precursor directo del burnout y un indicador crítico de deuda social.. 37